

OSCAT LIBRARY Dokumentation

by

The WWW.OSCAT.DE Documentation Project

Bibliothek Version 2.74

Inhaltsverzeichnis

1	<u>Rechtsgrundlagen</u>	15
1.1	<u>Haftungsausschluss</u>	15
1.2	<u>Lizenzbedingungen</u>	15
1.3	<u>Eingetragene Markenzeichen</u>	16
1.4	<u>Bestimmungsgemäßer Gebrauch</u>	16
1.5	<u>Sonstiges</u>	16
2	<u>Einleitung</u>	17
2.1	<u>Ziele</u>	17
2.2	<u>Konventionen</u>	18
2.3	<u>Testumgebung</u>	19
2.4	<u>Globale Konstanten</u>	20
2.5	<u>Aktualität</u>	22
2.6	<u>Support</u>	22
3	<u>Diverse Funktionen</u>	23
3.1	<u>ESR_COLLECT</u>	23
3.2	<u>ESR_MON_B8</u>	25
3.3	<u>ESR_MON_R4</u>	26
3.4	<u>ESR_MON_X8</u>	27
3.5	<u>OSCAT_VERSION</u>	28
3.6	<u>STATUS_TO_ESR</u>	28
4	<u>Mathematik</u>	30
4.1	<u>ACOSH</u>	30
4.2	<u>ACOTH</u>	30
4.3	<u>AGDF</u>	31
4.4	<u>ASINH</u>	31
4.5	<u>ATAN2</u>	31
4.6	<u>ATANH</u>	32
4.7	<u>CEIL</u>	32
4.8	<u>CEIL2</u>	33
4.9	<u>CMP</u>	33
4.10	<u>COSH</u>	34
4.11	<u>COTH</u>	34
4.12	<u>D_TRUNC</u>	35
4.13	<u>DEG</u>	35
4.14	<u>DIFFER</u>	36

4.15	<u>ERF</u>	36
4.16	<u>ERFC</u>	37
4.17	<u>EVEN</u>	37
4.18	<u>EXP10</u>	38
4.19	<u>EXPN</u>	38
4.20	<u>FACT</u>	39
4.21	<u>FIB</u>	39
4.22	<u>FLOOR</u>	40
4.23	<u>FLOOR2</u>	40
4.24	<u>FRACT</u>	41
4.25	<u>GAMMA</u>	41
4.26	<u>GAUSS</u>	42
4.27	<u>GAUSSCD</u>	42
4.28	<u>GDF</u>	43
4.29	<u>GOLD</u>	43
4.30	<u>HYPOT</u>	44
4.31	<u>INC</u>	44
4.32	<u>LANGEVIN</u>	45
4.33	<u>MAX3</u>	46
4.34	<u>MIN3</u>	46
4.35	<u>MID3</u>	47
4.36	<u>MODR</u>	47
4.37	<u>MUL_ADD</u>	48
4.38	<u>NEG</u>	48
4.39	<u>RAD</u>	49
4.40	<u>RDM</u>	49
4.41	<u>RDM2</u>	50
4.42	<u>RDMDW</u>	50
4.43	<u>RND</u>	51
4.44	<u>ROUND</u>	52
4.45	<u>SGN</u>	52
4.46	<u>SIGMOID</u>	53
4.47	<u>SIGN_I</u>	53
4.48	<u>SIGN_R</u>	54
4.49	<u>SINC</u>	54
4.50	<u>SINH</u>	54
4.51	<u>SQRTN</u>	55
4.52	<u>TANH</u>	55
4.53	<u>WINDOW</u>	56
4.54	<u>WINDOW2</u>	56

5	<u>Array</u>	57
5.1	<u>_ARRAY_ABS</u>	57
5.2	<u>_ARRAY_ADD</u>	58
5.3	<u>_ARRAY_INIT</u>	58
5.4	<u>_ARRAY_MEDIAN</u>	59

5.5	<u>ARRAY_MUL</u>	60
5.6	<u>ARRAY_SHUFFLE</u>	61
5.7	<u>ARRAY_SORT</u>	62
5.8	<u>ARRAY_AVG</u>	62
5.9	<u>ARRAY_GAV</u>	63
5.10	<u>ARRAY_HAV</u>	64
5.11	<u>ARRAY_MAX</u>	64
5.12	<u>ARRAY_MIN</u>	65
5.13	<u>ARRAY_SDV</u>	66
5.14	<u>ARRAY_SPR</u>	66
5.15	<u>ARRAY_SUM</u>	67
5.16	<u>ARRAY_TREND</u>	68
5.17	<u>ARRAY_VAR</u>	68
5.18	<u>IS_SORTED</u>	69

6	<u>Komplexe Mathematik</u>	70
6.1	<u>EINLEITUNG</u>	70
6.2	<u>CABS</u>	70
6.3	<u>CACOS</u>	71
6.4	<u>CACOSH</u>	71
6.5	<u>CADD</u>	71
6.6	<u>CARG</u>	72
6.7	<u>CASIN</u>	72
6.8	<u>CASINH</u>	73
6.9	<u>CATAN</u>	73
6.10	<u>CATANH</u>	73
6.11	<u>CCON</u>	74
6.12	<u>CCOS</u>	74
6.13	<u>CCOSH</u>	74
6.14	<u>CDIV</u>	75
6.15	<u>CEXP</u>	75
6.16	<u>CINV</u>	75
6.17	<u>CLOG</u>	76
6.18	<u>CMUL</u>	76
6.19	<u>CPOL</u>	76
6.20	<u>CPOW</u>	77
6.21	<u>CSET</u>	77
6.22	<u>CSIN</u>	78
6.23	<u>CSINH</u>	78
6.24	<u>CSQRT</u>	78
6.25	<u>CSUB</u>	79
6.26	<u>CTAN</u>	79
6.27	<u>CTANH</u>	79

7	<u>Arithmetik doppelter Genauigkeit</u>	80
7.1	<u>Einleitung</u>	80
7.2	<u>R2_ABS</u>	80
7.3	<u>R2_ADD</u>	81
7.4	<u>R2_ADD2</u>	81
7.5	<u>R2_MUL</u>	82
8	<u>Arithmetische Funktionen</u>	83
8.1	<u>F_LIN</u>	83
8.2	<u>F_LIN2</u>	83
8.3	<u>F_POLY</u>	84
8.4	<u>F_POWER</u>	84
8.5	<u>F_QUAD</u>	85
8.6	<u>FT_AVG</u>	85
8.7	<u>FT_MIN_MAX</u>	86
8.8	<u>FT_RMP</u>	87
8.9	<u>LINEAR_INT</u>	88
8.10	<u>POLYNOM_INT</u>	89
9	<u>Geometrische Funktionen</u>	91
9.1	<u>CIRCLE_A</u>	91
9.2	<u>CIRCLE_C</u>	91
9.3	<u>CONE_V</u>	92
9.4	<u>ELLIPSE_A</u>	92
9.5	<u>ELLIPSE_C</u>	93
9.6	<u>SPHERE_V</u>	93
9.7	<u>TRIANGLE_A</u>	94
10	<u>Vektor Mathematik</u>	95
10.1	<u>Einleitung</u>	95
10.2	<u>V3_ABS</u>	95
10.3	<u>V3_ADD</u>	96
10.4	<u>V3_ANG</u>	96
10.5	<u>V3_DPRO</u>	96
10.6	<u>V3_NORM</u>	97
10.7	<u>V3_NUL</u>	97
10.8	<u>V3_PAR</u>	98
10.9	<u>V3_REV</u>	98
10.10	<u>V3_SMUL</u>	99
10.11	<u>V3_SUB</u>	99
10.12	<u>V3_XANG</u>	100
10.13	<u>V3_XPRO</u>	100

10.14	<u>V3 YANG</u>	101
10.15	<u>V3_ZANG</u>	101
11	<u>Time & Date</u>	102
11.1	<u>DATE_ADD</u>	102
11.2	<u>DAY_OF_DATE</u>	103
11.3	<u>DAY_OF_MONTH</u>	103
11.4	<u>DAY_OF_YEAR</u>	103
11.5	<u>DAY_TO_TIME</u>	104
11.6	<u>DAYS_DELTA</u>	104
11.7	<u>DCF77</u>	105
11.8	<u>DST</u>	106
11.9	<u>EASTER</u>	107
11.10	<u>HOLIDAY</u>	107
11.11	<u>HOURL</u>	108
11.12	<u>HOURL_TO_TIME</u>	109
11.13	<u>HOURL_TO_TOD</u>	109
11.14	<u>LEAP_OF_DATE</u>	109
11.15	<u>LEAP_YEAR</u>	110
11.16	<u>LOCAL_TIME</u>	110
11.17	<u>LTIME</u>	111
11.18	<u>LTIME_TO_UTC</u>	112
11.19	<u>MINUTE</u>	112
11.20	<u>MINUTE_TO_TIME</u>	113
11.21	<u>MONTH</u>	113
11.22	<u>MULTIME</u>	114
11.23	<u>PERIOD</u>	114
11.24	<u>PERIOD2</u>	115
11.25	<u>RTC_2</u>	115
11.26	<u>RTC_MS</u>	116
11.27	<u>SECOND</u>	117
11.28	<u>SECOND_TO_TIME</u>	117
11.29	<u>SET_DATE</u>	118
11.30	<u>SET_DT</u>	119
11.31	<u>SET_TOD</u>	119
11.32	<u>SUN_POS</u>	120
11.33	<u>SUN_TIME</u>	121
11.34	<u>WEEKDAY</u>	122
11.35	<u>WORK_WEEK</u>	122
11.36	<u>YEAR</u>	123
11.37	<u>YEAR_BEGIN</u>	123
12	<u>String Funktionen</u>	124
12.1	<u>BYTE_TO_STRB</u>	124
12.2	<u>BYTE_TO_STRH</u>	124

12.3	<u>CAPITALIZE</u>	125
12.4	<u>CHARCODE</u>	125
12.5	<u>CHARNAME</u>	126
12.6	<u>CHR</u>	127
12.7	<u>CODE</u>	127
12.8	<u>COUNT_CHAR</u>	128
12.9	<u>DWORD_TO_STRB</u>	128
12.10	<u>DWORD_TO_STRF</u>	129
12.11	<u>DWORD_TO_STRH</u>	129
12.12	<u>EXEC</u>	130
12.13	<u>FILL</u>	130
12.14	<u>FIND_CHAR</u>	131
12.15	<u>FIND_CTRL</u>	131
12.16	<u>FIND_NONUM</u>	132
12.17	<u>FIND_NUM</u>	132
12.18	<u>FINDB</u>	133
12.19	<u>FINDB_NONUM</u>	133
12.20	<u>FINDB_NUM</u>	134
12.21	<u>FINDP</u>	134
12.22	<u>FIX</u>	135
12.23	<u>HEX_TO_DWORD</u>	135
12.24	<u>IS_ALNUM</u>	136
12.25	<u>IS_ALPHA</u>	136
12.26	<u>IS_CC</u>	137
12.27	<u>IS_CTRL</u>	137
12.28	<u>IS_HEX</u>	138
12.29	<u>IS_LOWER</u>	138
12.30	<u>IS_NCC</u>	139
12.31	<u>IS_NUM</u>	139
12.32	<u>IS_UPPER</u>	140
12.33	<u>ISC_ALPHA</u>	140
12.34	<u>ISC_CTRL</u>	141
12.35	<u>ISC_HEX</u>	142
12.36	<u>ISC_LOWER</u>	142
12.37	<u>ISC_NUM</u>	143
12.38	<u>ISC_UPPER</u>	143
12.39	<u>LOWERCASE</u>	144
12.40	<u>MESSAGE_8</u>	144
12.41	<u>MIRROR</u>	146
12.42	<u>REAL_TO_STRF</u>	146
12.43	<u>REPLACE_ALL</u>	147
12.44	<u>REPLACE_CHARS</u>	147
12.45	<u>REPLACE_UML</u>	148
12.46	<u>TICKER</u>	148
12.47	<u>TO_LOWER</u>	149
12.48	<u>TO_UML</u>	149
12.49	<u>TO_UPPER</u>	150
12.50	<u>TRIM</u>	151

12.51	TRIM1	151
12.52	TRIME	151
12.53	UPPERCASE	152
13	Speicherbausteine	153
13.1	FIFO_16	153
13.2	FIFO_32	154
13.3	STACK_16	154
13.4	STACK_32	155
14	Takt Generatoren	156
14.1	B_TRIG	156
14.2	CLK_1MS	156
14.3	CLK_2MS	157
14.4	CLK_4MS	157
14.5	CLK_8MS	157
14.6	CLK_DIV	157
14.7	CLK_N	159
14.8	CLK_PRG	160
14.9	CLK_PULSE	160
14.10	D_TRIG	161
14.11	GEN_BIT	162
14.12	GEN_SQ	164
14.13	SEQUENCE_4	165
14.14	SEQUENCE_8	168
14.15	TONOF	169
14.16	TP_R	170
15	Logik Bausteine	171
15.1	BCD_TO_INT	171
15.2	BIT_COUNT	171
15.3	BIT_LOAD_B	172
15.4	BIT_LOAD_DW	172
15.5	BIT_LOAD_W	173
15.6	BIT_OF_DWORD	173
15.7	BYTE_OF_BIT	174
15.8	BYTE_OF_DWORD	174
15.9	BYTE_TO_BIT	175
15.10	CHECK_PARITY	175
15.11	CRC_CHECK	176
15.12	CRC_GEN	177
15.13	DEC_2	179
15.14	DEC_4	180

15.15	<u>DEC_8</u>	181
15.16	<u>DW_TO_REAL</u>	182
15.17	<u>DWORD_OF_BYTE</u>	183
15.18	<u>INT_TO_BCD</u>	183
15.19	<u>MATRIX</u>	184
15.20	<u>MUX_2</u>	186
15.21	<u>MUX_4</u>	186
15.22	<u>PARITY</u>	187
15.23	<u>REAL_TO_DW</u>	188
15.24	<u>REVERSE</u>	188
15.25	<u>SHL1</u>	189
15.26	<u>SHR1</u>	189
15.27	<u>SWAP_BYTE</u>	190
15.28	<u>SWAP_BYTE2</u>	190
15.29	<u>WORD_OF_BYTE</u>	190
15.30	<u>WORD_OF_DWORD</u>	191
16	<u>Latches, Flip-Flop und Schieberegister</u>	192
16.1	<u>COUNT_BR</u>	192
16.2	<u>FF_D2E</u>	193
16.3	<u>FF_D4E</u>	194
16.4	<u>FF_DRE</u>	195
16.5	<u>FF_JKE</u>	196
16.6	<u>SELECT_8</u>	197
16.7	<u>LATCH</u>	198
16.8	<u>LATCH4</u>	198
16.9	<u>SHR_4E</u>	199
16.10	<u>SHR_4UDE</u>	200
16.11	<u>SHR_8PLE</u>	201
16.12	<u>SHR_8UDE</u>	202
16.13	<u>STORE_8</u>	203
16.14	<u>TOGGLE</u>	204
17	<u>Signalgeneratoren</u>	205
17.1	<u>GEN_RDM</u>	205
17.2	<u>GEN_RDT</u>	206
17.3	<u>GEN_RMP</u>	206
17.4	<u>GEN_SIN</u>	207
17.5	<u>GEN_SQR</u>	209
17.6	<u>PWM_DC</u>	210
17.7	<u>PWM_PW</u>	211
17.8	<u>RMP_B</u>	211
17.9	<u>RMP_SOFT</u>	214
17.10	<u>RMP_W</u>	215

18	<u>Signalverarbeitung</u>	216
18.1	<u>AIN</u>	216
18.2	<u>AIN1</u>	217
18.3	<u>AOUT</u>	219
18.4	<u>AOUT1</u>	220
18.5	<u>BYTE_TO_RANGE</u>	221
18.6	<u>DELAY</u>	221
18.7	<u>DELAY_4</u>	222
18.8	<u>FADE</u>	224
18.9	<u>MIX</u>	225
18.10	<u>MUX_R2</u>	225
18.11	<u>MUX_R4</u>	226
18.12	<u>OFFSET</u>	226
18.13	<u>OFFSET2</u>	228
18.14	<u>OVERRIDE</u>	229
18.15	<u>RANGE_TO_BYTE</u>	230
18.16	<u>RANGE_TO_WORD</u>	230
18.17	<u>SCALE</u>	231
18.18	<u>SCALE_B</u>	231
18.19	<u>SCALE_B2</u>	232
18.20	<u>SCALE_B4</u>	234
18.21	<u>SCALE_B8</u>	235
18.22	<u>SCALE_D</u>	236
18.23	<u>SCALE_R</u>	236
18.24	<u>SCALE_X2</u>	237
18.25	<u>SCALE_X4</u>	238
18.26	<u>SCALE_X8</u>	239
18.27	<u>SEL2_OF_3</u>	240
18.28	<u>SEL2_OF_3B</u>	240
18.29	<u>SH</u>	241
18.30	<u>SH_1</u>	242
18.31	<u>SH_2</u>	243
18.32	<u>SH_T</u>	245
18.33	<u>STAIR</u>	246
18.34	<u>STAIR2</u>	247
18.35	<u>WORD_TO_RANGE</u>	249
19	<u>Sensorik</u>	250
19.1	<u>MULTI_IN</u>	250
19.2	<u>RES_NI</u>	251
19.3	<u>RES_PT</u>	252
19.4	<u>RES_SI</u>	253
19.5	<u>SENSOR_INT</u>	254
19.6	<u>TEMP_NI</u>	255
19.7	<u>TEMP_PT</u>	256
19.8	<u>TEMP_SI</u>	257

20	<u>Messbausteine</u>	258
20.1	<u>ALARM_2</u>	258
20.2	<u>BAR_GRAPH</u>	259
20.3	<u>CALIBRATE</u>	261
20.4	<u>CYCLE_TIME</u>	262
20.5	<u>METER</u>	262
20.6	<u>METER_STAT</u>	264
20.7	<u>M_D</u>	265
20.8	<u>M_T</u>	266
20.9	<u>M_TX</u>	267
20.10	<u>ONTIME</u>	268
20.11	<u>T_PLC_MS</u>	269
20.12	<u>T_PLC_US</u>	270
20.13	<u>TC_MS</u>	272
20.14	<u>TC_S</u>	272
20.15	<u>TC_US</u>	272
21	<u>Umrechnungen</u>	274
21.1	<u>ASTRO</u>	274
21.2	<u>C_TO_F</u>	275
21.3	<u>C_TO_K</u>	275
21.4	<u>ENERGY</u>	275
21.5	<u>F_TO_C</u>	276
21.6	<u>F_TO_OM</u>	276
21.7	<u>F_TO_PT</u>	277
21.8	<u>KMH_TO_MS</u>	277
21.9	<u>LENGTH</u>	277
21.10	<u>MS_TO_KMH</u>	279
21.11	<u>K_TO_C</u>	279
21.12	<u>OM_TO_F</u>	279
21.13	<u>PRESSURE</u>	280
21.14	<u>PT_TO_F</u>	281
21.15	<u>SPEED</u>	281
21.16	<u>TEMPERATURE</u>	282
22	<u>Regelungstechnik</u>	283
22.1	<u>CONTROL_SET1</u>	283
22.2	<u>CONTROL_SET2</u>	284
22.3	<u>DEAD_BAND</u>	286
22.4	<u>DEAD_ZONE</u>	286
22.5	<u>DEAD_ZONE2</u>	287
22.6	<u>FT_DERIV</u>	288
22.7	<u>FT_IMP</u>	288
22.8	<u>FT_INT</u>	289

22.9	FT_PI	290
22.10	FT_PID	293
22.11	FT_PT1	295
22.12	FT_PT2	297
22.13	FT_TN16	298
22.14	FT_TN64	298
22.15	FT_TN8	299
22.16	HYST_1	300
22.17	HYST_2	301
22.18	HYST_3	301
23	Stellglieder	303
23.1	ACTUATOR_2P	303
23.2	ACTUATOR_3P	304
23.3	ACTUATOR_PUMP	306
23.4	ACTUATOR_UD	307
24	Heizung Lüftung und Klima	309
24.1	AIR_DENSITY	309
24.2	AIR_ENTHALPY	309
24.3	BOILER	310
24.4	BURNER	312
24.5	DEW_CON	316
24.6	DEW_RH	316
24.7	DEW_TEMP	317
24.8	HEAT_INDEX	317
24.9	HEAT_METER	318
24.10	HEAT_TEMP	319
24.11	LEGIONELLA	321
24.12	SDD	324
24.13	TEMP_EXT	325
24.14	WATER_CP	327
24.15	WATER_DENSITY	327
24.16	WATER_ENTHALPY	328
24.17	WCT	328
25	Elektro	329
25.1	CLICK	329
25.2	DIMM_I	331
25.3	F_LAMP	332
25.4	PULSE_LENGTH	333
25.5	SWITCH_I	334
25.6	SWITCH_X	334

25.7	<u>TIMER_1</u>	335
25.8	<u>TIMER_2</u>	336
26	<u>Jalousiesteuerung</u>	339
26.1	<u>Einleitung</u>	339
26.2	<u>BLIND_ACTUATOR</u>	340
26.3	<u>BLIND_CONTROL</u>	342
26.4	<u>BLIND_INPUT</u>	344
26.5	<u>BLIND_NIGHT</u>	346
26.6	<u>BLIND_SCENE</u>	348
26.7	<u>BLIND_SECURITY</u>	350
26.8	<u>BLIND_SHADE</u>	352
27	<u>Automation</u>	356
27.1	<u>DRIVER_1</u>	356
27.2	<u>DRIVER_4</u>	357
27.3	<u>FT_PROFILE</u>	357
27.4	<u>INC_DEC</u>	360
27.5	<u>INTERLOCK</u>	361
27.6	<u>PARSET</u>	362
27.7	<u>PARSET2</u>	363
27.8	<u>SIGNAL</u>	364
27.9	<u>SIGNAL_4</u>	365
27.10	<u>SRAMP</u>	366
27.11	<u>TUNE</u>	368
27.12	<u>TUNE2</u>	369
28	<u>Geräte Treiber</u>	370
28.1	<u>IRTRANS</u>	370
28.1.1	<u>IRTRANS_1</u>	370
28.1.2	<u>IRTRANS_4</u>	372
28.1.3	<u>IRTRANS_8</u>	372
28.1.4	<u>IRTRANS_DECODE</u>	373
29	<u>Netzwerk und Kommunikation</u>	376
29.1	<u>HTML_DECODE</u>	376
29.2	<u>HTML_ENCODE</u>	377
29.3	<u>IP4_CHECK</u>	377
29.4	<u>IP4_DECODE</u>	378
29.5	<u>IP4_TO_STRING</u>	379
29.6	<u>IS_IP4</u>	379

29.7	<u>IS_URLCHR</u>	380
29.8	<u>STRING_TO_URL</u>	380
29.9	<u>URL_DECODE</u>	381
29.10	<u>URL_ENCODE</u>	381
29.11	<u>URL_TO_STRING</u>	382

30	<u>BUFFER Management</u>	383
-----------	---	------------

30.1	<u>_BUFFER_CLEAR</u>	383
30.2	<u>_BUFFER_INIT</u>	384
30.3	<u>_BUFFER_INSERT</u>	384
30.4	<u>STRING_TO_BUFFER</u>	385
30.5	<u>BUFFER_SEARCH</u>	386
30.6	<u>BUFFER_TO_STRING</u>	387

1 Rechtsgrundlagen

1.1 Haftungsausschluss

Die in der OSCAT Bibliothek enthaltenen Softwaremodule sind in der Absicht angeboten als Entwicklungsvorlage und Leitfaden zur Softwareentwicklung für SPS nach IEC61131-3 zu dienen. Eine Funktionsgarantie wird von den Entwicklern nicht übernommen und wird explizit ausgeschlossen. Da die in der Bibliothek enthaltenen Softwaremodule ohne jegliche Kosten bereitgestellt werden, besteht keinerlei Gewährleistung, soweit dies gesetzlich zulässig ist. Sofern nicht explizit schriftlich vereinbart, stellen die Copyright-Inhaber und / oder Dritte die Softwaremodule so zur Verfügung, „wie es ist“, ohne irgendeine Gewährleistung, weder ausdrücklich noch implizit, einschließlich - aber nicht begrenzt - auf Marktreife oder Verwendbarkeit für einen bestimmten Zweck. Das volle Risiko und die volle Verantwortung bezüglich Qualität, Fehlerfreiheit und Leistungsfähigkeit der Softwaremodule liegt beim Anwender selbst. Sollte sich die Bibliothek, oder Teile der Bibliothek als fehlerhaft erweisen, liegen die Kosten für notwendigen Service, Reparatur und / oder Korrektur beim Anwender selbst. Sollten Teile oder die gesamte Bibliothek zur Erstellung von Anwendungssoftware verwendet werden, oder in Softwareprojekten eingesetzt werden, so haftet der Anwender für die Fehlerfreiheit, Funktion und Qualität der Anwendung. Eine Haftung durch OSCAT ist grundsätzlich ausgeschlossen. Der Anwender der OSCAT Bibliothek hat durch geeignete Tests und Freigaben und Qualitätssicherungsmaßnahmen dafür zu sorgen, dass durch eventuelle Fehler in der Bibliothek von OSCAT keine Schäden entstehen können. Die vorstehenden Lizenzbedingungen und Haftungsausschlüsse gelten gleichermaßen für die Softwarebibliothek, sowie die in diesem Handbuch angebotenen Beschreibungen und Erläuterungen, auch wenn dies nicht explizit erwähnt wird.

1.2 Lizenzbedingungen

Die Verwendung der OSCAT Bibliothek ist kostenfrei und kann ohne Lizenzvereinbarung für private oder gewerbliche Zwecke eingesetzt werden. Eine Verbreitung der Bibliothek ist ausdrücklich erwünscht, hat aber kostenfrei und unter Hinweis auf unsere Webpage WWW.OSCAT.DE zu erfolgen. Wird die Bibliothek in elektronischer

Form zum Download bereitgestellt, oder auf Datenträgern verbreitet, so ist sicherzustellen, dass ein deutlich erkennbarer Hinweis auf OSCAT und ein Weblink zu WW.W.OSCAT.DE entsprechend enthalten sind.

1.3 Eingetragene Markenzeichen

Alle in dieser Beschreibung benutzen Markennamen werden ohne Verweis auf deren Eintragung beziehungsweise Besitzer verwendet. Die Existenz solcher Rechte kann daher nicht ausgeschlossen werden. Die benutzten Markennamen sind Eigentum des jeweiligen Besitzers. Eine Verwendung der Beschreibung für kommerzielle Zwecke ist daher nicht gestattet, auch nicht auszugsweise.

1.4 Bestimmungsgemäßer Gebrauch

Die in der OSCAT Bibliothek enthaltenen und in dieser Dokumentation beschriebenen Softwaremodule sind ausschließlich für Fachpersonal mit einer Ausbildung in SPS Programmierung. Die Anwender sind verantwortlich für die Einhaltung aller geltenden Normen und Vorschriften die bei der Anwendung zum tragen kommen. OSCAT weist weder im Handbuch noch in der Software auf diese Normen und Vorschriften hin.

1.5 Sonstiges

Alle Rechtsverbindlichen Regelungen befinden sich ausschließlich im Kapitel 1 dieses Handbuchs. Eine Ableitung oder Bezug von rechtlichen Ansprüchen aufgrund des Inhalts dieses Manuals außer den Bestimmungen in Kapitel 1 ist gänzlich ausgeschlossen.

2 Einleitung

2.1 Ziele

OSCAT steht für "Open Source Community for Automation Technology".

OSCAT erstellt eine Open Source Bibliothek nach dem IEC61131-3 Standard, welche auf herstellerspezifische Funktionen verzichtet und deshalb auf alle IEC61131-3 kompatiblen Speicherprogrammierbaren Steuerungen portiert werden kann. Auch wenn Entwicklungen für SPS unter dem Einsatz von herstellerspezifischen Bibliotheken meist effizient zu lösen sind und diese Bibliotheken auch teilweise kostenfrei zur Verfügung gestellt werden, ergeben sich doch große Nachteile durch ihren Einsatz:

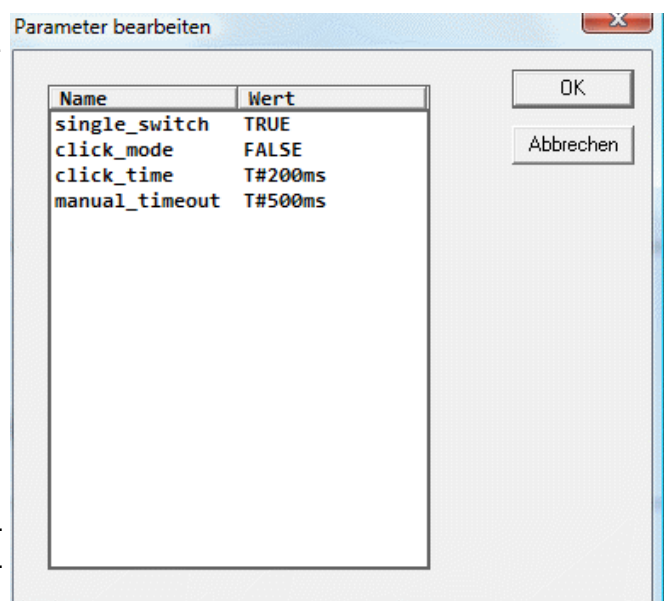
1. Die Bibliotheken der Hersteller sind praktisch alle geschützt und der Source Code ist nicht frei zugänglich, was im Fehlerfall eine Fehlersuche und auch die Behebung des Fehlers enorm schwierig, oft sogar unmöglich macht.
2. Die grafische Erstellung von Programmen kann mit herstellerspezifischen Bibliotheken schnell unübersichtlich, ineffizient und fehleranfällig werden, weil vorhandene Funktionen wegen des fehlenden Source Codes den eigentlichen Bedürfnissen nicht angepasst und erweitert werden können.
3. Ein Wechsel der Hardware, insbesondere der Wechsel zu einem anderen Hersteller, ist durch die geschützten Bibliotheken verhindert und die Vorteile, die ein Standard wie IEC61131 bieten würde, werden so eingeschränkt. An einen Austausch einer herstellerspezifischen Bibliothek mit der eines Wettbewerbers ist ausgeschlossen, denn die Bibliotheken der Hersteller unterscheiden sich enorm in Umfang und Inhalt.
4. Das Verständnis komplexer Module ist ohne einen Einblick in den Sourcecode oft sehr schwierig. Dadurch werden Programme ineffizient und fehleranfällig.

OSCAT will mit der quelloffenen OSCAT Bibliothek einen mächtigen und umfassenden Standard für die Programmierung von SPS schaffen, der im Source Code zur Verfügung steht und durch vielfältige Anwendungen ausführlich verifiziert und getestet wurde. Weiterhin fließen durch die Vielzahl der Anwendungen umfangreiche Erkenntnisse und Anregungen in die Bibliothek ein. Dadurch kann die Bibliothek als sehr praxisnah bezeichnet werden. OSCAT versteht seine Bibliothek als Entwicklungsvorlage und nicht als ausgereiftes Produkt. Der Nutzer ist selbst verantwortlich dafür, eventuell in seiner Anwendung verwendete Module mit geeigneten Verfahren zu testen und die notwendige Fehlerfreiheit, Qualität und Funktionalität zu verifizie-

ren. An dieser Stelle sei noch einmal auf die Lizenzbedingungen und den Haftungsausschluss in dieser Dokumentation hingewiesen.

2.2 Konventionen

1. Direkte Manipulationen im Speicher:
Funktionen, die Eingangswerte über Pointer verändern, wie zum Beispiel `_Array_Sort`, beginnen alle mit einem Unterstrich „_“. `_Array_Sort` sortiert ein Array direkt im Speicher, was den gravierenden Vorteil hat, dass ein eventuell sehr großes Array erst gar nicht der Funktion übergeben werden muss und deshalb Speicher in der Größe des Arrays und die Zeit zum Kopieren eingespart wird. Es wird allerdings nur erfahrenen Anwendern empfohlen diese Funktionen einzusetzen, da eine Fehlanwendung zu gravierenden Fehlern und Abstürzen führen kann! Bei Anwendung von Funktionen die mit einem „_“ beginnen ist besondere Sorgfalt angebracht und insbesondere darauf zu achten, dass die Aufrufparameter niemals undefinierte Werte annehmen können.
2. Namensgebung bei Funktionen:
Funktionsbausteine mit Zeitverhalten, wie etwa die Funktion `PT1` werden durch die Namensgebung `FT_Bausteinname` (`FT_PT1`) beschrieben. Funktionen ohne Zeitbezug sind mit `F_Bausteinname` angegeben.
3. Logische Gleichungen:
Innerhalb dieses Handbuchs werden die logischen Verknüpfungen `&` für UND bzw. AND, `+` für ODER bzw. OR, `/A` für ein negiertes A und `#` für XOR (exklusives ODER) verwendet.
4. Setup-Werte für Bausteine:
Damit die Anwendung und Programmierung übersichtlich bleibt und komplexe Funktionen einfacher dargestellt werden können, haben viele der Bausteine der OSCAT Bibliothek einstellbare Parameter, die bei Anwendung durch einen Doppelklick auf das grafische Symbol des Bausteins bearbeitet werden können. Ein Doppelklick auf das Symbol öffnet eine Dialogbox, die das Bearbeiten der Setup-Werte erlaubt. Wird eine Funktion mehrfach verwendet, so können damit je Baustein die Setup-Werte einzeln festgelegt werden. Die Bearbeitung



durch Doppelklick funktioniert unter CoDeSys ausschließlich in CFC. In ST müssen alle Parameter, auch die Setup-Parameter, im Aufruf übergeben werden. Die Setup-Parameter werden einfach nach den normalen Eingängen angefügt. Die Parameter werden in der grafischen Oberfläche durch Doppelklick bearbeitet und dann wie Konstanten unter IEC61131 eingegeben. Es ist Dabei zu beachten dass Zeiten mit T#200ms und TRUE und FALSE in Großbuchstaben geschrieben sein müssen.

5. Error- und Status-Reporting (ESR):

Komplexere Bausteine erhalten zum großen Teil einen Error- oder Status-Ausgang. Ein Error-Ausgang ist 0, falls kein Fehler bei der Ausführung auftritt. Tritt jedoch im Baustein ein Fehler auf, so nimmt dieser Ausgang einen Wert im Bereich 1 .. 99 an und meldet somit einen Fehler mit Fehlernummer. Ein Status- oder Error-Sammelmodul kann diese Meldungen sammeln und mit einem Zeitstempel versehen, in einer Datenbank bzw. Array speichern, oder per TCP/IP an übergeordnete Systeme weiterleiten. Ein Ausgang des Typs Status ist kompatibel zu einem Error-Ausgang mit identischer Funktion. Jedoch meldet ein Status-Ausgang nicht nur Fehler, sondern führt auch über Aktivitäten des Bausteins Protokoll. Werte zwischen 1 .. 99 sind weiterhin Fehlermeldungen. Zwischen 100 .. 199 befinden sind Meldungen über Zustandsveränderungen. Der Bereich 200 .. 255 ist reserviert für Debug-Meldungen. Mit dieser, innerhalb der OSCAT Bibliothek standardisierten Funktionalität, wird eine einfache und übergreifende Möglichkeit geboten, Betriebszustandsmeldungen und Fehlermeldungen auf einfache Weise zu integrieren, ohne die Funktion eines Systems zu beeinflussen. Bausteine, die dieses Verfahren unterstützen, werden ab der Revision 1.4 mit der Kennzeichnung „ESR-Fähig“ gekennzeichnet. Weitere Informationen zu den ESR-Bausteinen finden Sie im Abschnitt „Diverse Funktionen“.

2.3 Testumgebung

Die OSCAT Bibliothek wird unter CoDeSys entwickelt und auf verschiedenen Systemen getestet.

Die Testumgebung umfasst folgende Systeme:

1. Beckhoff BX 9000
mit TwinCAT PLC Control Version 2.10.0
2. Beckhoff CX 9001-1001
mit TwinCAT PLC Control Version 2.10.0
3. Wago 750-841
mit Wago CoDeSys Version 2.3.8.1

4. Möller EC4P222
mit CoDeSys Version 2.3.9.5
5. CoDeSys Simulation auf I386 mit CoDeSys 2.3.9.5
6. CoDeSys Simulation auf i386 mit Codesys 3.2 mit SP WIN V3
7. S7 und STEP7: Die OSCAT Bibliothek wird seit Version 1.5 auf STEP7 übersetzt und auch verifiziert.
8. PCWORX / MULTIPROG: Die OSCAT Bibliothek wird seit Version 2.6 auf MULTIPROG übersetzt und verifiziert.

Wir sind stetig darum bemüht die OSCAT Bibliothek auch auf weiteren Testumgebungen zu testen.

2.4 Globale Konstanten

Die OSCAT Bibliothek versucht globale Variablen zu vermeiden, um möglichst einfach in fremde Umgebungen integriert werden zu können. Globale Variablen sind nicht für Funktion oder den Datenaustausch zwischen Bausteinen nötig. Das Setup und die Konfiguration ist vollständig innerhalb der Module realisiert, um höchste Modularität und Portabilität zu gewährleisten. Für physikalische und mathematische Konstanten haben wir uns aus Gründen der Übersichtlichkeit jedoch dazu entschlossen, Globale Konstanten zu verwenden.

Mathematische Konstanten:

PI1 : REAL (Kreiszahl π 3.1415...)

PI2 : REAL (Kreiszahl $\pi * 2 = 6.283185...$)

PI05 : REAL (Kreiszahl $\pi / 2 = 1.5707963267949...$)

E1 : REAL (Eulersche Zahl 2.71828...)

Physikalische Konstanten:

C0 : REAL (Lichtgeschwindigkeit 299792458 m/s)

EL : REAL (Elementarladung in Coulomb = $A * s$)

GN : REAL (Erdbeschleunigung 9.80665 m / s²)

TK : REAL (absoluter Nullpunkt -273.15 °C)

UG : REAL (Universelle Gaskonstante = 8.314472 J/Mol/K)

PN : REAL (Normaldruck = 101325 Pa)

Setup Parameter:

EXTENDED_ASCII : BOOL := TRUE.

EXTENDED_ASCII ist per Default auf TRUE gesetzt und erlaubt es den String Funktionen einen erweiterten Zeichensatz zu berücksichtigen. Im Erweiterten Zeichensatz sind zum Beispiel Umlaute wie Ä, Ö, Ü zu finden. Wird EXTENDED_ASCII = FALSE gesetzt so ignorieren die String Funktionen den erweiterten Zeichensatz. EXTENDED_ASCII berücksichtigt den Zeichensatz nach ISO 8859-1 (Latin1).

Als Beispiel die Funktion UPPERCASE():

UPPERCASE('abcäöü') = 'ABCÄÖÜ' wenn EXTENDED_ASCII = TRUE

UPPERCASE('abcäöü') = 'ABCäöü' wenn EXTENDED_ASCII = FALSE

STRING_LENGTH : INT := 80.

STRING_LENGTH ist per Default 255, und wird von STRING Funktionen benutzt um eine Bereichsüberschreitung bei Verarbeitung von STRINGS zu vermeiden. STRING_LENGTH legt auch innerhalb der OSCAT LIB die maximale STRING Länge fest welche bei häufiger Nutzung des Datentyps STRING zu hohem Speicherverbrauch führen kann. Wenn in einer Anwendung nur kurze STRINGS verarbeitet werden müssen so kann über diese SETUP Konstante die STRING Länge entsprechend reduziert werden. Wir empfehlen STRINGS innerhalb der Anwendung unter Zuhilfenahme dieser Konstante zu definieren.

NEUER_STRING : STRING(STRING_LENGTH).

Durch diese Definition ist auch der neu definierte STRING automatisch auf die Länge von STRING_LENGTH definiert und kann wenn nötig global geändert werden.

CHAR_NAME_1 : STRING(255).

CHAR_NAME_2 : STRING(255).

CHAR_NAME_3 : STRING(255).

CHAR_NAME_4 : STRING(255).

CHARNAME_1 .. 4 hinterlegen die HTML Zeichennamen und den zugehörigen Code.

NETWORK_BUFFER_SIZE : UINT := 4000.

Mit Hilfe des Setup Parameters NETWORK_BUFFER_SIZE wird die Obergrenze der Datenstruktur NETWORK_BUFFER festgelegt. Die Vorgabewert kann bei Bedarf durch diese Konstante geändert werden.

2.5 Aktualität

OSCAT aktualisiert diese Beschreibung fortlaufend. Es wird empfohlen sich die jeweils aktuellste Version von der OSCAT Homepage unter www.OSCAT.DE zu laden. Hier wird das jeweils aktuellste Manual zum Download bereitgestellt. Neben dem Manual stellt OSCAT auch eine detaillierte Revision Historie bereit. Die "OSCAT Revision History" listet alle Revisionen der einzelnen Bausteine mit Änderungen und ab welcher Release der Bibliothek dieser Baustein enthalten ist.

2.6 Support

Support wird durch die Vielzahl der Anwender selbst im Forum unter www.OSCAT.DE zur Verfügung gestellt. Ein Anspruch auf Support besteht aber generell nicht, auch dann nicht, wenn sich herausstellen sollte, dass die Bibliothek oder Teile der Bibliothek fehlerhaft sind. Der im Rahmen des OSCAT Forums bereitgestellte Support wird von Anwendern freiwillig und untereinander bereitgestellt.

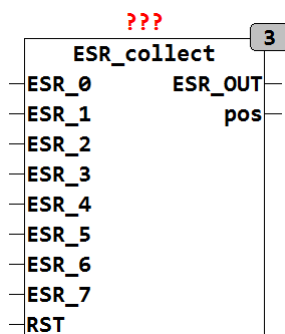
Updates der Bibliothek und der Dokumentation werden in der Regel einmal im Monat auf der Homepage von OSCAT unter www.OSCAT.DE zur Verfügung gestellt. Ein Anspruch auf Wartung, Fehlerbehebung und Softwarepflege jedweder Art besteht generell nicht und ist als freiwillige Leistung von OSCAT anzusehen.

Bitte senden Sie bei Support Anfragen keine email an OSCAT. Anfragen können schneller und effektiver bearbeitet werden wenn die Anfragen in unserem Forum gestellt werden.

3 Diverse Funktionen

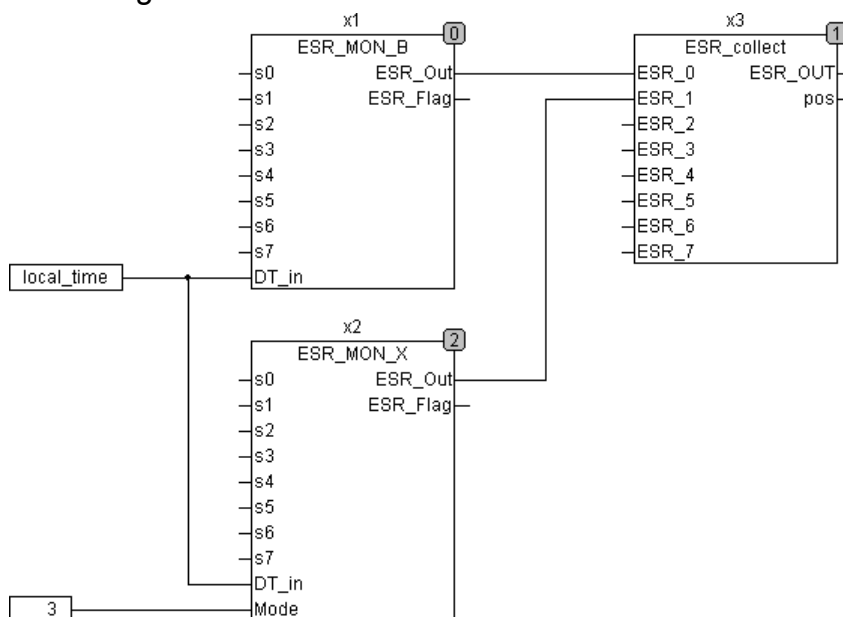
3.1 ESR_COLLECT

Type	Funktionsbaustein
Input	ESR_0..7 : ESR_DATA (ESR_Eingänge) RST : BOOL (Asynchroner Reset Eingang)
Output	ESR_OUT : ESR_DATA (Array mit dem ESR-Protokoll) POS : INT (Position des neusten ESR Protokolls im Array)



ESR_COLLECT sammelt ESR-Daten von bis zu 8 ESR-Bausteinen und speichert das Protokoll in einem Array. Der Ausgang POS zeigt an, an welcher Position des Arrays ESR_OUT sich die momentan letzte ESR-Meldung befindet. Sammelt der Baustein mehr als 64 Meldungen, so werden die ältesten Meldungen überschrieben. Mit dem Asynchronen Reset Eingang kann der Baustein jederzeit zurückgesetzt werden. Durch einen Reset werden alle gesammelten Reset Daten gelöscht und der Positionszeiger auf Null gestellt.

Das folgende Beispiel demonstriert, wie ESR_COLLECT mit ESR-Bausteinen zusammen geschaltet wird.



Der Ausgang ESR_OUT ist vom Typ ESR_DATA und setzt sich wie folgt zusammen:

.TYP	.ADRESS	.DS	.TS	.DATA[0..7]	
1	Label	Date	TIME	Status, 1 Byte	ESR Error
2	Label	Date	TIME	Status, 1 Byte	ESR Status
3	Label	Date	TIME	Status, 1 Byte	ESR Debug
10	Label	Date	TIME	not used	Boolean input low transition
11	Label	Date	TIME	not used	Boolean input high transition
20	Label	Date	TIME	Byte 0 - 3 Real Value	Real Value change

Die ESR Daten enthalten folgende Informationen:

ESR_DATA.TYP Datentyp siehe vorstehende Tabelle

ESR_DATA.ADRESS bis zu 10 Zeichen langer String der die Datenquelle
Bezeichnet

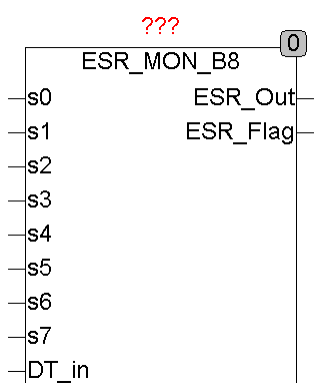
ESR_DATA.DS Datums Stempel von Typ DATETIME

ESR_DATA.TS Zeitstempel vom Typ TIME (SPS Timer)

ESR_DATA.DATA bis zu 8 Byte Datenblock

3.2 ESR_MON_B8

Type	Funktionsbaustein
Input	S0..7 : BOOL (Signaleingänge) DT_IN : DATE_TIME (Zeit- Datum-Eingang für Zeitstempel)
Output	ESR_OUT : ESR_Data (ESR_Datenausgang) ESR_FLAG : BOOL (TRUE, wenn ESR-Daten vorhanden sind)
Config	A0..7 : STRING(10) (Bezeichnung der Eingänge)



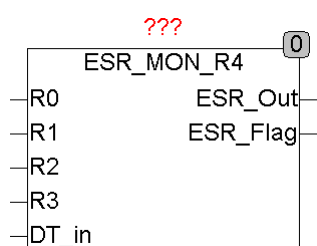
ESR_MON_B8 überwacht bis zu 8 Binäre Signale auf Änderungen, und versieht sie mit einem Zeitstempel und einer Bezeichnung. Die gesammelten Meldungen werden gepuffert und an einen Protokollbaustein über ESR_OUT weitergereicht. Der Ausgang ESR_FLAG wird auf TRUE gesetzt, wenn Meldungen vorhanden sind.

Die ESR-Daten am Ausgang setzen sich wie folgt zusammen:

.TYP	11 steigende Flanke, 10 fallende Flanke
.ADRESS	Adresse Byte der ESR-Datenaufzeichnung
.LINE	Liniennummer (Eingang) der ESR-Datenaufzeichnung
.DS	Datumsstempel vom Typ DATE_TIME
.DT	Zeitstempel vom Typ TIME (SPS-Timer)
.Data	Datenblock von 8 Byte unbelegt

3.3 ESR_MON_R4

Type	Funktionsbaustein
Input	R0..3 : REAL (Signaleingänge) DT_IN : DATE_TIME (Zeit- Datum-Eingang für Zeitstempel)
Output	ESR_OUT : ESR_Data (ESR_Datenausgang) ESR_FLAG : BOOL (TRUE, wenn ESR-Daten vorhanden sind)
Config	A0..3 : STRING(10) (Signaladresse der Eingänge) S0..3 : REAL (Schwellenwerte für Signaländerung)

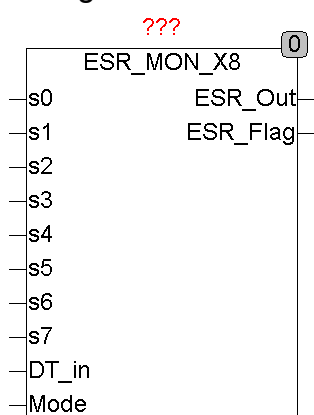


ESR_MON_R4 überwacht bis zu 4 Analoge Signale auf Änderungen und versieht sie mit einem Zeitstempel und der Signaladresse des entsprechenden Eingangs. Die gesammelten Meldungen werden gepuffert und an einen Protokollbaustein über ESR_OUT weitergereicht. Der Ausgang ESR_FLAG wird auf TRUE gesetzt, wenn Meldungen vorhanden sind. Eine Änderung eines Eingangs wird nur dann aufgezeichnet, wenn sich der Eingang um mehr als durch den Schwellenwert S vorgegebenen Wert geändert hat.

.TYP	20 Gleitpunktwert
.ADRESS	Adresse Byte der ESR-Datenaufzeichnung
.LINE	Liniennummer (Eingang) der ESR-Datenaufzeichnung
.DS	Datumsstempel vom Typ DATE_TIME
.DT	Zeitstempel vom Typ TIME (SPS-Timer)
.Data	Datenblock 4 Byte Real Wert

3.4 ESR_MON_X8

Type	Funktionsbaustein
Input	S0..7 : Byte (Statuseingänge) DT_IN : DATE_TIME (Zeit-Datum-Eingang für Zeitstempel) Mode : Byte (legt die zu verarbeitende Art von Meldungen fest)
Output	ESR_OUT : Array[0..7] of ESR_DATA (gesammelte Meldungen) ESR_FLAG : BOOL (TRUE, wenn Meldungen vorhanden sind)
Config	A0..7 : STRING(10) (Signaladresse der Eingänge)



ESR_MON_X8 sammelt Status-Meldungen von bis zu 8 ESR kompatiblen Bausteinen ein, versieht sie mit einem Zeitstempel, Datumsstempel, Eingangsnummer und einer Bausteinadresse. Die gesammelten Meldungen werden gepuffert und an einen Protokollbaustein weitergereicht. Wenn Meldungen zur Weitergabe vorhanden sind, wird dies mit dem Signal ESR_FLAG signalisiert. Am Eingang DT_IN sollte die aktuelle Uhrzeit, die für den Zeitstempel der Meldungen dient anliegen. Der Eingang MODE legt fest, welche Statusmeldungen weitergereicht werden sollen.

- Mode = 1, nur Fehlermeldungen werden verarbeitet.
- Mode = 2, Status- und Fehlermeldungen werden verarbeitet.
- Mode = 3, Fehler, Status und Debug-Meldungen werden verarbeitet.

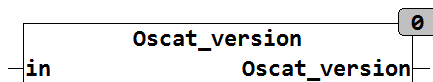
Wenn der Eingang MODE nicht beschaltet wird, werden automatisch alle Meldungen verarbeitet.

Die ESR-Daten am Ausgang setzen sich wie folgt zusammen:

.TYP	1=Fehler, 2=Status, 3=Debug
.ADRESS	Adresse Byte der ESR-Datenaufzeichnung
.LINE	Liniennummer (Eingang) der ESR-Datenaufzeichnung
.DS	Datumsstempel vom Typ DATE_TIME
.DT	Zeitstempel vom Typ TIME (SPS-Timer)
.Data	Data Byte 0 enthält die Statusmeldung

3.5 OSCAT_VERSION

Type Funktion : DWORD
 Input IN : BOOL (wenn TRUE liefert der Baustein das Release Datum)
 Output (Version der Bibliothek)

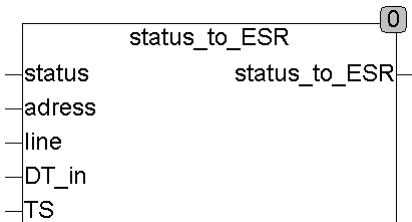


OSCAT_VERSION gibt wenn IN = FALSE die aktuelle Versionsnummer als DWORD zurück. Wird IN auf TRUE gesetzt so wird das Release Datum der aktuellen Version als DWORD zurückgegeben.

Beispiel: OSCAT_VERSION(FALSE) = 201 für Version 2.60
 DWORD_TO_DATE(OSCAT_VERSION(TRUE)) = 2008-1-1

3.6 STATUS_TO_ESR

Type Funktion : ESR_DATA
 Input STATUS : BYTE (Status Byte)
 ADRESS : Byte (Adresse, Byte)
 LINE : Byte (Eingangsnummer)
 DT_IN : DATE_TIME (Zeit-Datum-Eingang)
 TS : TIME (Zeit für Zeitstempel)
 Output ESR_DATA (ESR-Datenblock)



STATUS_TO_ESR erzeugt einen ESR Datensatz aus den Eingangswerten.

Ein STATUS im Bereich zwischen 1 .. 100 ist eine Fehlermeldung und wird als Typ 1 gekennzeichnet.

Status 100 .. 199 wird als Typ 2 gekennzeichnet und 200 .. 255 wird als Typ 3 gekennzeichnet (Debug-Information).

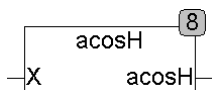
Die ESR-Daten am Ausgang setzen sich wie folgt zusammen:

.TYP	1=Fehler, 2=Status, 3=Debug
.ADRESS	Adresse Byte der ESR-Datenaufzeichnung
.LINE	Liniennummer (Eingang) der ESR-Datenaufzeichnung
.DS	Datumsstempel vom Typ DATE_TIME
.DT	Zeitstempel vom Typ TIME (SPS-Timer)
.Data	Datenblock von 8 Byte

4 Mathematik

4.1 ACOSH

Type Funktion : REAL
 Input X : REAL (Eingangswert)
 Output REAL (Ausgangswert)

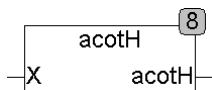


ACOSH Berechnet den Arcus Cosinus Hyperbolicus nach folgender Formel:

$$\text{ACOSH} = \text{LN}(X + \text{SQRT}(\text{EXPT}(X,2)-1))$$

4.2 ACOTH

Type Funktion : REAL
 Input X : REAL (Eingangswert)
 Output REAL (Ausgangswert)

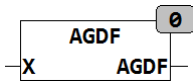


ACOTH Berechnet den Arcus Cotangens Hyperbolicus nach folgender Formel:

$$\text{ACOTH} = \text{LN}((X+1)/(X-1))/2$$

4.3 AGDF

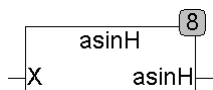
Type Funktion : REAL
 Input X : REAL (Eingangswert)
 Output REAL (Inverse Gundermannfunktion)



AGDF berechnet die Inverse Gundermannfunktion.
 Die Berechnung erfolgt nach der Formel $GDF := \text{LN}((1+\text{SIN}(X))/\text{COS}(X))$.

4.4 ASINH

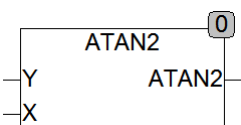
Type Funktion : REAL
 Input X : REAL (Eingangswert)
 Output REAL (Ausgangswert)



ASINH Berechnet den Arcus Sinus Hyperbolicus nach folgender Formel:
 $\text{ASINH} = \text{LN}(X + \text{SQRT}(\text{EXPT}(X,2)+1))$

4.5 ATAN2

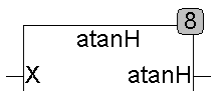
Type Funktion : REAL
 Input X : REAL (Eingangswert)
 Output REAL (Ausgangswert)



ATAN2 Berechnet den Winkel von Koordinaten (Y, X) in RAD. Das Ergebnis liegt zwischen $-\pi$ und $+\pi$

4.6 ATANH

Type Funktion : REAL
 Input X : REAL (Eingangswert)
 Output REAL (Ausgangswert)

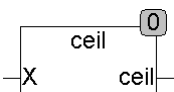


ATANH Berechnet den Arcus Tangens Hyperbolicus nach folgender Formel:

$$\text{ATANH} = \text{LN}((1+X)/(1-X)) / 2$$

4.7 CEIL

Type Funktion : INT
 Input X : REAL (Eingangswert)
 Output INT (Ausgangswert)

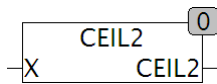


Die Funktion CEIL gibt den kleinsten ganzzahligen Wert größer gleich X zurück.

Beispiel: $\text{CEIL}(3.14) = 4$
 $\text{CEIL}(-3.14) = -3$
 $\text{CEIL}(2) = 2$

4.8 CEIL2

Type Funktion : DINT
 Input X : REAL (Eingangswert)
 Output DINT (Ausgangswert)

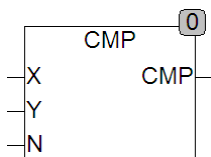


Die Funktion CEIL2 gibt den kleinsten ganzzahligen Wert größer gleich X zurück.

Beispiel: $\text{CEIL2}(3.14) = 4$
 $\text{CEIL2}(-3.14) = -3$
 $\text{CEIL2}(2) = 2$

4.9 CMP

Type Funktion : BOOL
 Input X, Y : REAL (Eingangswerte)
 N : INT (Anzahl der Stellen die verglichen werden sollen)
 Output BOOL (Ergebnis)



CMP vergleicht 2 REAL Werte ob die ersten N Stellen gleich Sind.

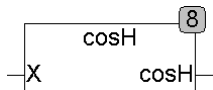
Beispiele:

$\text{CMP}(3.140, 3.149, 3) = \text{TRUE}$ $\text{CMP}(3.140, 3.149, 4) = \text{FALSE}$
 $\text{CMP}(0.015, 0.016, 1) = \text{TRUE}$ $\text{CMP}(0.015, 0.016, 2) = \text{FALSE}$

Bei der Funktion CMP ist zu beachten das durch die Duale Kodierung der Zahlen eine 0.1 im Dezimalsystem nicht unbedingt immer als 0.1 im Binärsystem dargestellt werden kann. Vielmehr kann es vorkommen das die 0.1 etwas kleiner oder größer dargestellt wird weil die Auflösung der Zahlencodierung im Binärsystem nicht exakt eine 0.1 zulässt. Aus diesem Grund kann die Funktion nicht immer zu 100% einen Unterschied von 1 an der letzten Stelle erkennen. Weiterhin ist zu beachten das ein Datentyp REAL mit 32 Bit nur eine Auflösung von 7 - 8 Dezimalstellen bietet.

4.10 COSH

Type Funktion : REAL
 Input X : REAL (Eingangswert)
 Output REAL (Ausgangswert)

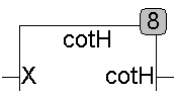


COSH Berechnet den Cosinus Hyperbolicus nach folgender Formel:

$$\text{COSH} = (\text{EXPT}(E, X) + \text{EXPT}(E, -X)) / 2$$

4.11 COTH

Type Funktion : REAL
 Input X : REAL (Eingangswert)
 Output REAL (Ausgangswert)



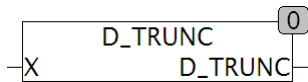
COTH Berechnet den Cotangens Hyperbolicus nach folgender Formel:

$$\text{COTH} = (\text{EXPT}(E, 2 * X) + 1) / (\text{EXPT}(E, 2 * X) - 1)$$

für Eingangswerte größer 20 oder kleiner als -20 liefert COTH den Näherungswert +1 beziehungsweise -1 was einer Genauigkeit besser 8 Stellen entspricht und somit unterhalb der Auflösung des Typs REAL liegt.

4.12 D_TRUNC

Type Funktion : DINT
 Input X : REAL (Eingangswert)
 Output DINT (Ausgangswert)



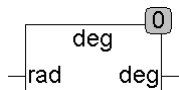
D_TRUNC liefert den ganzzahligen Teil eines REAL Wertes als DINT. Die IEC Routine TRUNC() unterstützt nicht auf allen Systemen ein TRUNC nach DINT so dass wir aus Gründen der Kompatibilität diese Routine nachgebildet haben. Leider liefert auch ein REAL_TO_DINT nicht auf allen Systemen dasselbe Ergebnis. D_TRUNC prüft welches Ergebnis die IEC Funktionen liefern und nutzt die passende Funktion um ein brauchbares Ergebnis zu liefern.

D_TRUNC(1.6) = 1

D_TRUNC(-1.6) = -1

4.13 DEG

Type Funktion : REAL
 Input Rad : REAL (Winkel in Bogenmaß)
 Output REAL (Winkel in Grad)



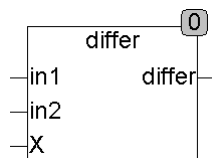
Die Funktion DEG konvertiert einen Winkelwert von Bogenmaß in Grad. Dabei wird berücksichtigt das der Eingang nicht größer als 2π sein darf. Wenn RAD größer als 2π ist wird entsprechend 2π abgezogen bis der Eingang RAD zwischen 0 und 2π liegt.

DEG(π) = 180 Grad, DEG(3π) = 180 Grad

DEG(0) = 0 Grad, DEG(2π) = 0 Grad

4.14 DIFFER

Type	Funktion : BOOL
Input	In1 : REAL (Wert 1) In2 : REAL (Wert 2) X : REAL (Mindestunterschied in1 zu in2)
Output	BOOL (TRUE, wenn sich in1 und in2 um mehr als x voneinander unterscheiden)



Die Funktion DIFFER wird TRUE, wenn in1 und in2 sich durch mehr als X voneinander unterscheiden.

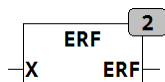
$\text{DIFFER} = \text{ABS}(\text{IN1} - \text{IN2}) > \text{X};$

Beispiel: Differ(100, 120, 10) ergibt TRUE

Differ(100, 110, 15) ergibt FALSE

4.15 ERF

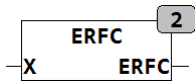
Type	Funktion : REAL
Input	X : REAL (Eingangswert)
Output	REAL (Ergebnis)



Die Funktion ERF berechnet die Fehlerfunktion von X.

4.16 ERFC

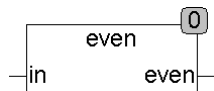
Type Funktion : REAL
 Input X : REAL (Eingangswert)
 Output REAL (Ergebnis)



Die Funktion ERFC berechnet die inverse Fehlerfunktion von X.

4.17 EVEN

Type Funktion : BOOL
 Input in : DINT (Eingangswert)
 Output BOOL (TRUE, wenn in gerade)

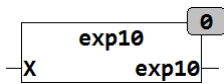


Die Funktion EVEN wird TRUE, wenn der Eingang in gerade ist und FALSE für ungerade in.

Beispiel: EVEN(2) ergibt TRUE
 EVEN(3) ergibt FALSE

4.18 EXP10

Type Funktion : REAL
 Input X : REAL (Eingangswert)
 Output REAL (Exponentialfaktor zur Basis 10)



Die Funktion EXP10 liefert den Exponentialwert zur Basis 10.

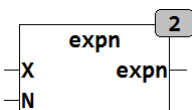
$$\text{EXP10}(2) = 100$$

$$\text{EXP10}(0) = 1$$

$$\text{EXP10}(3.14) = 1380.384265$$

4.19 EXPN

Type Funktion : REAL
 Input X : REAL (Eingangswert)
 N : INT (Exponentialwert)
 Output REAL (Ergebnis X^N)



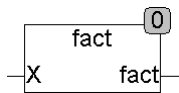
EXPN berechnet den Exponentialwert von X^N für Ganzzahlige N. EXPN ist speziell für SPS ohne Floating Point Unit geschrieben und ist ca. 30 mal schneller als die IEC Standard Funktion EXPT().

$$\text{EXPT}(10,-2) = 0.01$$

$$\text{EXPT}(1.5,2) = 2.25$$

4.20 FACT

Type Funktion : DINT
 Input X : INT (Eingangswert)
 Output DINT (Fakultät von X)



Die Funktion FACT berechnet die Fakultät von X.

Sie ist definiert für Eingangswerte von 0 .. 12. Für Werte kleiner Null und größer 12 wird das Ergebnis -1. für die Fakultät von größeren Zahlen ist die Funktion GAMMA geeignet.

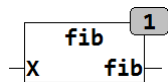
Für natürliche Zahlen X ist: $X! = 1*2*3*...*(X-1)*X$, $0! = 1$

Fakultäten für negative oder nicht ganze Zahlen sind nicht definiert.

Beispiel: $1! = 1$
 $2! = 1*2 = 2$
 $5! = 1*2*3*4*5 = 120$

4.21 FIB

Type Funktion : DINT
 Input X : INT (Eingangswert)
 Output DINT (Fibonacci Zahl)



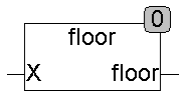
FIB berechnet die Fibonacci Zahl. Die Fibonacci Zahl ist wie folgt definiert:

$FIB(0) = 0$, $FIB(1) = 1$, $FIB(2) = 1$, $FIB(3) = 2$, $FIB(4) = 3$, $FIB(5) = 5$

Die Fibonacci Zahl von X ist gleich der Summe der Fibonacci Zahlen von X-1 und X-2. Die Funktion kann die Fibonacci Zahlen bis 46 berechnen, ist $X < 0$ oder Größer als 46 gibt die Funktion -1 zurück.

4.22 FLOOR

Type Funktion : INT
 Input X : REAL (Eingangswert)
 Output INT (Ausgangswert)



Die Funktion FLOOR gibt den größten ganzzahligen Wert kleiner gleich X zurück.

Beispiel: $\text{FLOOR}(3.14) = 3$
 $\text{FLOOR}(-3.14) = -4$
 $\text{FLOOR}(2) = 2$

4.23 FLOOR2

Type Funktion : DINT
 Input X : REAL (Eingangswert)
 Output DINT (Ausgangswert)

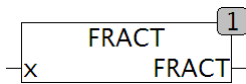


Die Funktion FLOOR2 gibt den größten ganzzahligen Wert kleiner gleich X zurück.

Beispiel: $\text{FLOOR2}(3.14) = 3$
 $\text{FLOOR2}(-3.14) = -4$
 $\text{FLOOR2}(2) = 2$

4.24 FRACT

Type Funktion : REAL
 Input X : REAL (Eingangswert)
 Output REAL (Nachkommateil von X)



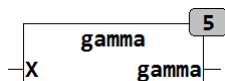
Die Funktion FRACT liefert den Nachkommateil von X.

Beispiel: FRACT(3.14) ergibt 0.14.

Für X größer oder kleiner als $\pm 2.14 \cdot 10^9$ liefert FRACT stets ein Null zurück. Das die Auflösung eines 32Bit REAL maximal bei 8 Dezimalstellen liegt kann aus Zahlen größer oder kleiner als $\pm 2.14 \cdot 10^9$ kein Nachkommateil ermittelt werden, weil dieser auch in einer REAL Variable nicht abgespeichert werden kann.

4.25 GAMMA

Type Funktion : REAL
 Input X : REAL (Eingangswert)
 Output REAL (Gamma Funktion)



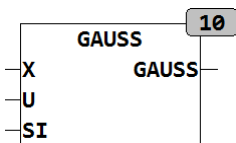
Die Funktion GAMMA berechnet die Gamma Funktion nach der Näherungsformel von Stirling.

$$\text{GAMMA} := \text{SQRT}(2 \cdot \pi \cdot X) \cdot \text{EXPT}(X / E, X).$$

Die Gamma Funktion kann für Ganzzahlige X als Ersatz für Die Fakultät verwendet werden. Der Relative Fehler der Stirling Funktion ist für $X > 9$ kleiner als 1% und für $X > 84$ kleiner als 0.1%

4.26 GAUSS

Type Funktion : REAL
 Input X : REAL (Eingangswert)
 U : REAL (Lokalität der Funktion)
 SI : REAL (Sigma, Spreizung der Funktion)
 Output REAL (Gauss Funktion)



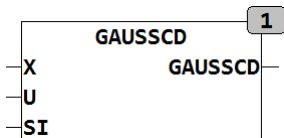
Die Funktion GAUSS berechnet die Normalverteilung nach folgender Formel:

$$\text{GAUSS} := 1/(\text{SI} \cdot \text{SQRT}(2 \cdot \text{PI})) * \text{EXP}(1/2 * ((x-u)/\text{SI})^2).$$

Die Normalverteilung ist die Dichtefunktion Normalverteilter Zufallsgrößen. Mit den Parameter U = 0 und SI = 1 entspricht sie der Standard Normalverteilung.

4.27 GAUSSCD

Type Funktion : REAL
 Input X : REAL (Eingangswert)
 U : REAL (Lokalität der Funktion)
 SI : REAL (Sigma, Spreizung der Funktion)
 Output REAL (Gauss Verteilungsfunktion)



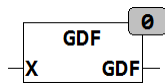
Die Funktion GAUSSCD berechnet die Verteilungsfunktion zur Normalverteilung nach folgender Formel:

$$\text{GAUSSCD} := 1/2 * (1 + \text{ERF}((X-U)/(\text{SI} \cdot \text{SQRT}(\text{PI}))).$$

Die Normalverteilung ist die Dichtefunktion Normalverteilter Zufallsgrößen. Mit den Parameter U = 0 und SI = 1 entspricht sie der Standard Normalverteilung. Die Verteilungsfunktion (Cumulative Distribution Function).

4.28 GDF

Type Funktion : REAL
 Input X : REAL (Eingangswert)
 Output REAL (Gundermannfunktion)



GDF berechnet die Gundermannfunktion.

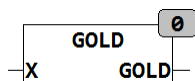
Die Berechnung erfolgt nach der Formel $GDF := 2 * ATAN(E^X) - \pi/2$.

Das Ergebnis von GDF liegt zwischen $-\pi/2$ und $+\pi/2$.

$GDF(0) = 0$

4.29 GOLD

Type Funktion : REAL
 Input X : REAL (Eingangswert)
 Output REAL (Ergebnis der Goldenen Funktion)

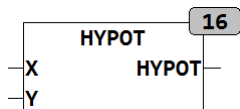


GOLD berechnet das Ergebnis der goldenen Funktion. $GOLD(1)$ ergibt den Goldenen Schnitt, und $GOLD(0)$ ergibt 1. $GOLD(X) * GOLD(-X)$ ergibt immer 1. $GOLD(X)$ ist das positive Ergebnis der Quadratischen Gleichung und $-GOLD(-X)$ ist das negative Ergebnis der Quadratischen Gleichung.

Die Berechnung erfolgt nach der Formel $GOLD := (X + SQRT(X^2 + 4)) / 2$.

4.30 HYPOT

Type Funktion : REAL
 Input X : REAL (X - Wert)
 Y : REAL (Y - Wert)
 Output REAL (Länge der Hypotenuse)

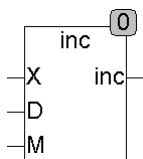


Die Funktion HYPOT berechnet die Hypotenuse eines rechtwinkligen Dreiecks nach dem Satz des Pythagoras.

$$\text{HYPOT} = \text{SQRT}(X^2 + Y^2)$$

4.31 INC

Type Funktion : INT
 Input X : INT (Eingangswert)
 D : INT (Wert, der zum Eingangswert addiert wird)
 M : INT (Maximalwert für den Ausgang)
 Output INT (Ausgangswert)



INC addiert zum Eingang X den Wert D und stellt sicher, dass der Ausgang INC nicht über den Wert M läuft. Ist das Ergebnis aus der Addition von X und D größer als M, so wird wieder bei 0 begonnen. Die Funktion ist vor allem sinnvoll beim adressieren von Arrays und Pufferspeichern. Auch beim Positionieren von Absolutwert Winkelgebern kann sie eingesetzt werden. INC kann auch mit einem negativen D zum decrementieren benutzt werden, dabei stellt INC sicher, dass das Ergebnis nicht unter Null läuft. Wird von Null eins abgezogen, beginnt INC wieder bei M.

$\text{INC} := X + D$, weil D maximal den Wert M annehmen kann.

Wird $\text{INC} > M$, so beginnt INC wieder bei 0.

Wird $INC < 0$ so beginnt INC wieder bei M

Beispiel: $INC(3, 2, 5)$ ergibt 5

$INC(4, 2, 5)$ ergibt 0

$INC(0, -1, 7)$ ergibt 7

4.32 LANGEVIN

Type Funktion : REAL

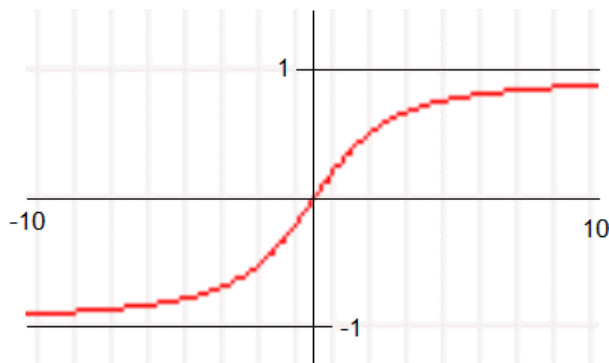
Input X : REAL (Eingangswert)

Output REAL (Ausgangswert)



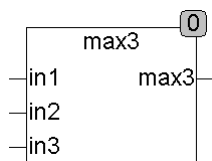
Die Langevin Funktion ist der Sigmoidfunktion sehr ähnlich, nähert sich aber langsamer den Grenzwerten an. Im Gegensatz zur Sigmoidfunktion liegen die Grenzwerte bei -1 und +1. Die Langevin Funktion ist vor allem auf CPUs ohne Gleitkommaeinheit deutlich schneller als die Sigmoidfunktion.

Die folgende Grafik verdeutlicht den Verlauf der Langevin Funktion:



4.33 MAX3

Type Funktion : REAL
 Input in1 : REAL (Eingang 1)
 in2 : REAL (Eingang 1)
 in3 : REAL (Eingang 1)
 Output REAL (Maximalwert der 3 Eingänge in1, in2, in3)

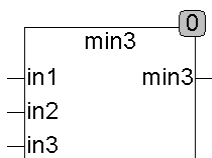


Die Funktion MAX3 liefert den Maximalwert von 3 Eingängen. Grundsätzlich sollte die im Standard Funktionsumfang nach IEC61131-3 enthaltene Funktion MAX mit einer variablen Anzahl von Eingängen ausgestattet sein. Da aber in einigen Systemen die Funktion MAX nur 2 Eingänge unterstützt wird die Funktion MAX3 angeboten.

Beispiel: $\text{MAX3}(1,3,2) = 3$.

4.34 MIN3

Type Funktion : REAL
 Input in1 : REAL (Eingang 1)
 in2 : REAL (Eingang 1)
 in3 : REAL (Eingang 1)
 Output REAL (Minimalwert der 3 Eingänge in1, in2, in3)

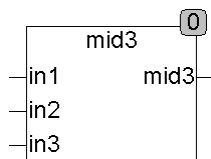


Die Funktion MIN3 liefert den Minimalwert von 3 Eingängen. Grundsätzlich sollte die im Standard Funktionsumfang nach IEC61131-3 enthaltene Funktion MIN mit einer variablen Anzahl von Eingängen ausgestattet sein. Da aber in einigen Systemen die Funktion MIN nur 2 Eingänge unterstützt wird die Funktion MIN3 angeboten.

Beispiel: $\text{MIN3}(1,3,2) = 1$.

4.35 MID3

Type Funktion : REAL
 Input in1 : REAL (Eingang 1)
 in2 : REAL (Eingang 1)
 in3 : REAL (Eingang 1)
 Output REAL (Mittlerer Wert der 3 Eingänge in1, in2, in3)

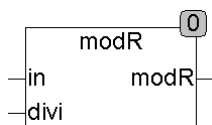


Die Funktion MID3 liefert den mittleren Wert von 3 Eingängen, nicht aber den Mathematischen Mittelwert.

Beispiel: $\text{MID3}(1,5,2) = 2$.

4.36 MODR

Type Funktion : REAL
 Input IN : REAL (Dividend)
 DIVI : REAL (Divisor)
 Output REAL (Rest der Division)

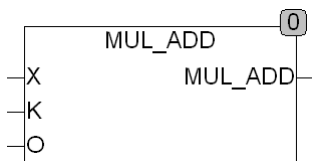


Die Funktion MODR liefert den Rest einer Division analog der Standardfunktion MOD, allerdings für REAL-Zahlen. MODR nutzt intern die Standard Funktion TRUNC() welche den Eingangswert in einen INTEGER vom Typ DINT wandelt. Hierbei kann es zu einem Überlauf kommen weil DINT maximal $\pm 2.14 \cdot 10^9$ speichern kann. Der Wertebereich von MODR ist deshalb auf $\pm 2.14 \cdot 10^9$ begrenzt.

Beispiel: $\text{MODR}(5.5, 2.5)$ ergibt 0.5.

4.37 MUL_ADD

Type Funktion : REAL
 Input X : REAL (Eingangswert)
 K : REAL (Multiplikator)
 O : REAL (Offset)
 Output : REAL (Ausgangswert)



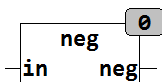
MUL_ADD multipliziert den Eingangswert X mit K und addiert O hinzu.

$MUL_ADD = X * K + O$.

MUL_ADD(0.5, 10, 2) ergibt $0.5 * 10 + 2 = 7$.

4.38 NEG

Type Funktion : INT
 Input IN : INT (Eingangswert)
 Output INT (Ausgangswert)



NEG liefert am Ausgang den Eingangswert IN multipliziert mit -1.

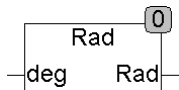
NEG(-3) = 3

NEG(15) = -15

NEG(0) = 0

4.39 RAD

Type Funktion : REAL
 Input DEG : REAL (Winkel in Grad)
 Output REAL (Winkel in Bogenmaß)

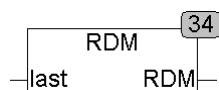


Die Funktion RAD konvertiert einen Winkelwert von Gradmaß in Bogenmaß. Dabei wird berücksichtigt das DEG nicht größer als 360 sein darf. Wenn DEG größer als 360 ist wird solange 360 abgezogen bis DEG zwischen 0 und 360 liegt.

Rad(0) = 0
 RAD(180) = π
 RAD(360) = 0
 RAD(540) = π

4.40 RDM

Type Funktion : REAL
 Input LAST : REAL (Letzter berechneter Wert)
 Output REAL (Zufallszahl zwischen 0 und 1)



RDM berechnet eine Pseudo-Zufallszahl. Dazu wird der SPS-interne Timer ausgelesen und in eine Pseudo-Zufallszahl überführt. Da RDM als Funktion und nicht als Funktionsblock geschrieben wurde, kann es keine Daten zwischen 2 Aufrufen speichern und muss deshalb mit Vorsicht angewendet werden. Wird RDM nur einmal je Zyklus aufgerufen, liefert sie hinreichend gute Zahlen. Wird sie aber mehrfach innerhalb eines Zyklus aufgerufen, so liefert sie dieselbe Zahl, weil höchstwahrscheinlich der SPS Timer noch auf demselben Wert steht. Wird die Funktion mehrfach innerhalb eines Zyklus benötigt, so muss ihr bei jedem Aufruf eine unterschiedliche Startzahl (LAST) übergeben werden. Wird Sie hingegen nur einmal je Zyklus aufgerufen, genügt der Aufruf RDM(0). Als Startzahl kann bei jedem Aufruf die letzte ermittelte Zahl von RDM verwendet werden. Die von RDM gelieferten Zufallszahlen liegen zwischen 0 und 1, die 1 nicht enthalten ($0 \leq \text{Zufallszahl} < 1$).

4.41 RDM2

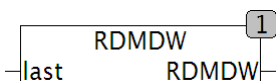
Type	Funktion : INT
Input	LAST : INT (Letzter berechneter Wert)
	LOW : INT (niedrigster generierter Wert)
	HIGH : INT (höchster generierter Wert)
Output	INT (Zufallszahl zwischen LOW und HIGH)



RDM2 erzeugt einen Integer Random Wert im Bereich von LOW bis HIGH, wobei LOW und HIGH im Wertebereich enthalten sind. Wird die Funktion nur einmal je Zyklus benutzt kann der Eingangswert LAST auf 0 bleiben. Die Funktion RDM2 benutzt die SPS interne Zeitbasis zur Erzeugung der Zufallszahl. Da RDM2 eine als LAST einen Integer benutzt welcher das letzte Ergebnis zwischen LOW und HIGH darstellt kann es zu einer Situation führen die darin resultiert das RDM2 innerhalb eines Zyklus solange sicher SPS Timer nicht verändert immer das gleiche Ergebnis liefert. Dies passiert immer dann wenn zufällig das Ergebnis auch dem Startwert entspricht. Da dann derselbe Startwert wieder verwendet wird wird innerhalb des gleichen Zyklus auch wieder das gleiche Ergebnis Zustande kommen. Dieser Fall tritt umso öfter auf je kleiner der durch LOW und HIGH bestimmte Range für das Ergebnis ist. Man kann diesen Effekt leicht umgehen indem man als Startwert einen Schleifenzähler verwendet der definitiv bei jedem Aufruf einen neuen Wert aufweist, oder noch besser einen Schleifenzähler addiert mit dem letzten Ergebnis als Startwert verwendet.

4.42 RDMDW

Type	Funktion : DWORD
Input	LAST : DWORD (Letzter berechneter Wert)
Output	DWORD (Zufälliges Bitmuster)

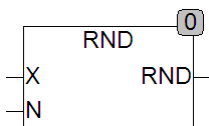


RDMDW berechnet eine Pseudo-Zufallszahl mit 32 Bit Länge im Format DWORD. Dazu wird der SPS-interne Timer ausgelesen und in eine Pseudo-Zufallszahl über-

führt. Da RDMDW als Funktion und nicht als Funktionsblock geschrieben wurde, kann es keine Daten zwischen 2 Aufrufen speichern und muss deshalb mit Vorsicht angewendet werden. Wird RDMDW nur einmal je Zyklus aufgerufen, liefert sie hinreichend gute Zahlen. Wird sie aber mehrfach innerhalb eines Zyklus aufgerufen, so liefert sie dieselbe Zahl, weil höchstwahrscheinlich der SPS Timer noch auf demselben Wert steht. Wird die Funktion mehrfach innerhalb eines Zyklus benötigt, so muss ihr bei jedem Aufruf eine unterschiedliche Startzahl (LAST) übergeben werden. Wird Sie hingegen nur einmal je Zyklus aufgerufen, genügt der Aufruf RDMDW(0). Als Startzahl kann bei jedem Aufruf die letzte ermittelte Zahl von RDMDW verwendet werden. Das von RDMDW gelieferte Ergebnis ist ein Zufälliges 32Bit breites Bit Muster.

4.43 RND

Type Funktion : REAL
 Input X : REAL (Eingangswert)
 N : Integer (Anzahl der Stellen)
 Output REAL (Gerundeter Wert)

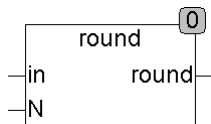


Die Funktion RND rundet den Eingangswert IN auf N Stellen. Folgt der letzten Stelle ein Digit größer 5 wird die letzte Stelle aufgerundet. RND nutzt intern die Standard Funktion TRUNC() welche den Eingangswert in einen INTEGER vom Typ DINT wandelt. Hierbei kann es zu einem Überlauf kommen weil DINT maximal $\pm 2.14 \cdot 10^9$ speichern kann. Der Wertebereich von RND ist deshalb auf $\pm 2.14 \cdot 10^9$ begrenzt. Siehe hierzu auch die Funktion ROUND welche den Eingangswert auf N Nachkommastellen rundet.

Beispiel: $\text{RND}(355.55, 2) = 360$
 $\text{RND}(3.555, 2) = 3.6$
 $\text{ROUND}(3.555, 2) = 3.56$

4.44 ROUND

Type Funktion : REAL
 Input IN : REAL (Eingangswert)
 N : Integer (Anzahl der Nachkommastellen)
 Output REAL (Gerundeter Wert)

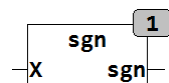


Die Funktion ROUND rundet den Eingangswert in auf N Nachkommastellen. Folgt der letzten Stelle ein Digit größer 5 wird die letzte Stelle aufgerundet. ROUND nutzt intern die Standard Funktion TRUNC() welche den Eingangswert in einen INTEGER vom Typ DINT wandelt. Hierbei kann es zu einem Überlauf kommen weil DINT maximal $\pm 2.14 \cdot 10^9$ speichern kann. Der Wertebereich von ROUND ist deshalb auf $\pm 2.14 \cdot 10^9$ begrenzt.

Beispiel: $\text{ROUND}(3.555, 2) = 3.56$

4.45 SGN

Type Funktion : INT
 Input X : REAL (Eingangswert)
 Output INT (Signum des Eingangs X)



Die Funktion SGN berechnet das Signum von X.

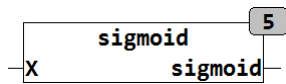
SGN = +1 wenn $X > 0$

SGN = 0 wenn $X = 0$

SGN = -1 wenn $X < 0$

4.46 SIGMOID

Type Funktion : INT
 Input X : REAL (Eingangswert)
 Output REAL (Ergebnis der Sigmoidfunktion)

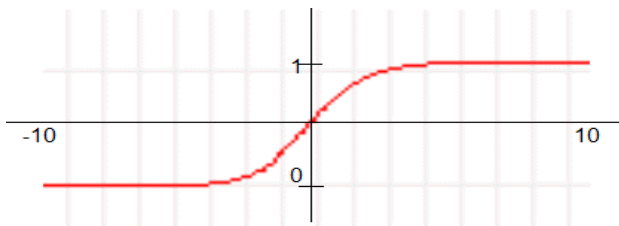


Die Sigmoidfunktion wird auch Schwanenhals- Funktion genannt und wird durch die folgende Gleichung beschrieben:

$$\text{SIGMOID} = 1 / (1 + \text{EXP}(-X))$$

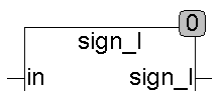
Die Sigmoidfunktion wird häufig als Aktivierungsfunktion verwendet. Durch seinen Verlauf eignet sich die Sigmoidfunktion für weiche Schaltübergänge.

Die folgende Grafik veranschaulicht den Verlauf der Sigmoidfunktion:



4.47 SIGN_I

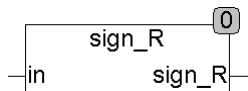
Type Funktion : BOOL
 Input IN : INT (Eingangswert)
 Output BOOL (TRUE, wenn der Eingang negativ ist)



Die Funktion SIGN_I liefert TRUE zurück, wenn der Eingangswert negativ ist. Die Eingangswerte sind vom Typ INT.

4.48 SIGN_R

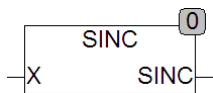
Type Funktion : BOOL
 Input IN : REAL(Eingangswert)
 Output BOOL (TRUE, wenn der Eingang negativ ist)



Die Funktion SIGN_R liefert TRUE zurück, wenn der Eingangswert negativ ist. Die Eingangswerte sind vom Typ REAL.

4.49 SINC

Type Funktion : REAL
 Input X : REAL (Eingangswert)
 Output REAL (Ausgangswert)

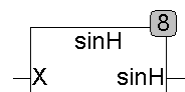


SINC berechnet den Sinus Cardinalis oder die Spaltfunktion.

$\text{SINC} = \sin(X) / X$. Mit $\text{SINC}(0) = 1$.

4.50 SINH

Type Funktion : REAL
 Input X : REAL (Eingangswert)
 Output REAL (Ausgangswert)

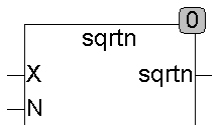


SINH Berechnet den Sinus Hyperbolicus nach folgender Formel:

$\text{SINH} = (\text{EXPT}(E,X) - \text{EXPT}(E,-X))/2$

4.51 SQRTN

Type Funktion : REAL
 Input X : REAL (Eingangswert)
 Output REAL (Ausgangswert)

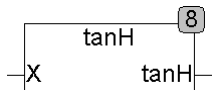


SQRTN Berechnet die N-fache Wurzel aus X nach folgender Formel:

$$\text{SQRTN} = \text{EXPT}(X, 1/N)$$

4.52 TANH

Type Funktion : REAL
 Input X : REAL (Eingangswert)
 Output REAL (Ausgangswert)

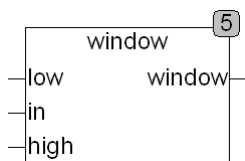


TANH Berechnet den Tangens Hyperbolicus nach folgender Formel:

$$\text{TANH} = -2 / (\text{EXPT}(E, -2 * X) + 1) + 1$$

4.53 WINDOW

Type Funktion : BOOL
 Input LOW : REAL (unterer Grenzwert)
 IN : REAL (Eingangswert)
 HIGH : REAL (oberer Grenzwert)
 Output BOOL (TRUE, wenn $in < HIGH$ und $in > LOW$)

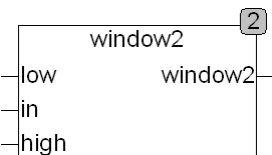


Die Funktion WINDOW testet, ob der Eingangswert IN innerhalb der Grenzen, die durch LOW und HIGH definiert sind, liegt.

WINDOW ist genau dann TRUE, wenn $IN < HIGH$ und $IN > LOW$ ist.

4.54 WINDOW2

Type Funktion : BOOL
 Input LOW : REAL (unterer Grenzwert)
 IN : REAL (Eingangswert)
 HIGH : REAL (oberer Grenzwert)
 Output BOOL (TRUE, wenn $in \leq HIGH$ und $in \geq LOW$)

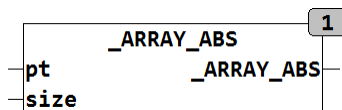


Die Funktion WINDOW2 testet, ob der Eingangswert $IN \leq HIGH$ und $IN \geq LOW$ ist. Im Gegensatz zur Funktion WINDOW die TRUE liefert wenn IN innerhalb der Grenzen LOW und HIGH liegt liefert WINDOW2 FALSE wenn IN außerhalb der Grenzen LOW und HIGH liegt.

5 Array

5.1 _ARRAY_ABS

Type Funktion : BOOL
 Input PT : Pointer (Zeiger auf das Array)
 SIZE : UINT (Größe des Arrays)
 Output BOOL (TRUE)



Die Funktion `_ARRAY_ABS` rechnet die Elemente eines beliebigen Array of REAL in den Absolutwert um. Beim Aufruf wird der Funktion ein Pointer auf das zu initialisierende Array und dessen Größe in Bytes übergeben. Unter CoDeSys lautet der Aufruf: `_ARRAY_ABS(ADR(Array), SIZEOF(Array), INIT)`, wobei Array der Name des zu manipulierenden Arrays ist. ADR ist eine Standardfunktion, die den Pointer auf das Array ermittelt und SIZEOF ist eine Standardfunktion, die die Größe des Arrays ermittelt. Die Funktion liefert nur TRUE zurück. Das durch den Pointer angegebene Array wird direkt im Speicher manipuliert.

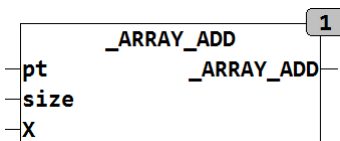
Diese Art der Bearbeitung von Arrays ist äußerst effizient, da kein zusätzlicher Speicher benötigt wird und keine Übergabewerte kopiert werden müssen.

Aufruf: `_ARRAY_ABS(ADR(bigarray), SIZEOF(bigarray))`

Beispiel: `[0,-2,3,-1-5]` wird umgewandelt in `[0,2,3,1,5]`

5.2 **_ARRAY_ADD**

Type Funktion : BOOL
 Input PT : Pointer (Zeiger auf das Array)
 SIZE : UINT (Größe des Arrays)
 X : REAL (zu addierender Wert)
 Output BOOL (TRUE)



Die Funktion `_ARRAY_ADD` addiert zu jedem Element eines beliebigen Array of REAL den Wert X. Beim Aufruf wird der Funktion ein Pointer auf das zu initialisierende Array und dessen Größe in Bytes übergeben. Unter CoDeSys lautet der Aufruf: `_ARRAY_ADD(ADR(Array), SIZEOF(Array), INIT)`, wobei Array der Name des zu manipulierenden Arrays ist. ADR ist eine Standardfunktion, die den Pointer auf das Array ermittelt und SIZEOF ist eine Standardfunktion, die die Größe des Arrays ermittelt. Die Funktion liefert nur TRUE zurück. Das durch den Pointer angegebene Array wird direkt im Speicher manipuliert.

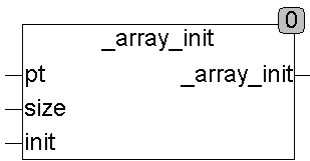
Diese Art der Bearbeitung von Arrays ist äußerst effizient, da kein zusätzlicher Speicher benötigt wird und keine Übergabewerte kopiert werden müssen.

Aufruf: `_ARRAY_ADD(ADR(bigarray), SIZEOF(bigarray), X)`

Beispiel: `[0,-2,3,-1-5] ; X = 3` wird umgewandelt in `[3,1,6,2,-2]`

5.3 **_ARRAY_INIT**

Type Funktion : BOOL
 Input PT : Pointer (Zeiger auf das Array)
 SIZE : UINT (Größe des Arrays)
 INIT : REAL (Initialwert)
 Output BOOL (TRUE)



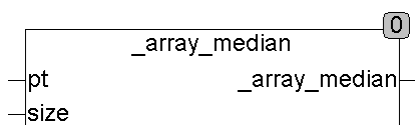
Die Funktion `_ARRAY_INIT` initialisiert ein beliebiges Array of REAL mit einem Initialwert. Beim Aufruf wird der Funktion ein Pointer auf das zu initialisierende Array und dessen Größe in Bytes übergeben. Unter CoDeSys lautet der Aufruf: `_ARRAY_INIT(ADR(Array), SIZEOF(Array), INIT)`, wobei Array der Name des zu manipulierenden Arrays ist. ADR ist eine Standardfunktion, die den Pointer auf das Array ermittelt und SIZEOF ist eine Standardfunktion, die die Größe des Arrays ermittelt. Die Funktion liefert nur TRUE zurück. Das durch den Pointer angegebene Array wird direkt im Speicher manipuliert.

Diese Art der Bearbeitung von Arrays ist äußerst effizient, da kein zusätzlicher Speicher benötigt wird und keine Übergabewerte kopiert werden müssen.

Beispiel: `_ARRAY_INIT(ADR(bigarray), SIZEOF(bigarray), 0)`
initialisiert bigarray mit 0.

5.4 `_ARRAY_MEDIAN`

Type Funktion : REAL
Input PT : Pointer (Zeiger auf das Array)
 SIZE : UINT (Größe des Arrays)
Output REAL (Medianwert des Arrays)



Die Funktion `_ARRAY_MEDIAN` ermittelt den Medianwert eines beliebigen Arrays of REAL. Beim Aufruf wird der Funktion ein Pointer auf das Array und dessen Größe in Bytes übergeben. Unter CoDeSys lautet der Aufruf: `_ARRAY_MEDIAN(ADR(Array), SIZEOF(Array))`, wobei Array der Name des zu manipulierenden Arrays ist. ADR() ist eine Standardfunktion, die den Pointer auf das Array ermittelt und SIZEOF() ist eine Standardfunktion, die die Größe des Arrays ermittelt. Um den Medianwert zu ermitteln wird das durch den Pointer referenzierte Array direkt im Speicher sortiert und bleibt nach beenden der Funktion sortiert. Die Funktion `_ARRAY_MEDIAN` verändert also den Inhalt des Arrays.

Diese Art der Bearbeitung von Arrays ist äußerst effizient, da kein zusätzlicher Speicher benötigt wird und keine Übergabewerte kopiert werden müssen.

Sollte ein Array bearbeitet werden, das nicht verändert werden darf, so ist es vor Übergabe des Pointer und Aufruf der Funktion in ein temporäres Array zu kopieren.

Beispiel: `_ARRAY_MEDIAN(ADR(bigarray), SIZEOF(bigarray))`

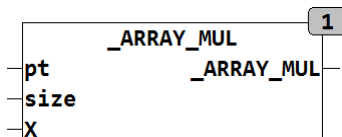
Medianwert:

Der Medianwert ist der Mittlere Wert einer sortierten Wertemenge. Median von (12, 0, 4, 7, 1) ist 4. Nach dem Ausführen der Funktion bleibt das Array sortiert im Speicher zurück (0, 1, 4, 7, 12).

Wenn das Array eine gerade Anzahl von Elementen enthält ist der Medianwert der Mittelwert aus den beiden mittleren Werten des Arrays.

5.5 `_ARRAY_MUL`

Type	Funktion : BOOL
Input	PT : Pointer (Zeiger auf das Array)
	SIZE : UINT (Größe des Arrays)
	X : REAL (Multiplikator)
Output	BOOL (TRUE)



Die Funktion `_ARRAY_MUL` multipliziert jedes Element eines beliebigen Array of REAL mit den Wert X. Beim Aufruf wird der Funktion ein Pointer auf das zu initialisierende Array und dessen Größe in Bytes übergeben. Unter CoDeSys lautet der Aufruf: `_ARRAY_MUL(ADR(Array), SIZEOF(Array), INIT)`, wobei Array der Name des zu manipulierenden Arrays ist. ADR ist eine Standardfunktion, die den Pointer auf das Array ermittelt und SIZEOF ist eine Standardfunktion, die die Größe des Arrays ermittelt. Die Funktion liefert nur TRUE zurück. Das durch den Pointer angegebene Array wird direkt im Speicher manipuliert.

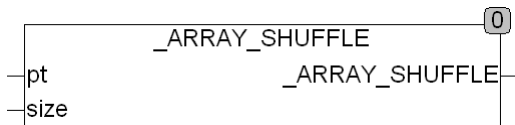
Diese Art der Bearbeitung von Arrays ist äußerst effizient, da kein zusätzlicher Speicher benötigt wird und keine Übergabewerte kopiert werden müssen.

Aufruf: `_ARRAY_MUL(ADR(bigarray), SIZEOF(bigarray), X)`

Beispiel: `[0,-2,3,-1-5]` ; X = 3 wird umgewandelt in `[0,-6,9,-3,-15]`

5.6 **_ARRAY_SHUFFLE**

Type Funktion : BOOL
 Input PT : Pointer (Zeiger auf das Array)
 SIZE : UINT (Größe des Arrays)
 Output BOOL (Returns TRUE)



Die Funktion `_ARRAY_SHUFFLE` vertauscht die Elemente eines beliebigen Arrays of REAL nach einem Zufallsprinzip. Beim Aufruf wird der Funktion ein Pointer auf das Array und dessen Größe in Bytes übergeben. Unter CoDeSys lautet der Aufruf: `_ARRAY_SHUFFLE(ADR(Array), SIZEOF(Array))`, wobei Array der Name des zu manipulierenden Arrays ist. `ADR()` ist eine Standardfunktion, die den Pointer auf das Array ermittelt und `SIZEOF()` ist eine Standardfunktion, die die Größe des Arrays ermittelt. Das durch den Pointer referenzierte Array wird direkt im Speicher manipuliert und steht nach beenden der Funktion direkt zur Verfügung. Die Funktion `_ARRAY_SHUFFLE` verändert also den Inhalt des Arrays.

Diese Art der Bearbeitung von Arrays ist äußerst effizient, da kein zusätzlicher Speicher benötigt wird und keine Übergabewerte kopiert werden müssen.

Sollte ein Array bearbeitet werden, das nicht verändert werden darf, so ist es vor Übergabe des Pointer und Aufruf der Funktion in ein temporäres Array zu kopieren.

Beispiel: `_ARRAY_SHUFFLE(ADR(bigarray), SIZEOF(bigarray))`

Ein Aufruf der Funktion `_ARRAY_SHUFFLE` könnte ein Array wie folgt verändern. Da die Funktion einen Pseudo Random Algorithmus verwendet wird das Ergebnis bei jedem Aufruf ein anderes sein, die Ergebnisse sind nicht reproduzierbar, auch nicht durch einen Neustart des Programms oder der Steuerung.

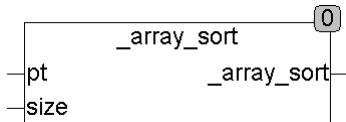
Ausgangsarray: (0,1,2,3,4,5,6,7,8,9)

Ergebnis: (5,0,3,9,7,2,1,8,4,6)

Das Ergebnis ist aber nicht wiederholbar, die Funktion liefert nach jedem Aufruf oder Auch Neustart eine neue Reihenfolge.

5.7 _ARRAY_SORT

Type Funktion : BOOL
 Input PT : Pointer (Zeiger auf das Array)
 SIZE : UINT (Größe des Arrays)
 Output BOOL (TRUE)



Die Funktion `_ARRAY_SORT` sortiert ein beliebiges Array of REAL in aufsteigender Reihenfolge. Beim Aufruf wird der Funktion ein Pointer auf das zu sortierende Array und dessen Größe in Bytes übergeben. Unter CoDeSys lautet der Aufruf: `_ARRAY_SORT(ADR(Array), SIZEOF(Array))`, wobei Array der Name des zu sortierenden Arrays ist. `ADR` ist eine Standardfunktion, die den Pointer auf das Array ermittelt und `SIZEOF` ist eine Standardfunktion, die die Größe des Arrays ermittelt.

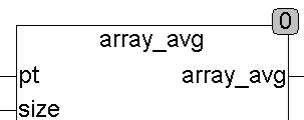
Die Funktion liefert nur TRUE zurück. Das durch den Pointer angegebene Array wird direkt im Speicher Manipuliert.

Diese Art der Bearbeitung von Arrays ist äußerst effizient, da kein zusätzlicher Speicher benötigt wird und keine Übergabewerte kopiert werden müssen.

Beispiel: `_ARRAY_SORT(ADR(bigarray), SIZEOF(bigarray))`

5.8 ARRAY_AVG

Type Funktion : REAL
 Input PT : Pointer (Zeiger auf das Array)
 SIZE : UINT (Größe des Arrays)
 Output REAL (Mittelwert des Arrays)



Die Funktion `ARRAY_AVG` ermittelt den Mittelwert eines beliebigen Arrays of REAL. Bei Aufruf wird der Funktion ein Pointer auf das Array und dessen Größe in Bytes übergeben. Unter CoDeSys lautet der Aufruf: `ARRAY_AVG(ADR(Array), SIZEOF(Ar-`

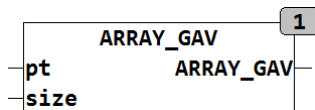
ray)), wobei Array der Name des zu durchsuchenden Arrays ist. ADR ist eine Standardfunktion, die den Pointer auf das Array ermittelt und SIZEOF ist eine Standardfunktion, die die Größe des Arrays ermittelt. Um den Maximalwert zu ermitteln wird das durch den Pointer referenzierte Array direkt im Speicher durchsucht. Die Funktion ARRAY_AVG verändert den Inhalt des Arrays nicht.

Diese Art der Bearbeitung von Arrays ist äußerst effizient, da kein zusätzlicher Speicher benötigt wird und keine Übergabewerte kopiert werden müssen.

Beispiel: `ARRAY_AVG(ADR(bigarray), SIZEOF(bigarray))`

5.9 ARRAY_GAV

Type Funktion : REAL
 Input PT : Pointer (Zeiger auf das Array)
 SIZE : UINT (Größe des Arrays)
 Output REAL (Mittelwert des Arrays)



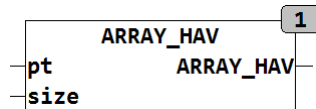
Die Funktion ARRAY_GAV ermittelt den geometrischen Mittelwert eines beliebigen Arrays of REAL. Bei Aufruf wird der Funktion ein Pointer auf das Array und dessen Größe in Bytes übergeben. Unter CoDeSys lautet der Aufruf: `ARRAY_GAV(ADR(Array), SIZEOF(Array))`, wobei Array der Name des zu durchsuchenden Arrays ist. ADR ist eine Standardfunktion, die den Pointer auf das Array ermittelt und SIZEOF ist eine Standardfunktion, die die Größe des Arrays ermittelt. Um den Maximalwert zu ermitteln wird das durch den Pointer referenzierte Array direkt im Speicher durchsucht. Die Funktion ARRAY_GAV verändert den Inhalt des Arrays nicht.

Diese Art der Bearbeitung von Arrays ist äußerst effizient, da kein zusätzlicher Speicher benötigt wird und keine Übergabewerte kopiert werden müssen.

Beispiel: `ARRAY_GAV(ADR(bigarray), SIZEOF(bigarray))`

5.10 ARRAY_HAV

Type Funktion : REAL
 Input PT : Pointer (Zeiger auf das Array)
 SIZE : UINT (Größe des Arrays)
 Output REAL (Mittelwert des Arrays)



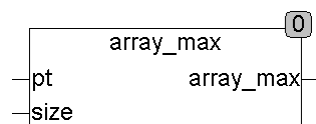
Die Funktion ARRAY_HAV ermittelt den harmonischen Mittelwert eines beliebigen Arrays of REAL. Bei Aufruf wird der Funktion ein Pointer auf das Array und dessen Größe in Bytes übergeben. Unter CoDeSys lautet der Aufruf: ARRAY_HAV(ADR(Array), SIZEOF(Array)), wobei Array der Name des zu durchsuchenden Arrays ist. ADR ist eine Standardfunktion, die den Pointer auf das Array ermittelt und SIZEOF ist eine Standardfunktion, die die Größe des Arrays ermittelt. Um den Maximalwert zu ermitteln wird das durch den Pointer referenzierte Array direkt im Speicher durchsucht. Die Funktion ARRAY_HAV verändert den Inhalt des Arrays nicht.

Diese Art der Bearbeitung von Arrays ist äußerst effizient, da kein zusätzlicher Speicher benötigt wird und keine Übergabewerte kopiert werden müssen.

Beispiel: ARRAY_HAV(ADR(bigarray), SIZEOF(bigarray))

5.11 ARRAY_MAX

Type Funktion : REAL
 Input PT : Pointer (Zeiger auf das Array)
 SIZE : UINT (Größe des Arrays)
 Output REAL (Maximalwert des Arrays)



Die Funktion ARRAY_MAX ermittelt den Maximalwert eines beliebigen Arrays of REAL. Bei Aufruf wird der Funktion ein Pointer auf das Array und dessen Größe in Bytes übergeben. Unter CoDeSys lautet der Aufruf: Array_MAX(ADR(Array), SIZEOF(Array)), wobei Array der Name des zu durchsuchenden Arrays ist. ADR ist eine Standard Funktion die den Pointer auf das Array ermittelt und SIZEOF ist eine

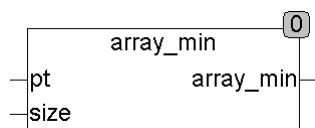
Standard Funktion die die Größe des Arrays ermittelt. Um den Maximalwert zu ermitteln wird das durch den Pointer referenzierte Array direkt im Speicher durchsucht. Die Funktion `ARRAY_MAX` verändert den Inhalt des Arrays nicht.

Diese Art der Bearbeitung von Arrays ist äußerst effizient, da kein zusätzlicher Speicher benötigt wird und keine Übergabewerte kopiert werden müssen.

Beispiel: `ARRAY_MAX(ADR(bigarray), SIZEOF(bigarray))`

5.12 ARRAY_MIN

Type Funktion : REAL
 Input PT : Pointer (Zeiger auf das Array)
 SIZE : UINT (Größe des Arrays)
 Output REAL (Minimalwert des Arrays)



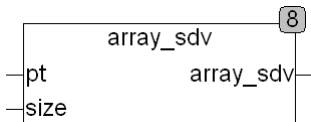
Die Funktion `ARRAY_MIN` ermittelt den Minimalwert eines beliebigen Arrays of REAL. Beim Aufruf wird der Funktion ein Pointer auf das Array und dessen Größe in Bytes übergeben. Unter CoDeSys lautet der Aufruf: `ARRAY_MIN(ADR(Array), SIZEOF(Array))`, wobei Array der Name des zu durchsuchenden Arrays ist. `ADR()` ist eine Standardfunktion, die den Pointer auf das Array ermittelt und `SIZEOF()` ist eine Standardfunktion, die die Größe des Arrays ermittelt. Um den Maximalwert zu ermitteln wird das durch den Pointer referenzierte Array direkt im Speicher durchsucht. Die Funktion `ARRAY_MIN` verändert den Inhalt des Arrays nicht.

Diese Art der Bearbeitung von Arrays ist äußerst effizient, da kein zusätzlicher Speicher benötigt wird und keine Übergabewerte kopiert werden müssen.

Beispiel: `ARRAY_MIN(ADR(bigarray), SIZEOF(bigarray))`

5.13 ARRAY_SDV

Type Funktion : REAL
 Input PT : Pointer (Zeiger auf das Array)
 SIZE : UINT (Größe des Arrays)
 Output REAL (Standardabweichung des Arrays)



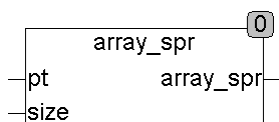
Die Funktion ARRAY_SDV ermittelt die Standardabweichung (Standard Deviation) eines beliebigen Arrays of REAL. Beim Aufruf wird der Funktion ein Pointer auf das Array und dessen Größe in Bytes übergeben. Unter CoDeSys lautet der Aufruf: ARRAY_SDV(ADR(Array), SIZEOF(Array)), wobei Array der Name des zu durchsuchenden Arrays ist. ADR() ist eine Standardfunktion, die den Pointer auf das Array ermittelt und SIZEOF() ist eine Standardfunktion, die die Größe des Arrays ermittelt. Um den Maximalwert zu ermitteln wird das durch den Pointer referenzierte Array direkt im Speicher durchsucht. Die Funktion ARRAY_SDV verändert den Inhalt des Arrays nicht.

Diese Art der Bearbeitung von Arrays ist äußerst effizient, da kein zusätzlicher Speicher benötigt wird und keine Übergabewerte kopiert werden müssen.

Beispiel: ARRAY_SDV(ADR(bigarray), SIZEOF(bigarray))

5.14 ARRAY_SPR

Type Funktion : REAL
 Input PT : Pointer (Zeiger auf das Array)
 SIZE : UINT (Größe des Arrays)
 Output REAL (Streuung des Arrays)



Die Funktion ARRAY_SPR ermittelt die Streuung eines beliebigen Arrays of REAL. Die Streuung ist der Maximalwert im Array minus dem Minimalwert des Arrays. Beim Aufruf wird der Funktion ein Pointer auf das Array und dessen Größe in Bytes über-

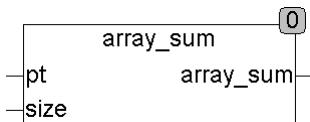
geben. Unter CoDeSys lautet der Aufruf: `ARRAY_SPR(ADR(Array), SIZEOF(Array))`, wobei `Array` der Name des zu durchsuchenden Arrays ist. `ADR` ist eine Standardfunktion, die den Pointer auf das Array ermittelt und `SIZEOF` ist eine Standardfunktion, die die Größe des Arrays ermittelt. Um den Maximalwert zu ermitteln wird das durch den Pointer referenzierte Array direkt im Speicher durchsucht. Die Funktion `ARRAY_SPR` verändert den Inhalt des Arrays nicht.

Diese Art der Bearbeitung von Arrays ist äußerst effizient, da kein zusätzlicher Speicher benötigt wird und keine Übergabewerte kopiert werden müssen.

Beispiel: `ARRAY_SPR(ADR(bigarray), SIZEOF(bigarray)) =`
`Maximalwert(bigarray) - Minimalwert(bigarray)`

5.15 ARRAY_SUM

Type Funktion : REAL
 Input PT : Pointer (Zeiger auf das Array)
 SIZE : UINT (Größe des Arrays)
 Output REAL (Summe aller Werte des Arrays)



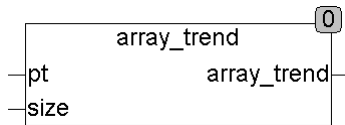
Die Funktion `ARRAY_SUM` ermittelt die Summe aller Werte eines beliebigen Arrays of REAL. Beim Aufruf wird der Funktion ein Pointer auf das Array und dessen Größe in Bytes übergeben. Unter CoDeSys lautet der Aufruf: `ARRAY_SUM(ADR(Array), SIZEOF(Array))` wobei `Array` der Name des zu durchsuchenden Arrays ist. `ADR()` ist eine Standardfunktion, die den Pointer auf das Array ermittelt und `SIZEOF()` ist eine Standardfunktion, die die Größe des Arrays ermittelt. Um den Maximalwert zu ermitteln wird das durch den Pointer referenzierte Array direkt im Speicher durchsucht. Die Funktion `ARRAY_SUM` verändert den Inhalt des Arrays nicht.

Diese Art der Bearbeitung von Arrays ist äußerst effizient, da kein zusätzlicher Speicher benötigt wird und keine Übergabewerte kopiert werden müssen.

Beispiel: `ARRAY_SUM(ADR(bigarray), SIZEOF(bigarray))`

5.16 ARRAY_TREND

Type Funktion : REAL
 Input PT : Pointer (Zeiger auf das Array)
 SIZE : UINT (Größe des Arrays)
 Output REAL (Trendentwicklung des Arrays)



Die Funktion ARRAY_TREND ermittelt den Trend der Werte eines beliebigen Arrays of REAL. Beim Aufruf wird der Funktion ein Pointer auf das Array und dessen Größe in Bytes übergeben. Unter CoDeSys lautet der Aufruf: ARRAY_TREND(ADR(Array), SIZEOF(Array)), wobei Array der Name des zu durchsuchenden Arrays ist. ADR() ist eine Standardfunktion, die den Pointer auf das Array ermittelt und SIZEOF() ist eine Standardfunktion, die die Größe des Arrays ermittelt. Um den Trend zu ermitteln wird das durch den Pointer referenzierte Array direkt im Speicher durchsucht. Die Funktion ARRAY_TREND verändert den Inhalt des Arrays nicht.

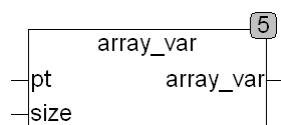
Diese Art der Bearbeitung von Arrays ist äußerst effizient, da kein zusätzlicher Speicher benötigt wird und keine Übergabewerte kopiert werden müssen.

Der Trend wird ermittelt, indem der Durchschnitt der unteren Hälfte der Werte des Arrays vom Durchschnitt der Werte der oberen Hälfte des Arrays abgezogen wird.

Beispiel: ARRAY_TREND(ADR(bigarray), SIZEOF(bigarray))

5.17 ARRAY_VAR

Type Funktion : REAL
 Input PT : Pointer (Zeiger auf das Array)
 SIZE : UINT (Größe des Arrays)
 Output REAL (Varianz des Arrays)



Die Funktion ARRAY_VAR ermittelt die Varianz eines beliebigen Arrays of REAL. Beim Aufruf wird der Funktion ein Pointer auf das Array und dessen Größe in Bytes

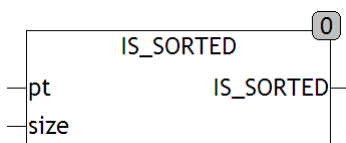
übergeben. Unter CoDeSys lautet der Aufruf: `ARRAY_VAR(ADR(Array), SIZEOF(Array))` wobei Array der Name des zu durchsuchenden Arrays ist. `ADR()` ist eine Standardfunktion, die den Pointer auf das Array ermittelt und `SIZEOF()` ist eine Standardfunktion, die die Größe des Arrays ermittelt. Um den Maximalwert zu ermitteln wird das durch den Pointer referenzierte Array direkt im Speicher durchsucht. Die Funktion `ARRAY_VAR` verändert den Inhalt des Arrays nicht.

Diese Art der Bearbeitung von Arrays ist äußerst effizient, da kein zusätzlicher Speicher benötigt wird und keine Übergabewerte kopiert werden müssen.

Beispiel: `ARRAY_VAR(ADR(bigarray), SIZEOF(bigarray))`

5.18 IS_SORTED

Type Funktion : BOOL
 Input PT : Pointer (Zeiger auf das Array)
 SIZE : UINT (Größe des Arrays)
 Output BOOL (TRUE)



Die Funktion `IS_SORTED` prüft ob ein beliebiges Array of REAL in aufsteigender Reihenfolge sortiert ist. Beim Aufruf wird der Funktion ein Pointer auf das zu sortierende Array und dessen Größe in Bytes übergeben. Unter CoDeSys lautet der Aufruf: `_ARRAY_SORT(ADR(Array), SIZEOF(Array))`, wobei Array der Name des zu sortierenden Arrays ist. `ADR` ist eine Standardfunktion, die den Pointer auf das Array ermittelt und `SIZEOF` ist eine Standardfunktion, die die Größe des Arrays ermittelt.

Die Funktion liefert TRUE zurück wenn das Array in aufsteigender Reihenfolge sortiert ist.

Diese Art der Bearbeitung von Arrays ist äußerst effizient, da kein zusätzlicher Speicher benötigt wird und keine Übergabewerte kopiert werden müssen.

Beispiel: `IS_SORTED(ADR(bigarray), SIZEOF(bigarray))`

6 Komplexe Mathematik

6.1 EINLEITUNG

Komplexe Zahlen werden mit dem definierten Typ COMPLEX abgebildet.

Der Typ COMPLEX setzt sich aus einem Realteil und einem Imaginärteil zusammen, beide Komponenten sind vom Typ REAL.

Die komplexe Zahl Z vom Typ COMPLEX besteht aus:

Z.RE Realteil vom Typ REAL.

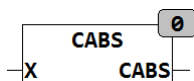
Z.IM Imaginärteil vom Typ REAL.

6.2 CABS

Type Funktion : REAL

Input X : COMPLEX (Eingangswert)

Output REAL (Ergebnis)

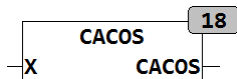


CABS berechnet die Länge des Vektors einer komplexen Zahl. Der Absolutwert wird auch Modul oder Magnitude genannt.

$$\text{CABS} = \text{SQRT}(\text{X.RE}^2 + \text{X.IM}^2)$$

6.3 CACOS

Type Funktion : COMPLEX
 Input X : COMPLEX (Eingangswert)
 Output COMPLEX (Ergebnis)

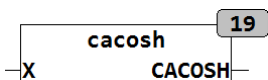


CACOS berechnet den Arcus Kosinus einer Komplexen Zahl.

Der Wertebereich des Ergebnisses liegt zwischen $[0, \pi]$ für den Realteil und $[-\infty, +\infty]$ für den Imaginärteil.

6.4 CACOSH

Type Funktion : COMPLEX
 Input X : COMPLEX (Eingangswert)
 Output COMPLEX (Ergebnis)

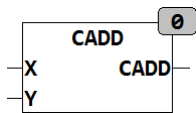


CACOSH berechnet den Arcus Kosinus Hyperbolicus einer Komplexen Zahl.

Der Wertebereich des Ergebnisses liegt zwischen $[-\pi, +\pi]$ für den Imaginärteil.

6.5 CADD

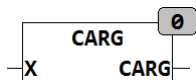
Type Funktion : COMPLEX
 Input X : COMPLEX (Eingangswert)
 Y : COMPLEX (Eingangswert)
 Output COMPLEX (Ergebnis)



CADD addiert zwei Komplexe Zahlen X und Y.

6.6 CARG

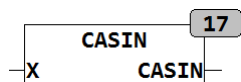
Type Funktion : REAL
 Input X : COMPLEX (Eingangswert)
 Output REAL (Ergebnis)



CARG berechnet den Winkel einer Komplexen Zahl im Koordinatensystem.
 Der Wertebereich des Ergebnisses liegt zwischen $[-\pi, +\pi]$.

6.7 CASIN

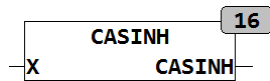
Type Funktion : COMPLEX
 Input X : COMPLEX (Eingangswert)
 Output COMPLEX (Ergebnis)



CASIN berechnet den Arcus Sinus einer Komplexen Zahl.
 Der Wertebereich des Ergebnisses liegt zwischen $[-\pi/2, +\pi/2]$ für den Realteil.

6.8 CASINH

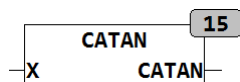
Type Funktion : COMPLEX
 Input X : COMPLEX (Eingangswert)
 Output COMPLEX (Ergebnis)



CASINH berechnet den Arcus Sinus Hyperbolicus einer Komplexen Zahl.
 Der Wertebereich des Ergebnisses liegt zwischen $[-\pi, +\pi]$ für den Imaginärteil.

6.9 CATAN

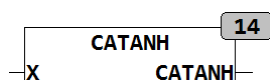
Type Funktion : COMPLEX
 Input X : COMPLEX (Eingangswert)
 Output COMPLEX (Ergebnis)



CATAN berechnet den Arcus Tangens einer Komplexen Zahl.
 Der Wertebereich des Ergebnisses liegt zwischen $[-\pi/2, +\pi/2]$ für den Realteil.

6.10 CATANH

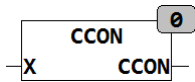
Type Funktion : COMPLEX
 Input X : COMPLEX (Eingangswert)
 Output COMPLEX (Ergebnis)



CATANH berechnet den Arcus Tangens Hyperbolicus einer Komplexen Zahl.
 Der Wertebereich des Ergebnisses liegt zwischen $[-\pi/2, +\pi/2]$ für den Imaginärteil.

6.11 CCON

Type Funktion : COMPLEX
 Input X : COMPLEX (Eingangswert)
 Output COMPLEX (Ergebnis)



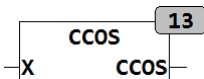
CCON berechnet die Konjugation einer komplexen Zahl.

$$\text{CCON.RE} = \text{X.RE}$$

$$\text{CCON.IM} = -\text{X.IM}$$

6.12 CCOS

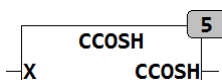
Type Funktion : COMPLEX
 Input X : COMPLEX (Eingangswert)
 Output COMPLEX (Ergebnis)



CCOS berechnet den Kosinus einer Komplexen Zahl.

6.13 CCOSH

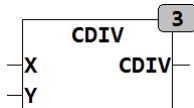
Type Funktion : COMPLEX
 Input X : COMPLEX (Eingangswert)
 Output COMPLEX (Ergebnis)



CCOSH berechnet den Kosinus Hyperbolicus einer Komplexen Zahl.

6.14 CDIV

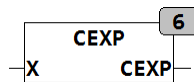
Type Funktion : COMPLEX
 Input X : COMPLEX (Eingangswert)
 Y : COMPLEX (Eingangswert)
 Output COMPLEX (Ergebnis)



CDIV dividiert zwei komplexe Zahlen X / Y .

6.15 CEXP

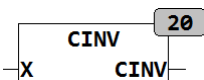
Type Funktion : COMPLEX
 Input X : COMPLEX (Eingangswert)
 Output COMPLEX (Ergebnis)



CEXP berechnet den komplexen Exponent zur Basis E, $CEXP = E^X$.

6.16 CINV

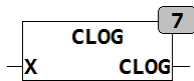
Type Funktion : COMPLEX
 Input X : COMPLEX (Eingangswert)
 Output COMPLEX (Ergebnis)



CINV berechnet der Kehrwert einer Komplexen Zahl, $CINV = 1/X$

6.17 CLOG

Type Funktion : COMPLEX
 Input X : COMPLEX (Eingangswert)
 Output COMPLEX(Ergebnis)



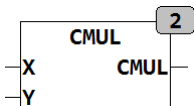
CLOG berechnet den natürlichen Logarithmus einer Komplexen Zahl zur Basis E.

$$\text{CLOG}(X) = \text{LOG}(e)(X).$$

Der Wertebereich des Ergebnisses liegt zwischen $[-\pi, +\pi]$ für den Imaginärteil.

6.18 CMUL

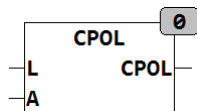
Type Funktion : COMPLEX
 Input X : COMPLEX (Eingangswert)
 Y : COMPLEX (Eingangswert)
 Output COMPLEX (Ergebnis)



CMUL Multipliziert zwei Komplexe Zahlen X und Y.

6.19 CPOL

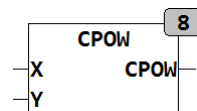
Type Funktion : COMPLEX
 Input L : REAL (Länge oder Raduis)
 A : REAL (Winkelwert)
 Output COMPLEX (Ergebnis)



CPOL erzeugt eine Komplexe Zahl aus der Polarform. Die Eingangswerte L und A spezifizieren die Länge (Radius) und den Winkel.

6.20 CPOW

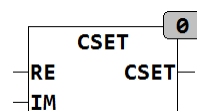
Type Funktion : COMPLEX
 Input X : COMPLEX (Eingangswert)
 Y : COMPLEX (Eingangswert 2)
 Output COMPLEX (Ergebnis)



CPOW berechnet die Potenz zweier Komplexer Zahlen, $CPOW = X^Y$.

6.21 CSET

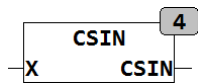
Type Funktion : COMPLEX
 Input RE : COMPLEX (Realer Eingangswert)
 IM : REAL (Imaginärer Eingangswert)
 Output COMPLEX (Ergebnis)



CSET erzeugt aus den beiden Eingangskomponenten RE und IM eine komplexe Zahl vom Typ COMPLEX.

6.22 CSIN

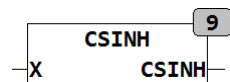
Type Funktion : COMPLEX
 Input X : COMPLEX (Eingangswert)
 Output COMPLEX (Ergebnis)



CSIN berechnet den Sinus einer Komplexen Zahl.

6.23 CSINH

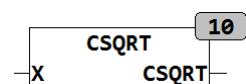
Type Funktion : COMPLEX
 Input X : COMPLEX (Eingangswert)
 Output COMPLEX (Ergebnis)



CSINH berechnet den Sinus Hyperbolicus einer Komplexen Zahl.

6.24 CSQRT

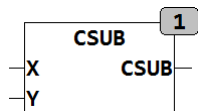
Type Funktion : REAL
 Input X : COMPLEX (Eingangswert)
 Output REAL (Ergebnis)



CSQRT berechnet die Quadratwurzel aus einer Komplexen Zahl.

6.25 CSUB

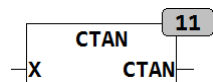
Type Funktion : COMPLEX
 Input X : COMPLEX (Eingangswert)
 Y : COMPLEX (Eingangswert)
 Output COMPLEX (Ergebnis)



CSUB Subtrahiert zwei komplexe Zahlen, $CSUB = X - Y$.

6.26 CTAN

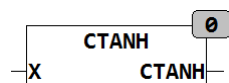
Type Funktion : COMPLEX
 Input X : COMPLEX (Eingangswert)
 Output COMPLEX (Ergebnis)



CTAN berechnet den Tangens einer Komplexen Zahl.

6.27 CTANH

Type Funktion : COMPLEX
 Input X : COMPLEX (Eingangswert)
 Output COMPLEX (Ergebnis)



CTANH berechnet den Tangens Hyperbolicus einer Komplexen Zahl.

7 Arithmetik doppelter Genauigkeit

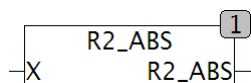
7.1 Einleitung

Gleitpunktzahlen werden im Format REAL abgespeichert. Ein gängiges Datenformat nach IEC754 benutzt dazu ein 24Bit breite Mantisse und einen 8Bit Exponenten. Daraus resultiert eine Genauigkeit von 7-8 Dezimalstellen. Normalerweise ist dies für Anwendungen in der Steuerungstechnik mehr als Ausreichend, kann aber in bestimmten Fällen zu einem Problem führen. Ein Typischer Fall der mit einfacher Genauigkeit nur unzureichend gelöst werden kann sind Verbrauchsmesser. Möchte man bis zu Mehreren Mwh (Megawattstunden) an Gesamtverbrauch Aufsummieren und dabei eine kleinste Leistung von 1mW (Milliwatt) im Abstand von 10ms Messen und berücksichtigen, so benötigt man eine Auflösung von $3.6 \cdot 10^7$ (entspricht 10MWs) und dabei würde man $1 \cdot 10^{-5}$ Ws aufaddieren wollen. Um dies zu bewerkstelligen benötigt man eine Auflösung von 12 Stellen.

Die von OSCAT implementierte Doppelte REAL Genauigkeit hat eine Auflösung von etwa 15 Stellen. Der hierfür Implementierte Datentyp REAL2 besteht aus R1 und RX, hierbei wird in RX der Wert mit den ersten 7-8 Stellen als Real gespeichert und der Rest in einem weiteren REAL R1. Dieser Datentyp hat den Vorteil das keine Wandlung von REAL2 zu REAL notwendig ist, vielmehr ist der Teil RX bereits der einfache REAL Wert.

7.2 R2_ABS

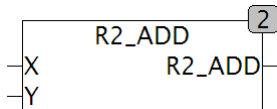
Type	Funktion : REAL2
Input	X : REAL2 (Eingangswert)
Output	REAL2 (Ergebnis mit Doppelter Genauigkeit)



R2_ABS liefert den Absolutwert von X in doppelter Genauigkeit.

7.3 R2_ADD

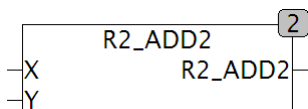
Type Funktion : REAL2
Input X : REAL2 (Eingangswert)
 Y : REAL (zu Addierender Wert)
Output REAL2 (Ergebnis mit Doppelter Genauigkeit)



R2_ADD addiert zu einem Wert mit doppelter Genauigkeit X einen Wert einfacher Genauigkeit Y. Das Ergebnis hat wieder Doppelte Genauigkeit.

7.4 R2_ADD2

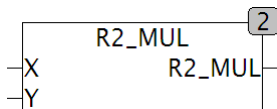
Type Funktion : REAL2
Input X : REAL2 (Eingangswert)
 Y : REAL2 (zu Addierender Wert)
Output REAL2 (Ergebnis mit Doppelter Genauigkeit)



R2_ADD2 addiert zu einem Wert doppelter Genauigkeit X einen anderen Wert doppelter Genauigkeit Y. Das Ergebnis hat wieder doppelte Genauigkeit.

7.5 R2_MUL

Type Funktion : REAL2
Input X : REAL2 (Eingangswert)
 Y : REAL (zu Addierender Wert)
Output REAL2 (Ergebnis mit Doppelter Genauigkeit)

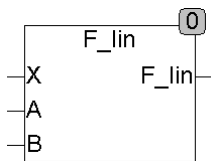


R2_MUL Multipliziert einen Wert Doppelter Genauigkeit X mit einem Wert einfacher Genauigkeit Y. Das Ergebnis hat wieder Doppelte Genauigkeit.

8 Arithmetische Funktionen

8.1 F_LIN

Type Funktion : REAL
Input X : REAL
 A : REAL
 B : REAL
Output REAL (F_LIN = A*X + B)

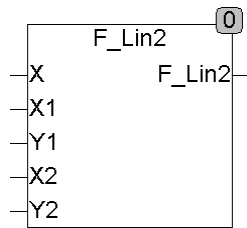


Die Funktion F_LIN Berechnet den Y-Wert einer linearen Gleichung.

$$F_LIN = A * X + B$$

8.2 F_LIN2

Type Funktion : REAL
Input X : REAL
 X1, Y1 : REAL (erste Koordinate)
 X2, Y2 : REAL (zweite Koordinate)
Output REAL (Wert auf der Geraden, die durch die obigen 2 Punkte
 definiert ist.)

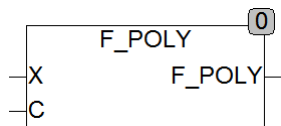


Die Funktion F_LIN2 Berechnet den Y-Wert einer linearen Gleichung.

Die Gerade ist dabei durch die Spezifikation zweier Koordinatenpunkte (X1,Y1; X2,Y2) definiert.

8.3 F_POLY

Type Funktion : REAL
 Input X : REAL (Eingang)
 C : ARRAY[0..7] of REAL (Polynomkoeffizienten)
 Output REAL (Ergebnis der Polynomgleichung)

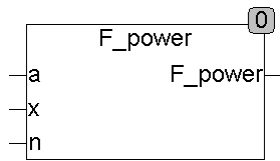


F_POLY errechnet ein Polynom 7ten Grades.

$$F_POLY = C[0] + C[1] * X^1 + C[2] * X^2 + ...C[7] * X^7$$

8.4 F_POWER

Type Funktion : REAL
 Input A : REAL
 X : REAL
 N : REAL
 Output REAL (Ergebnis der Gleichung $F_POWER = A * X^N$)



F_Power berechnet die Potenzfunktion nach der Gleichung $F_POWER = A * X^n$.

8.5 F_QUAD

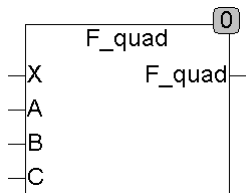
Type Funktion : REAL

Input X : REAL

X : REAL

A, B, C : REAL

Output REAL ($F_QUAD = A * X^2 + B * X + C$)



F_QUAD berechnet das Ergebnis einer quadratischen Gleichung nach der Formel $f_QUAD = A * X^2 + B * X + C$.

8.6 FT_AVG

Type Funktionsblock

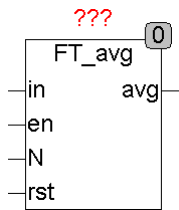
Input IN : REAL (Eingangssignal)

EN : BOOL (Freigabe Eingang)

N : INT (Anzahl der Werte über die der Mittelwert gebildet wird)

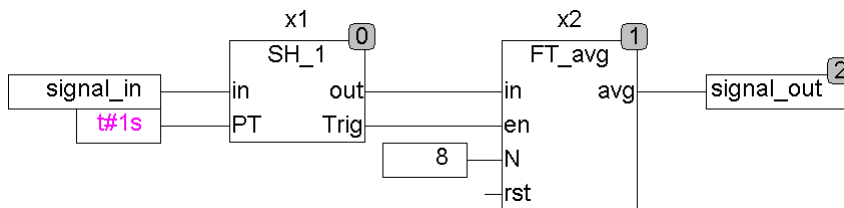
RST : BOOL (Reset Eingang)

Output REAL (Gleitender Mittelwert über die letzten N Werte)



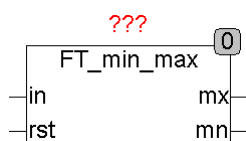
Der Funktionsblock FT_AVG berechnet einen gleitenden Mittelwert jeweils über die letzten N Werte. Durch den Eingang RST können die gespeicherten Werte gelöscht werden. N ist definiert von 0 .. 32. N=0 bedeutet das Ausgangssignal = Eingangssignal ist. N=5 bildet den Mittelwert über die letzten 5 Werte. Der Mittelwert wird maximal über 32 Werte gebildet. Durch den Eingang EN kann kontrolliert werden, wann der Eingang gelesen wird. Hierdurch kann auf einfache Weise ein Sample und Hold Baustein wie zum Beispiel SH_1 mit FT_AVG verknüpft werden. Beim ersten Aufruf von FT_AVG wird der Puffer mit dem Eingangssignal geladen, um zu vermeiden dass ein Ramp-up stattfindet.

Im folgenden Beispiel liest SH_1 einmal pro Sekunde den Eingangswert Signal_in und gibt diese Werte einmal pro Sekunde an FT_AVG weiter, welcher dann aus den letzten 8 Werten den Mittelwert bildet.



8.7 FT_MIN_MAX

Type	Funktionsblock
Input	IN : REAL (Eingangssignal)
	RST : BOOL (Reset Eingang)
Output	MX : REAL (Maximalwert des Eingangssignal)
	MN : REAL (Minimalwert des Eingangssignal)

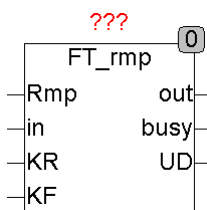


FT_MIN_MAX Speichert den Minimal- und Maximalwert eines Eingangssignals IN und stellt diese beiden Werte an den Ausgängen MN und MX zur Verfügung bis er

durch einen Reset wieder gelöscht wird. Ein Reset setzt MN und MX auf den zum Zeitpunkt des Resets anliegenden Eingangswert zurück.

8.8 FT_RMP

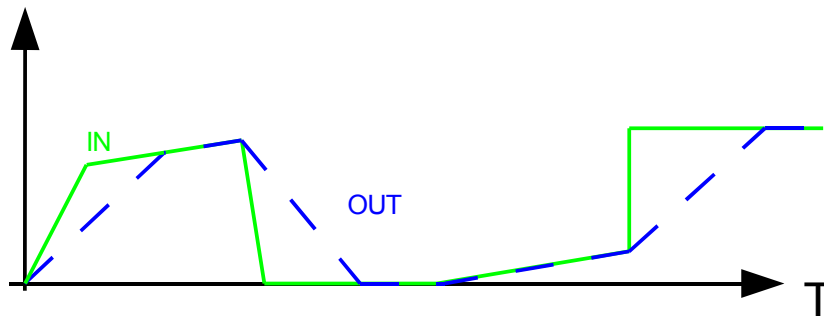
Type	Funktionsblock
Input	IN : REAL (Eingangssignal) KR : REAL (Geschwindigkeit des Anstiegs in 1 / Sekunden) KF : REAL (Geschwindigkeit des Abfalls in 1 / Sekunden)
Output	OUT : REAL (Ausgangssignal) BUSY : BOOL (Zeigt an ob Ausgang Steigt oder Fällt) UD : BOOL (TRUE, wenn Ausgang steigt und FALSE, wenn Ausgang fällt)



Der Ausgang OUT folgt dem Eingang mit einer linearen Rampe mit definierter Anstiegs- oder Abfall-Geschwindigkeit (KR und KF). $K = 1$ bedeutet, dass der Ausgang mit 1 Einheit pro Sekunde steigt oder fällt. Der K Faktor muss größer als 0 sein. Der Ausgang UD ist TRUE, wenn das Ausgangssignal steigt und FALSE, wenn es abfällt. Wenn der Ausgang den Eingangswert erreicht hat ist BUSY FALSE, ansonsten ist BUSY TRUE und zeigt an, dass eine steigende oder fallende Rampe aktiv ist.

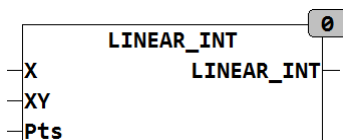
Das Ausgangssignal folgt solange dem Eingangssignal wie die Anstiegs- oder Abfall-Geschwindigkeit des Eingangssignals kleiner ist, als die durch KR und KF definierte maximale Anstiegs- oder Abfall-Geschwindigkeit. Verändert sich das Eingangssignal schneller, so läuft der Ausgang mit der Geschwindigkeit KR oder KF dem Eingangssignal hinterher. Die Rampenerzeugung ist Zeitecht, was bedeutet, dass FT_RMP zu jeder Ausführung berechnet wo der Ausgang stehen sollte und diesen Wert auf den Ausgang legt. Die Ausgangsveränderung ist also abhängig von der Zykluszeit und erfolgt nicht in gleichen Schritten. Wird eine Rampe aus lauter gleichen Schritten benötigt, so stehen die Bausteine RMP_B und RMP_W zur Verfügung.

Die folgende Grafik zeigt den Verlauf des Ausgangs in Abhängigkeit eines Eingangssignals:



8.9 LINEAR_INT

Type	Funktion
Input	X : REAL (Eingangssignal)
	XY : ARRAY[1..20,0..1] (Aufsteigend sortierte Wertepaare)
	PTS : INT (Anzahl der Wertepaare)
Output	REAL (Ausgangssignal)



LINEAR_INT ist ein linearer Interpolations Baustein. Eine beliebige Kennlinie wird mit maximal 20 Koordinatenwerten (X,Y) beschrieben und dadurch in bis zu 19 lineare Segmente zerlegt. Die Definition der Koordinatenwerte wird in einem Array übergeben welches die Kennlinie mit einzelnen X,Y Wertepaaren beschreibt. Die Wertepaare müssen aufsteigend nach dem X_Wert Sortiert sein. Wird ein X-Wert außerhalb des durch die Wertepaare beschriebenen Bereiches abgefragt, so wird das erste beziehungsweise letzte lineare Segment extrapoliert und der entsprechende Wert Ausgegeben. Um die Anzahl der Definitionspunkte flexibel zu halten wird am Eingang PTS die Anzahl der Punkte vorgegeben. Die mögliche Punktezahl liegt im Bereich 3 bis 20, wobei jeder Einzelpunkt mit X- und Y-Wert dargestellt ist.

Das folgende Beispiel zeigt die Definition für das Array XY und einige Werte:

VAR

BEISPIEL : ARRAY[1..4,0..1] := -10,-0.53, 10,0.53, 100,88.3, 200,122.2;

END_VAR

für obige Definition ergeben sich folgende Ergebnisse:

LINEAR_INT(0, Beispiel, 4) = 0;

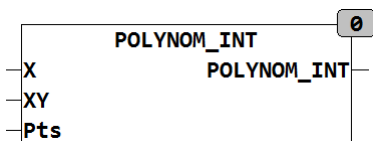
LINEAR_INT(30.0, Beispiel, 4) = 20.0344;

LINEAR_INT(66.41, Beispiel, 4) = 55.54229;

LINEAR_INT(800.0, Beispiel, 4) = 325.6;

8.10 POLYNOM_INT

Type	Funktion
Input	X : REAL (Eingangssignal) XY : ARRAY[1..5,0..1] (Aufsteigend sortierte Wertepaare) PTS : INT (Anzahl der Wertepaare)
Output	REAL (Ausgangssignal)



POLYNOM_INT interpoliert eine Anzahl von Wertepaaren mit einem Polynom N-fachen Grades. Die Anzahl der Wertepaare ist PTS und N entspricht der Anzahl der Wertepaare (PTS). Eine beliebige Kennlinie wird mit maximal 5 Koordinatenwerten (X,Y) beschrieben und intern mit einem Polynom beschrieben. Die Definition der Koordinatenwerte wird in einem Array übergeben welches die Kennlinie mit einzelnen X,Y Wertepaaren beschreibt. Die Wertepaare müssen aufsteigend nach dem X_Wert Sortiert sein. Wird ein X-Wert außerhalb des durch die Wertepaare beschriebenen Bereiches abgefragt, so wird dieser gemäß dem ermittelten Polygon berechnet, es ist hierbei aber zu beachten das durch ein Polynom höheren Grades Schwingungen oberhalb und unterhalb des Definitionsbereiches auftreten können und berechnete Werte in diesem Bereich meist nicht sinnvoll sind. Vor der Anwendung einer Polynominterpolation sind unbedingt Grundlagen hierzu, zum Beispiel in Wikipedia, nachzulesen. Um die Anzahl der Definitionspunkte flexibel zu halten wird am Eingang PTS die Anzahl der Punkte vorgegeben. Die mögliche Punktezahl liegt im Bereich 3 bis 5, wobei jeder Einzelpunkt mit X- und Y-Wert dargestellt ist. Ein Polynominterpolation mit mehr als 5 Punkten führt zu erhöhter Schwingneigung und ist aus diesem Grunde abzulehnen.

Das folgende Beispiel zeigt die Definition für das Array XY und einige Werte:

VAR

BEISPIEL : ARRAY[1..5,0..1] := -10,-0.53, 10,0.53, 100,88.3, 200,122.2;

END_VAR

für obige Definition ergeben sich folgende Ergebnisse:

POLYNOM_INT(0, Beispiel, 4) = -1.397069;

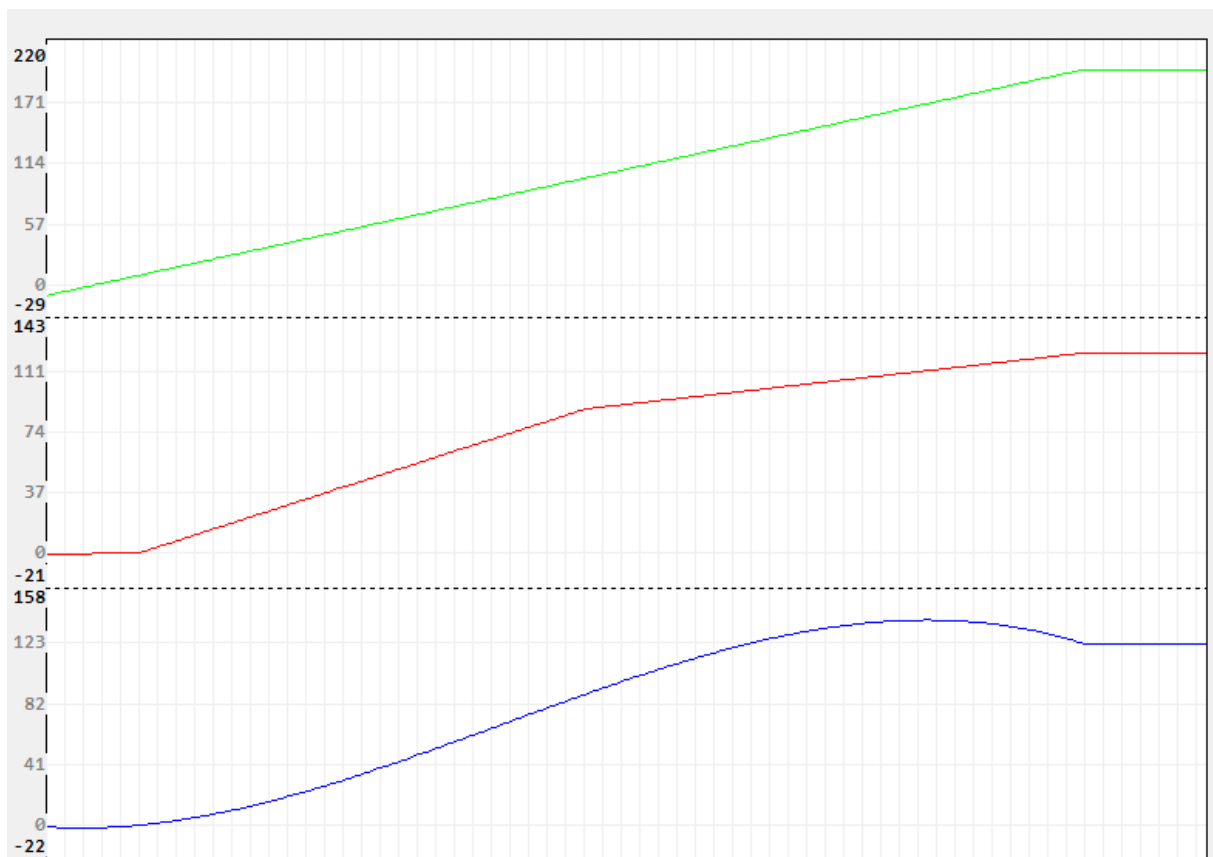
POLYNOM_INT(30.0, Beispiel, 4) = 11.4257;

POLYNOM_INT(66.41, Beispiel, 4) = 47.74527;

POLYNOM_INT(800.0, Beispiel, 4) = -19617.94;

Bei den Ergebnissen im Beispiel ist deutlich zu erkennen das der Wert von -19617.94 für den Eingang X = 800 keinen Sinn macht, da er außerhalb des definierten Bereichs von -10 bis +200 liegt.

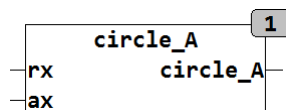
Die folgende Trace Aufzeichnung zeigt den Verlauf des Ausgangs zum Eingang. Hierbei sind deutlich die Überschwinger des Polygons gegenüber einer linearen Interpolation zu erkennen. Grün = Eingang X, Rot = lineare Interpolation, Blau = Polynom Interpolation.



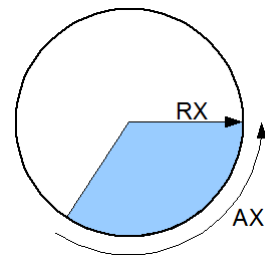
9 Geometrische Funktionen

9.1 CIRCLE_A

Type Funktion
 Input RX : REAL (Kreisradius)
 AX : REAL (Winkelsegment, 360 = gesamter Kreis)
 Output REAL (Fläche des Kreissegments)

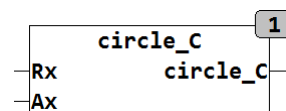


CIRCLE_A berechnet die Fläche eines Kreissegments mit den Winkel AX und dem Radius RX. Wird der Winkel AX = 360 gesetzt so wird die Kreisfläche berechnet.

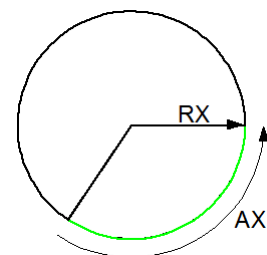


9.2 CIRCLE_C

Type Funktion
 Input RX : REAL (Kreisradius)
 AX : REAL (Winkelsegment, 360 = gesamter Kreis)
 Output REAL (Kreisbogenlänge bzw. Kreisumfang)

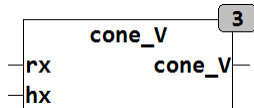


CIRCLE_C berechnet die Bogenlänge eines Kreisbogens mit den Winkel AX und dem Radius RX. Wird der Winkel AX = 360 gesetzt so wird der Kreisumfang berechnet.

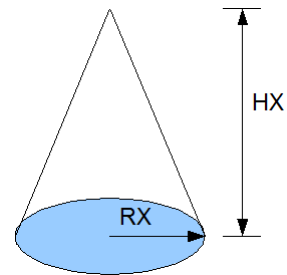


9.3 CONE_V

Type Funktion
 Input RX : REAL (Kreisradius der Grundfläche)
 HX : REAL (Höhe des Kegels)
 Output REAL (Volumen des Kegels)

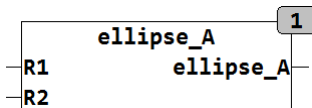


KONE_V berechnet das Volumen eines Kegels mit dem Radius RX und der Höhe HX.

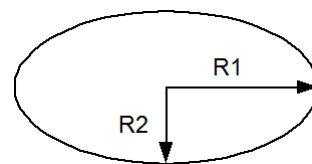


9.4 ELLIPSE_A

Type Funktion
 Input R1 : REAL (Radius 1)
 R2 : REAL (Radius 2)
 Output REAL (Fläche der Ellipse)

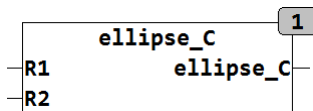


ELLIPSE_A berechnet die Fläche einer Ellipse die durch die Radien R1 und R2 definiert ist.

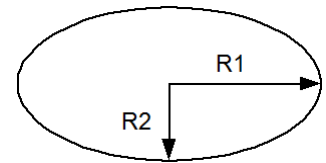


9.5 ELLIPSE_C

Type Funktion
 Input R1 : REAL (Radius 1)
 R2 : REAL (Radius 2)
 Output REAL (Umfang der Ellipse)

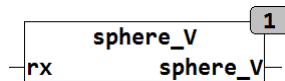


ELLIPSE_C berechnet den Umfang einer Ellipse die durch die Radien R1 und R2 definiert ist.



9.6 SPHERE_V

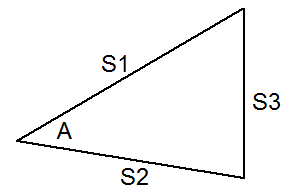
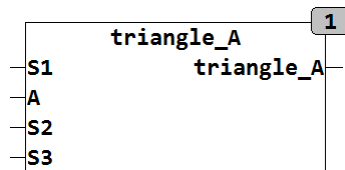
Type Funktion
 Input RX : REAL (Radius)
 Output REAL (Volumen der Kugel)



SPHERE_V berechnet das Volumen einer Kugel mit dem Radius RX.

9.7 TRIANGLE_A

Type	Funktion
Input	S1 : REAL (Seitenlänge 1) A : REAL (Winkel zwischen Seite 1 und Seite 2) S2 : REAL (Seitenlänge 2) S3 : REAL (Seitenlänge 3)
Output	REAL (Fläche des Dreiecks)



TRIANGLE_A berechnet die Fläche eines beliebigen Dreiecks. Das Dreieck kann wahlweise durch 2 Seiten und den durch die Seiten 1 und 2 aufgespannten Winkel (S1, S2 und A) definiert sein oder wenn $A = 0$ dann wird die Fläche aus den drei Seiten (S1, S2 und S3) berechnet.

10 Vektor Mathematik

10.1 Einleitung

Vektoren werden mit dem definierten Typ VEKTOR_3 abgebildet.

Der Typ VEKTOR_3 setzt sich aus 3 Komponenten X, Y und Z zusammen, alle Komponenten sind vom Typ REAL.

Der Vektor V vom Typ Vektor besteht aus:

V.X X Komponente vom Typ REAL.

V.Y Y Komponente vom Typ REAL.

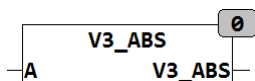
V.Z Z Komponente vom Typ REAL.

10.2 V3_ABS

Type Funktion

Input A : VECTOR_3 (Vektor mit den Koordinaten X, Y, Z)

Output REAL (Absolute Länge des Vektors)

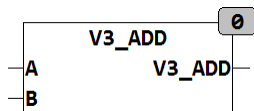


V3_ABS berechnet den Absolutwert (Länge) eines Vektors in einen dreidimensionalen Koordinatensystem.

V3_ABS(3,4,5) = 7.071...

10.3 V3_ADD

Type	Funktion
Input	A : VECTOR_3 (Vektor mit den Koordinaten X, Y, Z) B : VECTOR_3 (Vektor mit den Koordinaten X, Y, Z)
Output	VECTOR_3 (Vektor mit den Koordinaten X, Y, Z)

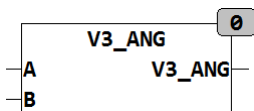


V3_ADD addiert 2 dreidimensionale Vektoren.

$$\text{V3_ADD}([3,4,5],[1,2,3]) = (4,6,8)$$

10.4 V3_ANG

Type	Funktion
Input	A : VECTOR_3 (Vektor mit den Koordinaten X, Y, Z) B : VECTOR_3 (Vektor mit den Koordinaten X, Y, Z)
Output	REAL (Winkel in Bogenmaß)



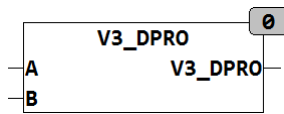
V3_ANG berechnet den Winkel zwischen 2 dreidimensionalen Vektoren

$$\text{V3_ANG}([1,0,0],[0,1,0]) = 1,57.. \text{ (PI / 2)}$$

10.5 V3_DPRO

Type	Funktion
Input	A : VECTOR_3 (Vektor mit den Koordinaten X, Y, Z) B : VECTOR_3 (Vektor mit den Koordinaten X, Y, Z)

Output REAL (Skalares Produkt)



V3_DPRO berechnet das Skalare Produkt zweier dreidimensionaler Vektoren.

$V3_DPRO([1,2,3],[3,1,2]) = 11$

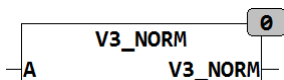
Das Skalare Produkt berechnet sich aus $A.X*B.X + A.Y*B.Y + A.Z*B.Z$

10.6 V3_NORM

Type Funktion

Input A : VECTOR_3 (Vektor mit den Koordinaten X, Y, Z)

Output VECTOR_3 (Vektor mit den Koordinaten X, Y, Z)



V3_NORM erzeugt aus einen beliebigen dreidimensionalen Vektor einen auf die Länge 1 Normierten Vektor mit gleicher Richtung. Ein Vektor mit der Länge 1 wird Einheitsvektor genannt

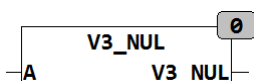
$V3_NORM(3,0,0) = (1,0,0)$

10.7 V3_NUL

Type Funktion

Input A : VECTOR_3 (Vektor mit den Koordinaten X, Y, Z)

Output BOOL (TRUE wenn Vektor ein Nullvektor ist)

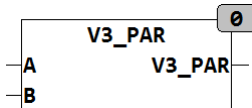


V3_NUL prüft ob der Vektor A ein Nullvektor ist. Ein Vektor ist dann ein Nullvektor wenn alle Komponenten (X, Y, Z) gleich Null sind.

$V3_NUL(0,0,0) = TRUE$

10.8 V3_PAR

Type	Funktion
Input	A : VECTOR_3 (Vektor mit den Koordinaten X, Y, Z) B : VECTOR_3 (Vektor mit den Koordinaten X, Y, Z)
Output	BOOL (TRUE wenn die beiden Vektoren parallel sind)



V3_PAR wird dann TRUE wenn die beiden Vektoren A und B Parallel sind. Ein Nullvektor ist Parallel zu jedem Vektor da er keine Richtung hat. Zwei Vektoren A und B die in Gegensätzliche Richtung zeigen sind Parallel.

$V3_PAR([1,1,1],[2,2,2]) = TRUE$

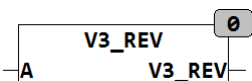
$V3_PAR([1,1,1],[-1,-1,-1]) = TRUE$

$V3_PAR([1,2,3],[0,0,0]) = TRUE$

$V3_PAR([1,2,3],[1,0,0]) = FALSE$

10.9 V3_REV

Type	Funktion
Input	A : VECTOR_3 (Vektor mit den Koordinaten X, Y, Z)
Output	VECTOR_3 (Vektor mit den Koordinaten X, Y, Z)

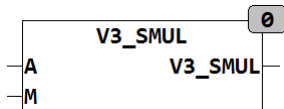


V3_REV erzeugt einen Vektor mit dem identischen Betrag von A aber mit entgegengesetzter Richtung. $A - V3_REV(A) = 0$.

$V3_REV(1,2,3) = (-1,-2,-3)$

10.10 V3_SMUL

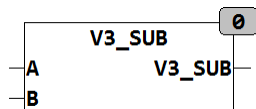
Type	Funktion
Input	A : VECTOR_3 (Vektor mit den Koordinaten X, Y, Z) M : REAL (Skalarer Multiplikator)
Output	VECTOR_3 (Vektor mit den Koordinaten X, Y, Z)



V3_SMUL Multipliziert einen dreidimensionalen Vektor A mit dem Skalar M.
 $V3_SMUL([1,2,3],10) = (10,20,30)$

10.11 V3_SUB

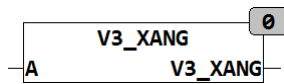
Type	Funktion
Input	A : VECTOR_3 (Vektor mit den Koordinaten X, Y, Z) B : VECTOR_3 (Vektor mit den Koordinaten X, Y, Z)
Output	VECTOR_3 (Vektor mit den Koordinaten X, Y, Z)



V3_SUB Subtrahiert den Vektor B von A
 $V3_SUB([3,3,3],[1,2,3]) = (2,1,0)$
 $V3_SUB([1,2,3],[1,-2,-3]) = (0,4,6)$

10.12 V3_XANG

Type	Funktion
Input	A : VECTOR_3 (Vektor mit den Koordinaten X, Y, Z)
Output	REAL (Winkel zur X-Achse)

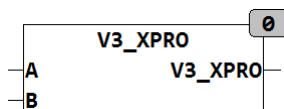


V3_XANG berechnet den Winkel zwischen Der X-Achse des Koordinatensystems und einem dreidimensionalen Vektor A in Bogenmaß

$V3_XANG(1,2,3) = 1.300..$

10.13 V3_XPRO

Type	Funktion
Input	A : VECTOR_3 (Vektor mit den Koordinaten X, Y, Z)
	B : VECTOR_3 (Vektor mit den Koordinaten X, Y, Z)
Output	VECTOR_3 (Vektor mit den Koordinaten X, Y, Z)



V3_XPRO berechnet das Kreuzprodukt zweier dreidimensionaler Vektoren A und B

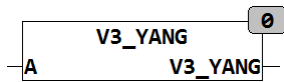
$V3_XPRO([1,2,3],[2,1,2]) = (1,4,-3)$

10.14 V3_YANG

Type Funktion

Input A : VECTOR_3 (Vektor mit den Koordinaten X, Y, Z)

Output REAL (Winkel zur Y-Achse)



V3_YANG berechnet den Winkel zwischen Der Y-Achse des Koordinatensystems und einem dreidimensionalen Vektor A in Bogenmaß

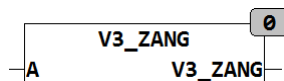
$V3_YANG(1,2,3) = 1.006..$

10.15 V3_ZANG

Type Funktion

Input A : VECTOR_3 (Vektor mit den Koordinaten X, Y, Z)

Output REAL (Winkel zur Z-Achse)



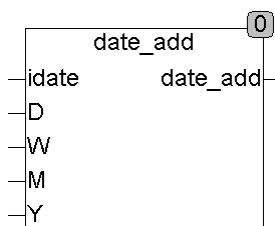
V3_ZANG berechnet den Winkel zwischen Der Z-Achse des Koordinatensystems und einem dreidimensionalen Vektor A in Bogenmaß

$V3_ZANG(1,2,3) = 0.640..$

11 Time & Date

11.1 DATE_ADD

Type	Funktion
Input	IDATE : DATE (Eingangsdatum) D : INT (zu addierende Tage) W : INT (zu addierende Wochen) M : INT (zu addierende Monate) Y : INT (zu addierende Jahre)
Output	DATE (Ergebnisdatum)



Die Funktion DATE_ADD addiert Tage, Wochen, Monate, und Jahre zu einem Datum hinzu. Es werden erst die angegebenen Tage und Wochen addiert, dann die Monate und zuletzt die Jahre.

Die Eingangswerte können sowohl positiv, wie auch negativ sein. Es kann also auch von einem Datum subtrahiert werden.

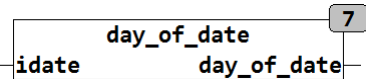
Vor allem bei negativen Eingangswerten ist zu beachten das das Datum beim addieren von negativen Werten wie zum Beispiel -3000 Tage nicht unter den 1.1.1970 läuft, dies würde einen Überlauf des Datentyps DATE zur folge haben und undefinierte Werte ergeben.

Beispiel: `DATE_ADD(1.1.2007,3,1,-1,-2) = 11.12.2005`

addiert 3 Tage und 1 Woche hinzu und zieht dann 1 Monat und 2 Jahre ab.

11.2 DAY_OF_DATE

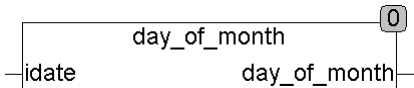
Type Funktion : DINT
 Input IDATE : DATE (Eingangsdatum)
 Output DINT (Tag im Monat des Eingangsdatums)



Die Funktion DAY_OF_DATE berechnet den Tag seit dem 1.1.1970. Das Ergebnis der Funktion ist vom Typ DINT weil der gesamte DATE Range 49710 Tage umfasst.

11.3 DAY_OF_MONTH

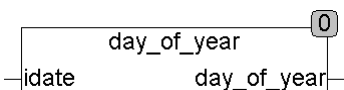
Type Funktion : INT
 Input IDATE : DATE (Eingangsdatum)
 Output INT (Tag im Monat des Eingangsdatums)



Die Funktion DAY_OF_MONTH berechnet den Tag des Monats aus dem Eingangsdatum IDATE.

11.4 DAY_OF_YEAR

Type Funktion : INT
 Input IDATE : DATE (Eingangsdatum)
 Output INT (Tag im Jahr des Eingangsdatums)



Die Funktion DAY_OF_YEAR berechnet den Tag des Jahres aus dem Eingangsdatum IDATE.

tum IDATE. Schaltjahre werden entsprechend dem gregorianischen Kalender berücksichtigt. Die Funktion ist definiert für die Jahre 1970 – 2099.

Beispiel: $\text{DAY_OF_YEAR}(31.12.2007) = 365$

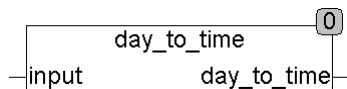
$\text{DAY_OF_YEAR}(31.12.2008) = 366$

11.5 DAY_TO_TIME

Type Funktion : TIME

Input Input : REAL (Anzahl Tage mit Nachkommastellen)

Output TIME (TIME)



Die Funktion DAY_TO_TIME berechnet einen Zeitwert (TIME) aus dem Eingangswert in Tagen als REAL.

Beispiel: $\text{DAY_TO_TIME}(1.1) = \text{T\#26h24m}$

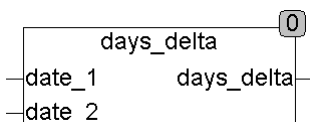
11.6 DAYS_DELTA

Type Funktion : DINT

Input DATE_1 : DATE (Datum1)

DATE_2 : DATE (Datum2)

Output DINT (Differenz der beiden Eingangsdatums in Tagen)



Die Funktion DAYS_DELTA berechnet die Differenz zweier Daten in Tagen.

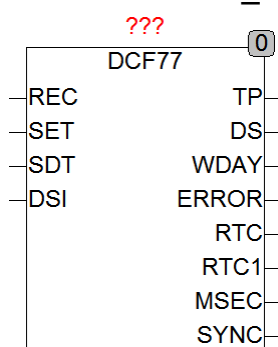
Beispiel: $\text{DAYS_DELTA}(10.1.2007, 1.1.2007) = -9$

$\text{DAYS_DELTA}(1.1.2007, 10.1.2007) = 9$

Das Ergebnis der Funktion ist vom Typ DINT weil der gesamte DATE Range 49710 Tage umfasst.

11.7 DCF77

Type	Funktionsblock
Input	REC : BOOL (Eingang für den DCF77 Empfänger) SET : BOOL (Asynchroner SET Eingang) SDT : DT (Anfangswert für RTC) DSI : BOOL (Sommerzeit Eingang)
Output	TP : BOOL (Puls zum Setzen von nachgeschalteten Uhren) DS : BOOL (TRUE, wenn Sommerzeit herrscht) WDAY : INT (Wochentag) ERROR : BOOL (TRUE, wenn kein gültiges Signal am Eingang rec) RTC : DT (Synchronisierte Weltzeit UTC) RTC1 : DT (Synchronisierte Lokalzeit) MSEC : INT (Millisekunden von RTC und RTC1) SYNC : BOOL (TRUE, wenn RTC und RTC1 mit DCF synchron sind)
Config	SYNC_TIMEOUT : TIME (Default = T#2m) TIME_OFFSET : INT (Zeit Offset für RTC1, Default = 1 Stunde) DST_EN : BOOL (Sommerzeit für RTC1, Default = TRUE)



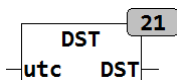
Die Funktion DCF77 dekodiert das Serielle Signal eines DCF77 Empfängers und steuert 2 interne Uhren RTC und RTC1, oder über den Ausgang TP auch externe (nachgeschaltete) Uhren. Ein Ausgang DS wird TRUE, wenn Sommerzeit herrscht. Der Ausgang WDAY gibt den Wochentag an (1 = Montag). Der Ausgang ERROR wird TRUE, wenn kein gültiges Signal Empfangen wird. Die Internen Uhren laufen aber trotzdem weiter, wenn Sie bereits synchronisiert sind. Ein weiterer Ausgang SYNC zeigt an dass die internen Uhren mit DCF77 synchronisiert sind und wird FALSE, wenn sie länger als durch die Setup-Variable SYNC_TIMEOUT festgelegte Zeit nicht mehr synchronisiert wurden. Die internen Uhren laufen in jedem Fall mit der Genauigkeit des SPS-Timers weiter. Durch einen Doppelklick auf das Symbol im CFC Editor können weitere Setup-Variablem definiert werden. Hierbei legt SYNC_TI-

MEOUT fest, nach welcher Zeit das Ausgangssignal SYNC, FALSE wird, wenn die internen Uhren RTC und RTC1 nicht mehr durch DCF77 synchronisiert wurden. Die Variable TIME_OFFSET legt die Zeitdifferenz der Lokalzeit (RTC1) von der UTC fest. Default ist 1 Stunde für MEZ (Mitteleuropäische Zeit). Die Variable TIME_OFFSET ist vom Typ INTEGER damit auch Zeitzonen mit negativem Offset (Westlich von Greenwich) möglich sind.

Durch DST_EN wird festgelegt, ob RTC1 automatisch auf Sommerzeit schalten soll oder nicht. Der Ausgang MSEC erweitert die auf RTC und RTC1 zur Verfügung gestellte Zeit um Millisekunden. Der Eingang SDT dient dazu die internen Uhren RTC und RTC1 auf einen definierten Anfangswert zu setzen damit sofort nach den Start eine gültige Uhrzeit zur Verfügung steht. Während des ersten Zyklus wird Datum und Zeit von SDT nach RTC kopiert, und läuft ab dem ersten Zyklus. Falls nötig kann mit den asynchronen Setz Eingang SET die interne UHR jederzeit neu gestellt werden. Sie wird aber nach einem Zyklus wieder von einem Gültigen DCF77 Signal überschrieben, ausser der Eingang SET bleibt auf TRUE. Sobald ein gültiges DCF77 Signal dekodiert wurde wird RTC und RTC1 auf die entsprechende genauere DCF77 Zeit synchronisiert. Am Eingang SDT kann zum Beispiel die Uhrzeit aus der in der SPS enthaltenen Hardware Uhr verwendet werden, es muss aber sichergestellt werden das DCF77 erst dann aufgerufen wird wenn bereits eine gültige Uhrzeit an SDT anliegt, den DCF77 liest diesen Wert nur ein einziges mal im ersten Zyklus ein, oder immer dann wenn der Eingang SET auf TRUE steht.

11.8 DST

Type	Funktion : BOOL
Input	UTC : DATE_TIME (Weltzeit)
	TIME_Zone_Offset : TIME (Differenz zur Weltzeit (UTC))
Output	BOOL (TRUE, wenn Sommerzeit)



Die Funktion DST überprüft, ob im Augenblick Sommerzeit herrscht, oder nicht. Sie kann dazu benutzt werden eine vorhandene nicht-Sommerzeit fähige Uhr sekunden-genau auf Sommer- und Winterzeit umzustellen.

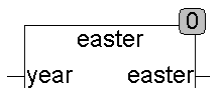
Die Funktion DST schaltet am letzten Sonntag des März um 01:00 UTC (02:00 MEZ) auf Sommerzeit (03:00 MESZ) und am letzten Sonntag des Oktober um 01:00 UTC (03:00 MESZ) auf 02:00 MEZ zurück. Der Ausgang von DST ist dann TRUE, wenn Sommerzeit herrscht.

Die Sommerzeit wird aufgrund von UTC (Weltzeit) berechnet. Eine Berechnung von Ortszeit nach Sommerzeit ist generell nicht möglich weil im letzten Sonntag des Okt-

obers die Stunde von 02:00 - 03:00 MEZ beziehungsweise MESZ doppelt existiert. Die Sommerzeit wird in allen Ländern der EU seit 1992 zur selben Sekunde nach Weltzeit umgestellt. In Mitteleuropa um 02:00, in England um 01:00 und in Griechenland um 04:00. Durch die Berechnung mithilfe der Weltzeit wird die Sommerzeit für alle Europäischen Zeitzone richtig berechnet.

11.9 EASTER

Type Funktion : DATE
 Input YEAR : INT (Jahr)
 Output DATE (Datum des Ostersonntag für das angegebene Jahr)



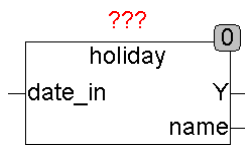
Die Funktion EASTER berechnet für ein gegebenes Jahr das Datum des Ostersonntags. Die meisten kirchlichen Feiertage haben einen festen Abstand von Ostern, sodass für den Fall, dass Ostern für ein Jahr bekannt ist, diese Feiertage auch einfach ermittelt werden können. EASTER wird auch im Modul HOLIDAY verwendet um Feiertage zu berechnen.

11.10 HOLIDAY

Type Funktionsblock
 Input DATE_IN : DATE (Eingangsdatum)
 Config SIZE : INT (maximale Anzahl der Feiertage)
 FEIERTAG : STRING (Namen der Feiertage)
 F_DAY : SINT (Monatstage der Feiertage)
 F_MONTH : SINT (Monate der Feiertage)
 F_USE : SINT (Feiertags Freigabe / Typ)
 FRIDAY : BOOL (TRUE wenn jeder Freitag ein Feiertag sein soll)
 SATURDAY : BOOL (TRUE wenn jeder Samstag ein Feiertag sein soll)
 SUNDAY : BOOL (TRUE wenn jeder Sonntag ein Feiertag sein soll)

Output Y : BOOL (TRUE, wenn DATE_IN ein Feiertag ist)

Name : STRING (Name des heutigen Feiertags)



Die Funktion HOLIDAY zeigt am Ausgang Y mit TRUE Feiertage an und liefert auch den Namen des aktuellen Feiertags am Ausgang NAME. HOLIDAY kann zusätzlich zu Feiertagen auch an den Wochentagen Freitag, Samstag oder Sonntag aktiv werden und am Ausgang Y TRUE liefern, abhängig davon ob die Setup Variablen FRIDAY, SATURDAY oder SUNDAY auf TRUE gesetzt sind. Mit Setup-Variablen werden Name und Datum von Feiertagen definiert und sind auch dort für andere Länder universell anpassbar. Feiertage können als festes Datum, mit einem Abstand von Ostern oder Wochentag vor einem festen Datum definiert werden. Als Vorgabe sind die deutschen Feiertage vordefiniert und können ganz einfach durch die Variable F_USE für einzelne Bundesländer ein oder Ausgeschaltet werden. Auch weitere lokale Feiertage können so definiert werden. Das Array Feiertag beinhaltet die Namen der Feiertage, die am Datum des Feiertags am Ausgang NAME als STRING zur Verfügung gestellt werden. Im Array F_DAY wird der jeweilige Tag im Monat oder Abstand von Ostern für die Feiertage definiert. Im Array F_MONTH steht der jeweilige Monat des Feiertags. Wenn in F_MONTH eine 0 steht, bedeutet dies, dass unter F_DAY nicht der Monatstag, sondern der Abstand von Ostern steht.

Im Array F_USE bedeutet eine 0, dass dieser Feiertag abgeschaltet ist. Eine 1 bedeutet, dass er eingeschaltet ist. Eine negative Zahl in F_USE gibt an, dass der angegebene Feiertag ein Wochentag vor dem in F_DAY und F_MONTH angegebenen Datum ist. -1 etwa für Montag und -7 für Sonntag.

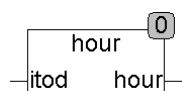
Beispiele sind die als Default im Modul enthaltenen deutschen Feiertage, die dort auch erweitert oder geändert werden können.

11.11 HOUR

Type Funktion : INT

Input ITOD : TIMEOFDAY (Tageszeit)

Output INT (aktuelle Stunde)

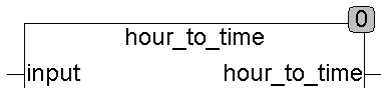


Die Funktion HOUR extrahiert die aktuelle Stunde aus der Tageszeit.

Beispiel: HOUR(22:55:13) = 22

11.12 HOUR_TO_TIME

Type Funktion : TIME
 Input INPUT : REAL (Anzahl Stunden mit Nachkommastellen)
 Output TIME (TIME)

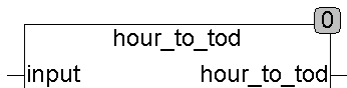


Die Funktion HOUR_TO_TIME berechnet einen Zeitwert (TIME) aus dem Eingangswert in Stunden als REAL.

Beispiel: HOUR_TO_TIME(1.1) = T#1h6m

11.13 HOUR_TO_TOD

Type Funktion : TIME
 Input INPUT : REAL (Anzahl Stunden mit Nachkommastellen)
 Output TIME (Tageszeit)

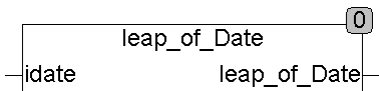


Die Funktion HOUR_TO_TOD berechnet eine Tageszeit (TIMEOFDAY) aus dem Eingangswert in Stunden als REAL.

Beispiel: HOUR_TO_TOD(12.1) = 12:06:00

11.14 LEAP_OF_DATE

Type Funktion : BOOL
 Input IDATE : DATE (Eingangsdatum)
 Output BOOL (TRUE, wenn IDATE in einem Schaltjahr liegt)

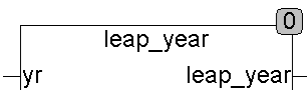


Die Funktion LEAP_OF_DATE testet, ob das Eingangsdatum in einem Schaltjahr liegt. Die Funktion berechnet, ob ein Datum innerhalb eines Schaltjahres liegt und gibt gegebenenfalls TRUE aus. Der Test hat Gültigkeit für den Zeitraum 1970 - 2099. im Jahr 2100 wird ein Schaltjahr angezeigt obwohl dies keines ist. Da aber der Wertebereich des Datums nach IEC61131-3 nur bis zum Jahr 2106 reicht wird auf diese Korrektur verzichtet.

Beispiel: LEAP_OF_YEAR(D#2004-01-12) = TRUE

11.15 LEAP_YEAR

Type Funktion : BOOL
 Input YR : INT (Jahreszahl)
 Output BOOL (TRUE, wenn das angegebene Jahr ein Schaltjahr ist)



Die Funktion LEAP_YEAR testet, ob das Eingangsjahr ein Schaltjahr ist und gibt gegebenenfalls TRUE aus. Der Test hat Gültigkeit für den Zeitraum 1970 - 2099. im Jahr 2100 wird ein Schaltjahr angezeigt obwohl dies keines ist. Da aber der Wertebereich des Datums nach IEC61131-3 nur bis zum Jahr 2106 reicht wird auf diese Korrektur verzichtet.

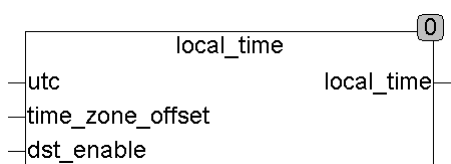
11.16 LOCAL_TIME

Die Funktion LOCAL_TIME wurde durch die Funktion LTIME ersetzt!
 Bitte in Zukunft nur noch LTIME einsetzen.

Type Funktion : DATE_TIME
 Input UTC : DATE_TIME (Weltzeit)
 TIME_ZONE_OFFSET : INT (Zeitdifferenz zu UTC in Stunden)

DST_ENABLE : BOOL (TRUE bei Sommerzeit)

Output DATE_TIME (Lokalzeit mit automatischer Sommerzeitschaltung)



Die Funktion LOCAL_TIME errechnet die Lokalzeit mit automatischer Umschaltung von Winter- auf Sommerzeit. Die Sommerzeit wird exakt am letzten Sonntag im März eingeschaltet und im Oktober entsprechend zurück auf Winterzeit gesetzt.

Mit LOCAL_TIME kann die SPS-Uhr auf UTC (Weltzeit) laufen und mit mehreren LOCAL_TIME Modulen können beliebig viele Weltzeiten errechnet werden.

11.17 LTIME

Type Funktionsblock

Input UTC : DATE_TIME (Weltzeit)

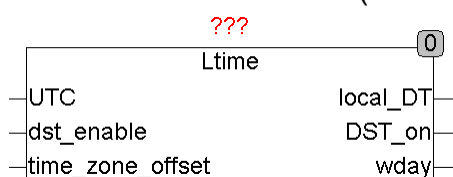
DST_ENABLE : BOOL (TRUE erlaubt Sommerzeit)

TIME_ZONE_OFFSET : INT (Zeitdifferenz zur Weltzeit in Stunden)

Output LOCAL_DT : DATE_TIME (Lokalzeit)

DST_ON : BOOL (TRUE, wenn Sommerzeit herrscht)

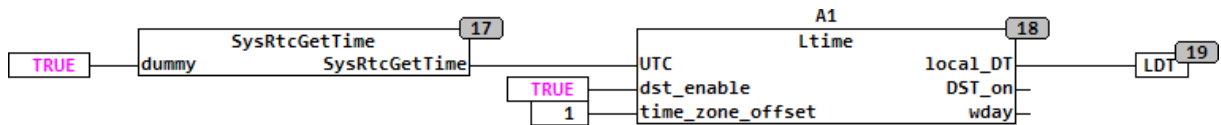
WDAY : INT (Wochentag 1 = Montag, 7 = Sonntag)



Der Funktionsblock LTIME errechnet aus der Weltzeit am Eingang UTC eine Lokalzeit (LOCAL_DT) mit automatischer Sommerzeitumschaltung falls DST_ENABLE auf TRUE steht. Ist DST_ENABLE FALSE wird die Lokalzeit ohne Sommerzeitumschaltung berechnet. Zusätzlich berechnet der Baustein noch den aktuellen Wochentag WDAY (1= Montag und 7 = Sonntag). DST_ON ist TRUE wenn am Ausgang LOCAL_DT Sommerzeit anliegt.

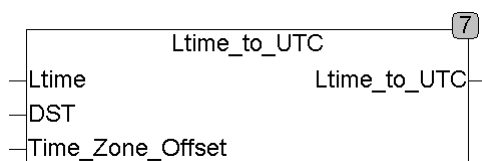
Dieser Funktionsblock benötigt UTC am Eingang, welche normalerweise von der SPS zur Verfügung gestellt wird und durch eine Routine des Herstellers gelesen werden kann.

Im folgenden Beispiel ist die Anwendung für eine WAGO 750-841 CPU dargestellt. Das Auslesen der internen Uhr wird durch die Herstelleroutine SYSRTCGETTIME erledigt. Die SPS-Uhr muss in diesem Fall auf Weltzeit eingestellt werden.



11.18 LTIME_TO_UTC

Type Funktion : DATE_TIME
 Input LTIME : DATE_TIME (Lokalzeit)
 DST : BOOL (TRUE, wenn Sommerzeit herrscht)
 TIME_ZONE_OFFSET : INT (Zeitdifferenz zur Weltzeit in Stunden)
 Output DATE_TIME (UTC, Weltzeit)



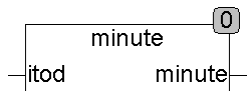
LTIME_TO_UTC errechnet UTC (Weltzeit) von einer vorgegebenen Lokalzeit. Die Weltzeit wird errechnet indem TIME_ZONE_OFFSET von der Lokalzeit LTIME subtrahiert wird. Wenn die Sommerzeit aktiv ist (DST = TRUE), wird eine weitere Stunde von LTIME abgezogen.

Anmerkung: Diese Funktion gilt nur innerhalb des europäischen Kontinents!

Für andere Regionen gelten unterschiedliche Regeln für die Sommerzeit.

11.19 MINUTE

Type Funktion : INT
 Input ITOD : TIMEOFDAY (Tageszeit)
 Output INT (aktuelle Minute)



Die Funktion MINUTE extrahiert die aktuelle Minute aus der Tageszeit.

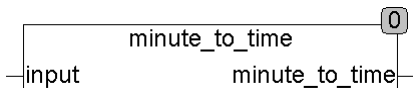
Beispiel: MINUTE(22:55:13) = 55

11.20 MINUTE_TO_TIME

Type Funktion : TIME

Input ITOD : REAL (Anzahl Minuten mit Nachkommastellen)

Output TIME (TIME)



Die Funktion MINUTE_TO_TIME berechnet einen Zeitwert (TIME) aus dem Eingangswert in Minuten als REAL.

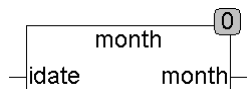
Beispiel: MINUTE_TO_TIME(122.5) = T#2h2m30s

11.21 MONTH

Type Funktion : INT

Input IDATE : DATE (Eingangsdatum)

Output INT (Monat im Jahr des Eingangsdatums)



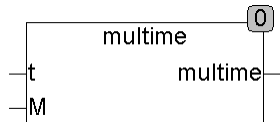
Die Funktion MONTH berechnet den Monat des Jahres aus dem Eingangsdatum IDATE.

Beispiel: MONTH(31.12.2007) = 12

MONTH(1.4.2008) = 4

11.22 MULTIME

Type Funktion : TIME
 Input T : TIME (Eingangszeit)
 M : REAL (Multiplikator)
 Output TIME (Ergebnis Eingangszeit Multipliziert mit M)

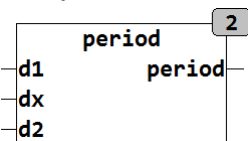


Die Funktion MULTIME Multipliziert einen Zeitwert mit einem Multiplikator.

Beispiel: MULTIME(T#1h10m, 2.5) = T#2h55m

11.23 PERIOD

Type Funktion : BOOL
 Input D1 : DATE (Perioden Beginn)
 DX : DATE (zu testendes Datum)
 D2 : DATE (Perioden Ende)
 Output BOOL (TRUE wenn DX innerhalb der Periode D1 .. D2 liegt)

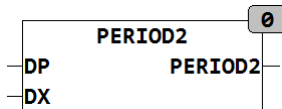


Die Funktion PERIOD prüft, ob ein Eingangsdatum DX größer gleich D1 und kleiner gleich D2 ist. Wenn das Datum DX im Zeitraum zwischen D1 und D2 (D1 und D2 eingeschlossen) liegt wird der Ausgang der Funktion TRUE gesetzt. PERIOD ignoriert dabei die Jahreszahlen in den Datumsangaben D1, D2 und DX. Die Prüfung erfolgt nur auf Monate und Tage, sodass diese Funktion für jedes Jahr funktioniert. Die zu Prüfende Periode kann auch über den 31. Dezember hinaus erstrecken, also zum Beispiel vom 1.9. – 15.3. Eine typische Anwendung ist die Prüfung, ob eine Heizperiode vorliegt. Damit PERIOD richtig rechnet dürfen die beiden Datumsangaben D1 und D2 nicht in einem Schaltjahr liegen. Es kann zum Beispiel immer das Jahr 2001 oder auch jedes beliebige andere Jahr das kein Schaltjahr ist verwendet werden.

PERIOD(1.10.2001, 12.11.2007, 31.3.2001) ergibt TRUE, weil das zu prüfende Datum innerhalb der Periode vom 1.10 - 31.3. liegt.

11.24 PERIOD2

Type Funktion : BOOL
 Input DP : ARRAY[0..3,0..1] of DATE (Perioden)
 DX : DATE (zu testendes Datum)
 Output BOOL (TRUE wenn DX innerhalb einer der Perioden liegt)



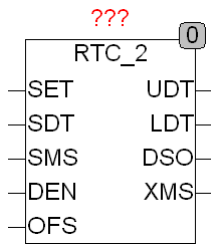
PERIOD2 prüft ob das Datum DX innerhalb einer von 4 spezifizierten Perioden liegt. Die Perioden werden in einem ARRAY[0..3,0..1] of DATE spezifiziert. Im Gegensatz zur Funktion PERIOD prüft PERIOD2 auch das Jahr. Die Perioden werden im ARRAY DP spezifiziert, wobei DP[N,0] das Anfangsdatum der Periode N DP[N,1] das Enddatum der Periode N ist.

Die Funktion Prüft nach der Formel: $DX \geq DP[N,0] \text{ AND } DX \leq DP[N,1]$. Es wird Dabei jeweils N=0 bis 3 geprüft. Wenn DX in eine der 4 Perioden fällt wird der Ausgang auf TRUE gesetzt.

Die einzelnen Perioden Müssen nicht sortiert vorliegen. PERIOD2 kann benutzt werden um Ferien oder Urlaubszeiten zu definieren. PERIOD2 prüft nicht wiederkehrende Perioden, wobei die Funktion PERIOD jährlich wiederkehrende Perioden Prüft.

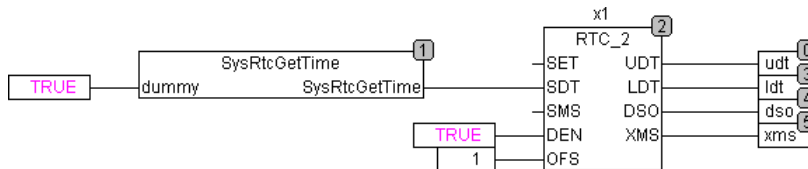
11.25 RTC_2

Type Funktionsblock
 Input SET : BOOL (Set Eingang)
 SDT : DT (Set Datum und Zeit)
 SMS : INT (Set Millisekunden)
 DEN : BOOL (Automatische Sommerzeitumstellung Ein)
 OFS : INT (Offset der Lokalzeit LDT von der Weltzeit UDT)
 Output UDT : DT (Datums und Zeit Ausgang für Weltzeit)
 LDT : DT (Lokalzeit)
 DSO : BOOL (Sommerzeit aktiv)
 XMS : INT (Millisekunden)



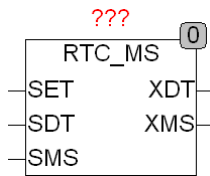
RTC_2 ist ein Uhrenbaustein der UTC und Lokale Zeit an den Ausgängen UDT und LDT zur Verfügung stellt. Die Uhrzeit wird automatisch beim ersten Start und immer dann wenn SET auf TRUE ist auf den Wert von SDT und SMS gesetzt. Wenn SET = FALSE läuft die Zeit selbständig weiter und liefert am Ausgang UDT das aktuelle Datum und die Uhrzeit für Weltzeit (UTC), sowie am Ausgang LDT die aktuelle Lokalzeit. Der Ausgang LDT entspricht UDT + OFS + Sommerzeit wenn diese aktuell ist. Die Sommerzeit wird wenn DEN TRUE ist automatisch am letzten Sonntag des März um 01:00 UTC (02:00 MEZ) auf Sommerzeit (03:00 MESZ) und am letzten Sonntag des Oktober um 01:00 UTC (03:00 MESZ) auf 02:00 MEZ zurück gestellt. Der Ausgang DSO ist dann TRUE, wenn Sommerzeit herrscht. Wenn DEN FALSE ist wird keine Sommerzeitumstellung vorgenommen. Die Genauigkeit der Uhr hängt vom Millisekunden Timer der SPS ab. Der Eingang OFS spezifiziert den Zeitversatz von LDT zu UDT, für MEZ ist dieser Wert 1 Stunde. OFS wird als Integer spezifiziert damit auch ein negativer Offset möglich ist. RTC_2 übernimmt beim Power Up automatisch die an SDT anliegende Startzeit und Datum. Der Ausgang XMS stellt die Millisekunden zur Verfügung und zählt in jeder Sekunde von 0 – 999.

Im folgenden Beispiel wird beim Start die Systemzeit übernommen.



11.26 RTC_MS

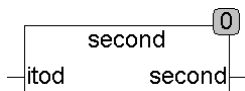
Type	Funktionsblock
Input	SET : BOOL (Set Eingang)
	SDT : DT (Set Datum und Zeit)
	SMS : INT (Set Millisekunden)
Output	XDT : DT (Datums und Zeit Ausgang)
	XMS : INT (Millisekunden Ausgang)



RTC_MS ist ein Uhrenbaustein mit einer Auflösung von Millisekunden und Datum. Die Uhrzeit wird automatisch beim ersten Start und immer dann wenn SET auf TRUE ist auf den Wert von SDT und SMS gesetzt. Wenn SET = FALSE läuft die Zeit selbstständig weiter und liefert am Ausgang XDT das aktuelle Datum und die Uhrzeit, sowie am Ausgang XMS die Millisekunden. Der Ausgang XMS zählt in jeder Sekunde von 0 - 999 und beginnt mit der nächsten Sekunde wieder bei 0. Die Genauigkeit der Uhr hängt vom Millisekunden Timer der SPS ab.

11.27 SECOND

Type Funktion : REAL
 Input ITOD : TOD (Tageszeit)
 Output REAL (Sekunden und Millisekunden der Tageszeit)

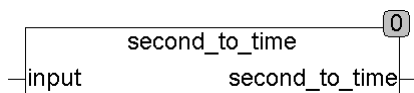


Die Funktion SECOND extrahiert den Sekundenanteil aus der Tageszeit

Beispiel: SECOND(22:10:12.331) = 12.331

11.28 SECOND_TO_TIME

Type Funktion : TIME
 Input INPUT : REAL (Anzahl Sekunden mit Nachkommastellen)
 Output TIME (TIME)

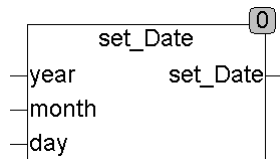


Die Funktion `SECOND_TO_TIME` berechnet einen Zeitwert (TIME) aus dem Eingangswert in Sekunden als REAL.

Beispiel: `SECOND_TO_TIME(63.123) = T#1m3s123ms`

11.29 SET_DATE

Type Funktion : DATE
 Input YEAR : INT (Jahreszahl)
 MONTH : INT (Monat)
 DAY : INT (Tag)
 Output DATE (Zusammengesetztes Datum)



Die Funktion `SET_DATE` berechnet einen Datum (DATE) aus den Eingangswerten Tag, Monat und Jahr. `SET_DATE` überprüft dabei nicht die Gültigkeit eines Datums. Zum Beispiel darf auch der 30. Februar gesetzt werden was natürlich den 1.3. oder bei einem Schaltjahr dem 2. März ergibt. `SET_DATE` kann deshalb auch benutzt werden um einen beliebigen Tag im Jahr zu erzeugen. Dies kann eine durchaus Sinnvolle Anwendung sein. In diesem Fall darf der Monat auch 0 betragen. Eine ungültige Monatsangabe ergibt immer ein Datum in Bezug auf den Januar. Ein ungültiger Monat (Monat < 1 oder Monat > 12) wird immer als Januar interpretiert.

Beispiel: `SET_DATE(2007,1,365) = 31.12.2007`

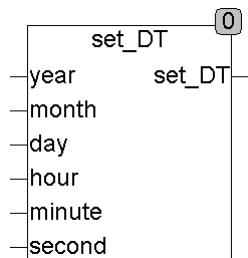
Beispiel: `SET_DATE(2007, 1, 22) = 22.1.2007`

11.30 SET_DT

Type Funktion : DATE_TIME

Input YEAR : INT (Jahreszahl)
 MONTH : INT (Monat)
 DAY : INT (Tag)
 HOUR : INT (Stunde)
 MINUTE : INT (Minute)
 SECOND : INT (Sekunden)

Output DATE_TIME (Zusammengesetztes Zeitdatum)



Die Funktion SET_DT berechnet einen Zeit-Datumswert (DATE_TIME) aus den Eingangswerten Tag, Monat, Jahr, Stunde, Minute und Sekunden.

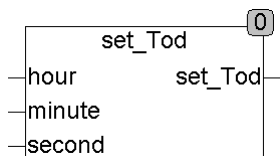
Beispiel: Set_DT(2007, 1, 22, 13, 10, 22) = DT#2007-1-22-13:10:22

11.31 SET_TOD

Type Funktion : TOD

Input HOUR : INT (Stunde)
 MINUTE : INT (Minute)
 SECOND : REAL (Sekunden und Millisekunden)

Output TOD (Ausgangswert Tageszeit)



Die Funktion SET_TOD berechnet eine Tageszeit (TOD) aus den Eingangswerten Stunde, Minute und Sekunden.

Beispiel: Set_TOD(13, 10, 22.33) = 13:10:22.330

11.32 SUN_POS

Type Funktionsblock

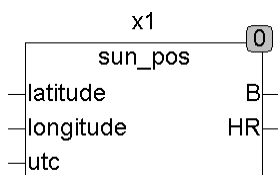
Input LATITUDE : REAL (Breitengrad des Bezugsortes)

LONGITUDE : REAL (Längengrad des Bezugsortes)

UTC : DATE_TIME (Weltzeit)

Output B : REAL (Sonnenazimut in Grad von Nord)

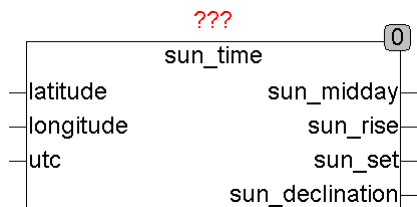
HR : REAL (Sonnenhöhe in Grad über Horizont mit Refraktion)



SUN_POS berechnet die exakte Position der Sonne zur aktuellen Zeit. Die Zeit wird als Weltzeit (UTC) angegeben. Lokalzeit muss vorher umgerechnet werden. Beim Sonnenstand ist die atmosphärische Refraktion für 1010mbar und 10°C bereits berücksichtigt. Die Genauigkeit ist besser als 0,1 Grad für den Zeitraum von 2000 bis 2050. Mögliche Anwendungen von SUN_POS sind die Nachführung von Solarpanels oder eine vom Sonnenstand abhängige Nachführung der Lamellen von Jalousien. SUN_POS ist ein aufwendiger Algorithmus der aber exakte Werte liefert. Um die Belastung einer SPS so gering wie möglich zu halten wird die Berechnung nur alle 10 Sekunden ausgeführt, was einer Ungenauigkeit von 0,04 Grad entspricht.

11.33 SUN_TIME

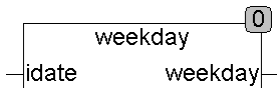
Type	Funktionsblock
Input	LATITUDE : REAL (Breitengrad des Bezugsortes) LONGITUDE : REAL (Längengrad des Bezugsortes) UTC : DATE_TIME (Weltzeit)
Output	SUN_MIDDAY : TOD (Uhrzeit, zu der die Sonne exakt im Süden steht) SUN_RISE : TOD (Zeit des Sonnenaufgangs) SUN_SET : TOD (Zeit des Sonnenuntergangs) SUN_DECLINATION : REAL (Sonnenwinkel, wenn die Sonne genau im Süden Steht)



Der Funktionsblock SUN_TIME ist ein Astrotimer. Er berechnet Sonnenaufgang und Sonnenuntergang für einen beliebigen Tag, definiert durch den Eingang UTC. Außer Sonnen Auf- und Untergang wird auch die Zeit des Sonnenazimut (Tageshöchststand im Süden) und der Sonnenwinkel im Azimut berechnet. Damit SUN_TIME unabhängig vom Einsatzort funktioniert werden alle Zeiten in UTC (Weltzeit) berechnet und können bei Bedarf wieder in Lokalzeit umgerechnet werden. zusätzlich zu den Zeiten für Sonnenaufgang und Sonnenuntergang berechnet der Baustein auch noch den Winkel der Sonneneinstrahlung über dem Horizont SUN_DECLINATION. SUM_TIME benutzt einen aufwendigen Algorithmus, um die Belastung einer SPS so gering wie möglich zu halten merkt sich der Baustein des aktuellen Tag und stellt sicher das die Werte für jeden Tag nur einmal berechnet werden. SUN_TIME wird für die Steuerung von Jalousien benutzt, um sie kurz vor Sonnenaufgang hochzuziehen, damit man im Schlafzimmer die Dämmerung genießen kann. Weitere Anwendungen sind im Gartenbau um Bewässerung anhand des Sonnen auf- und Untergangs zu steuern oder auch zum Nachführen von Solarpanels. Weitere Berechnungen des Sonnenstandes stellt der Baustein SUN_POS zur Verfügung.

11.34 WEEKDAY

Type Funktion : INT
 Input IDATE : DATE (Eingangsdatum)
 Output INT (Monat im Jahr des Eingangsdatums)



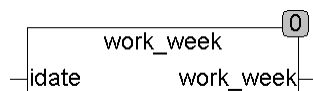
Die Funktion WEEKDAY berechnet den Wochentags aus dem Eingangsdatum IDATE.

Montag = 1... Sonntag = 7. Die Berechnung erfolgt gemäß ISO8601.

Beispiel: WEEKDAY(8.1.2007) = 1

11.35 WORK_WEEK

Type Funktion : INT
 Input IDATE : DATE (Eingangsdatum)
 Output INT (Arbeitswoche des Eingangsdatums)

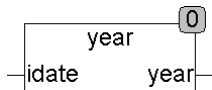


Die Funktion WORK_WEEK berechnet die Kalenderwoche aus dem Eingangsdatum IDATE. Die Kalenderwoche startet mit 1 für die erste Woche des Jahres. Der erste Donnerstag im Jahr liegt immer in der ersten Kalenderwoche. Wenn ein Jahr mit einem Donnerstag anfängt oder mit einem Donnerstag endet hat dieses Jahr 53 Kalenderwochen. Ist der erste Tag eines Jahres ein Dienstag, Mittwoch oder Donnerstag so beginnt die Kalenderwoche 1 bereits im Dezember des Vorjahres. Wenn der erste Tag eines Jahres Freitag, Samstag oder Sonntag ist so erstreckt sich die letzte Kalenderwoche des Vorjahres in den Januar. Die Berechnung erfolgt gemäß ISO8601.

Da die Arbeitswoche (Work Week) international nicht immer einheitlich Verwendung findet ist vor der Anwendung der Funktion zu klären ob die Work Week gemäß ISO8601 der in der Anwendung gewünschten Funktion entspricht.

11.36 YEAR

Type Funktion : INT
Input IDATE : DATE (Eingangsdatum)
Output INT (Jahr des Eingangsdatums)

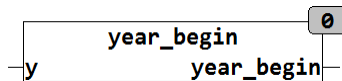


Die Funktion YEAR berechnet das entsprechende Jahr aus dem Eingangsdatum IDATE.

Beispiel: YEAR(31.12.2007) = 2007

11.37 YEAR_BEGIN

Type Funktion : DATE
Input Y : INT (Jahreszahl)
Output DATE (Datum des 1. Januars für die Jahreszahl)

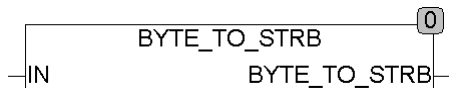


YEAR_BEGIN berechnet das Datum des 1. Januars für das Jahr Y.

12 String Funktionen

12.1 BYTE_TO_STRB

Type Funktion : STRING
 Input IN : BYTE (Eingangswert)
 Output STRING (Ergebnis STRING)

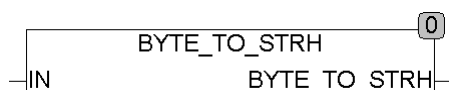


BYTE_TO_STRB konvertiert ein Byte in einen STRING fester Länge. Der Ausgangsstring ist exakt 8 Stellen lang und entspricht der bitweisen Schreibweise des Wertes IN. Der Ausgangsstring besteht aus den Zeichen '0' und '1'. Das niederwertigste Bit ist rechts im STRING. Falls ein STRING mit weniger als 8 Zeichen benötigt wird kann dieser mit der Standardfunktion RIGHT() entsprechend abgeschnitten werden. Der Aufruf RIGHT(BYTE_TO_STRB(X),4) ergibt einen STRING mit 4 Zeichen, die dem Inhalt der untersten 4 Bit von X entspricht.

Beispiel: BYTE_TO_STRB(3) = '00000011'

12.2 BYTE_TO_STRH

Type Funktion : STRING
 Input IN : BYTE (Eingangswert)
 Output STRING (Ergebnis STRING)



BYTE_TO_STRH konvertiert ein Byte in einen STRING fester Länge. Der Ausgangsstring ist exakt 2 Stellen lang und entspricht der Hexadezimalen Schreibweise des

Wertes von IN. Der Ausgangsstring besteht aus den Zeichen '0' .. '1' und 'A' .. 'F'. Das niederwertigste Zeichen steht rechts im STRING.

Beispiel: `BYTE_TO_STRH(15) = '0F'`

12.3 CAPITALIZE

Type Funktion : STRING
 Input STR : STRING (Eingangsstring)
 Output STRING (Ergebnis STRING)

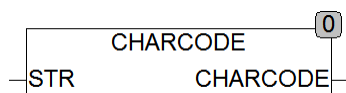


CAPITALIZE setzt alle Anfangsbuchstaben in STR auf Großbuchstaben. Bei der Konvertierung wird die Globale Setup Konstante EXTENDED_ASCII berücksichtigt. Wenn EXTENDED_ASCII = TRUE werden Zeichen des erweiterten ASCII Zeichensatzes nach ISO 8859-1 berücksichtigt.

`CAPITALIZE('hugo wallenstein') = 'Hugo Wallenstein'`

12.4 CHARCODE

Type Funktion : BYTE
 Input STR : STRING(10) (Eingangsstring)
 Output BYTE (Zeichencode)

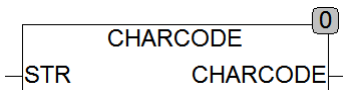


CHARCODE liefert den Byte Code eines Named Characters. Eine Liste der Codes mit Namen befindet sich unter der Funktion CHARNAME. Falls für den Namen in STR keine Named Character bekannt ist wird der Code des ersten Zeichens der Zeichenkette STR zurückgegeben. CHARCODE benutzt die globalen Variablen CHARNAME_1 .. 4 die die Liste der Namen mit Codes enthalten.

Beispiel: `CHARCODE('euro') = 128` und entspricht dem Zeichen €

12.5 CHARNAME

Type Funktion : STRING(10)
 Input STR : STRING (Eingangsstring)
 Output STRING (Ergebnis STRING)



CHARNAME ermittelt den Zeichennamen für einen Zeichencode.

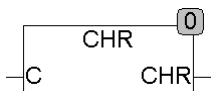
Beispiel: CHARNAME(128) = 'euro'

<i>char code</i>	<i>name</i>	<i>char code</i>	<i>name</i>	<i>char code</i>	<i>name</i>
"	34	quot	½	189	frac12
&	38	amp	¾	190	frac34
<	60	lt	¿	191	iquest
>	62	gt	À	192	Agrave
€	128	euro	Á	193	Aacute
	160	nbsp	Â	194	Acirc
¡	161	iexcl	Ã	195	Atilde
¢	162	cent	Ä	196	Auml
£	163	pound	Å	197	Aring
¤	164	curren	Æ	198	AElig
¥	165	yen	Ç	199	Ccedil
¦	166	brvbar	È	200	Egrave
§	167	sect	É	201	Eacute
¨	168	uml	Ê	202	Ecirc
©	169	copy	Ë	203	Euml
ª	170	ordf	Ì	204	Igrave
«	171	laquo	Í	205	Iacute
¬	172	not	Î	206	Icirc
¯	173	shy	Ï	207	Iuml
®	174	reg	Ð	208	ETH
—	175	macr	Ñ	209	Ntilde
°	176	deg	Ò	210	Ograve
±	177	plusmn	Ó	211	Oacute
²	178	sup2	Ô	212	Ocirc
³	179	sup3	Õ	213	Otilde
´	180	acute	Ö	214	Ouml
µ	181	micro	×	215	times
¶	182	para	Ø	216	Oslash
·	183	middot	Ù	217	Ugrave
¸	184	cedil	Ú	218	Uacute
¹	185	sup1	Û	219	Ucirc
º	186	ordm	Ü	220	Uuml
»	187	raquo	Ý	221	Yacute
			ß	223	szlig
			à	224	agrave
			á	225	aacute
			â	226	acirc
			ã	227	atilde
			ä	228	auml
			å	229	aring
			æ	230	aelig
			ç	231	ccedil
			è	232	egrave
			é	233	eacute
			ê	234	ecirc
			ë	235	euml
			ì	236	igrave
			í	237	iacute
			î	238	icirc
			ï	239	iuml
			ð	240	eth
			ñ	241	ntilde
			ò	242	ograve
			ó	243	oacute
			ô	244	ocirc
			õ	245	otilde
			ö	246	ouml
			÷	247	divide
			ø	248	oslash
			ù	249	ugrave
			ú	250	uacute
			û	251	ucirc
			ü	252	uuml
			ý	253	yacute
			þ	254	thorn
			ÿ	255	yuml

Falls für einen Code kein Name bekannt ist wird der Code als einzelnes Zeichen zurückgegeben. Für den Code 0 wird eine leere Zeichenkette zurückgegeben. CHAR-CODE benutzt die globalen Variablen CHARNAME_1 .. 4 die die Liste der Namen mit Codes enthalten.

12.6 CHR

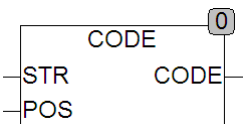
Type Funktion : STRING
 Input C : Byte (Eingangswert)
 Output STRING (Ergebnis STRING)



CHR formt ein ASCII Zeichen aus einem Byte und liefert es als einen ein Zeichen langen STRING.

12.7 CODE

Type Funktion : BYTE
 Input STR : STRING (Zeichenkette)
 POS : INT (Position an der das Zeichen gelesen wird)
 Output BYTE (Code des Zeichens an der Position POS)

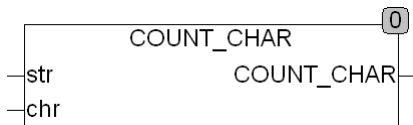


CODE ermittelt den Numerischen Code eines Zeichens an der Stelle POS in STR. wird CODE mit einer Position kleiner 1 oder größer der Länge von STR aufgerufen wird 0 zurückgegeben.

Beispiel: CODE('ABC 123',4) = 32 (Das Zeichen ' ' wird mit dem Wert 32 kodiert.

12.8 COUNT_CHAR

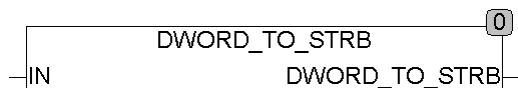
Type	Funktion : STRING
Input	STR : STRING (Zeichenkette) CHR : Byte (Suchzeichen)
Output	STRING (Ergebnis STRING)



COUNT_CHAR ermittelt wie oft das Zeichen CHR in der Zeichenkette STR vorkommt. Um auch nach Sonderzeichen und Steuerzeichen suchen zu können wird das Suchzeichen CHR als BYTE angegeben.

12.9 DWORD_TO_STRB

Type	Funktion : STRING
Input	IN : DWORD (Eingangswert)
Output	STRING (Ergebnis String)

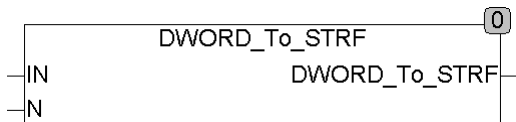


DWORD_TO_STRB konvertiert ein DWORD, Word oder Byte in einen STRING fester Länge. Der Ausgangsstring ist exakt 32 Stellen lang und entspricht der bitweisen Schreibweise des Wertes IN. Der Ausgangsstring besteht aus den Zeichen '0' und '1'. Das niederwertigste Bit steht links im STRING. DWORD_TO_STRB kann als Input Byte, Word und DWORD Typen verarbeiten. Der Ausgang ist aber unabhängig vom Eingangstyp immer ein STRING mit 32 Zeichen. Falls ein kürzerer STRING benötigt wird, kann dieser mit der Standard Funktion RIGHT() entsprechend abgeschnitten werden. Der Aufruf RIGHT(DWORD_TO_STRB(X),8) ergibt einen STRING mit 8 Zeichen die dem Inhalt des untersten Bytes von X entsprechen.

[illegible]

12.10 DWORD_TO_STRF

Type Funktion : STRING
 Input IN : DWORD (Eingangswert)
 N : Int (Länge des Ergebnis Strings)
 Output STRING (Ergebnis String)

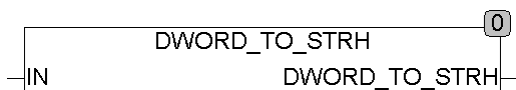


DWORD_TO_STRF konvertiert ein DWORD, Word oder Byte in einen STRING fester Länge. Der Ausgangsstring ist exakt N Stellen lang, wobei führende Nullen eingefügt werden oder führende Stellen abgeschnitten werden. Die maximale erlaubte Länge N ist 20 Digits.

Beispiel: DWORD_TO_STRF(5123, 6) = '005123'
 DWORD_TO_STRF(5123, 3) = '123'

12.11 DWORD_TO_STRH

Type Funktion : STRING
 Input IN : DWORD (Eingangswert)
 Output STRING (Ergebnis String)

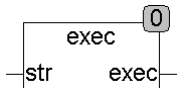


DWORD_TO_STRH konvertiert ein DWORD, Word oder Byte in einen STRING fester Länge. Der Ausgangsstring ist exakt 8 Stellen lang und entspricht der hexadezimalen Schreibweise des Wertes IN. Der Ausgangsstring besteht aus den Zeichen '0' .. '1' und 'A' .. 'F'. Das niederwertigste Hexadezimal-Zeichen steht rechts im STRING. DWORD_TO_STRH kann als Input Byte, Word und DWORD Typen verarbeiten. Der Ausgang ist aber unabhängig vom Eingangstyp immer ein STRING mit 32 Zeichen. Falls ein kürzerer STRING benötigt wird, kann dieser mit der Standardfunktion RIGHT() entsprechend abgeschnitten werden. Der Aufruf RIGHT(DWORD_TO_STRH(X),4) ergibt einen STRING mit 4 Zeichen die dem Inhalt der untersten 2 Bytes von X entsprechen.

Beispiel: DWORD_TO_STRH(127) = '0000007F'

12.12 EXEC

Type Funktion : STRING
 Input STR : STRING (Eingabe STRING)
 Output STRING (Ergebnis STRING)



Die Funktion EXEC arbeitet Mathematische Ausdrücke ab und liefert das Ergebnis als STRING zurück. Der Ausdruck darf nur ein einfacher Ausdruck mit einem Operator und ohne Klammern sein. Bei Fehlern, wie zum Beispiel einem Teilen durch Null liefert EXEC den Rückgabestring 'ERROR'.

Die zulässigen Operatoren sind: +, -, *, /, ^, SIN, COS, TAN, SQRT.

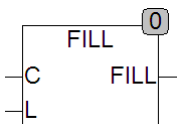
Als Zahlen sind REAL und Integer-Zahlen zulässig.

Beispiel: EXEC('3^2') = '9'

EXEC('4-2') = '2'

12.13 FILL

Type Funktion : STRING
 Input C : BYTE (Character Code)
 L : INT (Länge der Zeichenkette)
 Output STRING (Ergebnis STRING)



FILL erzeugt eine Zeichenkette bestehend aus dem Zeichen C mit der Länge L.

FILL(49,5) = '11111'

Die Funktion FILL wertet auch die Globale Setup Konstante STRING_LENGTH aus und begrenzt die maximale Länge L der Zeichenkette auf STRING_LENGTH.

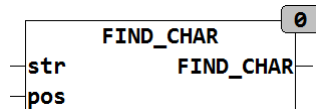
12.14 FIND_CHAR

Type Funktion : INT

Input STR : STRING (Eingabe STRING)

POS : INT (Startposition)

Output INT (Position des ersten Zeichens das kein Steuerzeichen ist)



FIND_CHAR durchsucht die Zeichenkette STR ab der Position POS und gibt die Position zurück an der das erste Zeichen steht das kein Steuerzeichen ist. Steuerzeichen sind alle Zeichen deren Wert kleiner 32 oder 127 ist. Bei der Prüfung wird die Globale Setup Konstante EXTENDED_ASCII berücksichtigt. Wenn EXTENDED_ASCII = TRUE ist werden Zeichen des erweiterten ASCII Zeichensatzes nach ISO 8859-1 berücksichtigt. Umlaute wie Ä,Ö,Ü werden nur dann berücksichtigt wenn die Globale Konstante EXTENDED_ASCII = TRUE ist. Wenn EXTENDED_ASCII = FALSE ist werden Zeichen des erweiterten Zeichensatzes mit einem Wert > 127 als Steuerzeichen interpretiert.

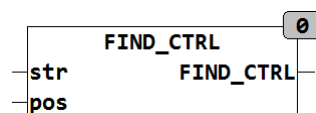
12.15 FIND_CTRL

Type Funktion : INT

Input STR : STRING (Eingabe STRING)

POS : INT (Startposition)

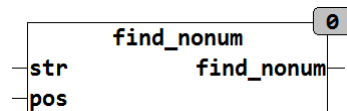
Output INT (Position des ersten Zeichens das ein Steuerzeichen ist)



FIND_CTRL durchsucht die Zeichenkette STR ab der Position POS und gibt die Position zurück an der das nächste Steuerzeichen steht. Steuerzeichen sind alle Zeichen deren Wert kleiner 32 oder 127 ist.

12.16 FIND_NONUM

Type Funktion : INT
 Input STR : STRING (Eingabestring)
 POS : INT (Position an der die Suche beginnt)
 Output INT (Position des ersten Buchstaben, der keine Zahl oder Punkt ist)



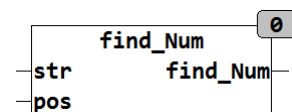
Die Funktion FIND_NONUM durchsucht STR ab der Startposition POS von links nach rechts und liefert die erste Stelle die keine Nummer ist zurück.

Nummern sind die Buchstaben "0..9" und ".".

Beispiel: FIND_NONUM('4+33',1) = 2

12.17 FIND_NUM

Type Funktion : INT
 Input STR : STRING (Eingabestring)
 POS : INT (Position an der die Suche beginnt)
 Output INT (Position des ersten Buchstaben, der eine Zahl oder Punkt ist)



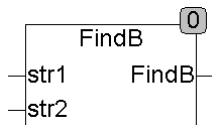
Die Funktion FIND_NUM durchsucht STR ab der Position POS von links nach rechts und liefert die erste Stelle die eine Nummer ist zurück.

Nummern sind die Buchstaben "0..9" und ".".

Beispiel: FIND_NONUM('4+33',1) = 1

12.18 FINDB

Type Funktion : INT
 Input STR1 : STRING (Eingabestring)
 STR2 : STRING (Suchstring)
 Output INT (Position des letzten Vorkommens von STR2 in STR1)



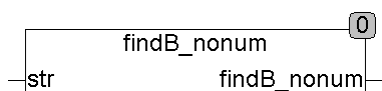
Die Funktion FINDB durchsucht STR1 auf das Vorkommen von STR2 und liefert die letzte Position von STR2 in STR1 zurück.

Falls STR2 nicht gefunden wird, wird eine 0 zurückgegeben.

Beispiel: FINDB('abs12fir12bus12', '12') = 14

12.19 FINDB_NONUM

Type Funktion : INT
 Input STR : STRING (Eingabestring)
 Output INT (Position des letzten Buchstaben, der keine Nummer ist)



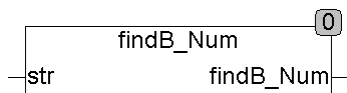
Die Funktion FINDB_NONUM durchsucht STR von rechts nach links und liefert die letzte Stelle die keine Nummer ist zurück.

Nummern sind die Buchstaben "0..9" und ".".

Beispiel: FINDB_NONUM('4+33+1') = 5

12.20 FINDB_NUM

Type Funktion : INT
 Input STR : STRING (Eingabestring)
 Output INT (Position des letzten Buchstaben, der eine Zahl oder Punkt ist)



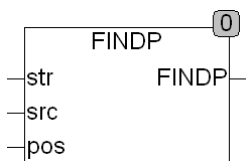
Die Funktion FINDB_NUM durchsucht STR von Rechts nach links und liefert die letzte Stelle die eine Nummer ist.

Nummern sind die Buchstaben "0..9" und ".".

Beispiel: FINDB_NUM('4+33+1hh') = 6

12.21 FINDP

Type Funktion : INT
 Input STR : STRING (Eingabestring)
 SRC : STRING (Suchstring)
 POS : INT (Position ab der gesucht wird)
 Output INT (Position des ersten Buchstaben des gefundenen Strings SRC)

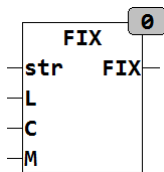


FINDP sucht in einer Zeichenkette STR ab der Position POS nach einer Zeichenkette SRC. Wird die Zeichenkette SRC gefunden so wird die Position des ersten Zeichens von SRC innerhalb von STR ausgegeben. Wird die Zeichenkette ab der Position POS nicht gefunden wird eine 0 ausgegeben. Wird eine Leere Zeichenkette als Suchstring vorgegeben liefert der Baustein das Ergebnis 0.

Beispiel: FINDP('ein Fuchs ist ein Tier','ein',1) = 1;
 FINDP('ein Fuchs ist ein Tier','ein',2) = 15;
 FINDP('ein Fuchs ist ein Tier','ein',16) = 0;

12.22 FIX

Type Funktion : STRING
 Input STR : STRING (Eingabestring)
 L : INT (feste Länge des Ausgabestrings)
 C : BYTE (Füllzeichen beim Auffüllen)
 M : INT (Mode zum Auffüllen)
 Output STRING (Zeichenkette mit fester Länge N)



FIX erzeugt eine Zeichenkette mit fester Länge N. Die Zeichenkette STR am Eingang wird auf die Länge N abgeschnitten beziehungsweise mit dem Füllzeichen C aufgefüllt. Wenn die Zeichenkette STR kürzer ist als die zu erzeugende Länge L wird abhängig von M die Zeichenkette mit dem Füllzeichen C aufgefüllt. Wenn M = 0 werden die Füllzeichen am Ende der Zeichenkette angehängt, ist M = 1 werden die Füllzeichen am Anfang angehängt, und wenn M = 2 wird die Zeichenkette zwischen Füllzeichen zentriert. Falls die Anzahl der nötigen Füllzeichen ungerade ist wird bei M = 2 am Ende ein Füllzeichen mehr als am Anfang angehängt. Die Funktion FIX wertet auch die Globale Setup Konstante STRING_LENGTH aus und begrenzt die maximale Länge L der Zeichenkette auf STRING_LENGTH.

12.23 HEX_TO_DWORD

Type Funktion : DWORD
 Input HEX : STRING (Hexadezimale Zeichenkette)
 Output DWORD (Ausgangswert)



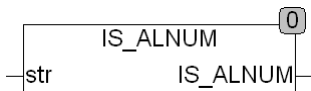
Die Funktion HEX_TO_DWORD konvertiert eine hexadezimale Zeichenkette in einen DWORD Wert. Die Hexadezimalen Zeichen sind '0'..'9' und 'A' .. 'F'. Die Kleinbuchstaben 'a' .. 'f' werden dabei nicht interpretiert. Der Rückgabewert ist nur dann Gültig wenn Die Zeichenkette HEX ausschließlich aus den Zeichen '0' .. '9' und 'A' .. 'F' besteht. Eventuelle Kleinbuchstaben können mit der Funktion UPPERCASE in Groß-

buchstaben gewandelt werden und eventuell enthaltene Leerzeichen können mithilfe der Funktion TRIM entfernt werden.

Beispiel: HEX_TO_DWORD('FF') ergibt 255.

12.24 IS_ALNUM

Type Funktion : BOOL
Input STR : STRING (Eingabestring)
Output BOOL (TRUE wenn STR nur Buchstaben oder Zahlen enthält)



IS_ALNUM testet ob in der Zeichenkette STR nur Buchstaben oder Zahlen enthalten sind. Wird ein falsches, nicht alphanumerisches Zeichen gefunden gibt die Funktion FALSE zurück. Enthält STR nur Buchstaben oder Zahlen ist das Ergebnis TRUE. Buchstaben sind die Zeichen A..Z und a..z, und Zahlen sind die Zeichen 0..9. Bei der Prüfung wird die Globale Setup Konstante EXTENDED_ASCII berücksichtigt. Wenn EXTENDED_ASCII = TRUE ist werden Zeichen des erweiterten ASCII Zeichensatzes nach ISO 8859-1 berücksichtigt. Umlaute wie Ä, Ö, Ü werden nur dann berücksichtigt wenn die Globale Konstante EXTENDED_ASCII = TRUE ist.

12.25 IS_ALPHA

Type Funktion : BOOL
Input STR : STRING (Eingabestring)
Output BOOL (TRUE wenn STR nur Buchstaben enthält)

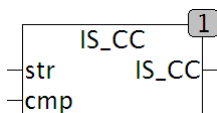


IS_ALPHA testet ob in der Zeichenkette STR nur Buchstaben enthalten sind. Wird ein falsches, nicht alphabetisches Zeichen gefunden gibt die Funktion FALSE zurück. Wenn in STR nur Buchstaben enthalten sind ist das Ergebnis TRUE. Buchstaben sind die Zeichen A..Z und a..z. Bei der Prüfung wird die Globale Setup Konstante EXTENDED_ASCII berücksichtigt. Wenn EXTENDED_ASCII = TRUE ist werden Zei-

chen des erweiterten ASCII Zeichensatzes nach ISO 8859-1 berücksichtigt. Umlaute wie Ä, Ö, Ü werden nur dann berücksichtigt wenn die Globale Konstante EXTENDED_ASCII = TRUE ist.

12.26 IS_CC

Type Funktion : BOOL
 Input STR : STRING (Eingabestring)
 CMP : STRING (Vergleichszeichen)
 Output BOOL (TRUE wenn STR nur die im STRING CMP aufgelisteten Zeichen enthält)



IS_CC testet ob in der Zeichenkette STR nur die in STR aufgelisteten Zeichen enthalten sind. Wird ein anderes Zeichen gefunden gibt die Funktion FALSE zurück.

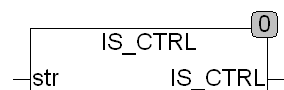
Beispiele:

IS_CC('3.14', '0123456789.') = TRUE

IS_CC('-3.14', '0123456789.') = FALSE

12.27 IS_CTRL

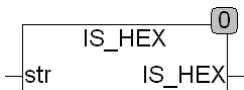
Type Funktion : BOOL
 Input STR : STRING (Eingabestring)
 Output BOOL (TRUE wenn STR nur Kontrollzeichen enthält)



IS_CTRL testet ob in der Zeichenkette STR nur Kontrollzeichen enthalten sind. Wird ein anderes Zeichen gefunden gibt die Funktion FALSE zurück. Sind in STR nur Kontrollzeichen enthalten gibt die Funktion TRUE zurück. Kontrollzeichen sind die Zeichen mit dem Dezimalcode 0..31 und 127.

12.28 IS_HEX

Type Funktion : BOOL
 Input STR : STRING (Eingabestring)
 Output BOOL (TRUE wenn STR nur Hexadezimalzeichen enthält)



IS_HEX testet ob in der Zeichenkette STR nur Hexadezimalzeichen enthalten sind. Wird ein anderes Zeichen gefunden gibt die Funktion FALSE zurück. Sind in STR nur Hexadezimalzeichen enthalten gibt die Funktion TRUE zurück. Hexadezimalzeichen sind die Zeichen mit dem Dezimalcode 0..9, a..f und A..F.

12.29 IS_LOWER

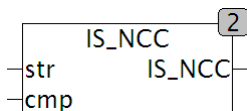
Type Funktion : BOOL
 Input STR : STRING (Eingabestring)
 Output BOOL (TRUE wenn STR nur Kleinbuchstaben enthält)



IS_LOWER testet ob in der Zeichenkette STR nur Kleinbuchstaben enthalten sind. Wird etwas anderes als ein Kleinbuchstabe gefunden gibt die Funktion FALSE zurück. Sind in STR nur Kleinbuchstaben enthalten gibt die Funktion TRUE zurück. Bei der Prüfung wird die Globale Setup Konstante EXTENDED_ASCII berücksichtigt. Wenn EXTENDED_ASCII = TRUE ist werden Zeichen des erweiterten ASCII Zeichensatzes nach ISO 8859-1 berücksichtigt. Umlaute wie Ä, Ö, Ü werden nur dann berücksichtigt wenn die Globale Konstante EXTENDED_ASCII = TRUE ist.

12.30 IS_NCC

Type Funktion : BOOL
 Input STR : STRING (Eingabestring)
 CMP : STRING (Vergleichszeichen)
 Output BOOL (TRUE wenn STR keine der im STRING CMP aufgelisteten
 Zeichen enthält)



IS_NCC testet ob in der Zeichenkette STR keine der in STR aufgelisteten Zeichen enthalten sind. Wird ein Zeichen aus CMP in STR gefunden gibt die Funktion FALSE zurück.

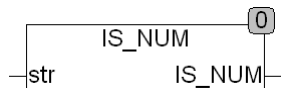
Beispiele:

IS_NCC('3.14', ', - +()') = TRUE

IS_NCC('-3.14', ', - +()') = FALSE

12.31 IS_NUM

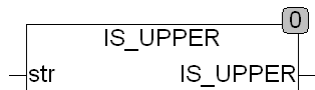
Type Funktion : BOOL
 Input STR : STRING (Eingabestring)
 Output BOOL (TRUE wenn STR keine Großbuchstaben enthält)



IS_NUM testet ob in der Zeichenkette STR nur Zahlen enthalten sind. Wird ein anderes Zeichen gefunden gibt die Funktion FALSE zurück. Sind in STR nur Zahlen enthalten gibt die Funktion TRUE zurück. Zahlen sind die Zeichen 0..9.

12.32 IS_UPPER

Type Funktion : BOOL
 Input STR : STRING (Eingabestring)
 Output BOOL (TRUE wenn STR nur Großbuchstaben enthält)



IS_UPPER testet ob in der Zeichenkette STR nur Großbuchstaben enthalten sind. Wird etwas anders als ein Großbuchstabe gefunden gibt die Funktion FALSE zurück. Sind in STR nur Großbuchstaben enthalten gibt die Funktion TRUE zurück. Bei der Prüfung wird die Globale Setup Konstante EXTENDED_ASCII berücksichtigt. Wenn EXTENDED_ASCII = TRUE ist werden Zeichen des erweiterten ASCII Zeichensatzes nach ISO 8859-1 berücksichtigt. Umlaute wie Ä, Ö, Ü werden nur dann berücksichtigt wenn die Globale Konstante EXTENDED_ASCII = TRUE ist.

12.33 ISC_ALPHA

Type Funktion : BOOL
 Input IN : BYTE (Zeichen)
 Output BOOL (TRUE IN ein Zeichen a..z, A..Z oder Umlaut ist))



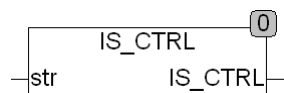
ISC_ALPHA testet ob das Zeichen IN ein alphabetisches Zeichen ist, Ist IN ein Zeichen A..Z, a..z oder ein beliebiger Umlaut gibt die Funktion TRUE zurück, wenn nicht gibt die Funktion FALSE zurück. Bei der Prüfung wird die Globale Setup Konstante EXTENDED_ASCII berücksichtigt. Wenn EXTENDED_ASCII = TRUE ist werden Zeichen des erweiterten ASCII Zeichensatzes nach ISO 8859-1 ausgewertet. Umlaute wie Ä, Ö, Ü werden nur dann berücksichtigt wenn die Globale Konstante EXTENDED_ASCII = TRUE ist.

Die folgende Tabelle Erläutert die Codes:

Code	EXTENDED_ASCII = TRUE	EXTENDED_ASCII = FALSE
0..64	FALSE	FALSE
65..90	TRUE	TRUE
91..96	FALSE	FALSE
97..122	TRUE	TRUE
123..191	FALSE	FALSE
192..214	TRUE	FALSE
215	FALSE	FALSE
216..246	TRUE	FALSE
247	FALSE	FALSE
248..255	TRUE	FALSE

12.34 ISC_CTRL

Type Funktion : BOOL
 Input IN : BYTE (Zeichen)
 Output BOOL (TRUE IN ein Zeichen 0..9 ist)



ISC_CTRL testet ob ein Zeichen IN ein Kontrollzeichen ist, Ist IN ein Kontrollzeichen gibt die Funktion TRUE zurück, wenn nicht gibt die Funktion FALSE zurück. Kontrollzeichen sind alle Zeichen mit dem Code < 32 oder 127.

12.35 ISC_HEX

Type Funktion : BOOL
 Input IN : BYTE (Zeichen)
 Output BOOL (TRUE IN ein Zeichen 0..9 ist)



ISC_HEX testet ob ein Zeichen IN ein Hexadezimaleres Zeichen ist, Ist IN ein Zeichen 0..9, A..F, a..f gibt die Funktion TRUE zurück, wenn nicht gibt die Funktion FALSE zurück.

Die Zeichen 0..9 haben die Codes (48..57)

Die Zeichen A..F haben die Codes (65..70)

Die Zeichen a..f haben die Codes (97..102)

12.36 ISC_LOWER

Type Funktion : BOOL
 Input IN : BYTE (Zeichen)
 Output BOOL (TRUE IN ein Zeichen 0..9 ist)



ISC_LOWER testet ob ein Zeichen IN ein Kleinbuchstabe ist, Ist IN ein Kleinbuchstabe gibt die Funktion TRUE zurück, wenn nicht gibt die Funktion FALSE zurück. Bei der Prüfung wird die Globale Setup Konstante EXTENDED_ASCII berücksichtigt. Wenn EXTENDED_ASCII = TRUE ist werden Zeichen des erweiterten ASCII Zeichensatzes nach ISO 8859-1 ausgewertet.

Die folgende Tabelle Erläutert die Zeichencodes:

Code	EXTENDED_ASCII = TRUE	EXTENDED_ASCII = FALSE
0..96, 123..223, 247, 255	FALSE	FALSE
97..122	TRUE	TRUE
224..246	TRUE	FALSE
248..254	TRUE	FALSE

12.37 ISC_NUM

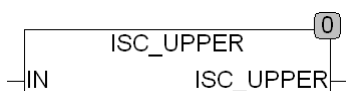
Type Funktion : BOOL
 Input IN : BYTE (Zeichen)
 Output BOOL (TRUE IN ein Zeichen 0..9 ist))



ISC_NUM testet ob ein Zeichen IN ein Numerisches Zeichen ist, Ist IN ein Zeichen 0..9 gibt die Funktion TRUE zurück, wenn nicht gibt die Funktion FALSE zurück. Die Zeichen von 0..9 haben die Zeichencodes (48..57).

12.38 ISC_UPPER

Type Funktion : BOOL
 Input IN : BYTE (Zeichen)
 Output BOOL (TRUE IN ein Zeichen 0..9 ist))



ISC_UPPER testet ob ein Zeichen IN ein Großbuchstabe ist, Ist IN ein Großbuchstabe gibt die Funktion TRUE zurück, wenn nicht gibt die Funktion FALSE zurück. Bei der Prüfung wird die Globale Setup Konstante EXTENDED_ASCII berücksichtigt.

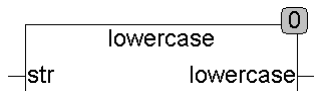
Wenn EXTENDED_ASCII = TRUE ist werden Zeichen des erweiterten ASCII Zeichensatzes nach ISO 8859-1 ausgewertet.

Die folgende Tabelle Erläutert die Zeichencodes:

Code	EXTENDED_ASCII = TRUE	EXTENDED_ASCII = FALSE
0..64, 91..191, 215, 223..255	FALSE	FALSE
65..90	TRUE	TRUE
192..214	TRUE	FALSE
216..222	TRUE	FALSE

12.39 LOWERCASE

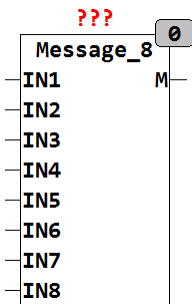
Type Funktion : STRING
 Input STR : STRING (Eingabestring)
 Output STRING (STRING in Kleinbuchstaben)



Die Funktion LOWERCASE wandelt den String STR in Kleinbuchstaben um. Bei der Konvertierung wird die Globale Setup Konstante EXTENDED_ASCII berücksichtigt. Wenn EXTENDED_ASCII = TRUE werden Zeichen des erweiterten ASCII Zeichensatzes nach ISO 8859-1 ausgewertet. Umlaute wie Ä, Ö, Ü werden nur dann berücksichtigt wenn die Globale Konstante EXTENDED_ASCII = TRUE ist. Eine Detaillierte Beschreibung der Codewandlung ist bei der Funktion TO_LOWER zu finden.

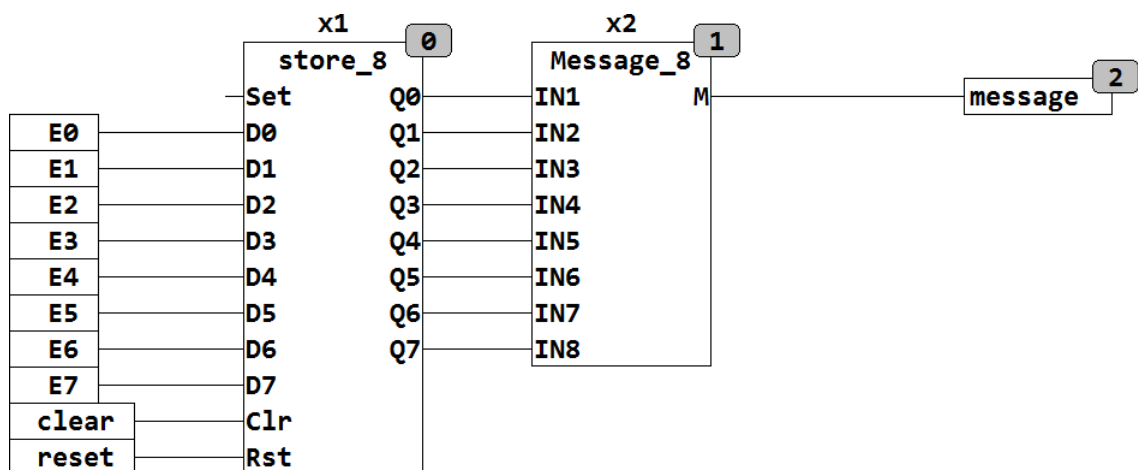
12.40 MESSAGE_8

Type Funktionsblock
 Input IN1..IN8 : BOOL (Auswahleingänge)
 Config S1..S8 : STRING (Nachrichtenvorgabe)
 Output M : STRING (Ausgangsstring)



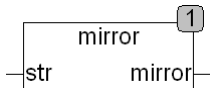
MESSAGE_8 erzeugt eine von 8 Nachrichten am Ausgang M. wenn keiner der Eingänge IN1..IN8 auf TRUE ist wird an M ein Leere Zeichenkette ausgegeben, ansonsten einer der in S1..S8 gespeicherten Nachrichten. Der Baustein gibt immer die Nachricht mit der höchsten Priorität aus. IN1 hat dabei die höchste Priorität und IN8 die niedrigste. MESSAGE_8 kann zusammen mit dem Baustein STORE_8 benutzt werden um Fehlerereignisse zu speichern und Anzuzeigen.

Im folgenden Beispiel werden bis zu 8 Fehlerereignisse (E0..E7) gespeichert und jeweils die am höchsten priorisierte am Ausgang M von MESSAGE_8 angezeigt. Mit dem Eingang CLEAR kann man durch Tasten jeweils die letzte Meldung löschen und zur nächsten anstehenden Fehlermeldung Weiterschalten. MIT dem Eingang RESET kann man alle anstehenden Fehlermeldungen löschen.



12.41 MIRROR

Type Funktion : STRING
 Input STR : STRING (Eingangsstring)
 Output STRING (Eingangsstring Rückwärts gelesen)

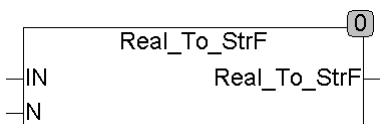


MIRROR liest die Zeichenkette an STR Rückwärts und gibt die Zeichen in umgekehrter Reihenfolge wieder aus.

Beispiel: MIRROR('Das ist ein Test') = 'tseT nie tsi saD'

12.42 REAL_TO_STRF

Type Funktion : STRING(20)
 Input IN : REAL (Eingangswert)
 N : INT (Anzahl der Nachkommastellen)
 Output STRING (Ausgangsstring)

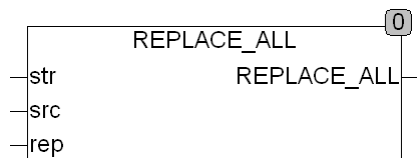


REAL_TO_STRF konvertiert einen REAL-Wert in einen STRING mit einer festen Anzahl von Nachkommastellen N. Bei der Konvertierung wird ausschließlich in Normales Zahlenformat XXX.NNN umgewandelt. Bei der Umwandlung wird IN auf N Stellen nach dem Komma gerundet und dann in einen String mit dem Format XXX.NNN gewandelt. Wenn N = 0 wird die REAL Zahl auf 0 Stellen hinter dem Komma gerundet und das Ergebnis als Integer ohne Punkt und Nachkommastellen ausgegeben. Wenn die Zahl IN kleiner ist als mit N Nachkommastellen erfasst werden können wird eine Null ausgegeben. Die Nachkommastellen werden immer auf N Stellen mit Nullen aufgefüllt. Die maximale Länge der Zeichenkette beträgt 20 Stellen.

Beispiel: REAL_TO_STRF(3.14159,4) = '3.1416'
 REAL_TO_STRF(3.14159,0) = '3'
 REAL_TO_STRF(0.04159,3) = '0.042'
 REAL_TO_STRF(0.001,2) = '0.00'

12.43 REPLACE_ALL

Type Funktion : STRING
 Input STR : STRING (Eingangs String)
 SRC : STRING (Suchstring)
 REP : STRING (Ersatzstring)
 Output STRING (Ausgangsstring)

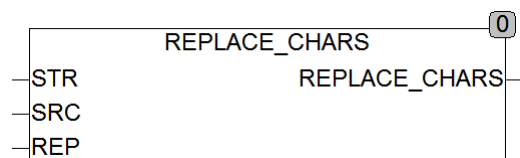


REPLACE_ALL ersetzt alle in der Zeichenkette STR vorkommenden Strings SRC mit REP. Eine leere Zeichenkette an SRC ergibt keine Suchergebnisse.

Beispiel: REPLACE_ALL('123BB456BB789BB','BB','/') = '123/456/789/'
 REPLACE_ALL('123BB456BB789BB','BB','') = '123456789'

12.44 REPLACE_CHARS

Type Funktion : STRING
 Input STR : STRING (Eingangsstring)
 SRC : STRING (Suchzeichen)
 REP : STRING (Ersatzzeichen)
 Output STRING (Ausgangsstring)



REPLACE_CHARS ersetzt alle Zeichen in SRC die in STR vorkommen mit den Zeichen an der gleichen Stelle in REP.

Beispiel: REPLACE_CHARS('abc123', '0123456789', 'ABCDEFGHIJ') = 'abcABC'

12.45 REPLACE_UML

Type Funktion : STRING
 Input STR : STRING (Eingangsstring)
 Output STRING (Ausgangsstring)

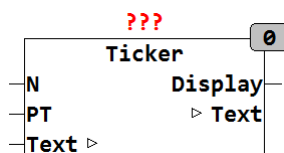


REPLACE_UML ersetzt Umlaute mit einer Kombination aus 2 Zeichen so dass das Ergebnis keine Umlaute mehr enthält. Die Groß und Kleinschreibung wird dabei beachtet. Wenn ein Wort nur aus Großbuchstaben besteht und darin ein Umlaut enthalten ist wird dieser mit einem Großbuchstaben gefolgt von einem Kleinbuchstaben ersetzt, im Falle eines ß welches keine Großschreibung kennt wird es immer mit zwei Kleinbuchstaben ersetzt. Wird die Funktion REPLACE_UML auf ein Wort das nur aus Großbuchstaben besteht angewendet muss anschließend mit der Funktion UPPERCASE() sichergestellt werden dass die Kleinbuchstaben wieder in Großbuchstaben gewandelt werden.

Ä > Ae, Ö > Oe, Ü > Ue, ä > ae, ö > oe, ü > oe, ß > ss.

12.46 TICKER

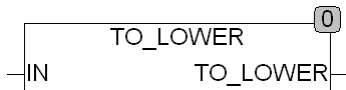
Type Funktionsblock
 Input N : INT (Länge des Display Strings)
 PT : TIME (Schiebedelay, Default = T#1s)
 TEXT : STRING (Eingangsstring)
 Output DISPLAY : STRING (Ausgangsstring)



TICKER erzeugt am Ausgang DISPLAY eine Laufschrift. Am Ausgang DISPLAY wird ein Teilstring von TEXT mit der Länge N ausgegeben. Display wird in einem Zeitraster von PT ausgegeben und beginnt bei jeder Ausgabe eine Stelle weiter von Links des Eingangsstrings TEXT. Die Laufschrift wird nur dann erzeugt wenn $N <$ als die Länge von TEXT ist. Wird $N \geq$ Länge von TEXT dann wird der String TEXT direkt am Ausgang DISPLAY dargestellt.

12.47 TO_LOWER

Type Funktion : BYTE
 Input IN : BYTE (Zeichen das konvertiert werden soll)
 Output BYTE (konvertiertes Zeichen)



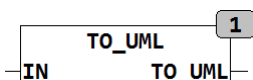
TO_LOWER wandelt einzelne Zeichen in Kleinbuchstaben um. Bei der Konvertierung wird die Globale Setup Konstante EXTENDED_ASCII berücksichtigt. Wenn EXTENDED_ASCII = TRUE werden Zeichen des erweiterten ASCII Zeichensatzes nach ISO 8859-1 berücksichtigt.

Die folgende Tabelle Erläutert die Codewandlung:

Code	EXTENDED_ASCII = TRUE	EXTENDED_ASCII = FALSE
0..64	0..64	0..64
65..90	97..122	97..122
91..191	91..191	91..191
192..214	224..246	192..214
215	215	215
216..222	248..254	216..254
223..255	223..255	223..255

12.48 TO_UML

Type Funktion : STRING(2)
 Input IN : BYTE (Zeichen das konvertiert werden soll)
 Output STRING(2) (konvertiertes Zeichen)



TO_UML wandelt einzelne Sonderzeichen des Zeichensatzes größer 127 in eine Kombination zweier Buchstaben um. Es handelt sich dabei um Zeichen des erweiterten ASCII Zeichensatzes nach ISO 8859-1(Latin1).

Es werden folgende Zeichen umgewandelt:

Ä >> Ae ä >> ae Ö >> Oe ö >> oe Ü >> Ue ü >> ue

ß >> ss

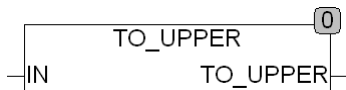
Alle andern Zeichen werden als STRING mit dem Zeichen IN zurückgegeben.

12.49 TO_UPPER

Type Funktion : BYTE

Input IN : BYTE (Zeichen das konvertiert werden soll)

Output BYTE (konvertiertes Zeichen)



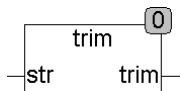
TO_UPPER wandelt einzelne Zeichen in Großbuchstaben um. Bei der Konvertierung wird die Globale Setup Konstante EXTENDED_ASCII berücksichtigt. Wenn EXTENDED_ASCII = TRUE werden Zeichen des erweiterten ASCII Zeichensatzes nach ISO 8859-1 berücksichtigt.

Die folgende Tabelle Erläutert die Codewandlung:

Code	EXTENDED_ASCII = TRUE	EXTENDED_ASCII = FASLE
0..64	0..64	0..64
65..90	97..122	97..122
91..191	91..191	91..191
192..214	224..246	192..214
215	215	215
216..222	248..254	216..254
223..255	223..255	223..255

12.50 TRIM

Type Funktion : STRING
 Input STR : STRING (Eingabestring)
 Output STRING (STR ohne Leerzeichen)

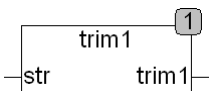


Die Funktion TRIM entfernt alle Leerzeichen aus STR.

Beispiel: TRIM('find BX12') = 'findBX12'

12.51 TRIM1

Type Funktion : STRING
 Input STR : STRING (Eingabestring)
 Output STRING (STR ohne doppelte Leerzeichen)

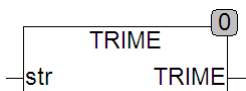


Die Funktion TRIM1 ersetzt mehrfache Leerzeichen mit nur einem Leerzeichen. Leerzeichen am Anfang und Am Ende von STR werden dabei komplett gelöscht.

Beispiel: TRIM1(' find BX12 ') = 'find BX12'

12.52 TRIME

Type Funktion : STRING
 Input STR : STRING (Eingabestring)
 Output STRING (Ausgabestring)



Die Funktion TRIME entfernt Leerzeichen am Anfang und Am Ende von STR. Leerzeichen innerhalb des Strings werden ignoriert, auch wenn Sie mehrfach vorkommen.

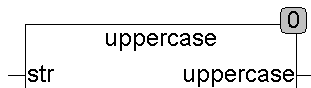
Beispiel: `TRIM1(' find BX12 ') = 'find BX12'`

12.53 UPPERCASE

Type Funktion : STRING

Input STR : STRING (Eingabestring)

Output STRING (STRING in Großbuchstaben)



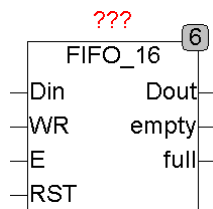
Die Funktion UPPERCASE wandelt alle Buchstaben in STR in Großbuchstaben um. Bei der Konvertierung wird die Globale Setup Konstante EXTENDED_ASCII berücksichtigt. Wenn EXTENDED_ASCII = TRUE werden Zeichen des erweiterten ASCII Zeichensatzes nach ISO 8859-1 berücksichtigt. Umlaute wie Ä,Ö,Ü werden nur dann berücksichtigt wenn die Globale Konstante EXTENDED_ASCII = TRUE ist. Eine Detaillierte Beschreibung der Codewandlung ist bei der Funktion TO_UPPER zu finden.

Beispiel: `UPPERCASE('find BX12') = 'FIND BX12'`.

13 Speicherbausteine

13.1 FIFO_16

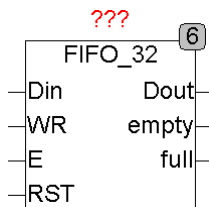
Type	Funktionsblock
Input	DIN : DWORD (Daten Eingang) WR : BOOL (TRUE = Schreiben und FALSE = Lesen) E : BOOL (Freigabe Eingang) RST : BOOL (Reset Eingang)
Output	DOUT : DWORD (Daten Ausgang) EMPTY : BOOL (EMPTY = TRUE bedeutet: Speicher ist Leer) FULL : BOOL (FULL = TRUE bedeutet: Speicher ist Voll)



FIFO_16 ist ein First-In First-Out Speicher mit 16 Speicherstellen für DWORD-Daten. Die beiden Ausgänge EMPTY und FULL zeigen an, wann der Speicher voll oder leer ist. Der Eingang RST löscht den gesamten Inhalt des Speichers. Der FIFO wird beschrieben, indem ein TRUE auf den Eingang WR, die Daten auf den Eingang DIN und ein TRUE-Puls auf den Eingang E gegeben werden.

13.2 FIFO_32

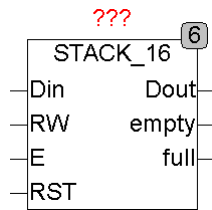
Type	Funktionsblock
Input	DIN : DWORD (Daten Eingang) WR : BOOL (TRUE = Schreiben und FALSE = Lesen) E : BOOL (Freigabe Eingang) RST : BOOL (Reset Eingang)
Output	DOUT : DWORD (Daten Ausgang) EMPTY : BOOL (EMPTY = TRUE bedeutet: Speicher ist Leer) FULL : BOOL (FULL = TRUE bedeutet: Speicher ist Voll)



FIFO_32 ist ein First-In First-Out Speicher mit 32 Speicherstellen für DWORD-Daten. Die beiden Ausgänge EMPTY und FULL zeigen an, wann der Speicher voll oder leer ist. Der Eingang RST löscht den gesamten Inhalt des Speichers. Der FIFO wird beschrieben, indem ein TRUE auf den Eingang WR, die Daten auf den Eingang DIN und ein TRUE Puls auf den Eingang E gegeben werden.

13.3 STACK_16

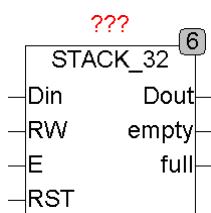
Type	Funktionsblock
Input	DIN : DWORD (Daten Eingang) WR : BOOL (TRUE = Schreiben und FALSE = Lesen) E : BOOL (Freigabe Eingang) RST : BOOL (Reset Eingang)
Output	DOUT : DWORD (Daten Ausgang) EMPTY : BOOL (EMPTY = TRUE bedeutet: Speicher ist Leer) FULL : BOOL (FULL = TRUE bedeutet: Speicher ist Voll)



STACK_16 ist ein Last-In First-Out Speicher mit 16 Speicherstellen für DWORD-Daten. Die beiden Ausgänge EMPTY und FULL zeigen an, wann der Speicher voll oder leer ist. Der Eingang RST löscht den gesamten Inhalt des Speichers. Der Stack wird beschrieben, indem ein TRUE auf den Eingang WR, die Daten auf den Eingang DIN und ein TRUE Puls auf den Eingang E gegeben werden.

13.4 STACK_32

Type	Funktionsblock
Input	DIN : DWORD (Daten Eingang)
	RW : BOOL (TRUE = Schreiben und FALSE = Lesen)
	E : BOOL (Freigabe Eingang)
	RST : BOOL (Reset Eingang)
Output	DOUT : DWORD (Daten Ausgang)
	EMPTY : BOOL (EMPTY = TRUE bedeutet: Speicher ist Leer)
	FULL : BOOL (FULL = TRUE bedeutet: Speicher ist Voll)

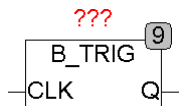


STACK_32 ist ein Last-In First-Out Speicher mit 16 Speicherstellen für DWORD-Daten. Die beiden Ausgänge EMPTY und FULL zeigen an, wann der Speicher voll oder leer ist. Der Eingang RST löscht den gesamten Inhalt des Speichers. Der Stack wird beschrieben, indem ein TRUE auf den Eingang WR, die Daten auf den Eingang DIN und ein TRUE Puls auf den Eingang E gegeben werden.

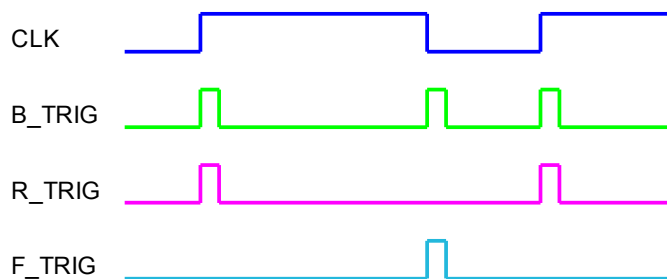
14 Takt Generatoren

14.1 B_TRIG

Type	Funktionsblock
Input	CLK : BOOL (Eingangssignal)
Output	Q : BOOL (Ausgangssignal)



Der Funktionsbaustein B_TRIG erzeugt nach einem Flankenwechsel am Eingang CLK einen Ausgangsimpuls für exakt einen SPS-Zyklus. Im Gegensatz zu den beiden Standardfunktionsbausteinen R_TRIG und F_TRIG, die jeweils nur bei fallender oder steigender Flanke einen Puls erzeugen, erzeugt B_TRIG bei fallender und steigender Flanke einen Ausgangspuls.



14.2 CLK_1MS

Der Baustein CLK_1ms wurde ab der Release 26 mit dem Baustein CLK_N ersetzt.

14.3 CLK_2MS

Der Baustein CLK_2ms wurde ab der Release 26 mit dem Baustein CLK_N ersetzt.

14.4 CLK_4MS

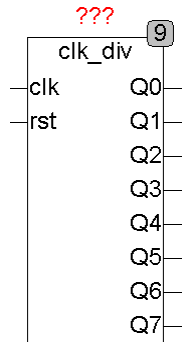
Der Baustein CLK_4ms wurde ab der Release 26 mit dem Baustein CLK_N ersetzt.

14.5 CLK_8MS

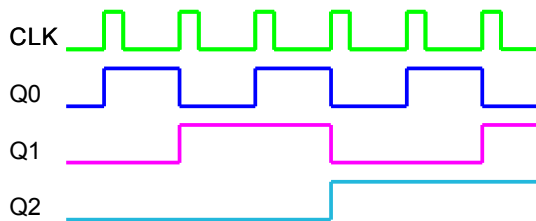
Der Baustein CLK_8ms wurde ab der Release 26 mit dem Baustein CLK_N ersetzt.

14.6 CLK_DIV

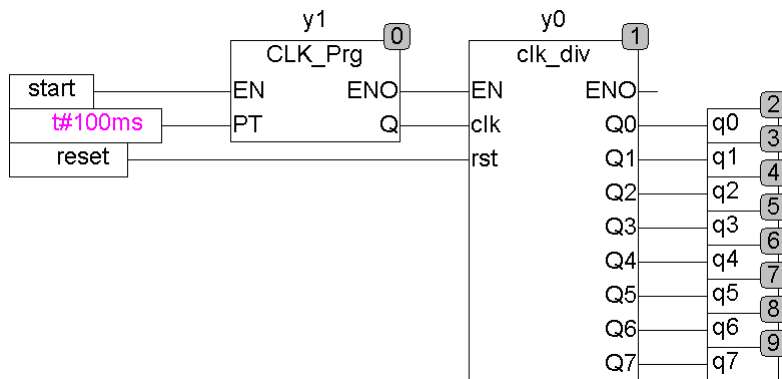
Type	Funktionsblock
Input	CLK : BOOL (Clock Eingang)
	RST : BOOL (Reset Eingang)
Output	Q0 : BOOL (Teiler Ausgang CLK / 2)
	Q1 : BOOL (Teiler Ausgang CLK / 4)
	Q2 : BOOL (Teiler Ausgang CLK / 8)
	Q3 : BOOL (Teiler Ausgang CLK / 16)
	Q4 : BOOL (Teiler Ausgang CLK / 32)
	Q5 : BOOL (Teiler Ausgang CLK / 64)
	Q6 : BOOL (Teiler Ausgang CLK / 128)
	Q7 : BOOL (Teiler Ausgang CLK / 256)

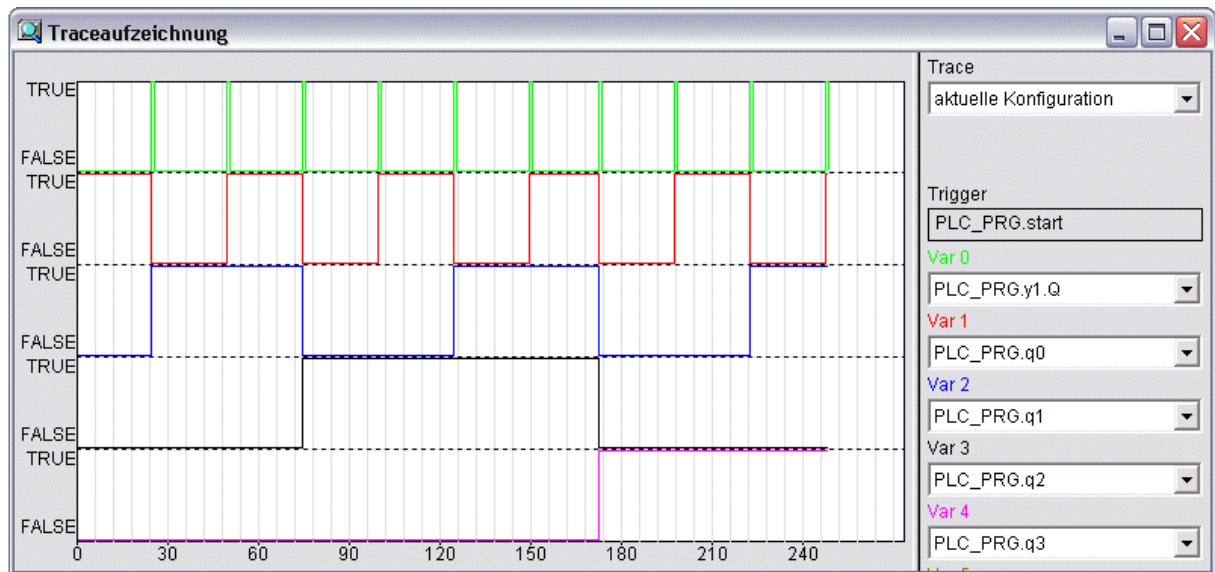


Der Funktionsbaustein CLK_DIV ist ein Teilerbaustein, der ein Eingangssignal CLK in 8 Stufen durch jeweils 2 teilt, sodass am Ausgang Q0 die halbe Frequenz des Eingangs CLK mit 50% Tastverhältnis zur Verfügung steht. Der Ausgang Q1 stellt die halbierte Frequenz von Q0 zur Verfügung und so weiter, bis an Q7 die Eingangsfrequenz geteilt durch 256 bereitsteht. Ein Reset Eingang RST setzt asynchron alle Ausgänge auf FALSE. CLK darf jeweils nur einen Zyklus auf TRUE sein, falls CLK dies nicht tut muss CLK über ein TP_R bereitgestellt werden.



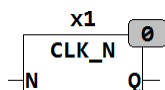
Das folgende Beispiel ist eine Testschaltung mit Startsignal über ENI / ENO Funktionalität Realisiert. Bild 2 zeigt eine entsprechende Traceaufzeichnung der Schaltung:





14.7 CLK_N

Type Funktionsblock
 Input N : INT (Clock Teiler)
 Output Q : BOOL (Taktausgang)

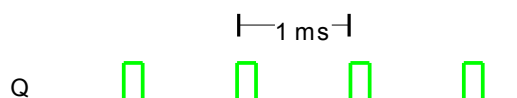


CLK_N erzeugt einen Impuls alle X Millisekunden basierend auf der SPS internen 1ms Referenz. Die Impulse sind exakt einen SPS Zyklus lang und werden alle 2^N Millisekunden erzeugt.

Die Periodendauer beträgt 1ms für N=0, 2ms für N=1, 4ms für N=2 usw.

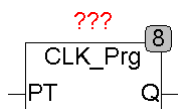
CLK_N ersetzt die Bausteine CLK_1ms, CLK_2ms, CLK_4ms und CLK_8ms aus älteren Bibliotheken.

Das nachfolgende Bild zeigt das Ausgangssignal für N=0:

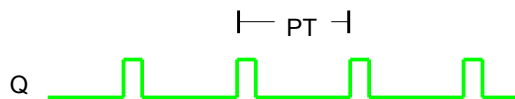


14.8 CLK_PRG

Type Funktionsblock
 Input PT : TIME (Zykluszeit)
 Output Q : BOOL (Taktausgang)

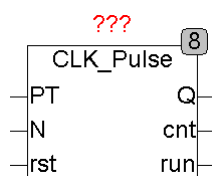


CLK_PRG erzeugt Taktpulse mit einer programmierbaren Taktperiode PT. Die Ausgangsimpulse sind jeweils nur einen SPS Zyklus.



14.9 CLK_PULSE

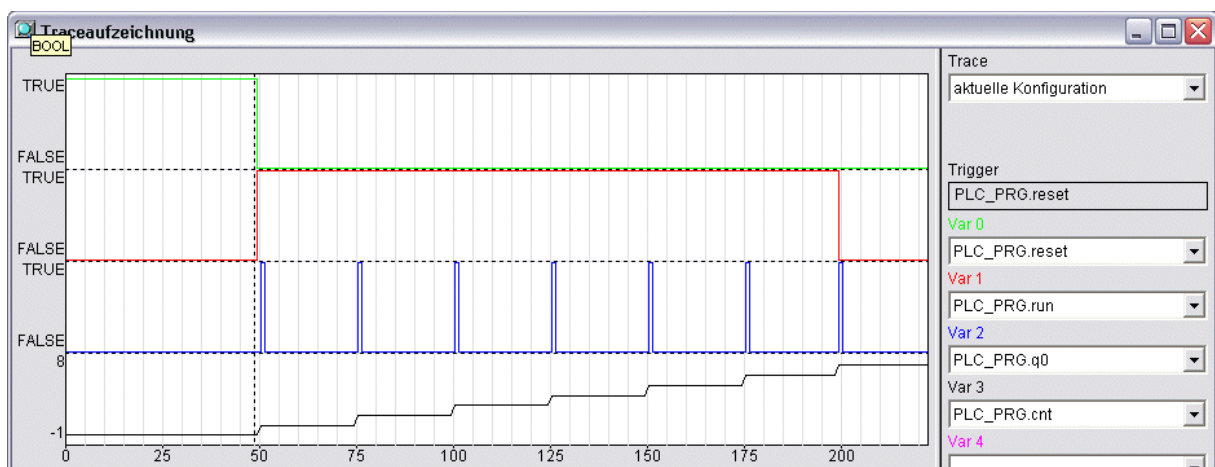
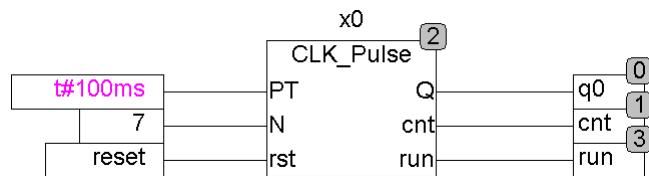
Type Funktionsblock
 Input PT : TIME (Zykluszeit)
 N : INT (Anzahl der zu erzeugenden Impulse)
 RST : BOOL (Reset)
 Output Q : BOOL (Taktausgang)
 CNT : INT (Zähler der Ausgangsimpulse)
 RUN : BOOL (TRUE, wenn Pulsgenerator läuft)



CLK_PULSE erzeugt eine definierte Anzahl von Taktpulsen mit einem programmierbaren Tastverhältnis. PT legt das Tastverhältnis fest und N die Anzahl der zu erzeugenden Impulse. Durch einen Reset Eingang RST kann der Generator jederzeit erneut gestartet werden. Der Ausgang CNT zählt die erzeugten Impulse und RUN = TRUE zeigt an, dass der Generator noch Impulse generiert. Ein Eingangswert N = 0

erzeugt eine unendliche Impulsfolge, Die Maximale Anzahl der Impulse ist auf 32767 begrenzt.

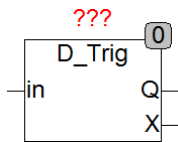
Das folgende Beispiel zeigt eine Anwendung von CLK_PULSE zur Erzeugung von 7 Impulsen mit einem Tastverhältnis von 100ms.



Die Traceaufzeichnung zeigt, wie der RESET (Grün) inaktiv wird und dadurch RUN (Rot) aktiv wird. Der Generator erzeugt dann 7 Impulse (Blau), wie am Eingang N spezifiziert. Der Ausgang CNT zählt dabei von 1 beim ersten Puls nach 7 beim letzten Puls. Nach Ablauf der Sequenz wird RUN wieder inaktiv und der Zyklus ist beendet, bis er durch einem neuen Reset wieder gestartet wird.

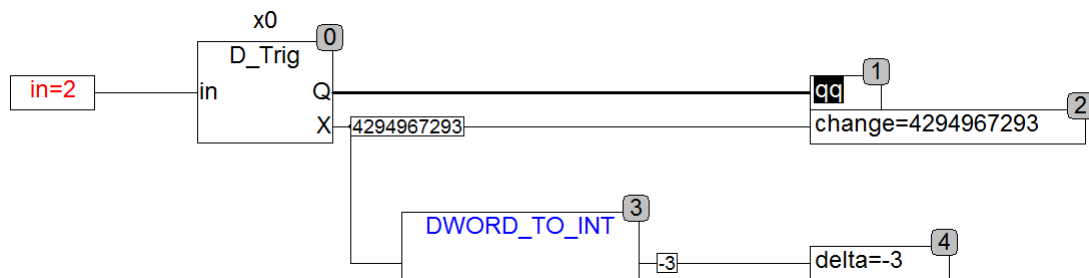
14.10 D_TRIG

Type	Funktionsblock
Input	IN : DWORD (Eingangssignal)
Output	Q : BOOL (Ausgangssignal)
	X : DWORD (Veränderung des Eingangssignals)



Der Funktionsbaustein D_TRIG erzeugt nach einer Veränderung am Eingang IN einen Ausgangsimpuls für exakt einen SPS Zyklus. Der Baustein funktioniert vergleichbar mit den Standardfunktionsbausteinen R_TRIG und F_TRIG und dem in der OSCAT Bibliothek enthaltenen Baustein B_TRIG. Während B_TRIG, R_TRIG und F_TRIG einen Booleschen Eingang überwachen, triggert der Baustein D_TRIG auf jede Veränderung des DWORD-Eingangs IN. Wenn sich der Eingangswert verändert hat, so wird der Ausgang Q für einen SPS Zyklus auf TRUE gesetzt und der Ausgang X gibt an, um welchen Wert sich der Eingang IN verändert hat. Der Eingang, sowie der Ausgang sind von Typ DWORD. Der Eingang kann auch WORD und BYTE Typen verarbeiten. Beim Ausgang X ist zu beachten, dass DWORD kein Vorzeichen hat und deshalb eine Veränderung um -1 am Eingang nicht -1 sondern die Zahl $2^{32}-2$ ausgibt. Mit der Standardfunktion DWORD_TO_INT kann der Ausgang X in einen Integer umgewandelt werden, welcher dann auch negative Veränderungen richtig darstellt.

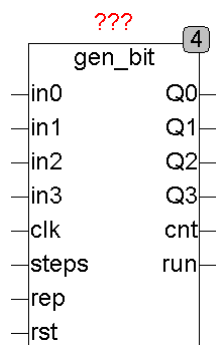
Das nachfolgende Beispiel zeigt die Anwendung von D_TRIG wenn sich der Eingang vom Wert 5 nach 2 verändert:



14.11 GEN_BIT

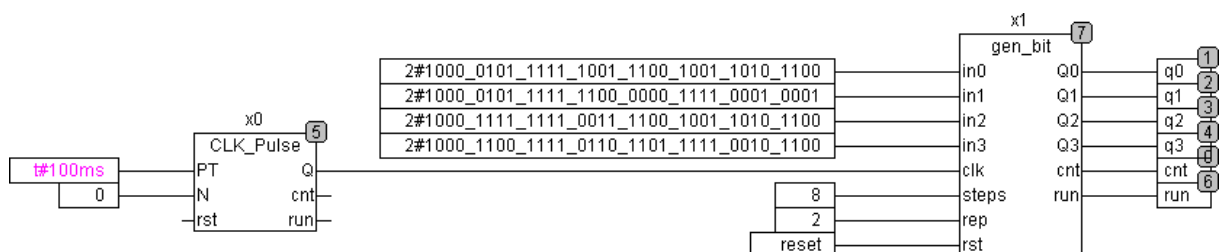
Type	Funktionsblock
Input	IN0 : DWORD (Bitsequenz für Q0)
	IN1 : DWORD (Bitsequenz für Q1)
	IN2 : DWORD (Bitsequenz für Q1)
	IN3 : DWORD (Bitsequenz für Q1)
	CLK : BOOL (Clock Eingang)
	STEPS : INT (Anzahl der zu erzeugenden Takte)

REP : INT
 RST : BOOL
 Output Q0 : BOOL (Bitsequenz Q0)
 Q1 : BOOL (Bitsequenz Q1)
 Q2 : BOOL (Bitsequenz Q2)
 Q3 : BOOL (Bitsequenz Q3)
 CNT : INT (Anzahl der bereits erzeugten Ausgangsbits)
 RUN : BOOL (TRUE, wenn der Sequenzer läuft)



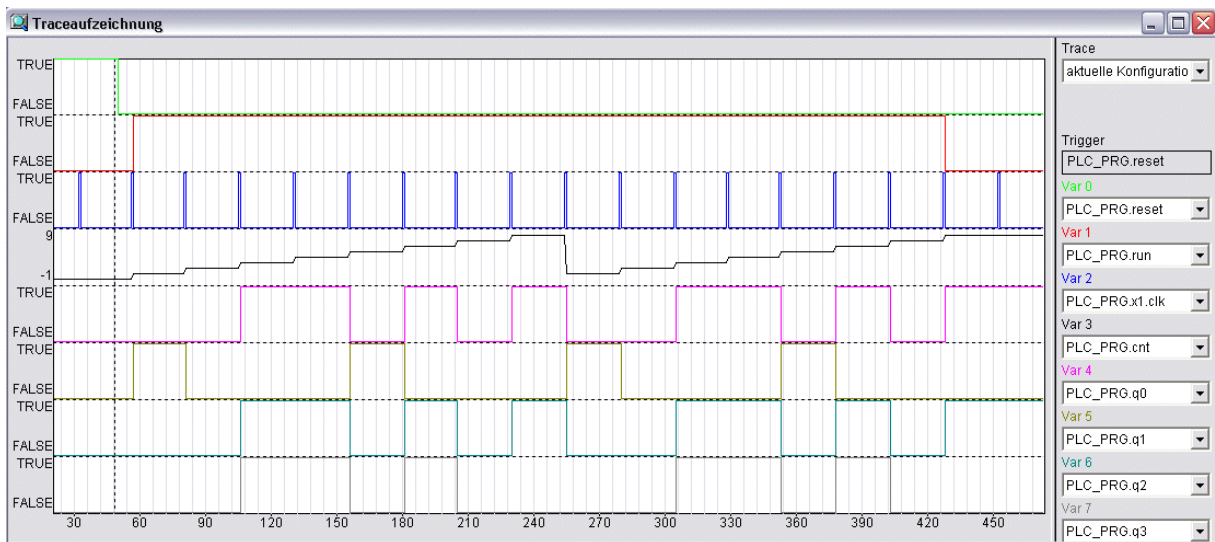
GEN_BIT ist ein frei programmierbarer Bitmustergenerator. An den Eingängen in0 .. in7 liegen die Bitmuster jeweils als DWORD an und werden durch den Eingang CLK je Taktimpuls beginnend von Bit 0 an aufsteigend an die Ausgänge Q0 .. Q3 geschoben. Nach dem ersten Taktimpuls am Eingang CLK liegt am Ausgang Q0 Bit 0 von IN0, an Q1 liegt Bit 0 von IN1 ... an Q7 liegt Bit 0 von IN3. Nach dem nächsten Taktimpuls am CLK Eingang wird jeweils das Bit 1 der Eingänge IN an die Ausgänge Q geschoben und so weiter, bis die Sequenz beendet ist. Der Eingang STEPS legt fest, wie viele Bits der Eingangs-DWORDS an die Ausgänge geschoben werden. Der Eingang REP legt fest, wie oft diese Sequenz wiederholt wird. Wird der Eingang auf 0 gesetzt, so wird die Sequenz fortlaufend wiederholt. Ein asynchroner Reset kann jederzeit den Sequenzer zurücksetzen. Die Ausgänge RUN und CNT zeigen an, welches Bit gerade am Ausgang anliegt und ob der Sequenzer läuft, oder die Sequenz (RUN inaktiv) beendet ist. Nachdem die Sequenzen abgelaufen sind bleibt das letzte Bitmuster an den Ausgängen vorhanden, bis ein Reset den Generator neu startet.

Beispiel:



In diesem Beispiel werden die untersten 8 Bits (Bit 0 .. 7) an den Eingängen IN auf die Ausgänge Q geschoben. Die Sequenz beginnt jeweils bei Bit 0 und endet bei Bit

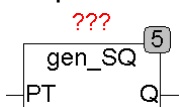
7 (8 Steps sind durch den Eingang 8 definiert). Diese Sequenz wird 2 mal (2 Wiederholungen am Eingang REP) wiederholt und dann gestoppt.



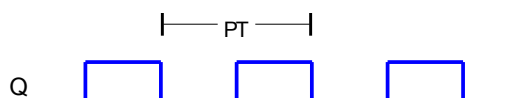
Die Trace Aufzeichnung zeigt das inaktiv werdende Reset Signal (Grün), welches den Generator startet und nach dem ersten Taktimpuls Bit 0 an die Ausgänge schiebt.

14.12 GEN_SQ

Type Funktionsblock
 Input PT : TIME (Periodendauer)
 Output Q : BOOL (Ausgangssignal)

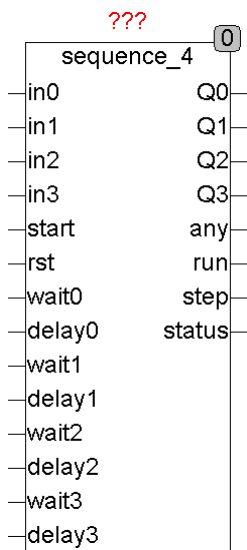


Gen_SQ ist ein Rechteckgenerator mit programmierbarer Periodendauer und einem festen Tastverhältnis von 50%. Der Eingang PT legt die Periodendauer fest und am Ausgang Q steht das Ausgangssignal zur Verfügung.



14.13 SEQUENCE_4

Type	Funktionsblock
Input	IN0..3 : BOOL (Freigabesignal für Q0..3) START : BOOL (Startflanke für den Sequenzer) RST : BOOL (Asynchroner Reset-Eingang) WAIT 0..3 : TIME (Wartezeit für das Eingangssignal an IN 0..3) DELAY 0..3 : TIME (Verzögerungszeit bis das Eingangssignal IN0..3 geprüft wird)
Output	Q 0..3 : BOOL (SteuerAusgänge) ANY : BOOL (ANY ist TRUE, wenn einer der Ausgänge Q0 .. Q3 aktiv ist) RUN : BOOL (RUN ist TRUE, wenn der Sequenzer gestartet wurde und läuft) STEP : INT (gibt den momentanen Schritt an) STATUS : BYTE (zu ESR kompatibler Status-Ausgang)



SEQUENCE_4 ist ein 4 Bit Sequenzer mit Steuereingängen. Nach einer steigenden Flanke an START wird RUN TRUE und der Sequenzer wartet die Zeit Wait0 auf ein TRUE-Signal am Eingang IN0. Nachdem das Signal am IN0 TRUE ist, wird der Ausgang Q0 gesetzt und die Zeit Delay0 abgewartet. Nach Ablauf der Zeit Delay0 wird im nächsten Zyklus für die Zeit wait1 auf ein Eingangssignal an in1 gewartet und Q0 bleibt solange TRUE, bis Q1 gesetzt wird. Das ganze wird solange wiederholt, bis alle 4 Zyklen abgelaufen sind. Falls während den Wartezeiten wait0..3 der entsprechende Eingang nicht TRUE wird, wird ein Fehler gesetzt, indem die entsprechende Error-Nummer am Ausgang STATUS angezeigt wird und abhängig von der Config-

Variable STOP_ON_ERROR wird der Sequenzer angehalten oder nicht. Der STATUS-Ausgang ist 110 für warten auf Startsignal und 111 wenn die Sequenz durchlaufen wird und zeigt mit 1 .. 4 Fehler an. Ein Error = 1 bedeutet, dass das Signal am Eingang in0 nicht aktiv wurde eine 2 entspricht in1 usw. Die Ausgänge RUN und STEP zeigen an, ob der Sequenzer läuft und in welchem Zyklus er sich gerade befindet. Der Ausgang ANY ist TRUE, wenn einer der Ausgänge Q0..Q3 TRUE ist.

Ein asynchroner Reset-Eingang kann den Sequenzer jederzeit zurücksetzen. Dieser Reset-Eingang kann auch mit einem der Ausgänge Q0..Q3 beschaltet werden um den Sequenzer vor dem vollen Ablauf zu stoppen. Der Sequenzer kann auch jederzeit mit einer steigenden Flanke am Eingang START wieder von neuem gestartet werden. Das gilt auch, wenn er eine Sequenz noch nicht beendet hat.

Sollte eine Flankenprüfung an einem oder mehreren Eingängen IN nicht nötig sein, so können sie einfach offen gelassen werden, denn der Vorgabe-Wert für diese Eingänge ist TRUE.

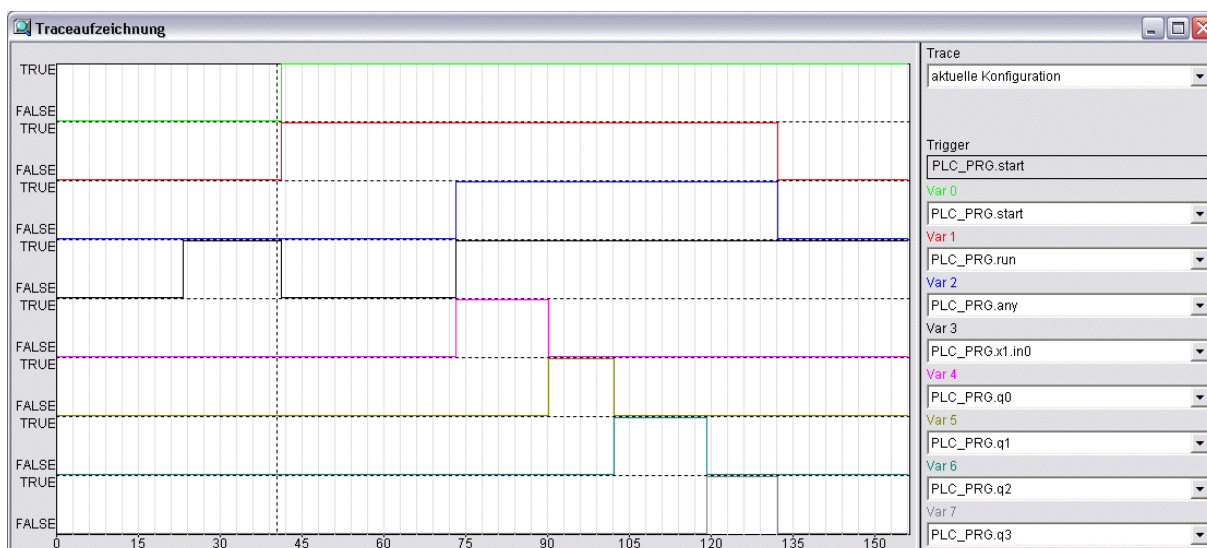
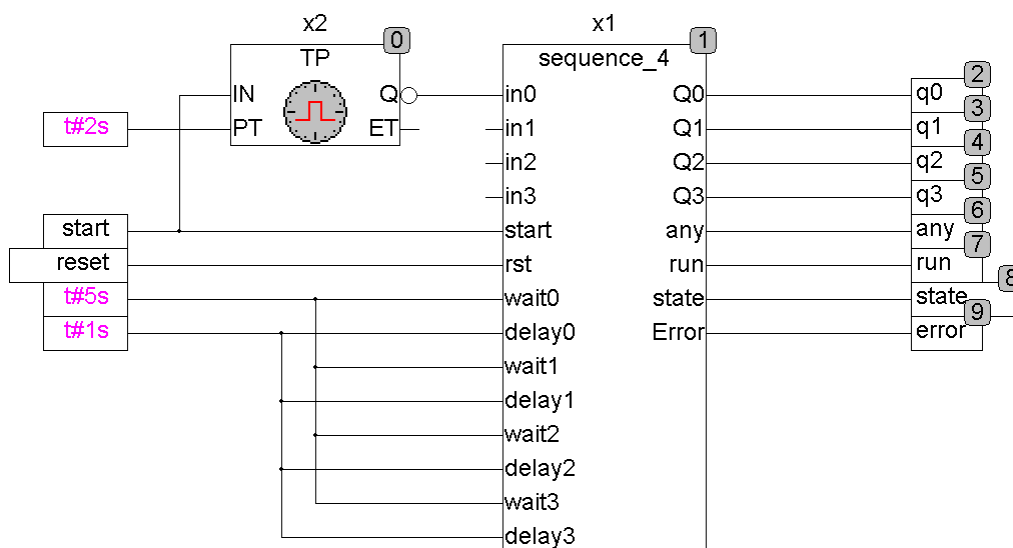
Der Ausgang Status ist ESR kompatibel und zeigt durch einen Wert 1- 4 an, dass ein Fehler aufgetreten ist. Ein Fehler tritt dann auf, wenn das entsprechende Eingangssignal an IN nicht während der entsprechenden Wartezeit auftritt.

Error = 1 bedeutet, dass in0 nicht innerhalb der Wartezeit aktiv geworden ist. Error 2 .. 4 entspricht den Eingängen 1 .. 3.

Ein Statuswert von 110 bedeutet Wartestellung und 111 bedeutet, dass gerade eine Sequenz durchlaufen wird.

Beispiel:

Im Folgenden Beispiel wird mit einer steigenden Flanke an Start der Sequenzer gestartet. Gleichzeitig wird ein Puls-Generator TP mit 2 Sekunden gestartet und damit das Startsignal mit 2 Sekunden Verzögerung auf den Eingang IN0 gelegt. Der Sequenzer setzt unmittelbar nach dem Startimpuls das Ausgangssignal RUN und wartet dann für maximal 5 Sekunden auf ein Signal an IN0. Mit der steigenden Flanke an IN0, dass nach 2 Sekunden von TP generiert wird, wird Q0 gesetzt und ein Delay von 1 Sekunde abgewartet. Damit ist der erste Schritt beendet und die restlichen Schritte werden ohne auf ein Eingangssignal an in1..3 zu warten ausgeführt. Die Vorgabewerte für die Eingänge IN sind TRUE, wenn sie unbeschaltet sind.

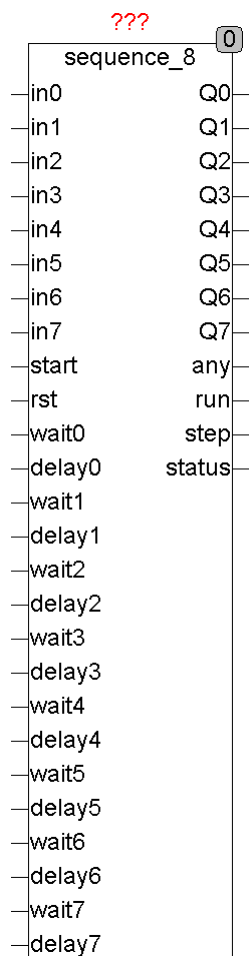


Die Traceaufzeichnung zeigt das Startsignal (Grün) und das Signal RUN (Rot). Nach 2 Sekunden wird die steigende Flanke am Eingang auf in0 und danach auf die Ausgangssignale Q0..3 und ANY gelegt.

Das Signal ANY (Blau) ist dann aktiv, wenn eines der Ausgangssignale aktiv ist und das Signal RUN (Rot) ist vom Start bis zum Ende aktiv.

14.14 SEQUENCE_8

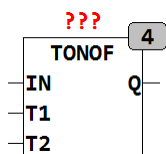
Type	Funktionsblock
Input	IN0..7 : BOOL (Freigabesignal für Q0..7) START : BOOL (Startflanke für den Sequenzer) RST : BOOL (Asynchroner Reset Eingang) WAIT 0..7 : TIME (Wartezeit für das Eingangssignal an IN 0..7) DELAY 0..7 : TIME (Verzögerungszeit, bis das Eingangssignal IN0..7 geprüft wird)
Output	Q 0..7 : BOOL (SteuerAusgänge) ANY : BOOL (Der Ausgang ANY ist TRUE, wenn einer der Ausgänge Q0 .. Q7 aktiv ist) RUN : BOOL (RUN ist TRUE, wenn der Sequenzer gestartet wurde und läuft) STEP : INT (gibt den momentanen Schritt an) STATUS : BYTE (0 wenn kein Fehler vorliegt, sonst > 0)



Eine Funktionsbeschreibung von SEQUENCE_8 ist unter SEQUENCE_4 zu finden. SEQUENCE_8 ist Funktionsidentisch mit SEQUENCE_4. Er hat 8 anstelle von 4 Kanälen. SEQUENCE_8 findet Anwendung in der OSCAT Bibliothek im Modul LE-GIONELLA.

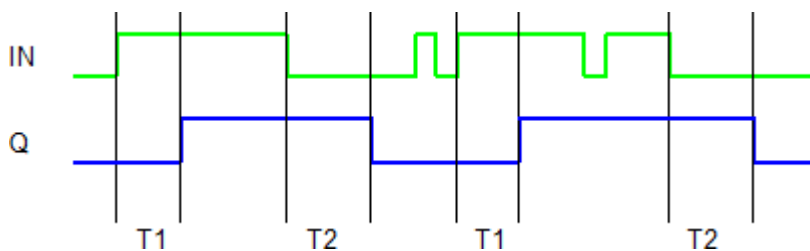
14.15 TONOF

Type Funktionsblock
 Input IN : BOOL (Eingangssignal)
 T1 : TIME (Einschaltverzögerung)
 T2 : TIME (Ausschaltverzögerung)
 Output Q : BOOL (Ausgangspuls)



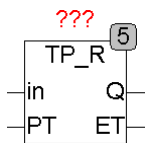
TONOF erzeugt eine Einschaltverzögerung T1 und eine Ausschaltverzögerung T2

Das steigende Flanke des Eingangssignals IN wird um T1 verzögert und die Fallende Flanke von IN wird um T2 verzögert.

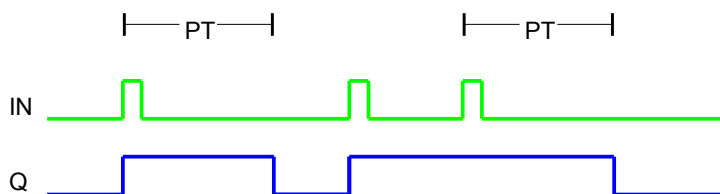


14.16 TP_R

Type	Funktionsblock
Input	IN : BOOL (Eingangssignal) PT : TIME (Impulsdauer)
Output	Q : BOOL (Ausgangspuls) ET : TIME (Zähle die Abgelaufene Zeit des Ausgangspulses)



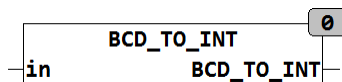
TP_R ist ein mehrfach triggerbarer Impulsgenerator. Im Gegensatz zum Standardbaustein TP kann dieser Baustein mehrfach getriggert werden und dadurch der Ausgangsimpuls verlängert werden. Der Ausgang Q bleibt nach dem letzten Triggerereignis (steigende Flanke an IN) für die Zeit PT auf ein. Während Q TRUE ist, kann jederzeit durch eine weitere Eingangsflanke an IN der Timer wieder getriggert werden und der Ausgangsimpuls dadurch verlängert werden. Im Gegensatz zu TOF wird bei TP_R die Zeit PT ab der letzten steigenden Flanke gemessen, unabhängig wie lange IN auf TRUE bleibt. Das bedeutet das der Ausgang Q nach Ablauf der Zeit PT gemessen von der letzten steigenden Flanke an IN auf FALSE geht, auch wenn der Eingang IN noch TRUE ist.



15 Logik Bausteine

15.1 BCD_TO_INT

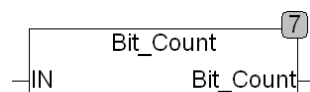
Type Funktion : INT
Input IN : BYTE (BCD codierter Eingang)
Output INT (Ausgangswert)



BCD_TO_INT konvertiert ein BCD codiertes Eingangs BYTE in einen Integer Wert.

15.2 BIT_COUNT

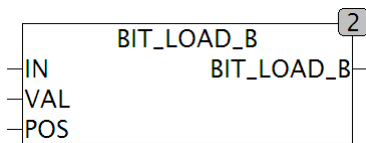
Type Funktion : INT
Input IN : DWORD (Eingang)
Output INT (Anzahl der Bits, welche in IN den Wert TRUE (1) besitzen)



BIT_COUNT ermittelt die Anzahl der Bits in IN, welche den Wert TRUE (1) besitzen. Der Eingang IN ist DWORD und Kann auch die Typen Byte und Word verarbeiten.

15.3 BIT_LOAD_B

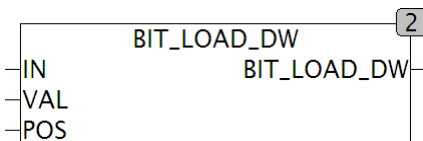
Type Funktion : BYTE
 Input IN : BYTE (Eingang)
 VAL : BOOL (Wert des zu ladenden Bits)
 POS : INT (Position des zu ladenden Bits)
 Output BYTE (Ausgang)



BIT_LOAD_B kopiert das am Eingang VAL anliegende Bit an die Position N im Byte IN. Das niederwertigste Bit B0 wird mit der Position 0 bezeichnet.

15.4 BIT_LOAD_DW

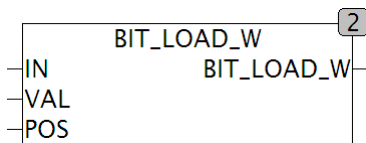
Type Funktion : DWORD
 Input IN : DWORD (Eingang)
 VAL : BOOL (Wert des zu ladenden Bits)
 POS : INT (Position des zu ladenden Bits)
 Output DWORD (Ausgang)



BIT_LOAD_DW kopiert das am Eingang VAL anliegende Bit an die Position N im DWORD IN. Das niederwertigste Bit B0 wird mit der Position 0 bezeichnet.

15.5 BIT_LOAD_W

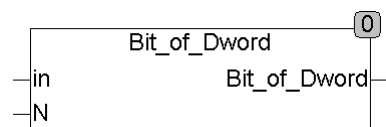
Type Funktion : WORD
 Input IN : WORD (Eingang)
 VAL : BOOL (Wert des zu ladenden Bits)
 POS : INT (Position des zu ladenden Bits)
 Output WORD (Ausgang)



BIT_LOAD_W kopiert das am Eingang VAL anliegende Bit an die Position N im WORD IN. Das niederwertigste Bit B0 wird mit der Position 0 bezeichnet.

15.6 BIT_OF_DWORD

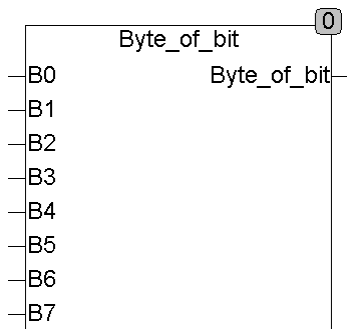
Type Funktion : BOOL
 Input IN : DWORD (Eingang)
 N : Byte (Nummer des Bits 0..31)
 Output BOOL (Ausgangsbit)



BIT_OF_DWORD extrahiert ein Bit aus dem DWORD am Eingang IN.
 Bit0 für N=0, Bit1 für N=1 usw.

15.7 BYTE_OF_BIT

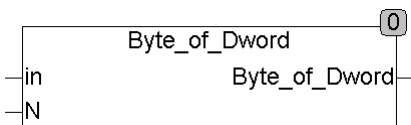
Type Funktion : BYTE
 Input B0 .. B7 : BOOL (Eingangs Bits)
 Output BYTE (Ausgangsbyte)



BYTE_OF_BIT setzt ein Byte aus 8 einzelnen Bits (B0 .. B7) zusammen.

15.8 BYTE_OF_DWORD

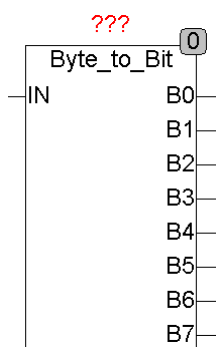
Type Funktion : BYTE
 Input IN : DWORD (Eingangs-DWORD)
 Output BYTE (Ausgangsbyte)



BYTE_OF_DWORD extrahiert ein Byte (B0 .. B3) aus einem DWORD. Die einzelnen Bytes werden mit 0 - 3 am Eingang IN ausgewählt.

15.9 BYTE_TO_BIT

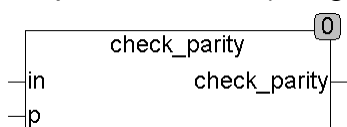
Type Funktionsbaustein
 Input IN : DWORD (Eingangs Byte)
 Output B0 .. B7 : BOOL (Ausgangs Bits)



BYTE_TO_BIT zerlegt ein Byte (IN) in seine einzelnen Bits (B0 .. B7). Der Eingang IN ist als DWORD definiert, um wahlweise Byte, Word oder DWORD am Eingang verarbeiten zu können. Wird ein Word oder DWORD am Eingang verwendet, so werden nur die Bits 0..7 verarbeitet. Ein DWORD kann dann mit dem Standardbefehl SHR um 8 Bits nach Rechts verschoben und so das nächste Byte verarbeitet werden.

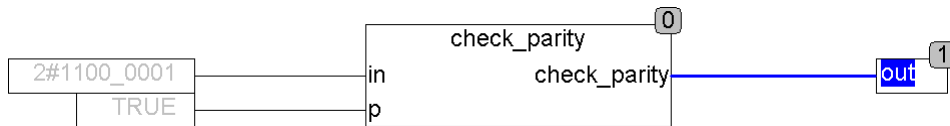
15.10 CHECK_PARITY

Type Funktion : BOOL
 Input IN : BYTE (Eingangsbyte)
 P : BOOL (Parity-Bit)
 Output BYTE (Ausgang ist bei gerader Parität TRUE)

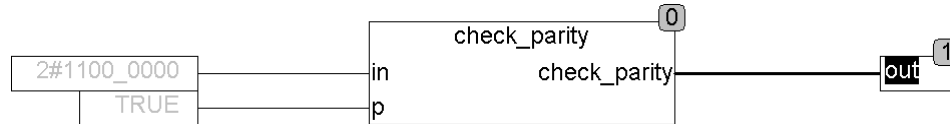


CHECK_PARITY überprüft ein Eingangsbyte IN und ein zugehöriges Parity-Bit P auf gerade Parität. Der Ausgang ist TRUE, wenn die Anzahl der Bits im Byte IN die den Wert TRUE besitzen zusammen mit dem Parity-Bit P eine gerade Anzahl ergeben.

Beispiel für Ausgang = TRUE:

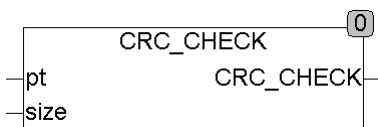


Beispiel für Ausgang = FALSE:



15.11 CRC_CHECK

Type	Funktion : BOOL
Input	PT : POINTER TO ARRAY OF BYTE (Eingangsbyte) SIZE : INT (Größe des Arrays)
Config	POLYNOM_L : DWORD (unterste 32 Bit des Polynoms) POLYNOM_32 : BOOL (Bit 32 des Polynoms)
Output	BOOL (Ausgang ist TRUE, wenn die CRC-Checksumme = 0)



CRC_CHECK prüft ein Array of Byte auf eine gültige CRC Checksumme und liefert TRUE, wenn die Checksumme 0 ergibt und der Datenblock fehlerfrei vorliegt. Beim Aufruf wird der Funktion ein Pointer auf das zu bearbeitende Array und dessen Größe in Bytes übergeben. Unter CoDeSys lautet der Aufruf: CRC_CHECK(ADR(Array), SIZEOF(Array)), wobei Array der Name des zu prüfenden Arrays ist. ADR ist eine Standardfunktion, die den Pointer auf das Array ermittelt und SIZEOF ist eine Standardfunktion, die die Größe des Arrays ermittelt. CRC_CHECK benötigt als Eingang das Datenarray mit angehängter CRC-Checksumme. Mit den Config-Variablen POLYNOM_L und POLYNOM_32 wird das CRC Polynom definiert. Das Polynom kann ein beliebiges POLYNOM von maximal 33 Bit Länge sein. Ein Polynom $X^3 + X^2 + 1$ wird mit 1101 dargestellt. Es können Polynome bis 33 Bit Länge (CRC 32) verarbeitet werden. Das 33te Bit wird dabei in POLYNOM_32 gespeichert. Weitere Infos und Daten zu gebräuchlichen Polynomen können unter CRC_GEN nachgeschlagen werden.

15.12 CRC_GEN

Type	Funktion : DWORD
Input	PT : POINTER TO ARRAY OF BYTE (Datenpaket) SIZE : INT (Größe des Arrays)
Config	POLYNOM_L : DWORD (unterste 32 Bits des Polynoms) POLYNOM_32 : BOOL (Bit 32 des Polynoms) INIT : DWORD (INIT Daten) XOR_OUT : DWORD (Letztes XOR des Ausgangs) REV_IN : BOOL (Eingangsdaten Bytes Umkehren) REV_OUT : (Ausgangsdaten Umkehren)
Output	DWORD (errechnete CRC-Checksumme)



CRC_GEN generiert eine CRC-Checksumme aus einem beliebig großen Array of Byte. Beim Aufruf wird der Funktion ein Pointer auf das zu bearbeitende Array und dessen Größe in Bytes übergeben. Unter CoDeSys lautet der Aufruf: CRC_GEN(ADR(Array), sizeof(Array)), wobei Array der Name des zu bearbeitenden Arrays ist. ADR ist eine Standardfunktion, die den Pointer auf das Array ermittelt und sizeof ist eine Standardfunktion, die die Größe des Arrays ermittelt. Das Polynom kann ein beliebiges POLYNOM bis maximal 33 Bit Länge sein. Ein Polynom $X^3 + X^2 + 1$ wird mit 1101 dargestellt. Es können Polynome bis 33 Bit Länge (CRC 32) verarbeitet werden. Das 33te Bit wird dabei in POLYNOM_32 gespeichert. Durch den Setup Wert INIT kann dem Datenwort eine beliebige Bitfolge vorangestellt werden, üblich sind hier 0000 und FFFF. Der Setup Parameter XOR_OUT legt fest mit welcher Bitfolge die Checksumme am Ende mit XOR verknüpft wird. Die weiter unten folgende CRC Tabelle gibt nähere Auskunft über die Verwendung von INIT und XOR_OUT. Wenn der Setup Wert REV_IN auf TRUE gesetzt wird, so wird jedes Byte im Datenstrom umgedreht, Bit0 wird zu Bit 7, Bit 1 wird zu Bit 6 und so weiter. mit REV_OUT auf TRUE gesetzt bewirkt dies die Umkehrung der generierten CRC Checksumme.

Gebräuchliche Polynome: Die Spalten P32 und POLYNOM_L geben die Werte an, die für das entsprechende Polynom in den Config-Variablen abzuspeichern sind. POLYNOM_L ist alternativ als Dezimalwert angegeben, um die umständliche 32-Bit-Zahl einfacher darzustellen.

CRC	Polynom	initial HEX	XOR_O UT	REV IN	REV OUT	P 3 2	Polynom_ L	Polynom_L (DEC)

CRC-4-ITU ITU G.704, p. 12	$x^4 + x + 1$					0	00000000 00000000 00000000 00010011	19
CRC-5-ITU ITU G.704, p. 9 Bluetooth	$x^5 + x^4 + x^2 + 1$					0	00000000 00000000 00000000 00110101	53
CRC-5-USB USB token packets	$x^5 + x^2 + 1$					0	00000000 00000000 00000000 00100101	37
CRC-6-ITU ITU G.704, p. 3	$x^6 + x + 1$					0	00000000 00000000 00000000 01000011	67
CRC-7 telecom, MMC	$x^7 + x^3 + 1$					0	00000000 00000000 00000000 10001001	137
CRC-8-ATM ATM HEC	$x^8 + x^2 + x + 1$	00	00	0	0	0	00000000 00000000 00000001 00000111	263
CRC-8-CCITT 1-Wire bus	$x^8 + x^7 + x^3 + x^2 + 1$					0	00000000 00000000 00000001 10001101	397
CRC-8 1-Wire bus Dallas/Maxim	$x^8 + x^5 + x^4 + 1$					0	00000000 00000000 00000001 00110001	305
CRC-8	$x^8 + x^7 + x^6 + x^4 + x^2 + 1$					0	00000000 00000000 00000001 11010101	469
CRC-8 SAE J1850	$x^8 + x^4 + x^3 + x^2 + 1$					0	00000000 00000000 00000001 00011101	285
CRC-10	$x^{10} + x^9 + x^5 + x^4 + x + 1$					0	00000000 00000000 00000110 00110011	1587
CRC-12 telecom sys- tems	$x^{12} + x^{11} + x^3 + x^2 + x + 1$					0	00000000 00000000 00011000 00001111	6159
CRC-15 CAN	$x^{15} + x^{14} + x^{10} + x^8 + x^7 + x^4 + x^3 + 1$					0	00000000 00000000 11000101 10011001	50585

CRC-CCITT XMODEM, PPP, IRDA, Bluetooth, V41	$x^{16} + x^{12} + x^5 + 1$	FFFF	0000	0	0	0	00000000 00000001 00010000 00100001	69665
CRC-16 X25	$x^{16} + x^{12} + x^5 + 1$	FFFF	FFFF	1	1	0	00000000 00000001 00010000 00100001	69665
CRC-16- CCITT KERMIT	$x^{16} + x^{12} + x^5 + 1$	0000	0000	1	1	0	00000000 00000001 00010000 00100001	69665
CRC-16-IBM "CRC-16" USB, ARC	$x^{16} + x^{15} + x^2 + 1$	0000	0000	1	1	0	00000000 00000001 10000000 00000101	98309
CRC-16 MODBUS	$x^{16} + x^{15} + x^2 + 1$	FFFF	0000	1	1	0	00000000 00000001 10000000 00000101	98309
CRC-24 Radix-64	$x^{24} + x^{23} + x^{18} + x^{17} + x^{14} + x^{11} + x^{10} + x^7 + x^6 + x^5 + x^4 + x^3 + x + 1$					0	00000001 10000110 01001100 11111011	25578747
CRC-32 MPEG2 IEEE 802.3	$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$	FFFF FFFF	FFFF FFFF	1	1	1	00000100 11000001 00011101 10110111	79764919
CRC-32C (Castagnoli)	$x^{32} + x^{28} + x^{27} + x^{26} + x^{25} + x^{23} + x^{22} + x^{20} + x^{19} + x^{18} + x^{14} + x^{13} + x^{11} + x^{10} + x^9 + x^8 + x^6 + 1$					1	00011110 11011100 01101111 01000001	517762881

15.13 DEC_2

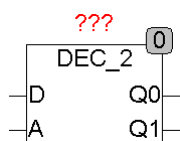
Type Funktionsbaustein

Input D : BOOL (Eingangs Bit)

A : BOOL (Adresse)

Output Q0 : BOOL (TRUE bei A=0)

Q1 : BOOL (TRUE bei A=1)

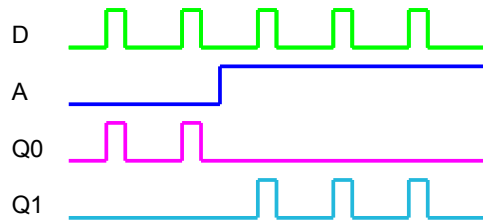


DEC_2 ist ein 2-Bit Dekodierbaustein. Ist A=0, so wird der Eingang D auf Ausgang Q0 geschaltet. Ist A=1, so wird D auf Q1 geschaltet. Mit anderen Worten: Q0=1 wenn D=1 und A=0.

Logische Verknüpfung:

$$Q0 = D \& \text{/A};$$

$$Q1 = D \& A$$



15.14 DEC_4

Type Funktionsbaustein

Input D : BOOL (Eingangs Bit)

A0 : BOOL (Adresse Bit0)

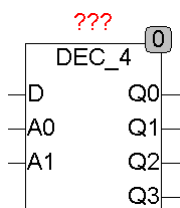
A1 : BOOL (Adresse Bit1)

Output Q0 : BOOL (TRUE bei A0=0 und A1=0)

Q1 : BOOL (TRUE bei A0=1 und A1=0)

Q2 : BOOL (TRUE bei A0=0 und A1=1)

Q3 : BOOL (TRUE bei A0=1 und A1=1)



DEC_4 ist ein 4-Bit Dekodierbaustein. Ist A0=0 und A1=0 wird der Eingang D auf Ausgang Q0 geschaltet. Wenn A0=1 und A1=1 wird D auf Q3 geschaltet. Mit anderen Worten: Q0=1, wenn D=1 und A0=0 und A1=0.

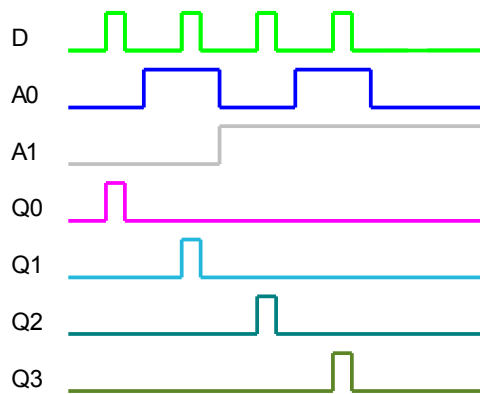
Logische Verknüpfung:

$$Q0 = D \& \text{/A0} \& \text{/A1}$$

$$Q1 = D \& A0 \& \text{/A1}$$

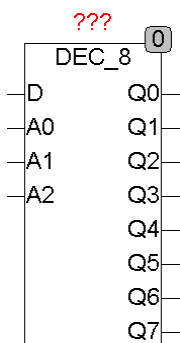
$$Q2 = D \& \text{/A0} \& A1$$

$$Q3 = D \& A0 \& A1$$



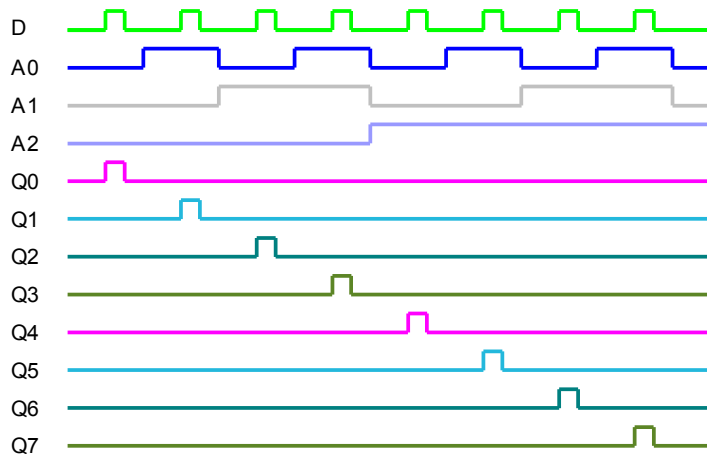
15.15 DEC_8

Type	Funktionsbaustein
Input	D : BOOL (Eingangs Bit)
	A0 : BOOL (Adresse Bit0)
	A1 : BOOL (Adresse Bit1)
	A2 : BOOL (Adresse Bit2)
Output	Q0 : BOOL (TRUE bei A0=0 und A1=0 und A2=0)
	Q1 : BOOL (TRUE bei A0=1 und A1=0 und A2=0)
	Q2 : BOOL (TRUE bei A0=0 und A1=1 und A2=0)
	Q3 : BOOL (TRUE bei A0=1 und A1=1 und A2=0)
	Q4 : BOOL (TRUE bei A0=0 und A1=0 und A2=1)
	Q5 : BOOL (TRUE bei A0=1 und A1=0 und A2=1)
	Q6 : BOOL (TRUE bei A0=0 und A1=1 und A2=1)
	Q7 : BOOL (TRUE bei A0=1 und A1=1 und A2=1)



DEC_8 ist ein 8-Bit Dekodierbaustein. Ist A0=0 und A1=0 und A2=0 wird der Eingang D auf Ausgang Q0 geschaltet, wenn A0=1 und A1=1 und A2=1 wird D auf Q3 geschaltet. Mit anderen Worten: Q0=1 wenn D=1 und A0=0 und A1=0 und A2=0.

Das folgende Schaubild verdeutlicht die Logik des Bausteins:

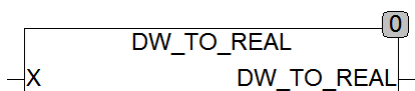


Logische Verknüpfung:

$$\begin{aligned}
 Q0 &= D \& \text{/A0} \& \text{/A1} \& \text{/A2} \\
 Q1 &= D \& A0 \& \text{/A1} \& \text{/A2} \\
 Q2 &= D \& \text{/A0} \& A1 \& \text{/A2} \\
 Q3 &= D \& A0 \& A1 \& \text{/A2} \\
 Q4 &= D \& \text{/A0} \& \text{/A1} \& A2 \\
 Q5 &= D \& A0 \& \text{/A1} \& A2 \\
 Q6 &= D \& \text{/A0} \& A1 \& A2 \\
 Q7 &= D \& A0 \& A1 \& A2
 \end{aligned}$$

15.16 DW_TO_REAL

Type Funktion : REAL
 Input IN : DWORD (Eingang)
 Output REAL (Ausgangswert)

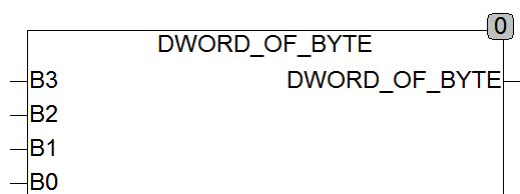


DW_TO_REAL kopiert das Bitmuster eines DWORD (IN) in einen REAL. Es werden dabei die einzelnen Bits kopiert ohne auf deren Bedeutung zu achten. Die Funktion REAL_TO_DW ist die Umkehrfunktion so das die Konvertierung von REAL_TO_DW

und anschließend DW_TO_REAL wiederum den Ausgangswert ergeben. Die IEC Standardfunktion DWORD_TO_REAL wandelt den Wert des DWORDS in einen REAL Wert.

15.17 DWORD_OF_BYTE

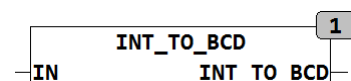
Type Funktion : DWORD
 Input B3 : Byte (Eingangs Byte 3)
 B2 : Byte (Eingangs Byte 2)
 B1 : Byte (Eingangs Byte 1)
 B0 : Byte (Eingangs Byte 0)
 Output DWORD (Ergebnis-DWORD)



DWORD_OF_BYTE setzt aus 4 separaten Bytes B0 .. B3 ein DWORD zusammen.
 EIN DWORD setzt sich zusammen wie folgt: B3-B2-B1-B0

15.18 INT_TO_BCD

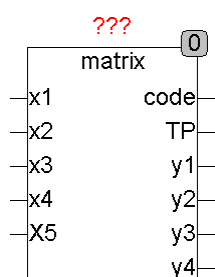
Type Funktion : BYTE
 Input IN : INT (Eingangswert)
 Output BYTE (BCD codierter Ausgangswert)



INT_TO_BCD konvertiert den Eingangswert IN in einen BCD codierten Ausgangswert.

15.19 MATRIX

Type	Funktionsblock
Input	X1 .. X5 : BOOL (Zeileneingänge)
Config	RELEASE : BOOL (Ein Tastencode wird beim Drücken und loslassen einer Taste erzeugt)
Output	CODE : Byte (Ausgang für Tastencode) TP : BOOL (TP ist für einen Zyklus TRUE, wenn ein neuer Tastencode ansteht) Y1 .. Y4 : BOOL (Zeilenausgänge)

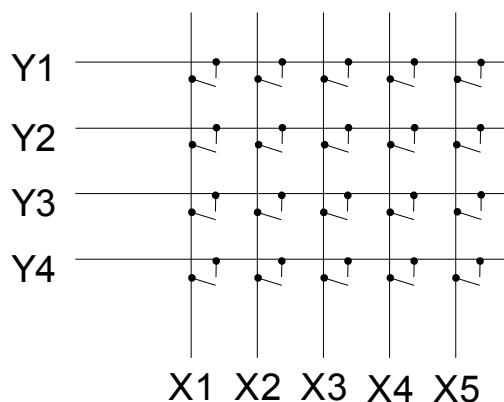


MATRIX ist ein Matrix-Tastatur-Controller für maximal 4 Spalten und 5 Zeilen. Mit jedem SPS Zyklus schaltet MATRIX den Spaltenausgang um eine Spalte weiter, so dass die Zeilen Y1 bis Y4 nacheinander abgefragt werden. Für jede Spalte werden die Zeileneingänge X1 bis X5 abgefragt und falls eine Taste gedrückt ist, wird der entsprechende Tastencode am Ausgang angezeigt. Der Ausgang TP ist genau dann einen Zyklus TRUE, wenn der Ausgang CODE einen neuen Wert anzeigt. Wenn die Config-Variable RELEASE auf TRUE gesetzt wird, dann wird für das Drücken und das Loslassen einer Taste jeweils ein Tastencode gesendet. Falls RELEASE auf FALSE gesetzt ist, wird nur beim Betätigen einer Taste ein Tastencode erzeugt. Der Tastencode des Ausgangs setzt sich wie folgt zusammen:

Bit	CODE Output
7	1 when key is pressed, 0 when key is released
6	Line number Bit 2
5	Line number Bit 1
4	Line number Bit 0
3	Always 0
2	Row number Bit 2

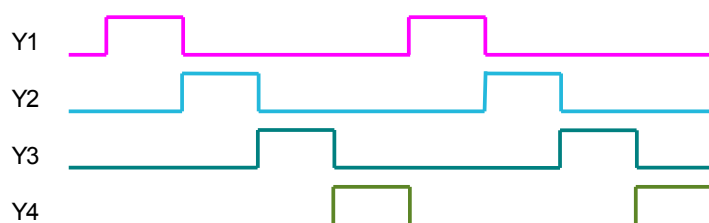
1	Row Number Bit 1
0	Row Number Bit 0

Der Matrixcontroller wird wie folgt beschaltet:



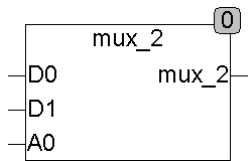
Bei dieser einfachen Beschaltung können bis zu 20 ($4 * 5$) Taster ausgewertet werden. Jedoch ist hierbei zu beachten, dass nur bedingt mehrere Tasten gleichzeitig gedrückt werden können. Der Controller kann mit dieser Beschaltung mehrere Tasten in einer Spalte sicher erkennen, jedoch nicht wenn Tasten an verschiedenen Spalten gleichzeitig gedrückt werden. Die Beschaltung kann jedoch erweitert werden, indem jeder einzelne Taster über Dioden entkoppelt wird und damit die Beeinflussung verschiedener Spalten untereinander verhindert wird. Bei der Beschaltung mit Dioden können beliebig viele Tasten gleichzeitig gedrückt und sicher Ausgewertet werden.

Die Ausgänge des Matrixcontrollers scannen kontinuierlich die Zeilen der Tastaturmatrix ab. Je SPS Zyklus wird eine Zeile eingelesen. Sind in einer Zeile mehrere Tasten gedrückt bzw. verändert worden, so werden die Änderungen als Codes über die folgenden Zyklen ausgegeben. Der Baustein merkt sich die einzelnen Tastencodes und gibt je Zyklus immer nur einen Code aus, sodass kein Code verloren gehen kann.



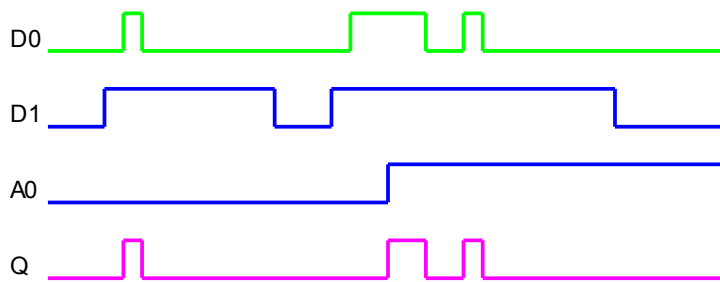
15.20 MUX_2

Type	Funktion : BOOL
Input	D0 : BOOL (Bit 0) D1 : BOOL (Bit 1) A0 : BOOL (Adresse)
Output	BOOL (D0, wenn A0=0 und D1, wenn A0=1)



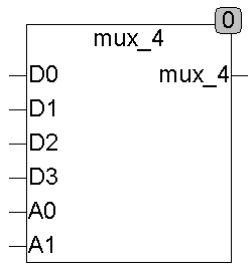
MUX_2 ist ein 2-Bit Multiplexer. Der Ausgang entspricht D0, wenn A0=0 und er entspricht D1, wenn A0=1.

Logische Verknüpfung: $\text{MUX}_2 = D0 \& \neg A0 + D1 \& A0$



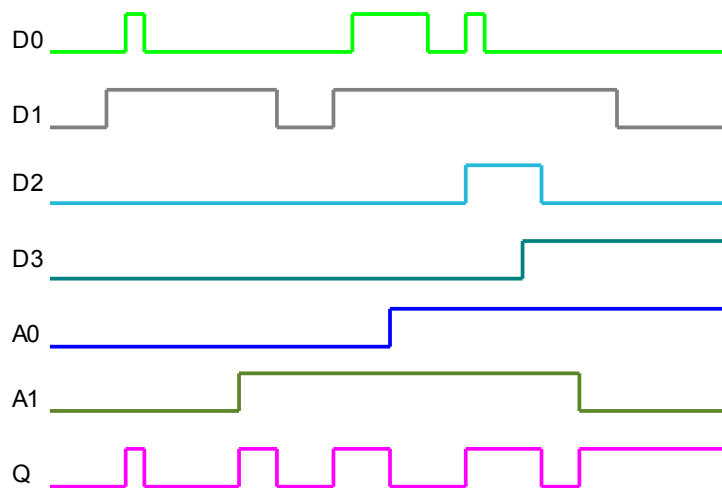
15.21 MUX_4

Type	Funktion : BOOL
Input	D0 : BOOL (Eingang 0) D1 : BOOL (Eingang 1) D2 : BOOL (Eingang 2) D3 : BOOL (Eingang 3)
Output	BOOL (D0, wenn A0=0 und A1 = 0, usw...)



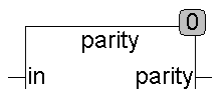
MUX_4 ist ein 4-Bit Multiplexer. Der Ausgang entspricht D0, wenn A0=0 und A1=0. Er entspricht D3, wenn A0=1 und A1=1.

Logische Verknüpfung:
$$\text{MUX}_4 = D0 \& \neg A0 \& \neg A1 + D1 \& A0 \& \neg A1 + D2 \& \neg A0 \& A1 + D3 \& A0 \& A1$$



15.22 PARITY

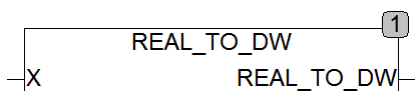
Type Funktion : BOOL
 Input IN : BYTE (Eingangs BYTE)
 Output BOOL (Ausgang ist TRUE, wenn Parität gerade ist)



PARITY bildet eine gerade Parität über das Eingangsbyte IN. Der Ausgang ist TRUE wenn die Anzahl der TRUE Bits im Byte (In) ungerade ist.

15.23 REAL_TO_DW

Type Funktion : DWORD
 Input IN : REAL (Eingang)
 Output DWORD (Ausgangswert)



REAL_TO_DW kopiert das Bitmuster eines REAL (IN) in ein DWORD. Es werden dabei die einzelnen Bits kopiert ohne auf deren Bedeutung zu achten. Die Funktion DW_TO_REAL ist die Umkehrfunktion so das die Konvertierung von REAL_TO_DW und anschließend DW_TO_REAL wiederum den Ausgangswert ergibt. Die IEC Standardfunktion REAL_TO_DWORD wandelt den REAL Wert in eine Ganzzahl und Rundet an der kleinsten Stelle des DWORD.

15.24 REVERSE

Type Funktion : BYTE
 Input IN : BYTE (Eingangs BYTE)
 Output BYTE(Ausgangs Byte)

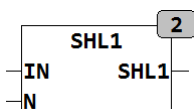


REVERSE dreht die Reihenfolge der Bits in einem Byte um. Bit7 von IN wird zu Bit 0, Bit 6 wird zu Bit 1 usw.

Beispiel: REVERSE(10011110) = 01111001

15.25 SHL1

Type Funktion : DWORD
 Input IN : DWORD (Eingangsdaten)
 N : INT (Anzahl der zu schiebenden Bits)
 Output DWORD (Ergebnis)

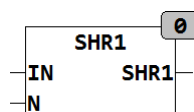


SHL1 schiebt das Eingangs DWORD um N Bits nach Links und füllt die N Rechten Bits mit 1en auf. Im Gegensatz zur IEC Standard Funktion SHL die beim Schieben mit Nullen aufgefüllt wird bei SHL1 mit Einsen Aufgefüllt.

Beispiel: SHL1(11110000,2) ergibt 11000011

15.26 SHR1

Type Funktion : DWORD
 Input IN : DWORD (Eingangsdaten)
 N : INT (Anzahl der zu schiebenden Bits)
 Output DWORD (Ergebnis)

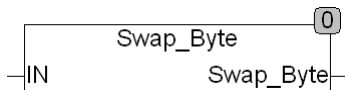


SHR1 schiebt das Eingangs DWORD um N Bits nach Rechts und füllt die N linken Bits mit 1en auf. Im Gegensatz zur IEC Standard Funktion SHL die beim Schieben mit Nullen aufgefüllt wird bei SHR1 mit Einsen Aufgefüllt.

Beispiel: SHR1(11110000,2) ergibt 11111100

15.27 SWAP_BYTE

Type Funktion : WORD
 Input IN : WORD (Eingangsdaten)
 Output WORD (Ergebnis)

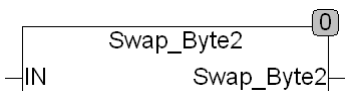


SWAP_BYTE tauscht das High und Low Byte in einem WORD.

Beispiel: SWAP_BYTE(16#33df) = 16#df33.

15.28 SWAP_BYTE2

Type Funktion : DWORD
 Input IN : DWORD (Eingangsdaten)
 Output DWORD (Ergebnis)

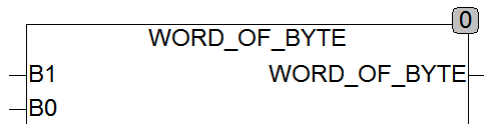


SWAP_BYTE2 kehrt die Reihenfolge der Bytes in einem DWORD um.

Beispiel: SWAP_BYTE2(16#33df1122) = 16#2211df33.

15.29 WORD_OF_BYTE

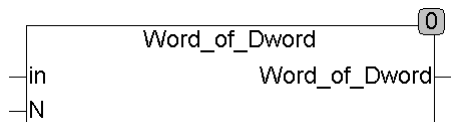
Type Funktion : WORD
 Input B1 : Byte (Eingangs Byte 1)
 B0 : Byte (Eingangs Byte 0)
 Output Word (Ergebnis Word)



WORD_OF_BYTE setzt aus 2 separaten Bytes B0 und B1 ein Word zusammen.

15.30 WORD_OF_DWORD

Type Funktion : WORD
Input IN : DWORD (Eingangs DWORD)
Output WORD (Ausgangs WORD)

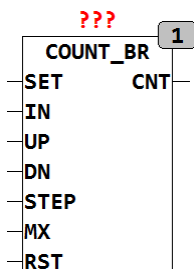


WORD_OF_DWORD extrahiert ein Word (W0 .. W1) aus einem DWORD.

16 Latches, Flip-Flop und Schieberegister

16.1 COUNT_BR

Type	Funktionsblock
Input	SET : BOOL (asynchroner Set) IN : BYTE (Vorgabewert für Set) UP : BOOL (Vorwärts Schalter flankengetriggert) DN : BOOL (Rückwärts Schalter flankengetriggert) STEP : BYTE (Schrittweite des Counters) MX : BYTE (Maximalwert des Counters) RST : BOOL (asynchroner Reset)
Output	CNT : BYTE (Ausgang)

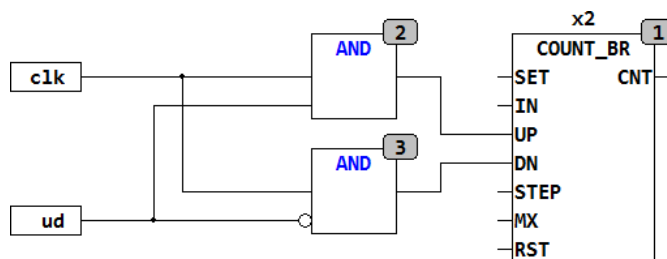


COUNT_BR ist ein Byte Zähler der von 0 bis MX zählt und dann wieder bei 0 beginnt. Der Zähler kann mittels 2 flankengetriggerten Eingängen UP und DN sowohl vorwärts als auch Rückwärts Zählen. beim Erreichen eines Endwerts 0 oder MX wird wieder bei 0 beziehungsweise MX weiter gezählt. Der Eingang STEP legt die Schrittweite des Zählers fest. Mit einem TRUE am Eingang SET wird der Zähler auf den an IN anliegenden Wert gesetzt. Ein Reset Eingang RST setzt den Zähler jederzeit auf 0.

	SET	IN	UP	DN	STEP	RST	CNT
Reset	-	-	-	-	-	1	0
Set	1	N	-	-	-	0	N

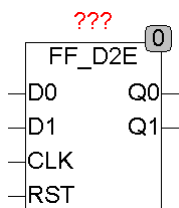
up	0	-	↑	0	N	0	CNT + N
down	0	-	0	↑	N	0	CNT - N

Falls die unabhängigen Eingänge UP und DN mit CLK und einen Steuereingang UP/DN ersetzt werden sollen kann dies mittels zwei AND Gattern vor den Eingängen erfolgen:

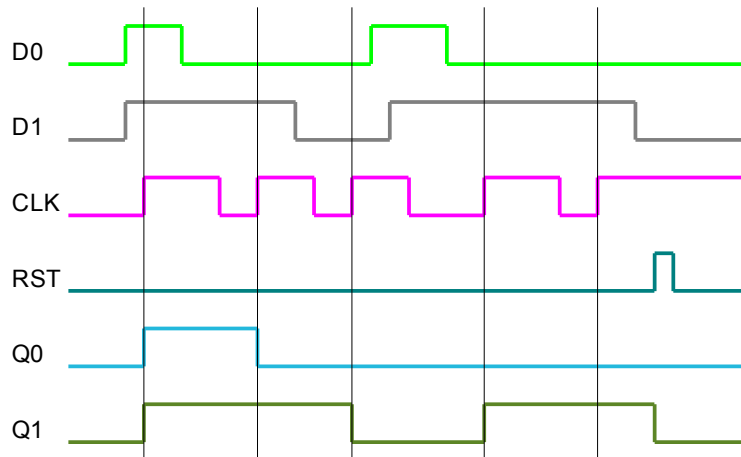


16.2 FF_D2E

Type	Funktionsblock
Input	D0 : BOOL (Data 0 in) D1 : BOOL (Data 1 in) CLK : BOOL (Takteingang) RST : BOOL (asynchroner Reset)
Output	Q0 : BOOL (Data 0 out) Q1 : BOOL (Data 1 out)

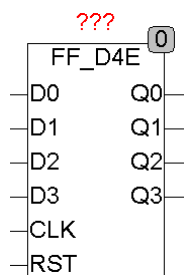


FF_D2E ist ein 2 Bit flankengetriggertes D-Flip-Flop mit asynchronem Reset-Eingang. Das D-Flip-Flop speichert die Werte am Eingang D mit einer steigenden Flanke am CLK Eingang.



16.3 FF_D4E

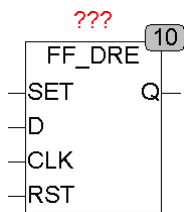
Type	Funktionsblock
Input	D0 : BOOL (Data 0 in)
	D1 : BOOL (Data 1 in)
	D2 : BOOL (Data 2 in)
	D3 : BOOL (Data 3 in)
	CLK : BOOL (Takteingang)
Output	RST : BOOL (asynchroner Reset)
	Q0 : BOOL (Data 0 Out)
	Q1 : BOOL (Data 1 Out)
	Q2 : BOOL (Data 2 Out)
	Q3 : BOOL (Data 3 Out)



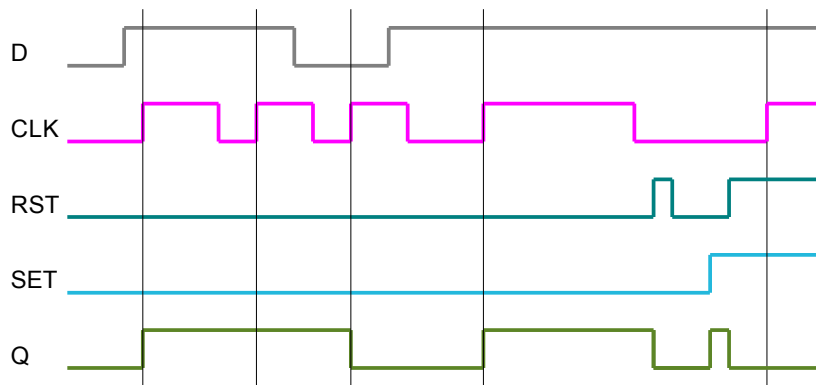
FF_D2E ist ein 4 Bit flankengetriggertes D-Flip-Flop mit asynchronem Reset-Eingang. Das D-Flip-Flop speichert die Werte am Eingang D mit einer steigenden Flanke an CLK. Detaillierte Angaben finden Sie beim Baustein FF_D2E.

16.4 FF_DRE

Type	Funktionsblock
Input	SET : BOOL (asynchroner Set) D : BOOL (Data in) CLK : BOOL (Takteingang) RST : BOOL (asynchroner Reset)
Output	Q : BOOL (Data Out)

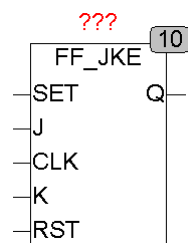


FF_DRE ist ein flankengetriggertes D-Flip-Flop mit asynchronem Set und Reset Eingang. Eine steigende Flanke an CLK speichert den Eingang D auf den Ausgang Q. Ein TRUE am SET oder RST-Eingang setzt oder löscht den Ausgang Q zu jeder Zeit unabhängig von CLK. Der Reset Eingang hat Vorrang vor den Set Eingang. Wenn beide aktiv (TRUE) sind wird ein Reset ausgeführt und Set ignoriert.

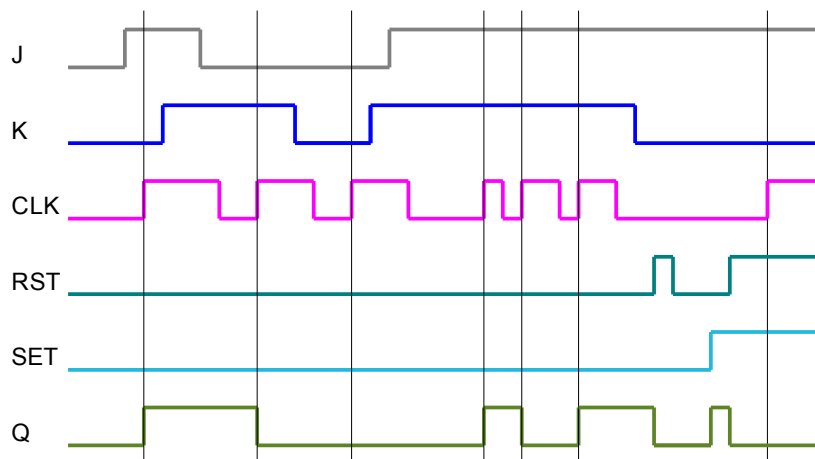


16.5 FF_JKE

Type	Funktionsblock
Input	SET : BOOL (asynchroner Set)
	J : BOOL (Takt-synchroner Set)
	CLK : BOOL (Takteingang)
	K : BOOL (Takt-synchroner Reset)
	RST : BOOL (asynchroner Reset)
Output	Q : BOOL (Ausgang)



FF_JKE ist ein flankengetriggertes JK-Flip-Flop mit asynchronen Set und Reset Eingängen. Das JK-Flip-Flop setzt den Ausgang Q, wenn bei einer steigenden Flanke von CLK der Input J TRUE ist. Q wird FALSE, wenn bei einer steigenden Taktflanke der Eingang K TRUE ist. Sind die beiden Eingänge J und K bei einer Steigenden Taktflanke TRUE, so wird der Ausgang negiert. Er schaltet bei jedem Takt das Ausgangssignal um.



16.6 SELECT_8

Type Funktionsblock

Input EN : BOOL (Enable für Ausgänge)

SET : BOOL (asynchroner Set)

IN : BYTE (Vorgabewert für Set)

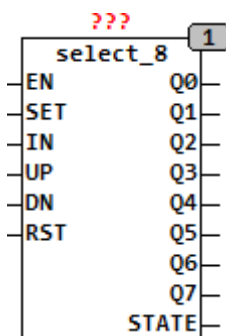
UP : BOOL (Vorwärts Schalter flankengetriggert)

DN : BOOL (Rückwärts Schalter flankengetriggert)

RST : BOOL (asynchroner Reset)

Output Q0 .. Q7 : BOOL (Ausgänge)

STATE : BYTE (Status Ausgang)

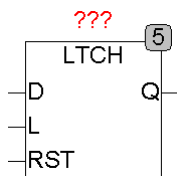


SELECT_8 setzt immer nur einen Ausgang auf TRUE solange EN auf TRUE ist. Der aktive Ausgang Q0..Q7 kann mittels des SET Eingangs und dem Wert am Eingang IN Selektiert werden. Ein TRUE an SET und ein Wert von 5 am Eingang IN setzen den Ausgang Q5 auf TRUE während alle anderen Ausgänge auf FALSE gesetzt werden. Ein TRUE am Eingang RST setzt Ausgang Q0 auf TRUE. Mit den Eingängen UP wird von einem Ausgang Qn auf Qn+1 weiter geschaltet, während der Eingang DN von einen Ausgang Qn auf Qn-1 schaltet. Der Eingang EN muss TRUE sein damit ein Ausgang TRUE wird, ist EN FALSE werden alle Ausgänge FALSE. Ein FALSE an EN beeinflusst aber nicht die Funktion der anderen Eingänge. So kann auch bei einem FALSE am Eingang EN mit UP oder DN hoch oder runter geschaltet werden. Die Eingänge UP und DN sind flankengetriggert und reagieren nur auf die steigende Flanke. Der Ausgang STATE zeigt immer an welcher Ausgang gerade selektiert ist.

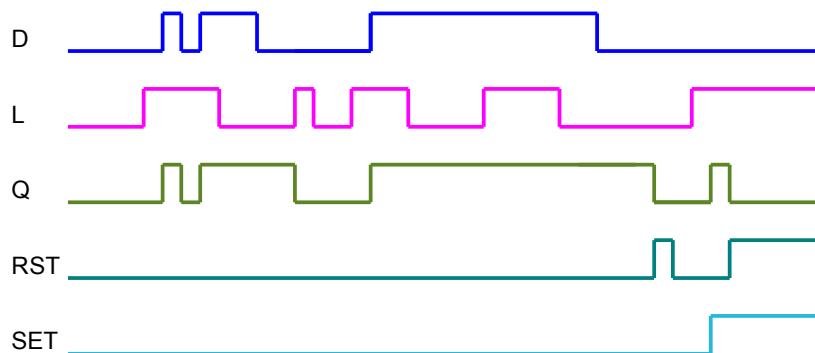
	EN	SET	IN	UP	DN	RST	Q	STATE
Reset	X	-	-	-	-	1	Q0 if EN=1	0
Set	X	1	N	-	-	0	QN if EN=1	N
up	X	0	-	↑	0	0	QN+1 if EN=1	N + 1
down	X	0	-	0	↑	0	QN-1 if EN=1	N - 1

16.7 LATCH

Type	Funktionsblock
Input	D : BOOL (Data in) L : BOOL (Latch enable Signal) RST : BOOL (asynchroner Reset)
Output	Q : BOOL (Data Out)

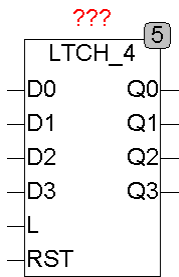


LTCH ist ein transparentes Speicherelement (Latch). Solange L TRUE ist, folgt Q dem Eingang D und mit der fallenden Flanke von L speichert der Ausgang Q das aktuelle Eingangssignal an D. Mit dem asynchronen Reset-Eingang kann das Latch jederzeit unabhängig von L gelöscht werden.



16.8 LATCH4

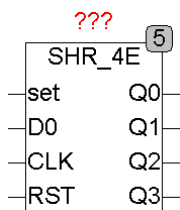
Type	Funktionsblock
Input	D0 .. D3 : BOOL (Data in) L : BOOL (Latch enable Signal) RST : BOOL (asynchroner Reset)
Output	Q0 .. Q3 : BOOL (Data Out)



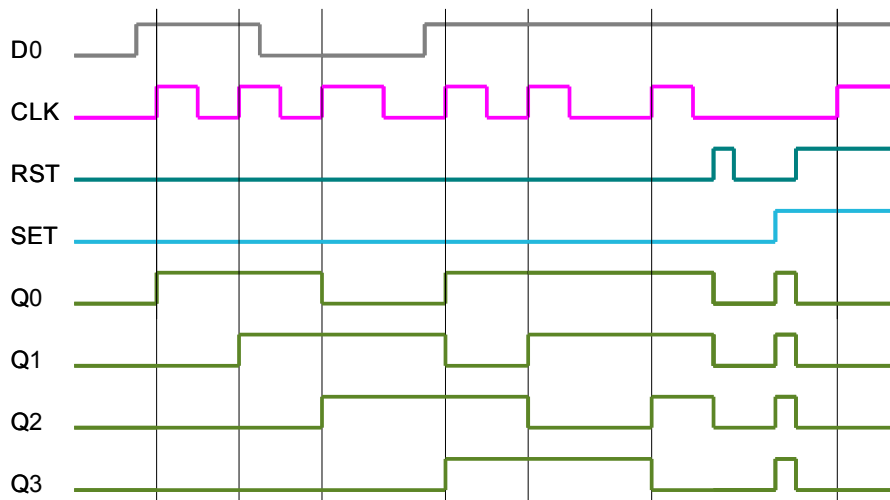
LTCH4 ist ein transparentes Speicherelement (Latch). Solange L TRUE ist folgen Q0 - Q3 den Eingängen D0 - D3 und mit der fallenden Flanke von L speichern die Ausgänge Q0 - Q3 das aktuelle Eingangssignal von D0 - D3. Mit dem asynchronen Reset-Eingang kann das Latch jederzeit unabhängig von L gelöscht werden. Weitere Erläuterungen und Angaben finden Sie beim Baustein LTCH.

16.9 SHR_4E

Type	Funktionsblock
Input	SET : BOOL (asynchroner Set)
	D0 : BOOL (Data Input)
	CLK : BOOL (Takteingang)
	RST : BOOL (asynchroner Reset)
Output	Q0 : BOOL (Data Out 0)
	Q1 : BOOL (Data Out 1)
	Q2 : BOOL (Data Out 2)
	Q3 : BOOL (Data Out 3)

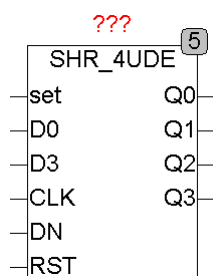


SHR_4E ist ein 4 Bit Shift Register mit asynchronem Set- und Reset-Eingang. Mit einer steigenden Flanke an CLK wird Q2 nach Q3 geschoben, Q1 nach Q2, Q0 nach Q1 und D0 nach Q0 gespeichert. Mit einem TRUE am Set-Eingang werden alle Ausgänge (Q0 .. Q3) auf TRUE gesetzt und mit RST werden alle auf FALSE gesetzt.

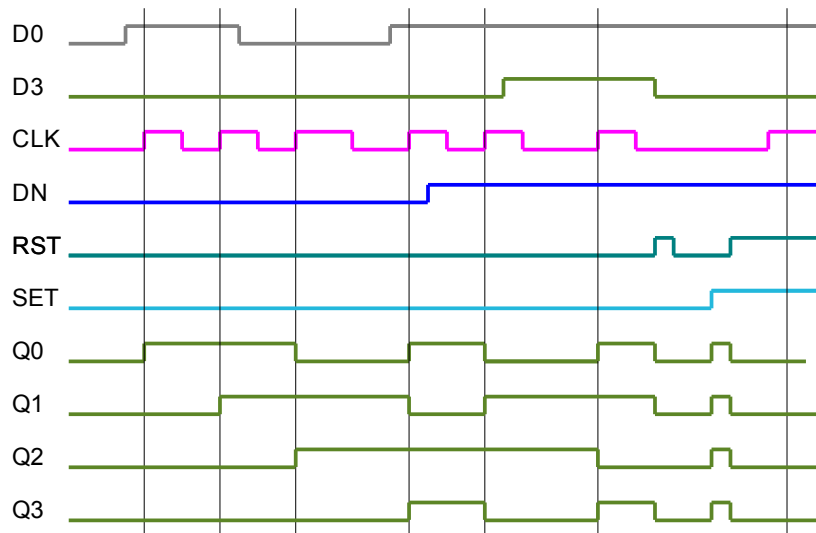


16.10 SHR_4UDE

Type	Funktionsblock
Input	SET : BOOL (asynchroner Set) D0 : BOOL (Data Input Bit 0) D3 : BOOL (Data Input Bit 3) CLK : BOOL (Takteingang) DN : BOOL (Steuereingang Up / Down, TRUE = Down) RST : BOOL (asynchroner Reset)
Output	Q0 : BOOL (Data Out 0) Q1 : BOOL (Data Out 1) Q2 : BOOL (Data Out 2) Q3 : BOOL (Data Out 3)

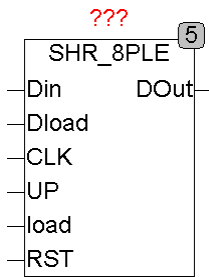


SHR_4UDE ist ein 4 Bit Schieberegister mit Up / Down Schieberichtungen. Mit einer steigenden Flanke an CLK wird Q2 nach Q3 geschoben, Q1 nach Q2, Q0 nach Q1 und D0 nach Q0 gespeichert. Die Schieberichtung kann mit einem TRUE am Eingang DN umgekehrt werden, dann wird D3 nach Q3 – nach Q2 – nach Q1 – nach Q0 geschoben. Mit einem TRUE am Set-Eingang werden alle Ausgänge (Q0 .. Q3) auf TRUE gesetzt und mit RST werden alle Eingänge auf FALSE gesetzt.



16.11 SHR_8PLE

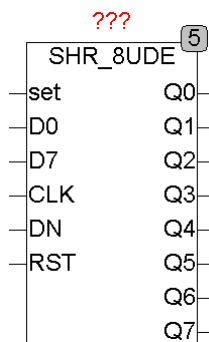
Type	Funktionsblock
Input	DIN : BOOL (Shift Data Input)
	DLOAD : Byte (Datenwort zum Parallel Load)
	CLK : BOOL (Taktingang)
	UP : BOOL (Steuereingang Up / Down, TRUE = Up)
	LOAD : BOOL (Steuereingang zum Laden des Registers)
	RST : BOOL (asynchroner Reset)
Output	DOUT : BOOL (Data Out)



SHR_8PLE ist ein 8 Bit Schieberegister mit Parallel Load und asynchronem Reset. Die Schieberichtung kann mit dem Eingang UP umgekehrt werden. Wenn UP=1, wird Bit 7 zuerst auf DOUT geschoben und wenn UP=0, wird Bit0 zuerst an DOUT geschoben. Für Up-Shift wird Bit 0 mit DIN geladen und bei Down-Shift wird Bit 7 mit DIN geladen. Am Eingang DLOAD liegt ein Byte Daten an, das bei Parallel Load (LOAD=1 und steigende Flanke an CLK) ins interne Register geladen wird. Im Falle von Parallel Load wird zuerst ein Shift durchgeführt und anschließend das Register geladen. Ein RST kann jederzeit asynchron das Register löschen. Eine eingehende Beschreibung eines Schieberegisters finden Sie beim Modul SHR_4E.

16.12 SHR_8UDE

Type	Funktionsblock
Input	SET : BOOL (asynchroner Set) D0 : BOOL (Data Input Bit 0) D3 : BOOL (Data Input Bit 3) CLK : BOOL (Takteingang) DN : BOOL (Steuereingang Up / Down, TRUE = Down) RST : BOOL (asynchroner Reset)
Output	Q0 .. Q7 : BOOL (Data Out)

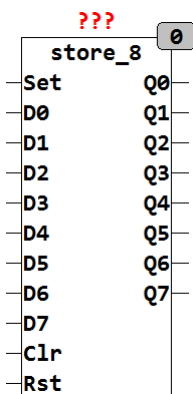


SHR_8UDE ist ein 8 Bit Schieberegister mit Up / Down Schieberichtung. Mit einer

steigenden Flanke an CLK werden die Daten Q0 nach Q7 um jeweils einen Schritt geschoben. Q0 wird anschließend mit D0 geladen. Die Schieberichtung kann mit einem TRUE am Eingang DN umgekehrt werden. Dann wird D7 nach Q6, Q5, Q4, Q3, Q2, Q1, Q0 geschoben und Q7 mit D7 geladen. Mit einem TRUE am Set-Eingang werden alle Ausgänge (Q0 .. Q3) auf TRUE gesetzt und mit RST werden alle Ausgänge auf FALSE gesetzt. Weitergehende Erläuterungen zu Schieberegistern finden Sie unter SHR_4E und speziell beim Modul SHR_4UDE, welches die gleiche Funktion für 4 Bits wie SHR_8UDE für 8 Bits erfüllt.

16.13 STORE_8

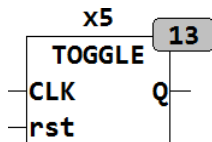
Type	Funktionsblock
Input	SET : BOOL (asynchroner Set)
	D0..D7 : BOOL (Data Input Bit 0..7)
	CLR : BOOL (Schrittweise Rücksetzen Eingang)
	RST : BOOL (Asynchroner Set Eingang)
Output	Q0..Q7 : BOOL (Ereignis Ausgänge)



STOTRE_8 ist ein 8-fach Ereignisspeicher. ein TRUE an einem der Eingänge D0..D7 setzt den entsprechenden Ausgang Q0..Q7. Die Asynchronen Set und Reset Eingänge (SET, RST) setzen alle Ausgänge gleichzeitig auf TRUE oder FALSE. ist während eines Resets einer der Eingänge TRUE wird nach dem Reset der entsprechende Ausgang sofort wieder auf TRUE gesetzt. Falls flankengetriggerte Eingänge gewünscht werden, so sind vor dem Baustein STRORE_8 TP_R Bausteine einzusetzen. Dies erlaubt es dem Anwender sowohl flanken- wie auch zustands- getriggerte Eingänge gleichzeitig zu verwenden. Der Eingang CLR löscht mit einer steigenden Flanke an CLR immer nur ein Ereignis, beginnend mit dem am höchsten priorisierten Ausgang der gerade TRUE ist. Wird mit CLR ein Ausgang Q gelöscht dessen Eingang D noch TRUE ist so wird der Ausgang Q mit dem nächsten Zyklus wieder auf TRUE gesetzt.

16.14 TOGGLE

Type	Funktionsblock
Input	CLK: BOOL (Takteingang) RST : BOOL (asynchroner Reset)
Output	Q : BOOL (Ausgang)

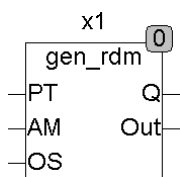


TOGGLE ist ein flankengetriggertes Toggle Flop-Flop mit asynchronem Reset-Eingang. Das TOGGLE Flip-Flop invertiert den Ausgang Q bei einer steigenden Flanke von CLK. Der Ausgang ändert bei jeder steigenden Flanke von CLK seinen Zustand.

17 Signalgeneratoren

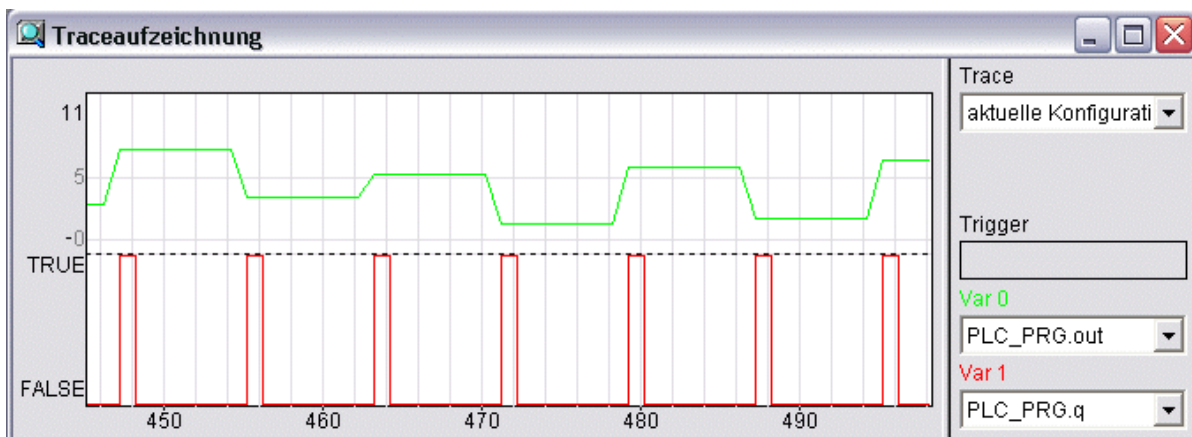
17.1 GEN_RDM

Type	Funktionsblock
Input	PT : TIME (Periodendauer) AM : REAL (Signal Amplitude) OS : REAL (Signal Offset)
Output	Q : BOOL (Binäres Ausgangssignal) OUT: REAL (Analoges Ausgangssignal)



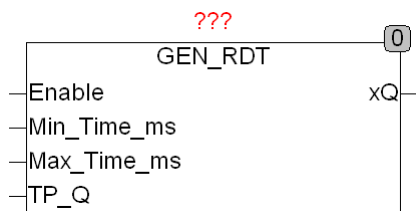
GEN_RDM ist ein Zufalls-Signalgenerator. Er erzeugt am Ausgang OUT einen neuen Wert in PT Zeitabständen. Der Ausgang Q wird für genau einen Zyklus TRUE, wenn sich der Ausgang OUT verändert hat. Der Eingang AM und OS legen die Amplitude und den Offset für den Ausgang OUT fest. Wenn die Eingänge OS und AM nicht beschaltet werden sind die Vorgabewerte 0 und 1.

Das folgende Beispiel zeigt eine Traceaufzeichnung für die Eingangswerte PT=100ms, AM=10 und OS=5. Der Ausgang erzeugt Werte alle 100ms im Bereich von 0 .. 10.



17.2 GEN_RDT

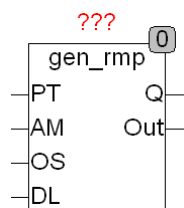
Type	Funktionsblock
Input	ENABLE : BOOL (Freigabeeingang) MIN_TIME_MS : TIME (Minimale Periodendauer) MAX_TIME_MS : TIME (Maximale Periodendauer) TP_Q : TIME (Pulsbreite des Ausgangspulses an XQ)
Output	XQ : BOOL (Binäres Ausgangssignal)



GEN_RDT erzeugt Impulse mit definierter Pulsbreite und Zufälligen Abstand. Die Ausgangsimpulse mit der Pulsbreite TP_Q werden in Zufälligen Zeitabständen TX erzeugt. TX schwankt zufällig zwischen der Zeit MIN_TIME_MS und MAX_TIME_MS. Der Baustein erzeugt nur Impulse an Ausgang XQ wenn der Eingang ENABLE auf TRUE ist.

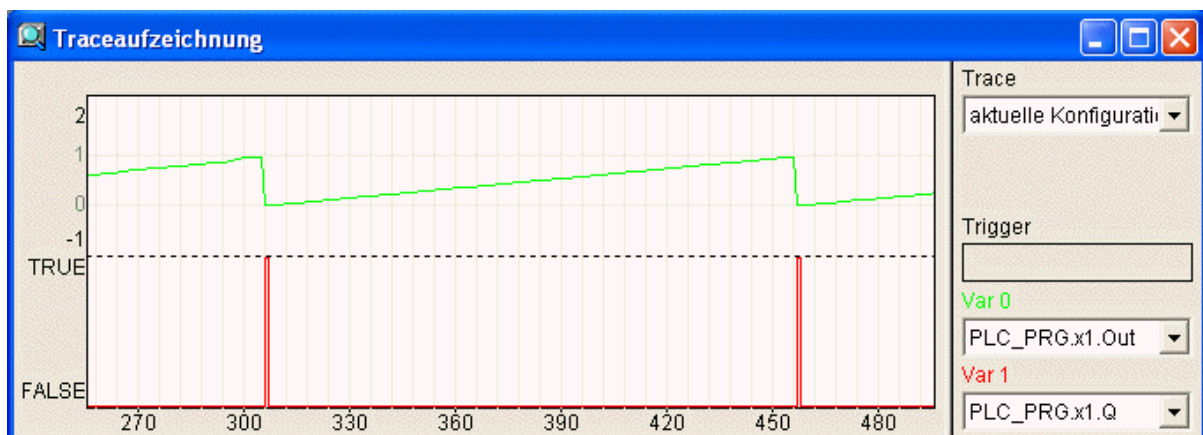
17.3 GEN_RMP

Type	Funktionsblock
Input	PT : TIME (Periodendauer) AM : REAL (Signal Amplitude) OS : REAL (Signal Offset) DL : REAL (Signal Verzögerung $0..1 * PT$)
Output	Q : BOOL (Binäres Ausgangssignal) OUT : REAL (Analoges Ausgangssignal)



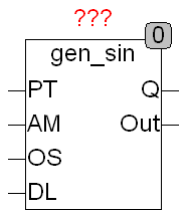
GEN_RMP ist ein Sägezahngenerator. Er erzeugt am Ausgang OUT eine Rampe mit der Dauer PT und wiederholt diese fortlaufend. Der Ausgang Q wird für genau einen Zyklus TRUE, wenn die Rampe am Ausgang OUT beginnt. Der Eingang AM und OS legen die Amplitude und den Offset für den Ausgang OUT fest. Wenn die Eingänge OS und AM nicht beschaltet werden sind die Vorgabewerte 0 und 1. Der Ausgang OUT erzeugt dann ein Sägezahnsignal von 0 .. 1. Der Eingang DL kann das Ausgangssignal um bis zu eine Periode (PT) verschieben und dient dazu, mehrere zueinander verschobene Signale, zu erzeugen. Eine 0 am Eingang DL bedeutet keine Verschiebung. Ein Wert zwischen 0 und 1 verschiebt das Signal um bis zu einer Periode.

Das folgende Beispiel zeigt eine Traceaufzeichnung für die Eingangswerte PT = 10s, AM = 1 und OS = 0.



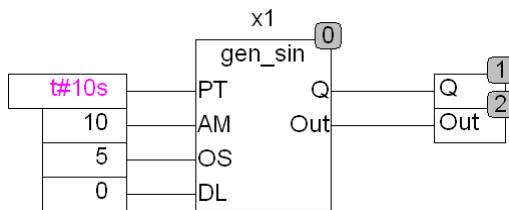
17.4 GEN_SIN

Type	Funktionsblock
Input	PT : TIME (Periodendauer) AM : REAL (Signal Amplitude) OS : REAL (Signal Offset) DL : REAL (Signal Verzögerung $0..1 * PT$)
Output	Q : BOOL (Binäres Ausgangssignal) OUT : REAL (Analoges Ausgangssignal)

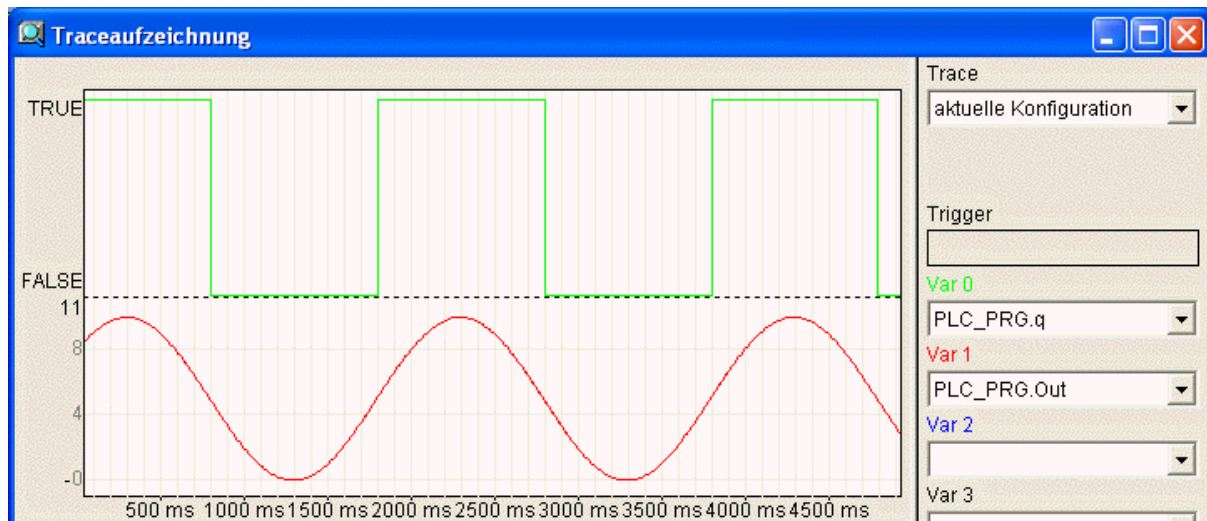


GEN_SIN ist ein Sinusgenerator mit programmierbarer Periodendauer, einstellbarer Amplitude und Signal Offset. Als Besonderheit kann auch noch ein Delay eingestellt werden, damit mit mehreren Generatoren überlappende Signale erzeugt werden können. Ein Binärer Ausgang Q stellt ein Logisches Signal zur Verfügung das Phasengleich mit dem Sinussignal erzeugt wird. Der Eingang DL gibt ein Delay für das Ausgangssignal vor. Das Delay wird spezifiziert mit $DL * PT$. Ein DL von 0.5 verzögert das Signal um eine halbe Periode.

Das folgende Beispiel zeigt GEN_SIN mit einer Traceaufzeichnung des Sinussignals und des binären Ausgangs Q.

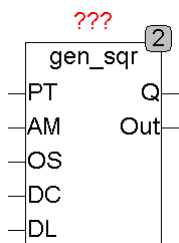


Obiges Beispiel Erzeugt ein Sinussignal mit 0.1 HZ ($PT = 10\text{ s}$) und einen unteren Spitzenwert von 0 und oberen Spitzenwert von 10.



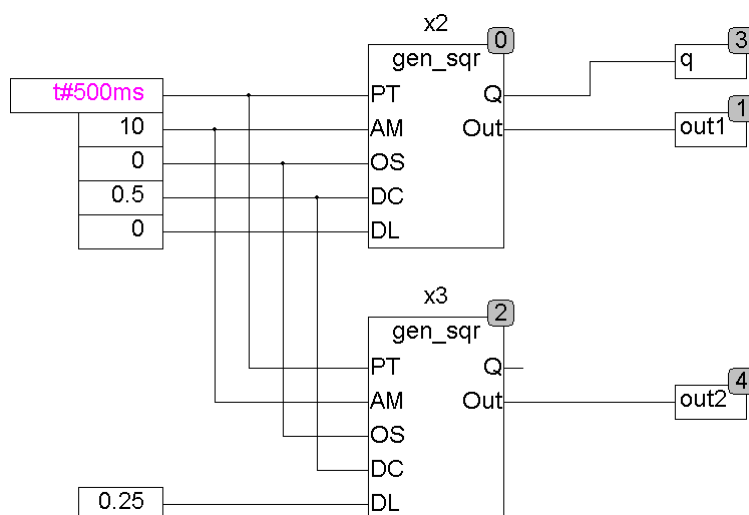
17.5 GEN_SQR

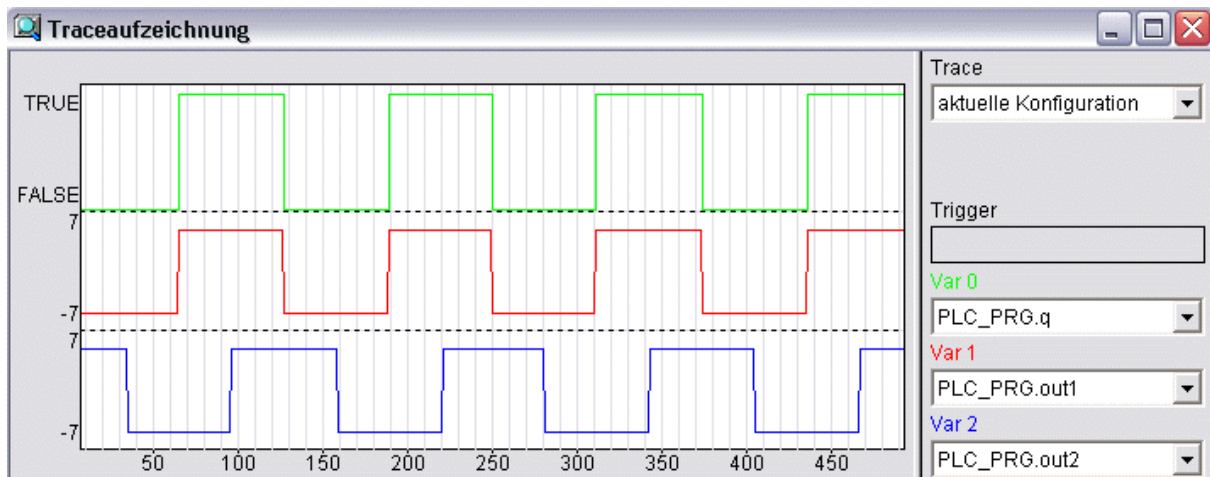
Type	Funktionsblock
Input	PT : TIME (Periodendauer)
	AM : REAL (Signal Amplitude)
	OS : REAL (Signal Offset)
	DC : REAL (Tastverhältnis 0..1)
	DL : REAL (Signal Verzögerung 0..1 * PT)
Output	Q : BOOL (Binäres Ausgangssignal)
	OUT : REAL (Analoges Ausgangssignal)



GEN_SQR ist ein Rechteckgenerator mit programmierbarer Periodendauer, einstellbarer Amplitude und Signal Offset sowie Tastverhältnis DC (Duty Cycle). Als Besonderheit kann auch noch ein Delay eingestellt werden, damit mit mehreren Generatoren überlappende Signale erzeugt werden können.

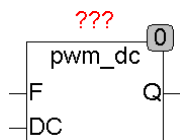
Das folgende Beispiel zeigt 2 GEN_SQR, wobei einer davon mit einem Delay von 0.25 ($\frac{1}{4}$ Periode) läuft. In der Traceaufzeichnung ist deutlich das Signal des ersten Generators und das verzögerte Signal des zweiten Generators zu sehen.





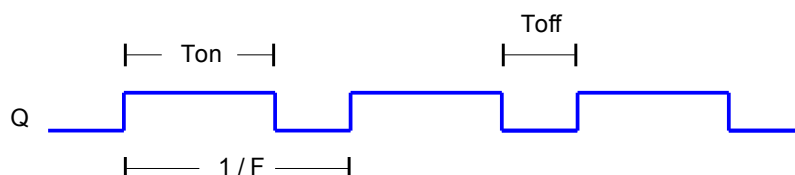
17.6 PWM_DC

Type Funktionsblock
 Input F : REAL (Ausgangsfrequenz)
 DC : REAL (Tastverhältnis 0..1)
 Output Q : BOOL (Ausgangssignal)



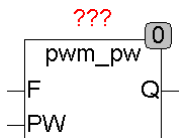
PWM_DC ist ein Duty-Cycle modulierter Frequenzgenerator. Der Generator erzeugt eine feste Frequenz F mit einem Tastverhältnis (TON / TOFF), welche über den Eingang DC moduliert (Eingestellt) werden kann. Ein Wert von 0.5 am Eingang DC erzeugt ein Tastverhältnis von 50%.

Das Folgende Signalbild zeigt ein Ausgangssignal mit einem Duty-Cycle von 2 / 1, was einem DC (Tastverhältnis) von 0.67 entspricht.

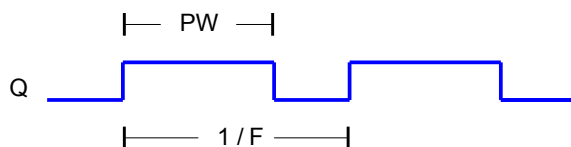


17.7 PWM_PW

Type	Funktionsblock
Input	F : REAL (Ausgangsfrequenz) PW : TIME (Impulsdauer High)
Output	Q : BOOL (Ausgangssignal)

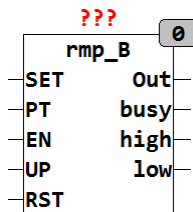


PWM_PW ist ein Puls breiten modulierter Frequenzgenerator. Der Generator erzeugt eine feste Frequenz F mit einem Tastverhältnis (TON / TOFF) das über den Eingang PW moduliert (eingestellt) werden kann. Der Eingang gibt die Zeit vor die das Signal auf TRUE bleibt.



17.8 RMP_B

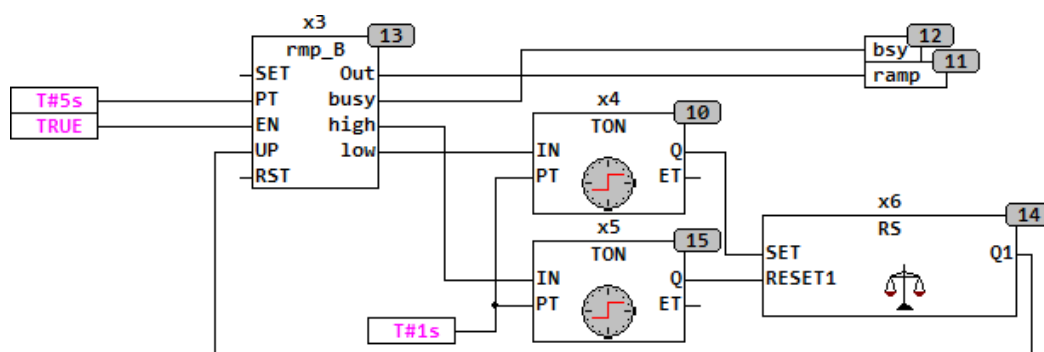
Type	Funktionsblock
Input	SET : BOOL (Set-Eingang) PT : TIME (Dauer einer Rampe 0..255) EN : BOOL (Freigabeeingang) UP : BOOL (Richtung UP=TRUE bedeutet Up) RST : BOOL (Reset-Eingang)
Output	OUT : Byte (Ausgangssignal) BUSY : BOOL (TRUE, wenn Rampe läuft) HIGH : BOOL (Maximaler Ausgangswert ist erreicht) LOW : BOOL (Minimaler Ausgangswert ist erreicht)

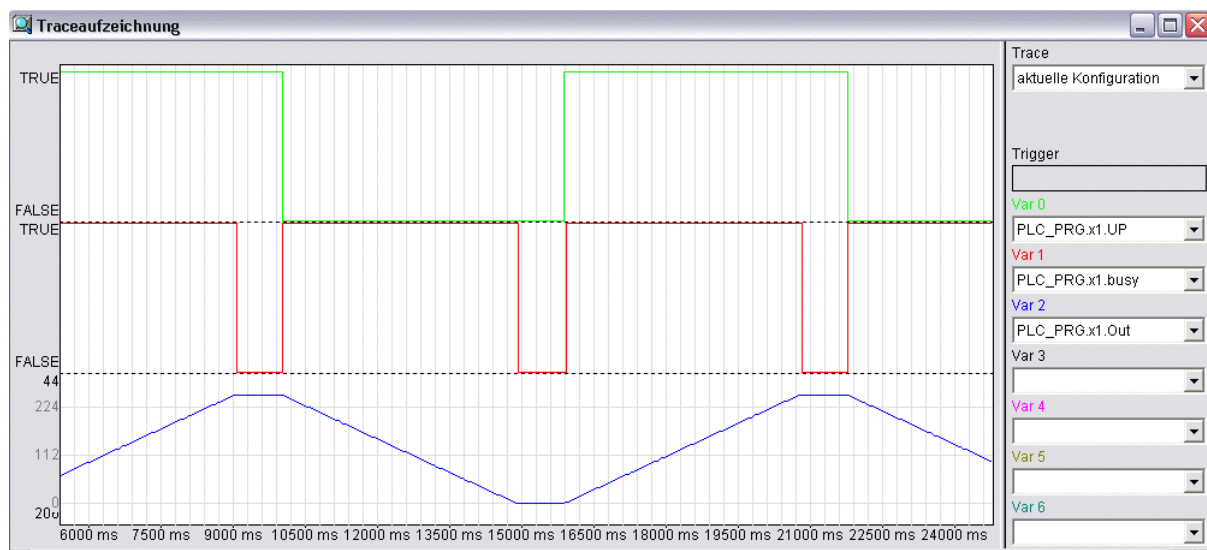


RMP_B ist ein Rampengenerator mit 8 Bit (1 Byte) Auflösung. Die Rampe von 0..255 wird in maximal 255 Schritte unterteilt und in einer Zeit von PT einmal komplett durchlaufen. Ein Freigabesignal EN schaltet den Rampengenerator an oder aus. Ein asynchroner Reset setzt jederzeit den Ausgang auf 0 und ein Impuls am Set-Eingang setzt den Ausgang auf 255. Mit einem UD-Eingang kann die Richtung AUF (UD = TRUE) oder Ab (UD = FALSE) vorgegeben werden. Der Ausgang BUSY = TRUE zeigt an, dass eine Rampe aktiv ist. BUSY = FALSE bedeutet der Ausgang ist stabil. Die Ausgänge HIGH und LOW werden TRUE wenn der Ausgang OUT das untere oder obere Limit (0 bzw. 255) erreicht hat.

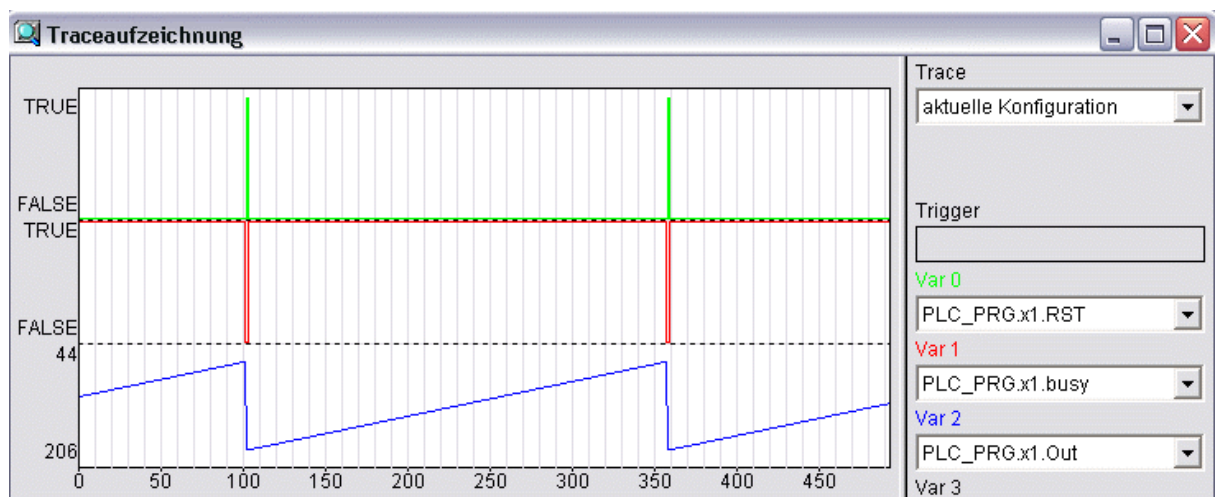
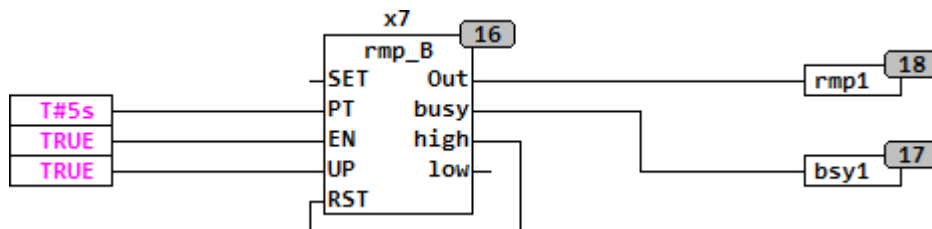
Beim festlegen von PT ist zu beachten, dass eine SPS mit 5ms Zykluszeit $256 \cdot 5 = 1275$ Millisekunden für eine Rampe benötigt. Wird die Zeit PT kürzer als die Zykluszeit mal 256 gewählt, wird die Flanke in entsprechend größere Sprünge übersetzt. Die Rampe wird in diesen Fall aus weniger als 256 Schritten je Zyklus zusammengesetzt. PT darf T#0s sein, dann schaltet der Ausgang zwischen Minimal- und Maximal-Wert hin und her.

Das folgende Beispiel zeigt eine Anwendung von RMP_B. Die Ausgänge HIGH und LOW Triggern die beiden TON (X4, X5) jeweils 1 Sekunde verzögert und schalten über das RS Flip-Flop (X6) den UP-Eingang des Rampengenerators um. Das Ergebnis ist eine Rampe von 5 Sekunden, gefolgt von einer Pause von 1 Sekunde und dann die umgekehrte Rampe von 5 Sekunden und wieder eine Pause von 1 Sekunde. In der Traceaufzeichnung ist der Verlauf der Signale zu erkennen.



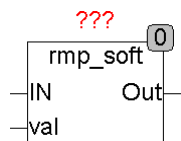


Ein weiteres Beispiel zeigt den Einsatz von RMP_B als Sägezahngenerator.



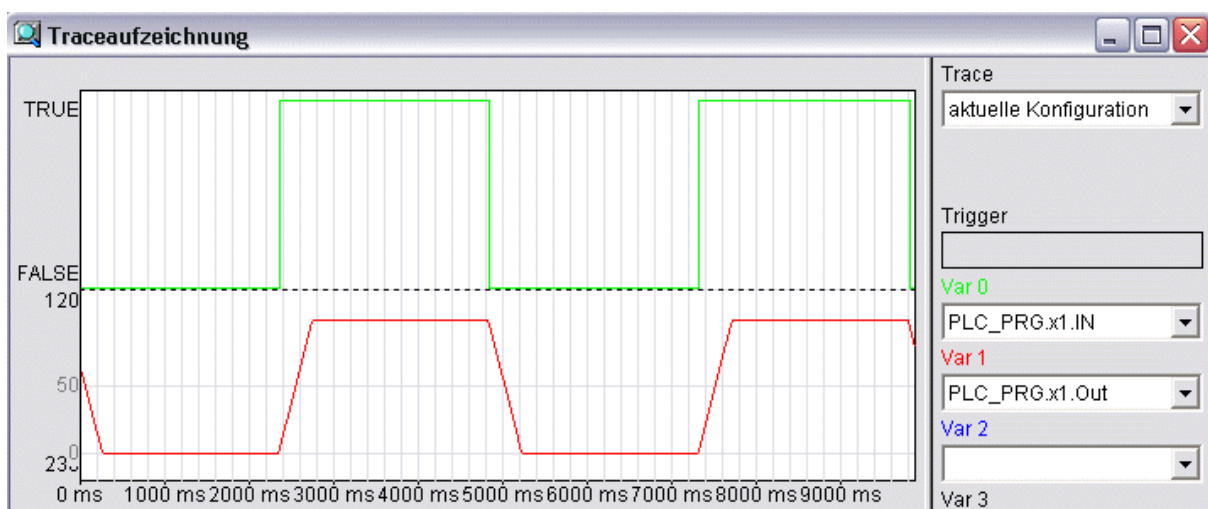
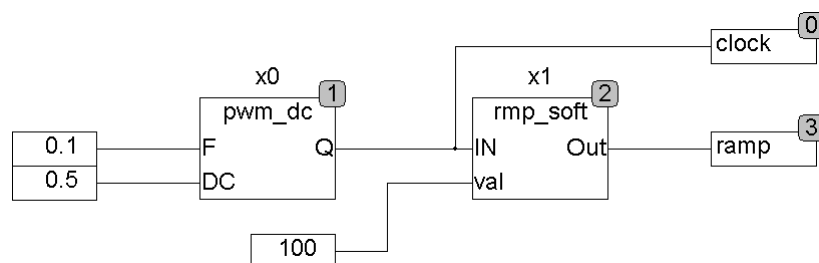
17.9 RMP_SOFT

Type	Funktionsblock
Input	IN : BOOL () VAL : Byte (Maximaler Ausgangswert)
Config	PT_ON : TIME (Anstiegszeit, Default ist 100 ms) PT_OFF : TIME (Abfallzeit, Default ist 100 ms)
Output	OUT : Byte (Ausgangssignal)



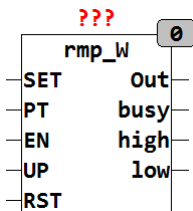
RMP_SOFT glättet die Rampen eines Eingangssignals IN. Das Signal Out folgt dem Eingangssignal IN, indem es bei steigender Flanke von IN eine Rampe von 0 .. VAL am Ausgang OUT erzeugt und bei fallender Flanke am Eingang eine Rampe von VAL .. 0 am Ausgang Out erzeugt. Die Anstiegszeit und Abfallzeit der Rampen sind durch Setup-Parameter im Modul RMP_SOFT definiert. Die Setup-Zeit PT_ON Gibt an, wie lange die Rampe von 0..255 dauert. Eine Rampe die durch VAL begrenzt ist, ist entsprechend kürzer. PT_OFF definiert entsprechend die fallende Rampe.

Beispiel:



17.10 RMP_W

Type	Funktionsblock
Input	SET : BOOL (Set Eingang) PT : TIME (Dauer einer Rampe 0..65535) EN : BOOL (Freigabeeingang) UP : BOOL (Richtung UP=TRUE bedeutet Up) RST : BOOL (Reset Eingang)
Output	OUT : Byte (Ausgangssignal) BUSY : BOOL (TRUE, wenn Rampe läuft) HIGH : BOOL (Maximaler Ausgangswert ist erreicht) LOW : BOOL (Minimaler Ausgangswert ist erreicht)



RMP_W ist ein Rampengenerator mit 16 Bit (2 Byte) Auflösung. Die Rampe von 0..65535 wird in maximal 65536 Schritte unterteilt und in einer Zeit von PT einmal komplett durchlaufen. Ein Freigabesignal EN schaltet den Rampengenerator an oder aus. Ein asynchroner Reset setzt jederzeit den Ausgang auf 0 und ein Impuls am Set-Eingang setzt den Ausgang auf 65535. Mit einem UD-Eingang kann die Richtung AUF (UD = TRUE) oder Ab (UD = FALSE) vorgegeben werden. Der Ausgang BUSY = TRUE zeigt an, dass eine Rampe aktiv ist. BUSY = FALSE bedeutet der Ausgang ist stabil. Die Ausgänge HIGH und LOW werden TRUE wenn der Ausgang OUT das untere oder obere Limit (0 bzw. 65535) erreicht hat.

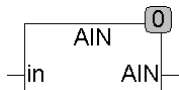
Beim festlegen von PT ist zu beachten, dass eine SPS mit 5ms Zykluszeit $65536 \cdot 5 = 327$ Sekunden für eine Rampe benötigt. Wird die Zeit PT kürzer als die Zykluszeit mal 65536 gewählt, wird die Flanke in entsprechend größere Sprünge übersetzt. Die Rampe wird in diesen Fall aus weniger als 65536 Schritten je Zyklus zusammengesetzt. PT darf T#0s sein, dann schaltet der Ausgang zwischen Minimal- und Maximal-Wert hin und her.

Eine detaillierte Beschreibung finden Sie beim Modul RMP_B. Die Funktion ist absolut identisch mit der Ausnahme, dass der Ausgang OUT 8 Bit anstelle von 16 Bit weit ist.

18 Signalverarbeitung

18.1 AIN

Type	Funktion
Input	IN : DWORD (Eingang vom A/D Wandler)
Output	REAL (Ausgangswert)
Config	BITS : Byte (Anzahl der Bits, 16 für ein komplettes Wort) SIGN : Byte (Sign Bit, 15 für Bit 15) LOW : REAL (kleinster Wert des Ausgangs) HIGH : REAL (größter Wert des Ausgangs)



Analoge Eingänge von A/D Wandlern liefern in der Regel ein WORD (16 Bit) oder DWORD (32 Bit), wobei sie selbst meist nicht 16 Bit oder 32 Bit Auflösung besitzen. Weiterhin digitalisieren A/D Wandler einen festen Eingangsbereich (z B. -10 .. + 10 V), was zum Beispiel mit den digitalen Werten 0 .. 65535 (Bei 16 Bit) repräsentiert wird. Die Funktion AIN wird durch Setup-Parameter Konfiguriert und rechnet die Ausgangswerte des A/D Wandlers entsprechend um, sodass nach dem Modul AIN ein REAL-Wert zur Verfügung steht, der mit dem echten gemessenen Wert übereinstimmt. Weiterhin kann das Modul ein Sign-Bit an beliebiger Stelle extrahieren und umrechnen. Durch einen Doppelklick auf den Baustein können mehrere Setup-Variablen gesetzt werden. Bits definiert wie viele Bits des Eingangs-DWORD verarbeitet werden sollen. Für einen 12 Bit Wandler ist dieser Wert 12. Es werden dann nur die Bits 0 – 11 ausgewertet. Sign definiert, ob ein Vorzeichenbit vorhanden ist und wo im Eingangswort es zu finden ist. Sign=255 bedeutet, dass kein Vorzeichenbit vorhanden ist und 15 bedeutet, dass Bit 15 im DWORD das Vorzeichen enthält. Der Vorgabewert für SIGN ist 255. LOW und HIGH definieren den kleinsten und höchsten Ausgangswert. Ist ein Sign-Bit definiert (SIGN < 255), dann müssen LOW und HIGH positiv sein. Ohne Sign-Bit können Sie sowohl positiv als auch negativ sein.

Beispiel:

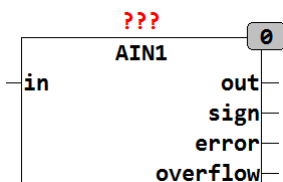
Ein 12 Bit A/D Wandler ohne Vorzeichen und Eingangsbereich 0 – 10 wird wie folgt definiert: Bits=12, Sign=255, LOW=0, HIGH=10.

Ein 14 Bit A/D Wandler mit Vorzeichen in Bit 14 und Eingangsbereich -10 - +10 wird folgendermaßen definiert: Bits=14, Sign=14, LOW=0, HIGH=+10.

Ein 24 Bit A/D Wandler ohne Vorzeichen und Eingangsbereich -10 - +10 wird folgendermaßen definiert: Bits=24, Sign=255, LOW=-10, HIGH=+10.

18.2 AIN1

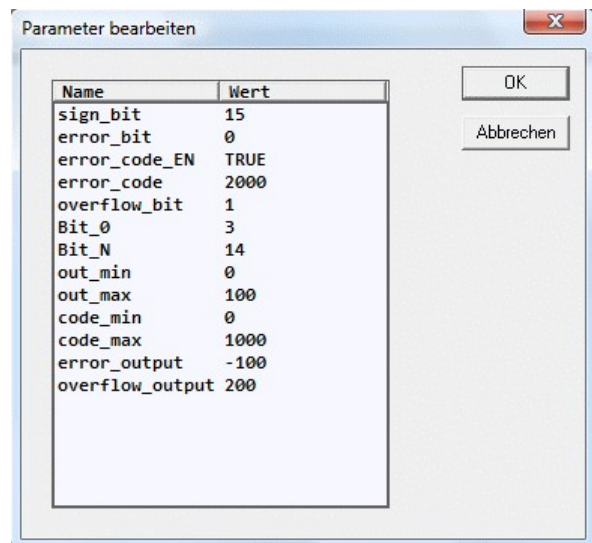
Type	Funktionsblock
Input	IN : DWORD (Eingang vom A/D Wandler)
Output	OUT : REAL (Ausgangswert)
Config	SIGN_BIT : INT (Bitnummer des Vorzeichens) ERROR_BIT : INT (Bitnummer des Fehlerbits) ERROR_CODE_EN : BOOL (Auswertung des Error Codes Ein) ERROR_CODE : DWORD (Fehlercode des Eingangs IN) OVERFLOW_BIT : INT (Bitnummer des Overflow Bits) OVERFLOW_CODE_EN : BOOL (Auswertung des Overflow Codes Ein) OVERFLOW_CODE : DWORD (Overflow Code des Eingangs IN) BIT_0 : INT (Bitnummer des niederwertigsten Datenbits) BIT_N : INT (Bitnummer des höchstwertigsten Datenbits) OUT_MIN : REAL (Ausgangswert bei CODE_MIN) OUT_MAX : REAL (Ausgangswert bei CODE_MAX) CODE_MIN : DWORD (Minimaler Eingangswert) CODE_MAX : DWORD (Maximaler Eingangswert) ERROR_OUTPUT : REAL (Ausgangswert der bei ERROR) OVERFLOW_OUTPUT: REAL (Ausgangswert der Bei OVERFLOW)



AIN1 setzt den Digitalen Ausgangswert eines A/D Wandlers in einen dem Messwert entsprechenden REAL Wert um. Der Baustein kann mittels Config Variablen auf die unterschiedlichsten Digitalwandler angepasst werden.

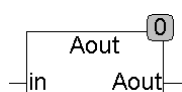
Ein SIGN_BIT legt fest an welchen Bit der D/A Wandler das Vorzeichen übermittelt. Wird diese Variable nicht definiert oder auf einen Wert größer 31 gesetzt so wird kein Vorzeichen ausgewertet. Der Inhalt des SIGN_BIT wird am Ausgang SIGN angezeigt. Wird ein ERROR_BIT spezifiziert, wird der Inhalt des Error Bits am Ausgang ERROR angezeigt. Manche A/D Wandler liefern anstatt eines Error Bits einen festgelegten Ausgangswert der außerhalb des spezifizierten Messbereichs liegt und Signalisieren dadurch einen Fehler. Mit der Config Variablen ERROR_CODE wird der entsprechende Error Code spezifiziert und mit ERROR_CODE_EN wird die Auswertung

des ERROR_CODE festgelegt. Wenn ERROR = TRUE wird am Ausgang OUT der Wert ERROR_OUTPUT ausgegeben. Mithilfe des OVERFLOW_BITS wird eine Bereichsüberschreitung des D/A Wandlers signalisiert und am Ausgang OVERFLOW ausgegeben. Mithilfe der Config Variablen OVERFLOW_CODE_EN und OVERFLOW_CODE kann ein bestimmter Code am Eingang IN abgefragt werden und bei Auftreten dieses Codes das Overflow Bit gesetzt werden. Zusätzlich zum OVERFLOW_BIT kann mittels CODE_MIN und CODE_MAX ein zulässiger Bereich für die Eingangsdaten spezifiziert werden. Wird dieser Bereich über- beziehungsweise unterschritten wird ebenfalls der OVERFLOW Ausgang gesetzt. Bei einem Überlauf wird am Ausgang OUT der Wert OVERFLOW_OUTPUT ausgegeben. Die Config Variablen BIT_0 und BIT_N legen fest wie der D/A Wandler den Messwert bereitstellt. Mit Bit_0 wird festgelegt bei welchem Bit das Datenwort beginnt und mit BIT_N an welchem Bit das Datenwort endet. Im obigen Beispiel wird das Datenwort von Bit 3 bis Bit 14 übertragen (Bit 3 = Bit 0 des Datenwortes und Bit 14 = Bit 12 des Datenwortes). Das empfangene Datenwort wird entsprechend den Config Variablen CODE_MIN, CODE_MAX und OUT_MIN, OUT_MAX umgerechnet und falls ein Vorzeichen vorhanden ist wird wenn SIGN = TRUE der Ausgangswert an OUT invertiert.



18.3 AOUT

Type	Funktion
Input	IN : REAL (Eingangswert)
Output	DWORD (Ausgangswort zum A/D Wandler)
Config	BITS : Byte (Anzahl der Bits, 16 für ein komplettes Wort)
	SIGN : Byte (Sign Bit, 15 für Bit 15)
	LOW : REAL (kleinster Wert des Eingangs)
	HIGH : REAL (größter Wert des Eingangs)



Eingänge von D/A Wandlern benötigen in der Regel ein WORD (16 Bit) oder DWORD (32 Bit), wobei Sie selbst meist nicht 16 Bit oder 32 Bit Auflösung haben. Weiterhin erzeugen D/A Wandler einen festen Ausgangsbereich (z B. -10 .. + 10 V) was zum Beispiel mit den digitalen Werten 0 .. 65535 (Bei 16 Bit) repräsentiert wird. Die Funktion AOUT wird durch Setup-Parameter konfiguriert und rechnet die Eingangswerte (IN) entsprechend um, sodass nach dem Modul AOUT ein digitaler Wert zur Verfügung steht, der am Ausgang des D/A Wandlers einen Wert erzeugt, der mit dem REAL-Wert IN übereinstimmt. Weiterhin kann das Modul ein Sign-Bit an beliebiger Stelle einfügen falls der D/A Wandler ein Sign-Bit benötigt. Durch einen Doppelklick auf den Baustein können mehrere Setup-Variablen gesetzt werden. Bits definiert wie viele Bits der D/A Wandler verarbeiten kann. Für einen 12 Bit Wandler ist dieser Wert 12. Es werden dann nur die Bits 0 – 11 belegt. Sign definiert, ob ein Vorzeichenbit benötigt wird und wo im Ausgangs-DWORD es zu platzieren ist. Sign=255 bedeutet, dass kein Vorzeichenbit benötigt wird, und 15 bedeutet das Bit 15 im DWORD das Vorzeichen enthält. LOW und HIGH definieren den kleinsten und höchsten Eingangswert. Ist ein Sign-Bit definiert, so müssen LOW und HIGH positiv sein. Ohne Sign-Bit können Sie sowohl positiv als auch negativ sein.

Beispiele:

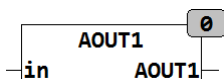
Ein 12 Bit D/A Wandler ohne Vorzeichen und Ausgangsbereich 0 – 10 wird wie folgt definiert: Bits=12, Sign=255, LOW=0, HIGH=10.

Ein 14 Bit D/A Wandler mit Vorzeichen in Bit 14 und Ausgangsbereich -10 - +10 wird folgendermaßen definiert: Bits=14, Sign=14, LOW=0, HIGH=+10.

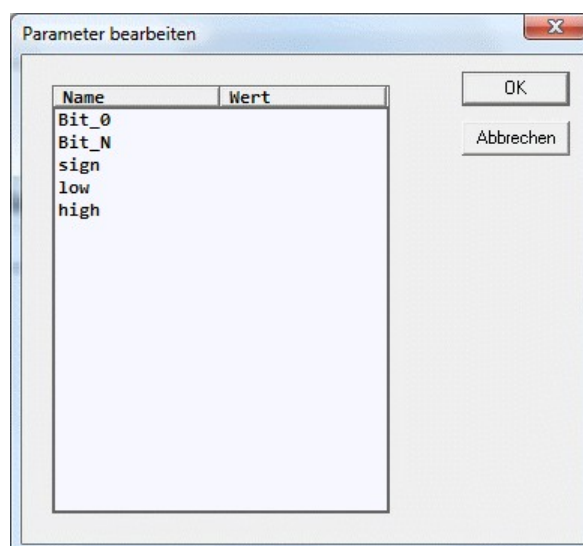
Ein 24 Bit D/A Wandler ohne Vorzeichen und Ausgangsbereich -10 - +10 wird folgendermaßen definiert: Bits=24, Sign=255, LOW=-10, HIGH=+10.

18.4 AOUT1

Type	Funktion
Input	IN : REAL (Eingangswert)
Output	DWORD (Ausgangswort zum A/D Wandler)
Config	BIT_0 : INT (Bitnummer des niederwertigsten Bits des Datenwortes)
	BIT_N : INT (Bitnummer des höchstwertigsten Bits des Datenwortes)
	SIGN : INT (Sign Bit, 15 für Bit 15)
	LOW : REAL (kleinster Wert des Eingangs)
	HIGH : REAL (größter Wert des Eingangs)

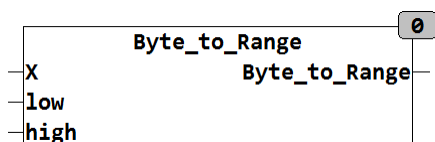


AOUT1 erzeugt aus dem REAL Eingangswert IN einen digitalen Ausgangswert für D/A Wandler oder andere Ausgangsbausteine die Digitale Daten verarbeiten. Mittels Config Variablen kann der digitale Ausgangswert an verschiedenste Bedürfnisse angepasst werden. Der Eingangswert IN wird mittels den Angaben in LOW und HIGH sowie der mit BIT_0 und BIT_N spezifizierten Länge Des Datenwortes umgewandelt und am Ausgang bereitgestellt. BIT_0 spezifiziert die Position des niderwertigsten (Bit0) Datenbits in den Ausgangsdaten und BIT_N spezifiziert die Position des höchstwertigen Datenbits in den Ausgangsdaten. Die Länge des Datenbereichs wird durch $\text{BIT_N} - \text{BIT_0} + 1$ automatisch errechnet. Wenn mit SIGN die Position eines Vorzeichen Bits angegeben wird so wird das Vorzeichen aus dem Eingangswert auf die spezifizierte Position von SIGN in den Ausgangsdaten kopiert.



18.5 BYTE_TO_RANGE

Type	Funktion
Input	IN : BYTE (Eingangswert)
Output	REAL (Ausgangswert)



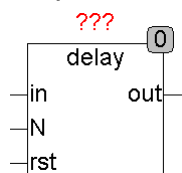
BYTE_TO_RANGE wandelt einen BYTE Wert in einen REAL. Ein Eingangswert von 0 entspricht dabei dem REAL Wert von LOW und eine Eingangswert von 255 entspricht dem Eingangswert von HIGH.

Um einen BYTE Wert von 0..255 in einen Prozentwert von 0..100 zu Wandeln wird der Baustein wie folgt aufgerufen:

BYTE_TO_RANGE(X,0,100)

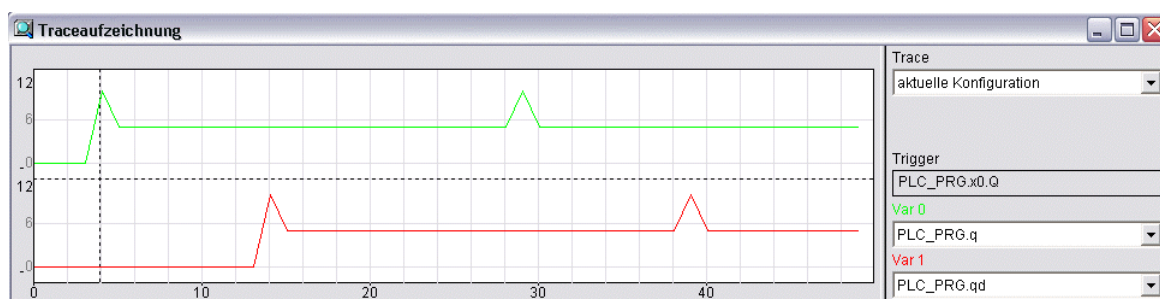
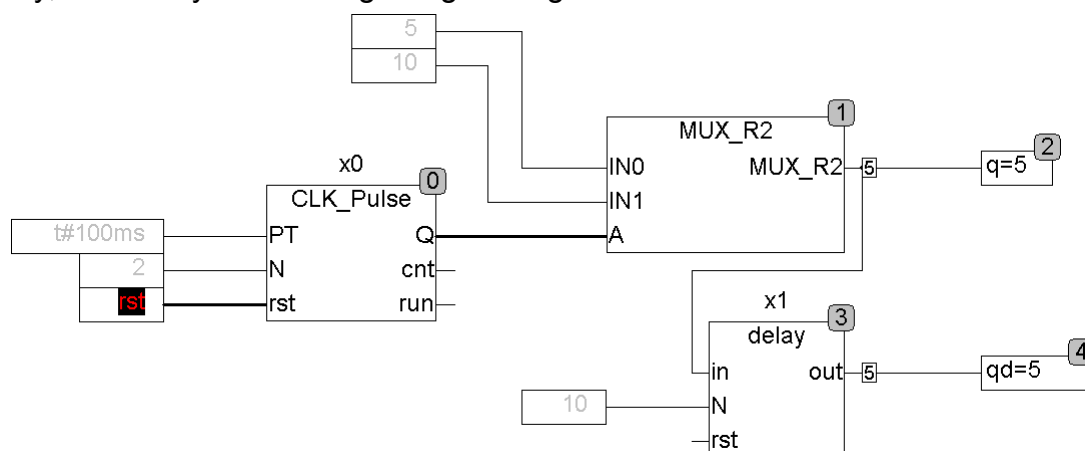
18.6 DELAY

Type	Funktionsblock
Input	IN : REAL (Eingangswert) N : INT (Anzahl der Verzögerungs Zyklen) RST : BOOL (asynchroner Reset)
Output	OUT : REAL (verzögerter Ausgangswert)



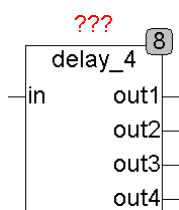
DELAY verzögert ein Eingangssignal (in) um N Zyklen. Der Eingang Reset ist asynchron und kann jederzeit den Delay Puffer löschen.

Das Beispiel zeigt einen Generator, der Impulse von 5 nach 10 erzeugt und ein Delay, das 10 Zyklen Verzögerung erzeugt.



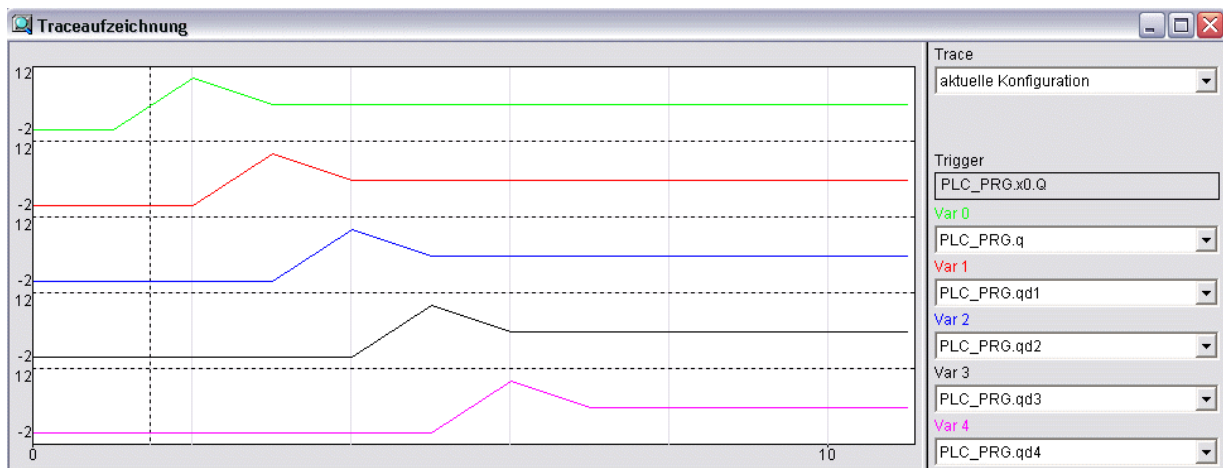
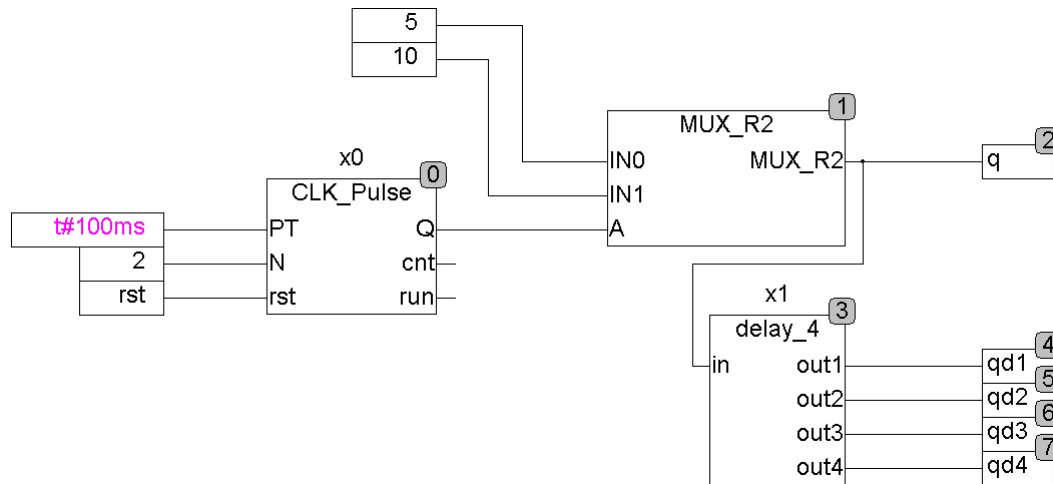
18.7 DELAY_4

Type Funktionsblock
 Input IN : REAL (Eingangswert)
 Output OUT1 : REAL (um 1 Zyklus verzögerter Ausgangswert)
 OUT2 : REAL (um 2 Zyklen verzögerter Ausgangswert)
 OUT3 : REAL (um 3 Zyklen verzögerter Ausgangswert)
 OUT4 : REAL (um 4 Zyklen verzögerter Ausgangswert)



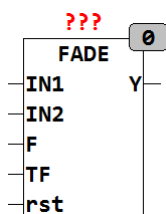
DELAY_4 verzögert ein Eingangssignal um maximal 4 Zyklen. An den Ausgängen Out1..4 stehen die letzten 4 Werte zur Verfügung. Out1 ist um einen Zyklus verzögert, Out2 um 2 Zyklen, Out3 um 3 Zyklen und Out4 um 4 Zyklen.

Beispiel:



18.8 FADE

Type	Funktionsblock
Input	IN1 : REAL (Eingangswert 1) IN2 : REAL (Eingangswert 2) F : REAL (Auswahl Eingang TRUE = IN2) TF : TIME (Überblendzeit) RST : BOOL (Asynchroner Reset)
Output	Y : REAL (Ausgangswert)



FADE wird benutzt um zwischen 2 Eingängen IN1 und IN2 mit einem weichen Übergang umzuschalten. Die Umschaltzeit wird dabei mit TF angegeben. Ein asynchroner Reset (RST) setzt den Baustein ohne Verzögerung auf IN1 wenn F = FALSE oder auf IN2 wenn F = TRUE. Ein Umschaltvorgang wird durch eine Wertänderung an F ausgelöst. Anschließend wird innerhalb der Zeit TF zwischen den beiden Eingängen umgeschaltet. Die Umschaltung erfolgt indem während der Umschaltzeit die beiden Eingänge gemischt werden. Am Anfang der Umschaltzeit stehen am Ausgang 0% des neuen Wertes und 100% des alten Wertes an. nach der halben Umschaltzeit (TF/2) ist der Ausgang jeweils 50% der beiden Eingangswerte ($Y = in1 \cdot 0.5 + in2 \cdot 0.5$). nach Ablauf der Zeit TF liegt dann am Ausgang der neue Wert zu 100% an.

Während der Umschaltzeit beträgt der Ausgang Y:

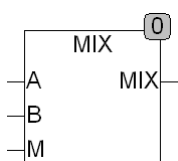
$$Y = TU/TF * IN1 + (1 - TU/TF) * IN2.$$

TU ist dabei die seit Beginn der Umschaltung vergangene Zeit.

Da der Ausgang von FADE dynamisch berechnet wird kann der Baustein auch zur Umschaltung von dynamischen Signalen verwendet werden. Die Umschaltung wird in bis zu 65535 Stufen eingeteilt, die jedoch durch die Zykluszeit der SPS begrenzt werden können. Eine SPS mit einer Zykluszeit von 10ms und einer TF von einer Sekunde wird lediglich in $1s/10ms = 100$ Stufen Umschalten.

18.9 MIX

Type	Funktion : REAL
Input	A : REAL (Eingangswert 1) B : REAL (Eingangswert 2) M : REAL (Mischungsverhältnis)
Output	REAL (Wert aus dem Mischungsverhältnis M zwischen A und B)

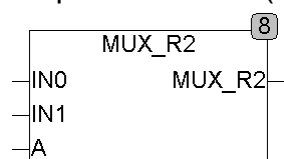


MIX stellt am Ausgang einen mit dem Mischungsverhältnis M gemischten Wert aus A und B zur Verfügung. Der Eingang M gibt den Anteil vom B in Bereich 0..1 an.

$$\text{MIX} = (M-1)*A + M*B$$

18.10 MUX_R2

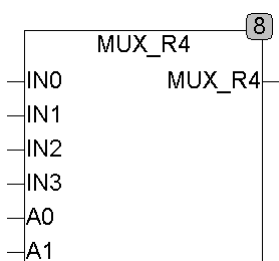
Type	Funktion
Input	IN0 : REAL (Eingangswert 0) IN1 : REAL (Eingangswert 1) A : BOOL (Adresseingang)
Output	REAL (IN0 wenn A=0, IN1 wenn A=1)



MUX_R2 wählt einen von 2 Eingangswerten aus. Die Funktion liefert den Wert von IN0 zurück, wenn A=0 und den Wert von IN1, wenn A=1.

18.11 MUX_R4

Type	Funktion
Input	IN0 : REAL (Eingangswert 0) IN1 : REAL (Eingangswert 1) IN2 : REAL (Eingangswert 0) IN3 : REAL (Eingangswert 1) A0 : BOOL (Adresseingang Bit 0) A1 : BOOL (Adresseingang Bit 1)
Output	REAL (IN0 wenn A0=0 und A1=0, IN3 wenn A0=1 und A3=1)



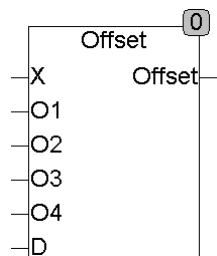
MUX_R4 wählt einen von 4 Eingangswerten aus.

Logische Verknüpfung:	IN0 wenn A0=0 & A1=0,
	IN1 wenn A0=1 & A1=0;
	IN2 wenn A0=0 & A1=1;
	IN3 wenn A0=1 & A1=1;

18.12 OFFSET

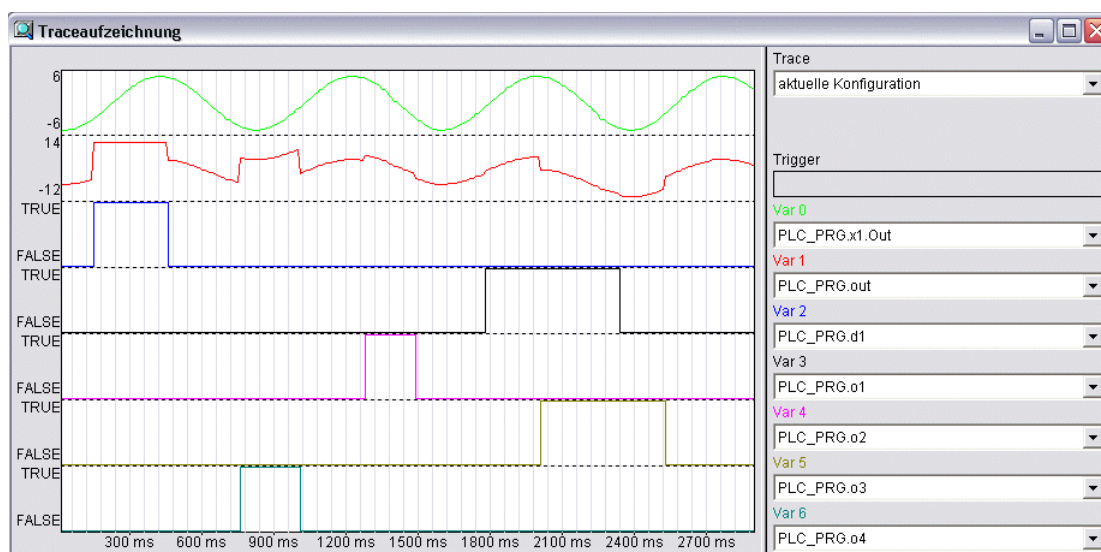
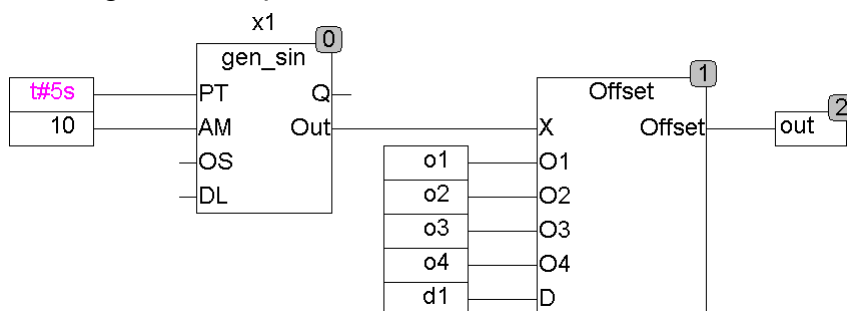
Type	Funktion
Input	X : REAL (Eingangssignal) O1 : BOOL (Enable Offset 1) O2 : BOOL (Enable Offset 2) O3 : BOOL (Enable Offset 3) D : BOOL (Enable Default)
Output	REAL (Ausgangswert mit Offset)

Config Offset_1 : REAL (Offset der addiert wird, wenn O1=TRUE)
 Offset_2 : REAL (Offset der addiert wird, wenn O2=TRUE)
 Offset_3 : REAL (Offset der addiert wird, wenn O3=TRUE)
 Offset_4 : REAL (Offset der addiert wird, wenn O4=TRUE)
 Default : REAL (Wird anstelle von X verwendet, wenn D=TRUE)



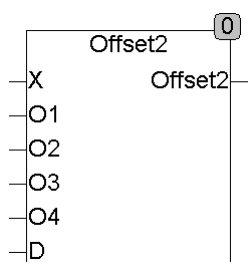
Die Funktion Offset addiert verschiedene Offsets zu einem Eingangssignal abhängig vom Binären Wert an O1 .. O4. Die Offsets können einzeln oder gleichzeitig addiert werden. Mit dem Eingang D kann ein Default-Wert anstelle des Eingangs X auf den Addierer geschaltet werden. Die Offsets und der Default-Wert werden über Config-Variablen definiert.

Das folgende Beispiel verdeutlicht die Funktionsweise von Offset:



18.13 OFFSET2

Type	Funktion
Input	X : REAL (Eingangssignal) O1 : REAL (Enable Offset 1) O2 : REAL (Enable Offset 2) O3 : REAL (Enable Offset 3) D : BOOL (Enable Default)
Output	REAL (Ausgangswert mit Offset)
Config	Offset_1 : REAL (Offset der addiert wird, wenn O1=TRUE) Offset_2 : REAL (Offset der addiert wird, wenn O2=TRUE) Offset_3 : REAL (Offset der addiert wird, wenn O3=TRUE) Offset_4 : REAL (Offset der addiert wird, wenn O4=TRUE) DEFAULT : REAL (Wird anstelle von X verwendet, wenn D=TRUE)

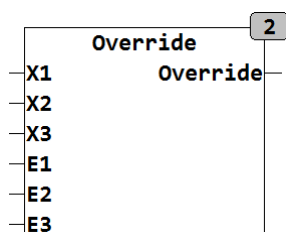


Die Funktion OFFSET2 addiert einen Offset zu einem Eingangssignal abhängig vom Binären Wert an O1 .. O4. Werden mehrere Offsets gleichzeitig selektiert, so wird der Offset mit der höchsten Nummer addiert und die anderen ignoriert. Werden O1 und O3 gleichzeitig TRUE, so wird nur Offset_3 addiert und nicht Offset_1. Mit dem Eingang D kann ein Default-Wert anstelle des Eingangs X auf den Addierer geschaltet werden. Die Offsets und der Defaultwert werden über Config-Variablen definiert.

Weitere Erläuterungen und ein Beispiel finden Sie unter Offset, welches sehr ähnliche Funktion aufweist. Offset2 addiert nur jeweils einen (den mit der höchsten Nummer) Offset, während Offset alle selektierten gleichzeitig addiert.

18.14 OVERRIDE

Type	Funktion
Input	X1 : REAL (Eingangssignal 1)
	X2 : REAL (Eingangssignal 2)
	X3 : REAL (Eingangssignal 3)
	E1 : BOOL (Enable Signal 1)
	E2 : BOOL (Enable Signal 2)
	E3 : BOOL (Enable Signal 3)
Output	REAL (Ausgangswert)



OVERRIDE liefert am Ausgang Y den Eingangswert (X1, X2, X3) dessen Absoluter Wert der größere von allen ist. Die Eingänge X1, X2 und X3 können jeder individuell mit den Eingängen E1, E2 und E3 freigeschaltet werden. wenn eines der Eingangssignale E1, E2 oder E3 auf FALSE steht wird der zugehörige Eingang X1, X2 oder X3 nicht berücksichtigt. Eine von vielen Anwendungsmöglichkeiten von OVERRIDE ist zum Beispiel die Abfrage von 3 Sensoren wobei der mit dem Höchsten Wert die anderen überschreibt. Mit den Eingängen E kann im Diagnosefall jeder Sensor einzeln abgefragt werden, oder ein defekter Sensor abgeschaltet werden.

Beispiel:

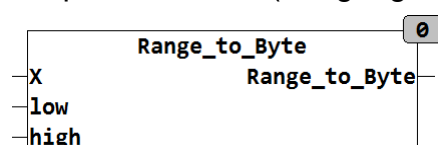
$\text{OVERRIDE}(10, -12, 11, \text{TRUE}, \text{TRUE}, \text{TRUE}) = -12$

$\text{OVERRIDE}(10, -12, 11, \text{TRUE}, \text{FALSE}, \text{TRUE}) = 11$

$\text{OVERRIDE}(10, -12, 11, \text{FALSE}, \text{FALSE}, \text{FALSE}) = 0$

18.15 RANGE_TO_BYTE

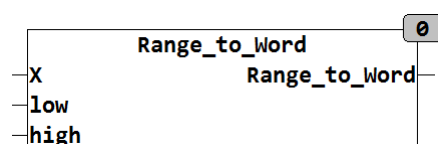
Type	Funktion
Input	X : REAL (Eingangswert)
	LOW : REAL (untere Bereichsgrenze)
	HIGH : REAL (Obere Bereichsgrenze)
Output	BYTE (Ausgangswert)



RANGE_TO_BYTE wandelt einen REAL Wert in einen BYTE Wert. Ein Eingangswert X der dem Wert LOW entspricht wird dabei in einen Ausgangswert von 0 gewandelt und ein Eingangswert X der dem Eingangswert HIGH entspricht wird in einen Ausgangswert von 255 gewandelt. Der Eingang X wird auf den Bereich von LOW bis HIGH begrenzt, ein Überlauf des BYTE Ausgangs kann deshalb nicht stattfinden.

18.16 RANGE_TO_WORD

Type	Funktion
Input	X : REAL (Eingangswert)
	LOW : REAL (untere Bereichsgrenze)
	HIGH : REAL (Obere Bereichsgrenze)
Output	WORD (Ausgangswert)



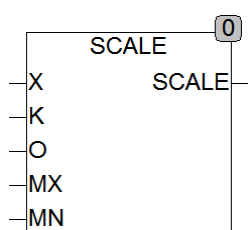
RANGE_TO_WORD wandelt einen REAL Wert in einen WORD Wert. Ein Eingangswert X der dem Wert LOW entspricht wird dabei in einen Ausgangswert von 0 gewandelt und ein Eingangswert X der dem Eingangswert HIGH entspricht wird in einen Ausgangswert von 65535 gewandelt. Der Eingang X wird auf den Bereich von LOW bis HIGH begrenzt, ein Überlauf des WORD Ausgangs kann deshalb nicht stattfinden.

18.17 SCALE

Type Funktion : REAL

Input X : Byte (Eingangswert)
 K : Byte (Multiplikator)
 O : REAL (Offset)
 MX : REAL (maximaler Ausgangswert)
 MN : REAL (minimaler Ausgangswert)

Output REAL (Ausgangswert)



SCALE Multipliziert den Eingang X mit K und addiert den Offset O. Der so errechnete Wert wird dann auf die Werte MN und MX begrenzt und das Ergebnis am Ausgang zur Verfügung gestellt.

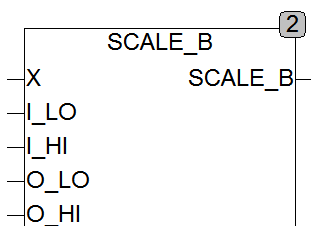
$$\text{SCALE} = \text{LIMIT}(\text{MN}, X * K + O, \text{MX})$$

18.18 SCALE_B

Type Funktion : REAL

Input X : DWORD (Eingangswert)
 I_LO : DWORD (Eingangswert min)
 I_HI : DWORD (Eingangswert max)
 O_LO : REAL (Ausgangswert min)
 O_HI : REAL (Ausgangswert max)

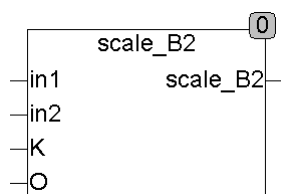
Output REAL (Ausgangswert)



SCALE_B skaliert einen Eingangswert BYTE und errechnet einen Ausgangswert in REAL. Der Eingangswert X wird dabei auf I_LO und I_HI begrenzt. SCALE_D(IN, 0, 255, 0, 100) skaliert einen Eingang mit 8 Bit Auflösung auf den Ausgang 0..100.

18.19 SCALE_B2

Type	Funktion : REAL
Input	IN1 : Byte (Eingangswert 1) IN2 : Byte (Eingangswert 2) K : REAL (Multiplikator) O : REAL (Offset)
Output	REAL (Ausgangswert)
Config	IN1_MIN : REAL (kleinster Wert für IN1) IN1_MAX : REAL (größter Wert für IN1) IN2_MIN : REAL (kleinster Wert für IN2) IN2_MAX : REAL (größter Wert für IN2)

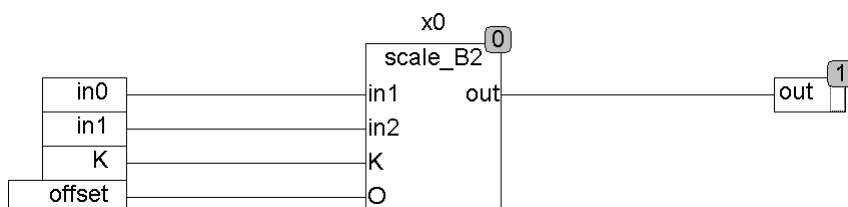


SCALE_B2 berechnet aus dem Eingangswert IN und den Setup-Werten IN_MIN und IN_MAX einen internen Wert, addiert alle internen Werte, multipliziert die Summe mit K und addiert den Offset O. Ein Eingangswert IN1=0 bedeutet IN1_MIN wird berücksichtigt, IN1=255 bedeutet IN1_MAX wird berücksichtigt. Wird K nicht beschaltet so ist der Multiplikator 1.

$$\text{Out} = \frac{(\text{in1} * (\text{IN1_MAX} - \text{IN1_MIN}) / 255 + \text{IN1_MIN}) + \text{in2} * (\text{IN2_MAX} - \text{IN2_MIN}) / 255 + \text{IN2_MIN}}{1} * K + O$$

SCALE_B2 kann zum Beispiel verwendet werden um Gesamtluftmengen in Lüftungsanlagen zu berechnen. Auch überall dort wo gesteuerte Mischer eingesetzt werden und die resultierende Gesamtmenge berechnet werden muss.

Beispiel:



IN0 ist eine Luftklappe, die die Luftmenge zwischen 100m³/h und 600m³/h für die Stellwerte in0 (0 – 255) regelt.

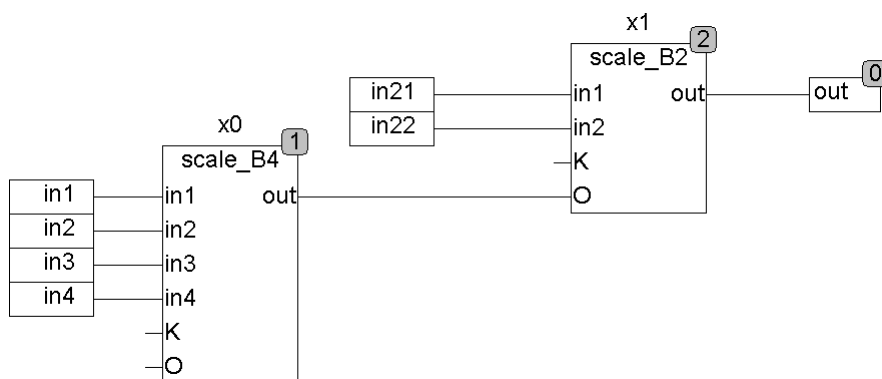
IN1 ist eine Abluftgerät, das die Abluftmenge von 0 m³/h bis 400 m³/h für die Stellwerte in1 von 0 – 255 regelt.

Die Setup-Werte für diese Anwendung sind: IN0_MIN = 100, IN0_MAX = 600, IN1_MIN = 0, IN1_MAX = -400.

Die resultierende Gesamtluftmenge bei K=1 und O=0 (kein Multiplikator und kein Offset) variiert dann von -300 (in0=0 und in1=255) bis +600 (in=255 und in1=0).

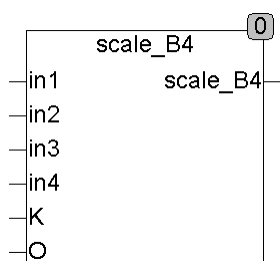
Für einen Eingangswert in0=128 (Klappe 50%) und IN1=128 (Lüfter auf 50%) ist der Ausgangswert $250 \text{ m}^3 - 200 \text{ m}^3 = 50 \text{ m}^3$.

Der Eingang Offset kann auch dazu benutzt werden um Bausteine zu kaskadieren.



18.20 SCALE_B4

Type	Funktion : REAL
Input	IN1.. IN4 : Byte (Eingangswerte) K : REAL (Multiplikator) O : REAL (Offset)
Output	REAL (Ausgangswert)
Config	IN1_MIN : REAL (kleinster Wert für IN1) IN1_MAX : REAL (größter Wert für IN1) IN2_MIN : REAL (kleinster Wert für IN2) IN2_MAX : REAL (größter Wert für IN2) IN3_MIN : REAL (kleinster Wert für IN3) IN3_MAX : REAL (größter Wert für IN3) IN4_MIN : REAL (kleinster Wert für IN4) IN4_MAX : REAL (größter Wert für IN4)



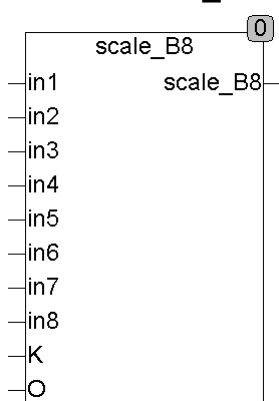
SCALE_B4 berechnet aus den Eingangswerten IN und den Setup-Werten IN_MIN und IN_MAX interne Werte, addiert alle internen Werte, multipliziert die Summe mit K und addiert den Offset O. Ein Eingangswert IN=0 bedeutet IN_MIN wird berücksichtigt, IN=255 bedeutet IN_MAX wird berücksichtigt. Wird K nicht beschaltet, so ist der Multiplikator 1.

$$\begin{aligned}
 \text{OUT} &= (\text{in1} * (\text{IN1_MAX} - \text{IN1_MIN}) / 255 + \text{IN1_MIN} \\
 &+ \text{in2} * (\text{IN2_MAX} - \text{IN2_MIN}) / 255 + \text{IN2_MIN} \\
 &+ \text{in3} * (\text{IN3_MAX} - \text{IN3_MIN}) / 255 + \text{IN3_MIN} \\
 &+ \text{in4} * (\text{IN4_MAX} - \text{IN4_MIN}) / 255 + \text{IN4_MIN}) * K + O
 \end{aligned}$$

SCALE_B4 kann zum Beispiel verwendet werden um Gesamtluftmengen in Lüftungsanlagen zu berechnen, oder überall dort, wo gesteuerte Mischer eingesetzt werden und die resultierende Gesamtmenge berechnet werden muss. Weitergehende Erläuterungen zur Funktionsweise finden Sie auch unter SCALE_B2.

18.21 SCALE_B8

Type	Funktion : REAL
Input	IN1 .. 8 : Byte (Eingangswerte) K : REAL (Multiplikator) O : REAL (Offset)
Output	REAL (Ausgangswert)
Config	IN_MIN : REAL (kleinster Wert für IN) IN_MAX : REAL (größter Wert für IN)



SCALE_B8 berechnet aus den Eingangswerten IN und den Setup-Werten IN_MIN und IN_MAX interne Werte, addiert alle internen Werte, multipliziert die Summe mit K und addiert den Offset O. Ein Eingangswert IN=0 bedeutet IN_MIN wird berücksichtigt, IN=255 bedeutet IN_MAX wird berücksichtigt. Wird K nicht beschaltet so ist der Multiplikator 1.

$$\begin{aligned}
 \text{OUT} &= (\text{in1} * (\text{IN1_MAX} - \text{IN1_MIN}) / 255 + \text{IN1_MIN} \\
 &+ \text{in2} * (\text{IN2_MAX} - \text{IN2_MIN}) / 255 + \text{IN2_MIN} \\
 &+ \text{in3} * (\text{IN3_MAX} - \text{IN3_MIN}) / 255 + \text{IN3_MIN} \\
 &+ \text{in4} * (\text{IN4_MAX} - \text{IN4_MIN}) / 255 + \text{IN4_MIN} \\
 &+ \text{in5} * (\text{IN5_MAX} - \text{IN5_MIN}) / 255 + \text{IN5_MIN} \\
 &+ \text{in6} * (\text{IN6_MAX} - \text{IN6_MIN}) / 255 + \text{IN6_MIN} \\
 &+ \text{in7} * (\text{IN7_MAX} - \text{IN7_MIN}) / 255 + \text{IN7_MIN} \\
 &+ \text{in8} * (\text{IN8_MAX} - \text{IN8_MIN}) / 255 + \text{IN8_MIN}) * K + O
 \end{aligned}$$

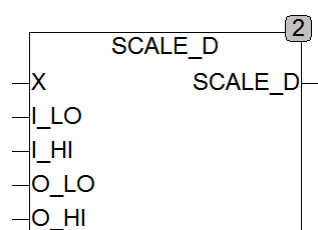
SCALE_B8 kann zum Beispiel verwendet werden um Gesamtluftmengen in Lüftungsanlagen zu berechnen, oder überall dort, wo gesteuerte Mischer eingesetzt werden und die resultierende Gesamtmenge berechnet werden muss. Weitergehende Erläuterungen zur Funktionsweise finden Sie auch unter SCALE_B2.

18.22 SCALE_D

Type Funktion : REAL

Input X : DWORD (Eingangswert)
 I_LO : DWORD (Eingangswert min)
 I_HI : DWORD (Eingangswert max)
 O_LO : REAL (Ausgangswert min)
 O_HI : REAL (Ausgangswert max)

Output REAL (Ausgangswert)



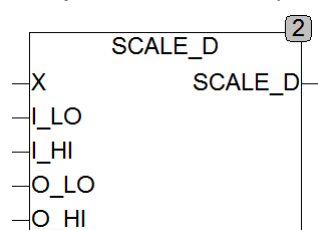
SCALE_D skaliert einen Eingangswert DWORD und errechnet einen Ausgangswert in REAL. Der Eingangswert X wird dabei auf I_LO und I_HI begrenzt. SCALE_D(IN, 0, 8191, 0, 100) skaliert einen Eingang mit 14 Bit Auflösung auf den Ausgang 0..100.

18.23 SCALE_R

Type Funktion : REAL

Input X : REAL (Eingangswert)
 I_LO : REAL (Eingangswert min)
 I_HI : REAL (Eingangswert max)
 O_LO : REAL (Ausgangswert min)
 O_HI : REAL (Ausgangswert max)

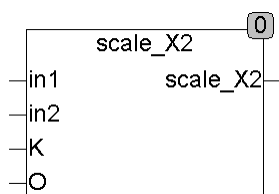
Output REAL (Ausgangswert)



SCALE_R skaliert einen Eingangswert REAL und errechnet einen Ausgangswert in REAL. Der Eingangswert X wird dabei auf I_LO und I_HI begrenzt. SCALE_D(IN, 4, 20, 0, 100) skaliert einen Eingang mit 4 .. 20 mA auf den Ausgang 0..100.

18.24 SCALE_X2

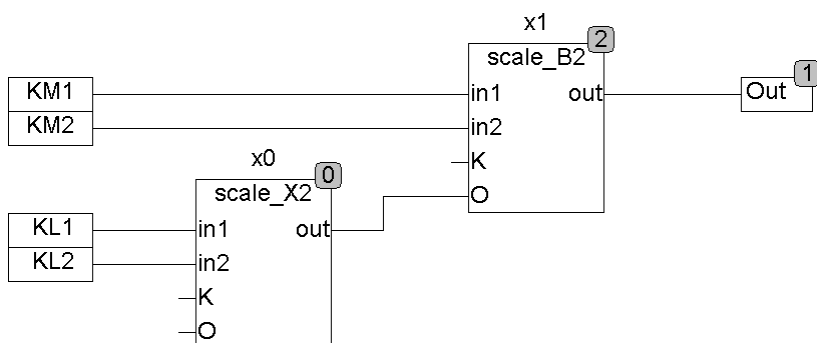
Type	Funktion
Input	IN1 .. 2 : BOOL (Eingangswerte)
	K : REAL (Multiplikator)
	O : REAL (Offset)
Output	REAL (Ausgangswert)
Config	IN_MIN : REAL (kleinster Wert für IN)
	IN_MAX : REAL (größter Wert für IN)



SCALE_X2 berechnet aus den Eingangswerten IN und den Setup-Werten IN_MIN und IN_MAX interne Werte, addiert alle internen Werte, multipliziert die Summe mit K und addiert den Offset O. Ein Eingangswert IN=FALSE bedeutet IN_MIN wird berücksichtigt, in=TRUE bedeutet IN_MAX wird berücksichtigt. Die Summe wird Mit K Multipliziert und Offset O addiert. Wird K nicht beschaltet so ist der Multiplikator 1.

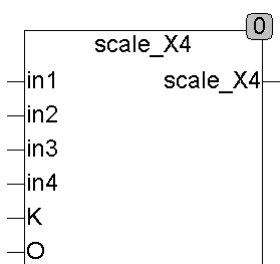
SCALE_X2 kann zum Beispiel verwendet werden um Gesamtluftmengen in Lüftungsanlagen zu berechnen, oder überall dort wo gesteuerte Klappen eingesetzt werden und die resultierende Gesamtmenge berechnet werden muss. Durch den Eingang Offset kann SCALE_X2 einfach mit den anderen SCALE Bausteinen kaskadiert werden.

Im folgenden Beispiel werden zwei motorische Klappen KM1 und km² mit 2 Ein/Aus Klappen KL1 und KL2 verknüpft und die resultierende Luftmenge berechnet.



18.25 SCALE_X4

Type	Funktion : REAL
Input	IN1 .. 4 : BOOL (Eingangswerte) K : REAL (Multiplikator) O : REAL (Offset)
Output	REAL (Ausgangswert)
Config	IN_MIN : REAL (kleinster Wert für IN) IN_MAX : REAL (größter Wert für IN)



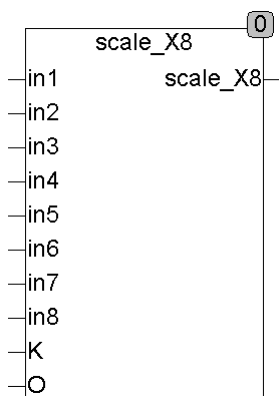
SCALE_X4 berechnet aus den Eingangswerten IN und den Setup-Werten IN_MIN und IN_MAX interne Werte, addiert alle internen Werte, multipliziert die Summe mit K und addiert den Offset O. Ein Eingangswert IN=FALSE bedeutet IN_MIN wird berücksichtigt, in=TRUE bedeutet IN_MAX wird berücksichtigt. Die Summe wird Mit K Multipliziert und Offset O addiert. Wird K nicht beschaltet so ist der Multiplikator 1.

SCALE_X4 kann zum Beispiel verwendet werden um Gesamtluftmengen in Lüftungsanlagen zu berechnen, oder überall dort wo gesteuerte Klappen eingesetzt werden und die resultierende Gesamtmenge berechnet werden muss. Durch den Ein-

gang Offset kann SCALE_X2 einfach mit den anderen SCALE Bausteinen kaskadiert werden. Weitere Erläuterungen und Beispiele finden Sie unter SCAE_X2.

18.26 SCALE_X8

Type	Funktion : REAL
Input	IN1 .. 8 : BOOL (Eingangswerte) K : REAL (Multiplikator) O : REAL (Offset)
Output	REAL (Ausgangswert)
Config	IN_MIN : REAL (kleinster Wert für IN) IN_MAX : REAL (größter Wert für IN)

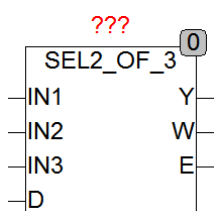


SCALE_X8 berechnet aus den Eingangswerten IN und den Setup-Werten IN_MIN und IN_MAX interne Werte, addiert alle internen Werte, multipliziert die Summe mit K und addiert den Offset O. Ein Eingangswert IN=FALSE bedeutet IN_MIN wird berücksichtigt, in=TRUE bedeutet IN_MAX wird berücksichtigt. Die Summe wird Mit K Multipliziert und Offset O addiert. Wird K nicht beschaltet so ist der Multiplikator 1.

SCALE_X8 kann zum Beispiel verwendet werden um Gesamtluftmengen in Lüftungsanlagen zu berechnen, oder überall dort wo gesteuerte Klappen eingesetzt werden und die resultierende Gesamtmenge berechnet werden muss. Durch den Eingang Offset kann SCALE_X2 einfach mit den anderen SCALE Bausteinen kaskadiert werden. Weitere Erläuterungen und Beispiele finden Sie unter SCAE_X2.

18.27 SEL2_OF_3

Type	Funktion : REAL
Input	IN1 : REAL (Eingangswert 1)
	IN2 : REAL (Eingangswert 2)
	IN3 : REAL (Eingangswert 3)
	D : REAL (Toleranzgrenze)
Output	Y : REAL (Ausgangswert)
	W : INT (Warnung)
	E : BOOL (Error Ausgang)



SEL2_OF_3 wertet 3 Eingänge (IN1 .. IN3) aus und prüft ob die Abweichung der Eingangswert kleiner oder gleich D ist. Der Mittelwert aus den 3 Eingängen wird am Ausgang Y ausgegeben. Die einzelnen Eingänge werden nur dann berücksichtigt wenn Sie nicht weiter als D von einem anderen Eingang entfernt liegen. Wird der Mittelwert nur von 2 Eingängen gebildet, so wird die Nummer des nicht berücksichtigten Eingangs am Ausgang W ausgegeben. Ist W = 0 werden alle 3 Eingänge berücksichtigt. Falls alle 3 Eingänge mehr als D voneinander abweichen wird der Ausgang W = 4 gesetzt und der Ausgang E = TRUE gesetzt. Der Ausgang Y wird in diesem Falle nicht verändert und bleibt auf dem letzten gültigen Wert stehen.

Eine typische Anwendung für den Baustein ist die Erfassung von 3 Sensoren die dieselbe Prozessgröße Messen um zum Beispiel Messfehler durch unterschiedliche Erfassung oder Drahtbruch zu verringern.

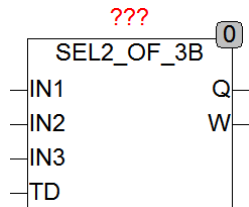
18.28 SEL2_OF_3B

Type	Funktion : BOOL
Input	IN1 : BOOL (Eingang 1)
	IN2 : BOOL (Eingang 2)
	IN3 : BOOL (Eingang 3)

TD : TIME (Delay für Ausgang W)

Output Q : BOOL (Ausgang)

W : BOOL (Warnmeldung)



SEL2_OF_3B wertet 3 redundante Binäre Eingänge aus und liefert am Ausgang Q den Wert den mindestens 2 der 3 Eingänge anliegen haben. Falls einer der 3 Eingänge einen anderen Wert als die beiden anderen hat wird der Ausgang W als Warnung gesetzt. Damit bei nicht exakt Zeitgleichen Umschalten der Eingänge der Ausgang W nicht anspricht kann für diesen eine Ansprechverzögerung TD festgelegt werden.

18.29 SH

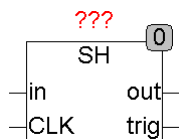
Type Funktionsbaustein

Input IN : REAL (Eingangssignal)

CLK : BOOL (Takteingang)

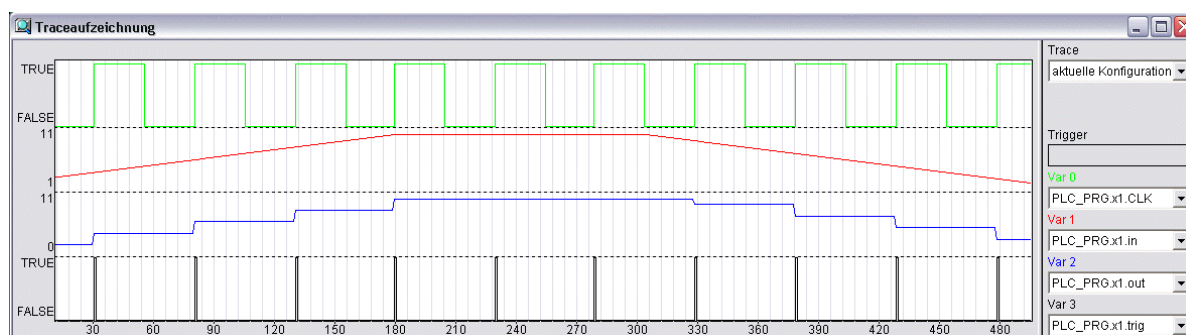
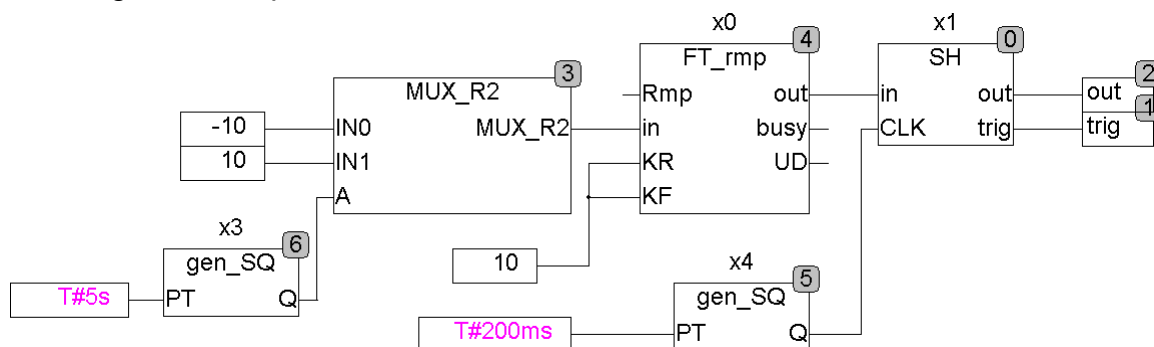
Output OUT : REAL (Ausgangssignal)

TRIG : BOOL (Trigger Output)



SH ist ein Sample and Hold Baustein. Er speichert bei jeder steigenden Flanke von CLK das Eingangssignal IN am Ausgang OUT. Nach jedem update von OUT bleibt TRIG für einen Zyklus TRUE.

Das Folgende Beispiel erläutert die Funktionsweise von SH:



18.30 SH_1

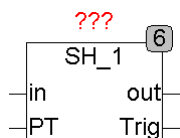
Type	Funktionsbaustein
------	-------------------

Input IN : REAL (Eingangssignal)

PT : TIME (Abtastzeit)

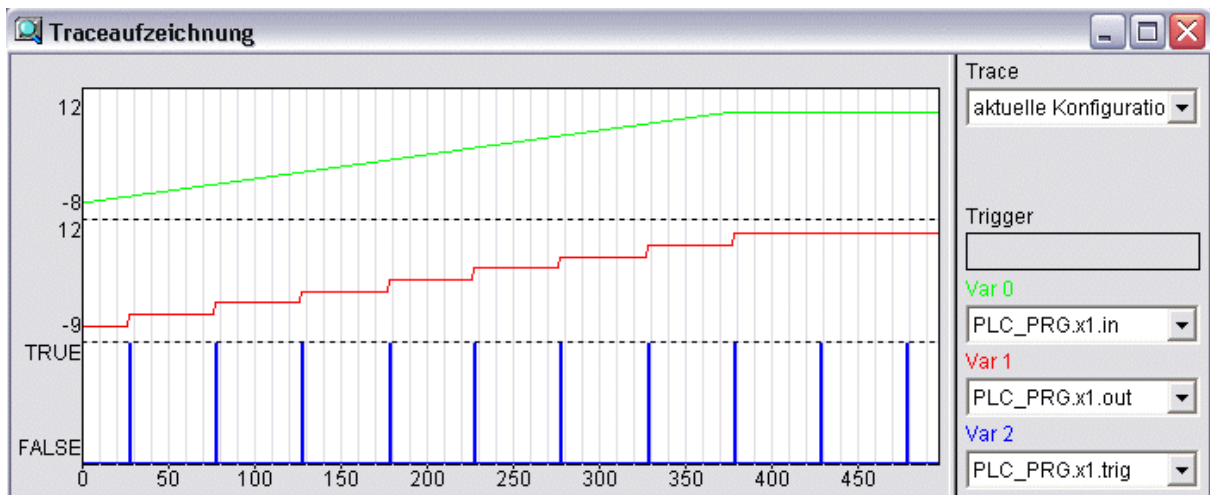
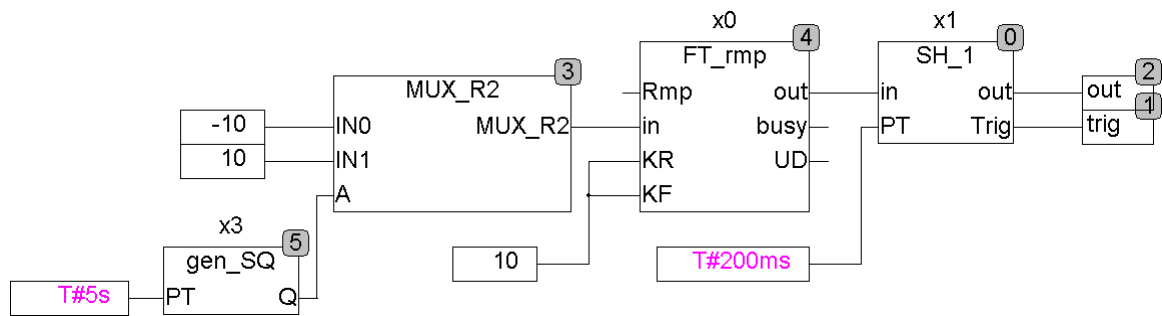
Output OUT : REAL (Ausgangssignal)

TRIG: BOOL (Trigger Output)



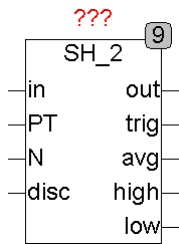
SH_1 ist ein Sample und Hold Baustein mit einstellbarer Abtastzeit. Er speichert alle PT das Eingangssignal IN am Ausgang OUT. Nach jedem update von OUT bleibt TRIG für einen Zyklus TRUE.

Das Folgende Beispiel erläutert die Funktionsweise von SH_1:



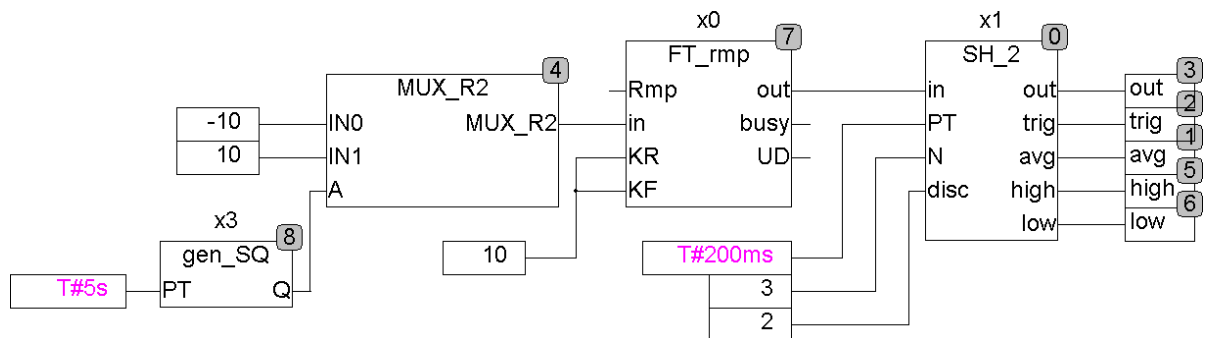
18.31 SH_2

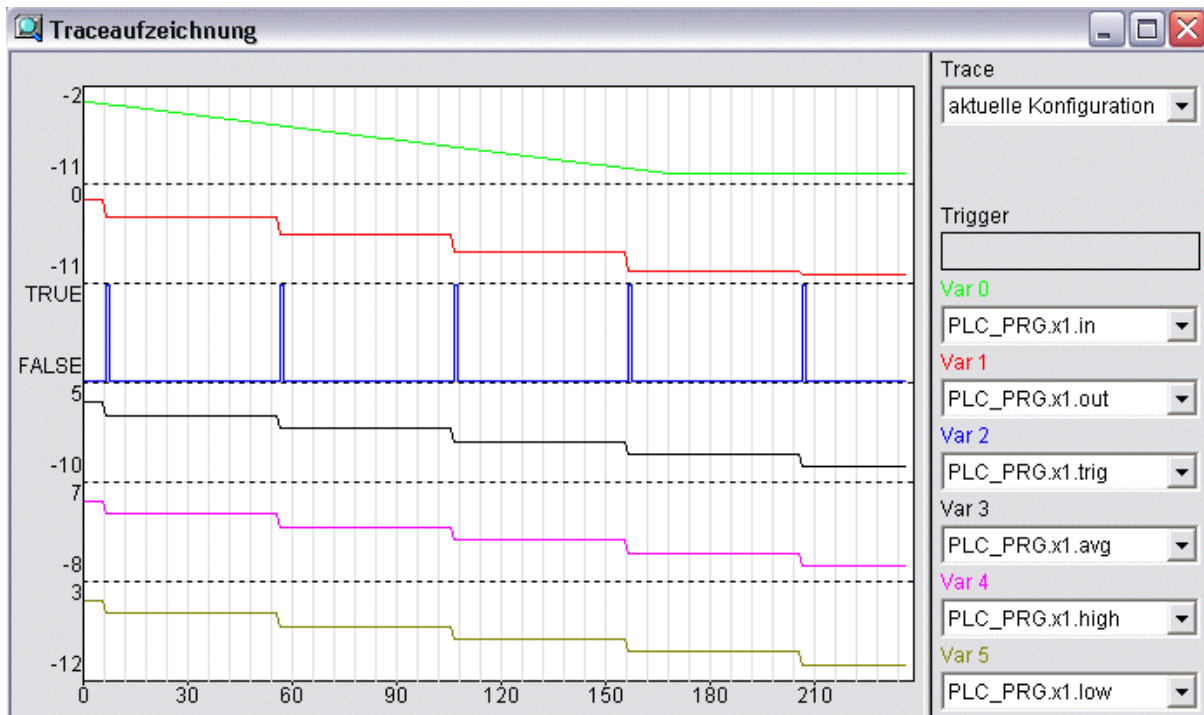
Type	Funktionsbaustein
Input	IN : REAL (Eingangssignal)
	PT : TIME (Abtastzeit)
	N : INT (Anzahl der Samples für Statistik)
Output	OUT : REAL (Ausgangssignal)
	TRIG : BOOL (Trigger Output)



SH_2 ist ein Sample and Hold Baustein mit einstellbarer Abtastzeit. Er speichert alle PT das Eingangssignal IN am Ausgang OUT. Nach jedem update von OUT bleibt TRIG für einen Zyklus TRUE. Zusätzlich zur Funktion eines Sample and Hold Bausteins bietet SH_2 bereits integrierte Funktionalität bezüglich der Statistik. Mit dem Eingang N kann spezifiziert werden, über wie viele Samples (maximal 16) ein Mittelwert, Minimalwert und Maximalwert gebildet werden. Als weitere Eigenschaft können aus den N Samples für die Statistik auch kleinste und größte Werte ignoriert werden, was sehr sinnvoll sein kann um Extremwerte zu ignorieren. Der Eingangswert DISC=0 bedeutet alle Samples benutzen, eine 1 bedeutet den niedrigsten Wert ignorieren, 2 bedeutet den niedrigsten und den höchsten Wert ignorieren usw. Wenn zum Beispiel N=5 und DISC=2, dann werden 5 Samples gesammelt, der niedrigste und der höchste Wert werden verworfen und über die 3 verbleibenden Samples wird der Mittelwert, Minimalwert und Maximalwert gebildet.

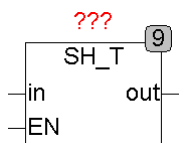
Das Folgende Beispiel erläutert die Funktionsweise von SH_2:





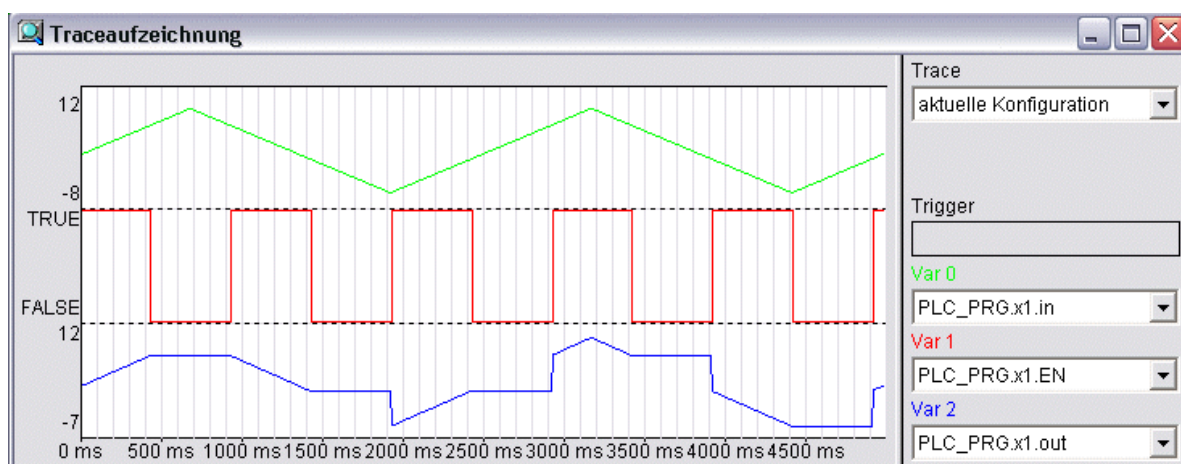
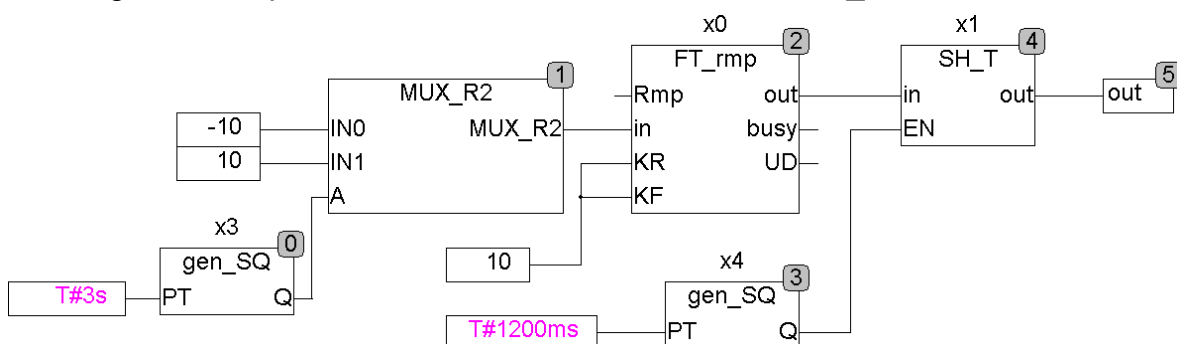
18.32 SH_T

Type Funktionsbaustein
 Input IN : REAL (Eingangssignal)
 EN : BOOL (enable Signal)
 Output OUT : REAL (Ausgangssignal)



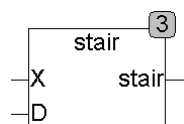
SH_T ist ein transparenter Sample and Hold Baustein. Das Eingangssignal in ist solange am Ausgang verfügbar, wie EN gleich TRUE ist. Mit einer fallenden Flanke von EN wird der Wert von in am Ausgang OUT gespeichert und bleibt solange bestehen bis EN wieder TRUE wird und dadurch wieder in auf OUT geschaltet wird.

Das folgende Beispiel verdeutlicht die Arbeitsweise von SH_T:



18.33 STAIR

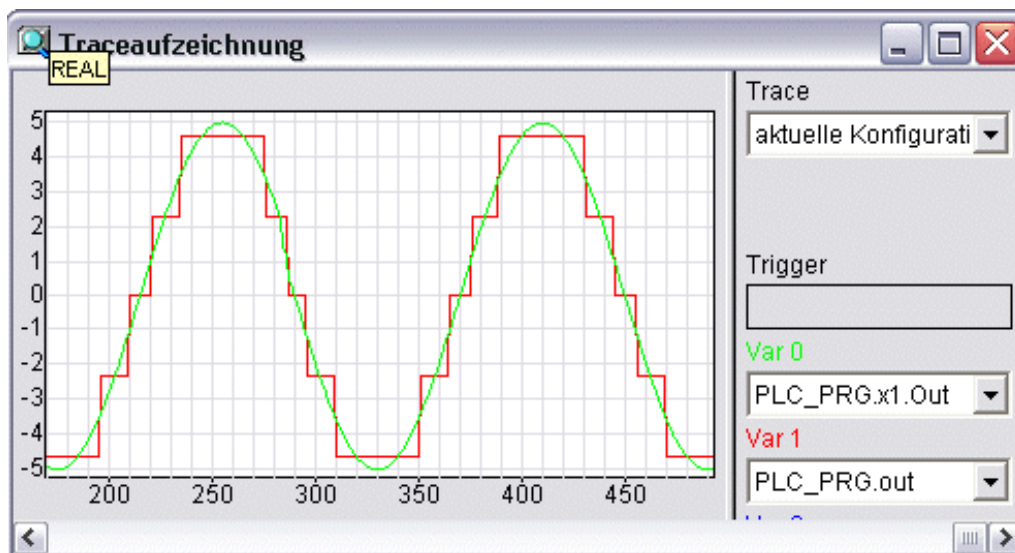
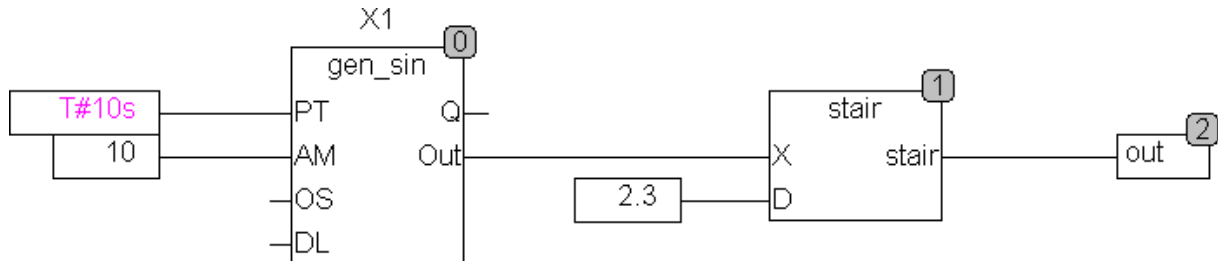
Type	Funktion
Input	X : REAL (Eingangssignal) D : REAL (Schrittweite des Ausgangssignals)
Output	REAL (Ausgangssignal)



Das Ausgangssignal von STAIR folgt dem Eingangssignal X mit einer Treppenfunktion. Die Höhe der Stufen ist vorgegeben durch D. Wird X = 0, so folgt das Ausgangssignal direkt dem Eingangssignal. STAIR ist jedoch nicht zum filtern von Eingangssi-

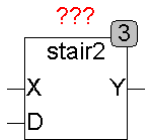
gnalen geeignet, denn wenn der Eingang um eine Treppenschwelle schwankt, schaltet der Ausgang zwischen zwei benachbarten Werten hin und her. Für diesen Zweck empfehlen wir den Einsatz von Stair2, der mit einer Hysterese arbeitet und instabile Zustände vermeidet.

Das folgende Beispiel verdeutlicht die Arbeitsweise von STAIR:



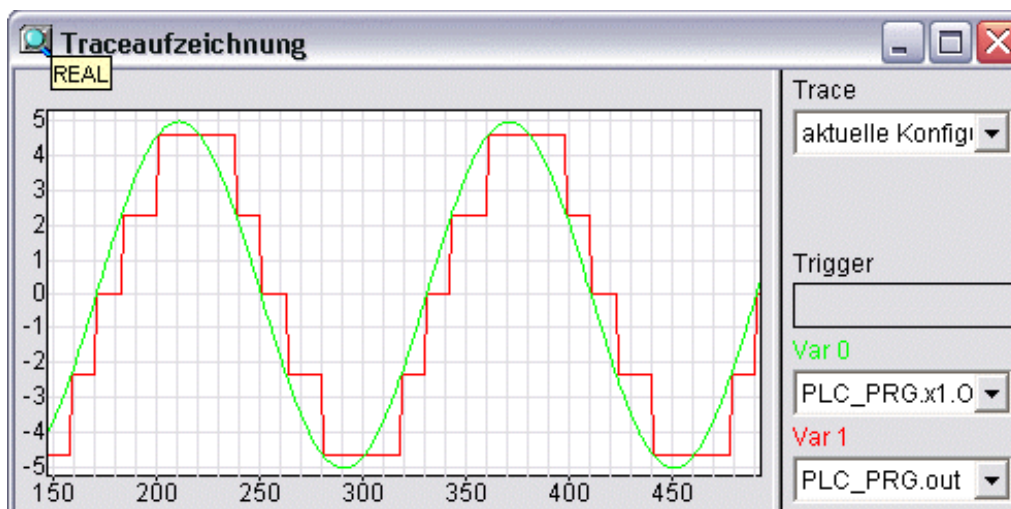
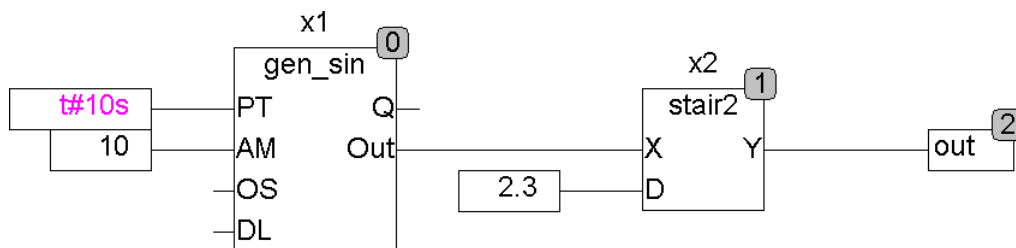
18.34 STAIR2

Type	Funktionsbaustein
Input	X : REAL (Eingangssignal) D : REAL (Schrittweite des Ausgangssignals)
Output	Y : REAL (Ausgangssignal)



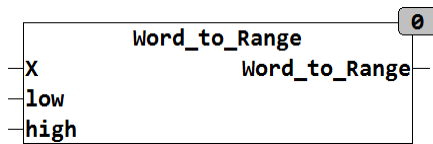
Das Ausgangssignal von STAIR2 folgt dem Eingangssignal X mit einer Treppenfunktion. Die Höhe der Stufen ist vorgegeben durch D. Wird $D = 0$, so folgt das Ausgangssignal direkt dem Eingangssignal. Das Signal folgt der Treppe aber mit einer Hysterese von D so dass ein verrauschtes Eingangssignal keine Sprünge zwischen Treppenwerten auslösen kann. STAIR2 ist auch als Eingangsfilter geeignet.

Das folgende Beispiel verdeutlicht die Arbeitsweise von STAIR2:



18.35 WORD_TO_RANGE

Type	Funktion
Input	IN : WORD (Eingangswert)
Output	REAL (Ausgangswert)



WORD_TO_RANGE wandelt einen WORD Wert in einen REAL Wert. Ein Eingangswert von 0 entspricht dabei dem REAL Wert von LOW und ein Eingangswert von 65535 entspricht dem Eingangswert von HIGH.

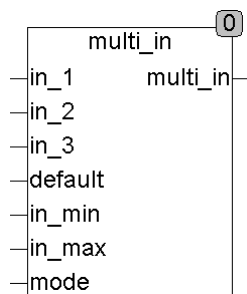
Um einen WORD Wert von 0..65535 in einen Prozentwert von 0..100 zu wandeln wird der Baustein wie folgt aufgerufen:

WORD_TO_RANGE(X,100,0)

19 Sensorik

19.1 MULTI_IN

Type	Funktion : REAL
Input	IN_1 : REAL (Eingang 1)
	IN_2 : REAL (Eingang 2)
	IN_3 : REAL (Eingang 3)
	DEFAULT : REAL (Vorgabewert)
	IN_MIN : REAL (unterer Grenzwert für Eingänge)
	IN_MAX : REAL (oberer Grenzwert für Eingänge)
	MODE : Byte (Auswahl des Betriebsmodus)
Output	REAL (Ausgangssignal)



MULTI_IN ist ein Sensorinterface, das bis zu 3 Sensoren einliest, auf Fehler überprüft und abhängig vom Eingangsmodus einen Ausgangswert berechnet.

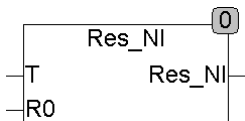
Mode	Funktion
0	MULTI_in = Durchschnitt der Eingänge in_1 .. 3
1	MULTI_in = Eingang in_1
2	MULTI_in = Eingang in_2
3	MULTI_in = Eingang in_3
4	MULTI_in = Default Eingang
5	MULTI_in = kleinster Wert der Eingänge in_1 .. 3

6	MULTI_in = größter Wert der Eingänge in_1 .. 3
7	MULTI_in = mittlerer Wert der Eingänge in_1..3
>7	MULTI_in = 0

Unabhängig vom eingestellten Mode werden Eingangswerte, die größer als IN_MAX oder kleiner als IN_MIN sind ignoriert. Ist keine Berechnung wie durch Mode vorgegeben mehr möglich, wird der Eingang Default als Ausgangswert benutzt. Multi_in wird eingesetzt, wenn verschiedene Sensoren den gleichen Wert messen und hohe Sicherheit und Zuverlässigkeit gefragt ist. Eine mögliche Anwendung ist die Messung der Außentemperatur an verschiedenen Stellen und die Überwachung auf Kabel oder Sensorbruch.

19.2 RES_NI

Type Funktion : REAL
 Input T : REAL (Temperatur in °C)
 R0 : REAL (Widerstand bei 0 °C)
 Output REAL (Widerstandswert)



RES_NI berechnet den Widerstand eines NI-Widerstandsfühlers aus den Eingangswerten T (Temperatur in °C) und R0 (Widerstand bei 0°C).

Die Berechnung erfolgt nach der Formel:

$$\text{RES_NI} = R0 + A \cdot T + B \cdot T^2 + C \cdot T^4$$

$$A = 0.5485$$

$$B = 0.665\text{E-}3$$

$$C = 2.805\text{E-}9$$

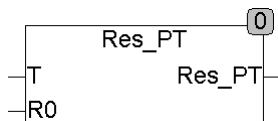
Die Berechnung ist geeignet für einen Temperaturbereich von -60 .. +180 °C.

Auszug aus DIN 43760 für Ni100

°C	R	°C	R	°C	R	°C	R	°C	R
-60	69,5	-10	94,6	40	123,0	90	154,9	140	190,9
-55	71,9	-5	97,3	45	126,0	95	158,3	145	194,8
-50	74,3	0	100,0	50	129,1	100	161,8	150	198,7
-45	76,7	5	102,8	55	132,2	105	165,3	155	202,6
-40	79,1	10	105,6	60	135,3	110	168,8	160	206,6
-35	81,6	15	108,4	65	138,5	115	172,4	165	210,7
-30	84,2	20	111,2	70	141,7	120	176,0	170	214,8
-25	86,7	25	114,1	75	145,0	125	179,6	175	219,0
-20	89,3	30	117,1	80	148,3	130	183,3	180	223,2
-15	91,9	35	120,0	85	151,6	135	187,1		

19.3 RES_PT

Type Funktion : REAL
 Input T : REAL (Temperatur in °C)
 R0 : REAL (Widerstand bei 0 °C)
 Output REAL (Widerstandswert)



RES_PT berechnet den Widerstand eines PT-Widerstandsfühlers aus den Eingangswerten T (Temperatur in °C) und R0 (Widerstand bei 0°C).

Die Berechnung erfolgt nach der Formel:

für Temperaturen > 0 °C

$$\text{RES_PT} = R0 * (1 + A*T + B*T^2)$$

und für Temperaturen < 0°C

$$\text{RES_PT} = R0 * (1 + A*T + B*T^2 + C*(T-100)*T^3)$$

$$A = 3.90802E-3$$

$$B = -5.80195E-7$$

$$C = -427350E-12$$

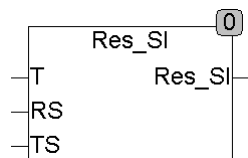
Die Berechnung ist geeignet für Temperaturen von -200 .. +850 °C.

Auszug aus DIN 43760 für Pt100

°C	R	°C	R	°C	R	°C	R	°C	R	°C	R
-200	18,49	-100	60,25	0	100,00	100	138,50	200	175,84	300	212,02
-195	20,65	-95	62,28	5	101,95	105	140,39	205	177,68	305	213,80
-190	22,80	-90	64,30	10	103,90	110	142,29	210	179,51	310	215,57
-185	24,94	-85	66,31	15	105,85	115	144,17	215	181,34	315	217,35
-180	27,08	-80	68,33	20	107,79	120	146,06	220	183,17	320	219,12
-175	29,20	-75	70,33	25	109,73	125	147,94	225	184,99	325	220,88
-170	31,32	-70	72,33	30	111,67	130	149,82	230	186,82	330	222,65
-165	33,43	-65	74,33	35	113,61	135	151,70	235	188,63	335	224,41
-160	35,33	-60	76,33	40	115,54	140	153,58	240	190,45	340	226,17
-155	37,63	-55	78,32	45	117,47	145	155,45	245	192,26	345	227,92
-150	39,71	-50	80,31	50	119,40	150	157,31	250	194,07	350	229,67
-145	41,79	-45	82,29	55	121,32	155	159,18	255	195,88	355	231,42
-140	43,87	-40	84,27	60	123,24	160	161,04	260	197,69	360	233,17
-135	45,94	-35	86,25	65	125,16	165	162,90	265	199,49	365	234,91
-130	48,00	-30	88,22	70	127,07	170	164,76	270	201,29	370	236,65
-125	50,06	-25	90,19	75	128,98	175	166,61	275	203,08	375	238,39
-120	52,11	-20	92,16	80	130,89	180	168,46	280	204,88	380	240,13
-115	54,15	-15	94,12	85	132,80	185	170,31	285	206,67	385	241,86
-110	56,19	-10	96,09	90	134,70	190	172,16	290	208,45	390	243,59
-105	58,22	-5	98,04	95	136,60	195	174,00	295	210,24	395	245,31

19.4 RES_SI

Type Funktion : REAL
 Input T : REAL (Temperatur in °C)
 R0 : REAL (Widerstand bei 0 °C)
 Output REAL (Widerstandswert)



RES_SI berechnet den Widerstand eines SI-Widerstandsfühlers aus den Eingangswerten T (Temperatur in °C) und RS, Widerstand bei TS in °C. Im Gegensatz zu den Bausteinen RES_NI und RES_PT deren R0 bei 0°C angegeben wird ist die Widerstandsangabe RS bei SI Fühler bei unterschiedlichen Temperaturen (z.B. 25°C bei KTY10). Deshalb hat der Baustein einen Eingang für RS und einen weiteren für TS.

Die Berechnung erfolgt nach der Formel:

$$RES_SI = RS + A \cdot (T - TS) + B \cdot (T - TS)^2$$

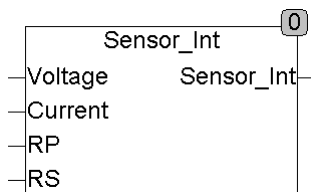
$$A = 7.64E-3$$

$$B = 1.66E-5$$

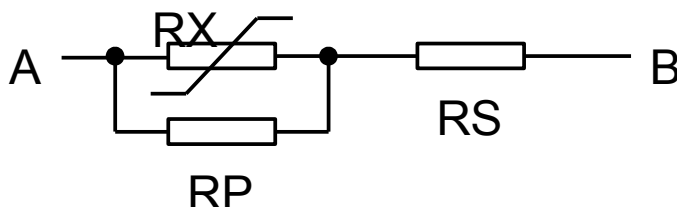
Die Berechnung ist geeignet für einen Temperaturbereich von -50 .. +150 °C.

19.5 SENSOR_INT

Type	Funktion : REAL
Input	VOLTAGE : REAL (gemessene Spannung in Volt) CURRENT : REAL (gemessener Strom in Ampere) RP : REAL (paralleler parasitärer Widerstand in Ohm) RS : REAL (serieller parasitärer Widerstand in Ohm)
Output	REAL (Widerstandswert des Sensors)



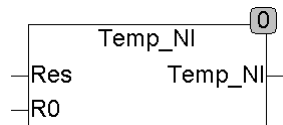
SENSOR_INT berechnet den Sensorwiderstand unter Berücksichtigung der Parasitären Widerstände, die die Messung normalerweise verfälschen. Der A/D Wandler misst entweder Strom bei fester Spannung oder Spannung bei festem Strom. Der daraus resultierende Widerstand ist aber nicht nur der Widerstand des Sensors, sondern setzt sich zusammen aus dem Widerstand des Sensors und 2 parasitären Widerständen RP und RS. Da die parasitären Widerstände konstant bleiben, können Sie wieder kompensiert werden und der wirkliche Widerstand des Sensors errechnet werden.



Der zwischen den Anschlüssen A und B gemessene Widerstand (gemessen durch Strom und Spannung) ist ein Gesamtwiderstand aus dem Sensorwiderstand parallel mit dem parasitären Widerstand RP und dem Leitungswiderstand RS. RS und RP werden kompensiert und der wirkliche Widerstand RX berechnet. Mit den Modulen TEMP_ kann dann zum Beispiel die exakte Temperatur berechnet werden.

19.6 TEMP_NI

Type Funktion : REAL
 Input RES : REAL (Widerstandswert in Ohm)
 R0 : REAL (Widerstand bei 0 °C)
 Output REAL (gemessene Temperatur)



RES_NI berechnet die Temperatur eines NI-Widerstandsfühlers aus den Eingangswerten RES (gemessener Widerstandswert) und R0 (Widerstand bei 0°C).

Die Berechnung ist geeignet für einen Temperaturbereich von -60 .. +180 °C und erfolgt nach folgender Formel:

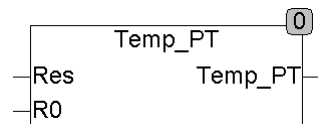
$$RES_NI = R0 + A \cdot T + B \cdot T^2 + C \cdot T^4$$

$$A = 0.5485; B = 0.665E-3; C = 2.805E-9$$

Auszug aus DIN 43760 für Ni100									
°C	R	°C	R	°C	R	°C	R	°C	R
-60	69,5	-10	94,6	40	123,0	90	154,9	140	190,9
-55	71,9	-5	97,3	45	126,0	95	158,3	145	194,8
-50	74,3	0	100,0	50	129,1	100	161,8	150	198,7
-45	76,7	5	102,8	55	132,2	105	165,3	155	202,6
-40	79,1	10	105,6	60	135,3	110	168,8	160	206,6
-35	81,6	15	108,4	65	138,5	115	172,4	165	210,7
-30	84,2	20	111,2	70	141,7	120	176,0	170	214,8
-25	86,7	25	114,1	75	145,0	125	179,6	175	219,0
-20	89,3	30	117,1	80	148,3	130	183,3	180	223,2
-15	91,9	35	120,0	85	151,6	135	187,1		

19.7 TEMP_PT

Type Funktion : REAL
 Input RES : REAL (gemessener Widerstandswert in Ohm)
 R0 : REAL (Widerstand bei 0 °C)
 Output REAL (gemessene Temperatur)



TEMP_PT berechnet die Temperatur eines PT-Widerstandsfühlers aus den Eingangswerten RES (gemessener Widerstandswert) und R0 (Widerstand bei 0°C). Wenn die Eingänge eine Temperatur außerhalb des Wertebereichs von -200 .. + 850°C ergeben wird am Ausgang die Temperatur +10000.0 °C ausgegeben.

Die Berechnung erfolgt nach der Formel:

für Temperaturen > 0 °C

$$RES_PT = R0 * (1 + A*T + B*T^2)$$

und für Temperaturen < 0°C

$$RES_PT = R0 * (1 + A*T + B*T^2 + C*(T-100)*T^3)$$

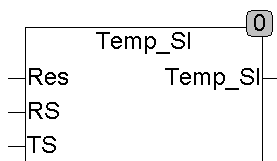
$$A = 3.90802E-3; B = -5.80195E-7; C = -427350E-12$$

Auszug aus DIN 43760 für Pt100

°C	R	°C	R	°C	R	°C	R	°C	R	°C	R
-200	18,49	-100	60,25	0	100,00	100	138,50	200	175,84	300	212,02
-195	20,65	-95	62,28	5	101,95	105	140,39	205	177,68	305	213,80
-190	22,80	-90	64,30	10	103,90	110	142,29	210	179,51	310	215,57
-185	24,94	-85	66,31	15	105,85	115	144,17	215	181,34	315	217,35
-180	27,08	-80	68,33	20	107,79	120	146,06	220	183,17	320	219,12
-175	29,20	-75	70,33	25	109,73	125	147,94	225	184,99	325	220,88
-170	31,32	-70	72,33	30	111,67	130	149,82	230	186,82	330	222,65
-165	33,43	-65	74,33	35	113,61	135	151,70	235	188,63	335	224,41
-160	35,33	-60	76,33	40	115,54	140	153,58	240	190,45	340	226,17
-155	37,63	-55	78,32	45	117,47	145	155,45	245	192,26	345	227,92
-150	39,71	-50	80,31	50	119,40	150	157,31	250	194,07	350	229,67
-145	41,79	-45	82,29	55	121,32	155	159,18	255	195,88	355	231,42
-140	43,87	-40	84,27	60	123,24	160	161,04	260	197,69	360	233,17
-135	45,94	-35	86,25	65	125,16	165	162,90	265	199,49	365	234,91
-130	48,00	-30	88,22	70	127,07	170	164,76	270	201,29	370	236,65
-125	50,06	-25	90,19	75	128,98	175	166,61	275	203,08	375	238,39
-120	52,11	-20	92,16	80	130,89	180	168,46	280	204,88	380	240,13
-115	54,15	-15	94,12	85	132,80	185	170,31	285	206,67	385	241,86
-110	56,19	-10	96,09	90	134,70	190	172,16	290	208,45	390	243,59
-105	58,22	-5	98,04	95	136,60	195	174,00	295	210,24	395	245,31

19.8 TEMP_SI

Type	Funktion : REAL
Input	RES : REAL (gemessener Widerstand in Ohm) RS : REAL (Widerstand bei 0 °C) TS : REAL (Temperatur bei der RS definiert ist)
Output	REAL (gemessene Temperatur)



TEMP_SI berechnet die Temperatur eines SI-Widerstandsfühlers aus den Eingangswerten RES (Widerstand in Ohm) und RS, Widerstand bei TS in °C. Im Gegensatz zu den Bausteinen TEMP_NI und TEMP_PT deren R0 bei 0°C angegeben wird, ist die Widerstandsangabe RS bei SI Fühler bei unterschiedlichen Temperaturen (z.B. 25°C bei KTY10) angegeben. Deshalb hat der Baustein einen Eingang für RS und einen weiteren für TS.

Die Berechnung erfolgt nach der Formel:

$$RES_SI = RS + A \cdot (T - TS) + B \cdot (T - TS)^2$$

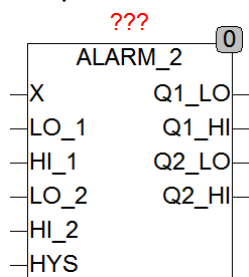
$$A = 7.64E-3; B = 1.66E-5$$

Die Berechnung ist geeignet für einen Temperaturbereich von -50 .. +150 °C.

20 Messbausteine

20.1 ALARM_2

Type	Funktionsbaustein
Input	X : REAL (Eingangswert)
	RST : BOOL (Reset Eingang für Alarm Ausgang)
Output	LOW : BOOL (TRUE, wenn $X < \text{TRIGGER_LOW}$)

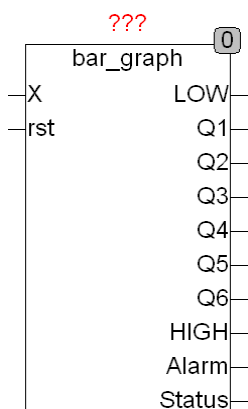


ALARM_2 prüft ob X die Grenzen HI_1 oder HI_2 nach oben übersteigt und setzt dabei die Ausgänge Q1_HI oder Q2_HI auf TRUE. Falls die Grenzen LO_1 oder LO_2 unterschritten werden wird entsprechend Q1_LO oder Q2_LO auf TRUE gesetzt. Die Ausgänge bleiben solange TRUE wie die entsprechende Grenze über- beziehungsweise unter-schritten wird. Um ein Flattern der Ausgänge zu unterbinden kann alternativ auch eine Hysterese HYS eingestellt werden. Wird HYS auf einen Wert > 0 gesetzt, so wird der entsprechende Ausgang erst dann gesetzt wenn die Grenze um mehr als $\text{HYS} / 2$ überschritten beziehungsweise unterschritten wird. Entsprechend muss der Eingang X die Grenze um mehr als $\text{HYS} / 2$ unter beziehungsweise überschreiten bevor die entsprechenden Ausgänge wieder gelöscht werden.

ALARM_2 kann zum Beispiel mit HI_1 und LO_1 den Pegel eines Flüssigkeitsbehälters steuern und mit HI_2 und LO_2 einen Alarm auslösen wenn ein kritischer Pegel unterschritten beziehungsweise überschritten wird.

20.2 BAR_GRAPH

Type	Funktionsbaustein
Input	X : REAL (Eingangswert) RST : BOOL (Reset Eingang für Alarm Ausgang)
Output	LOW : BOOL (TRUE, wenn $X < \text{TRIGGER_LOW}$) Q1 .. Q6 : BOOL (Triggerausgänge) HIGH : BOOL (TRUE, wenn $X \geq \text{TRIGGER_HIGH}$) ALARM : BOOL (Alarmausgang) STATUS : Byte (ESR Status Ausgang)
Config	TRIGGER_LOW : REAL (Triggerschwelle für LOW Output) TRIGGER_HIGH : REAL (Triggerschwelle für HIGH Output) ALARM_LOW : BOOL (Enable Alarm für Low Output) ALARM_HIGH : BOOL (Enable Alarm für High Output) LOG_SCALE : BOOL (Ausgang ist Logarithmisch wenn TRUE)

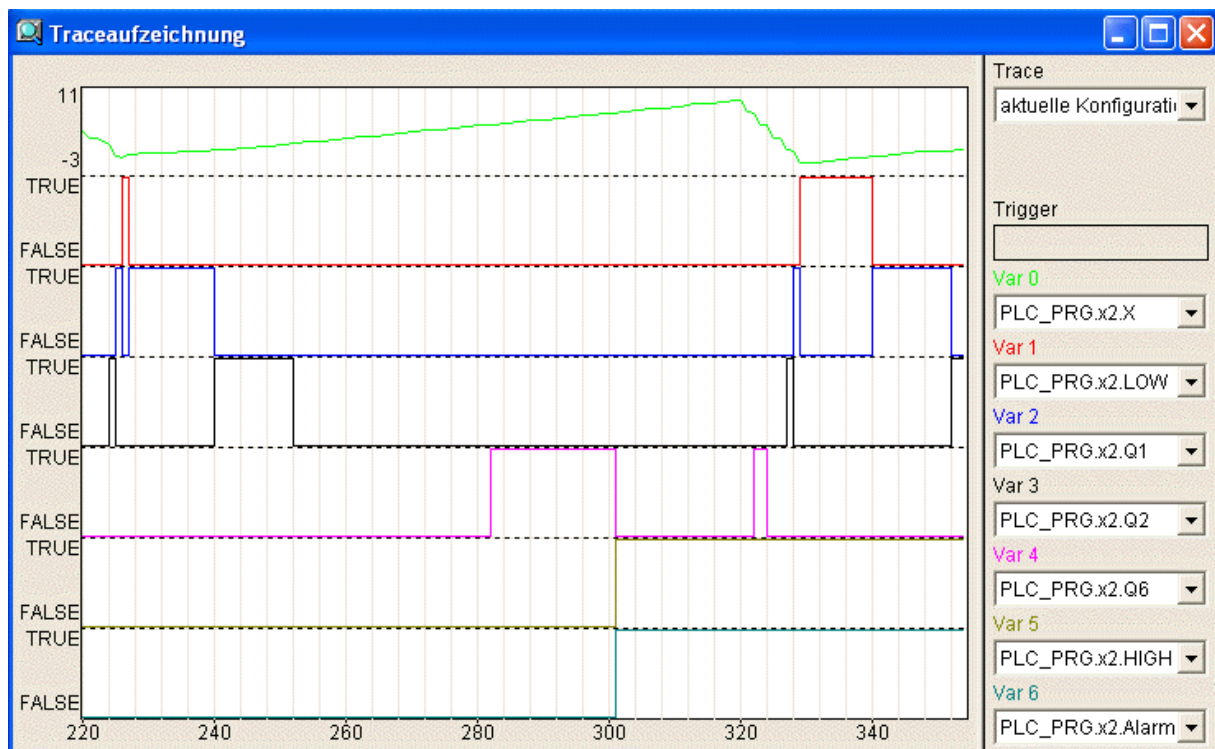
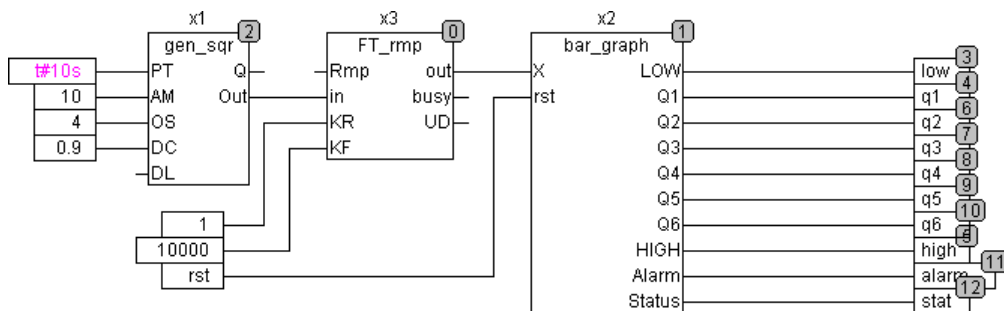


BAR_GRAPH ist ein Level Detector, der abhängig vom Eingangswert einen Ausgang aktiviert. Der Ansprechwert für die LOW und HIGH Ausgänge ist durch die Setup-Variablen TRIGGER_LOW und TRIGGER_HIGH einstellbar. LOW wird TRUE, wenn X kleiner als TRIGGER_LOW ist und HIGH wird TRUE, wenn X größer gleich TRIGGER_HIGH ist. Sind die Setup-Variablen ALARM_LOW und / oder ALARM_HIGH auf TRUE gesetzt, so wird bei unterschreiten von TRIGGER_LOW oder überschreiten von TRIGGER_HIGH der Ausgang ALARM auf TRUE gesetzt und der Ausgang LOW oder HIGH und ALARM bleibt solange auf TRUE, bis der Eingang RST TRUE wird und den Alarm zurücksetzt. Die Ausgänge Q1 bis Q6 unterteilen den Bereich zwischen TRIGGER_LOW und TRIGGER_HIGH in 7 gleiche Bereiche. Ist die Setup Variable LOG_SCALE gesetzt, so wird der Bereich zwischen TRIGGER_LOW und TRIGGER_HIGH logarithmisch aufgeteilt.

Der Ausgang Status ist ein ESR kompatibler Ausgang, der Zustände und Alarme an ESR Bausteine weiterreicht.

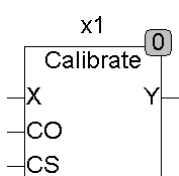
Status	
110	Eingang liegt zwischen Trigger_Low und Trigger_High
111	Eingang kleiner Trigger_Low, Ausgang LOW ist TRUE
112	Eingang größer Trigger_High, Ausgang HIGH ist TRUE
1	Eingang kleiner als Trigger_low und Alarm_Low ist TRUE
2	Eingang größer als Trigger_high und Alarm_High ist TRUE

Das folgende Beispiel zeigt einen Signalverlauf an Bar_Graph:



20.3 CALIBRATE

Type	Funktionsbaustein
Input	X : REAL (Eingangswert) CO : BOOL (Puls zum Speichern des Offsets) CS : BOOL (Puls zum Speichern des Verstärkungsfaktors)
Output	Y : REAL (kalibriertes Ausgangssignal)
Config	Y_OFFSET : REAL (Y Wert bei dem der Offset gesetzt wird) Y_SCALE : REAL (Y Wert bei dem der Verstärkungsfaktor gesetzt wird)



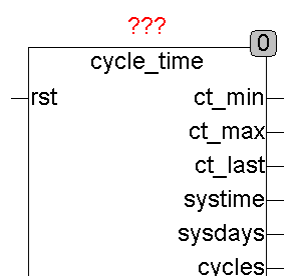
CALIBRATE dient zum Kalibrieren eines Analogen Signals. Um ein Kalibrieren zu ermöglichen müssen 2 Referenzwerte (Y_OFFSET und Y_SCALE) durch einen Doppelklick auf das Symbol des Bausteins gesetzt werden. Y_OFFSET ist der Ausgangswert, bei dem der Offset durch einen Puls an CO gesetzt wird und Y_SCALE ist der Wert, bei dem der Verstärkungsfaktor ermittelt wird. Eine Kalibrierung ist nur dann erfolgreich, wenn zuerst Offset und danach Verstärkung kalibriert werden.

Beispiel:

Ein Eingangssignal von 4..20 mA soll auf die Temperaturwerte von 0 .. 70 °C kalibriert werden. Hierzu werden die Config- Variablen Y_OFFSET = 0 und Y_SCALE = 70 gesetzt. Dann wird der Sensor in Eiswasser gelegt und nach der Ansprechzeit des Sensors ein Puls am Eingang C0 ausgelöst, der den Baustein veranlasst einen Korrekturwert für Offset zu errechnen und intern abzuspeichern. Als nächstes wird dann der Sensor mit 70 °C beaufschlagt und nach der Ansprechzeit ein Puls am Eingang CS ausgelöst, wodurch im Baustein ein Verstärkungsfaktor berechnet wird, der intern gespeichert wird. Die Kalibrationswerte werden permanent gespeichert. Das heißt, sie gehen auch dann nicht verloren, wenn ein Reset ausgeführt wird oder die Stromversorgung an der SPS ausgeschaltet wird.

20.4 CYCLE_TIME

Type	Funktionsbaustein
Input	RES : BOOL (Reset)
Output	CT_MIN : TIME (minimale gemessene Zykluszeit)
	CT_MAX : TIME (maximale gemessene Zykluszeit)
	CT_LAST : TIME (zuletzt gemessene Zykluszeit)
	SYSTIME : TIME (gesamte Laufzeit des Systems seit dem letzten Start)
	SYSDAYS : INT (Anzahl der Tage seit dem letzten Start)
	CYCLES : DWORD (Anzahl der Zyklen seit dem letzten Start)

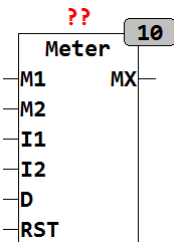


CYCLE_TIME überwacht die Zykluszeiten einer SPS und stellt dem Anwender eine Reihe von Informationen über Zykluszeiten und Laufzeiten zur Verfügung. Die Gesamtzahl der Zyklen wird ebenfalls gemessen. Hiermit kann der Anwender zum Beispiel sicherstellen, dass eine Funktion alle 100 Zyklen aufgerufen wird. Regelbausteine können Fehler melden, wenn die Zykluszeit zu lange wird und deshalb die Regelparameter nicht mehr garantiert werden können.

20.5 METER

Type	Funktionsbaustein
Input	M1 : REAL (Verbrauchswert 1)
	M2 : REAL (Verbrauchswert 2)
	I1 : BOOL (Freigabeeingang 1)
	I2 : BOOL (Freigabeeingang 2)
	D : REAL (Teiler für den Ausgang)
	RST : BOOL (Reset Eingang)

Output MX : REAL (Verbrauchswert)



METER ist ein Verbrauchszähler, der 2 unabhängige Eingänge (M1 und M2) über die Zeit aufaddiert. Die Verbrauchszählung wird durch die Eingänge I1 und I2 gesteuert. Mit einem Reset-Eingang RST kann der Zähler jederzeit zurückgesetzt werden. Der Wert M1 wird je Sekunde zum Ausgangswert addiert solange I1 auf TRUE steht und analog wird der Wert M2 je Sekunde zum Ausgang addiert wenn I2 TRUE ist. Sind I1 und I2 TRUE, so wird der Wert $M1 + M2$ je Sekunde einmal zum Ausgang hinzu addiert. Der Eingang D Teilt den Ausgang MX. Damit können z B. Wattstunden anstatt Wattsekunden gezählt werden. Der Baustein benutzt intern den OSCAT spezifischen Datentyp REAL2 der eine Auflösung von 15 Dezimalstellen erlaubt. Dadurch ist es dem Baustein möglich kleinste Verbrauchswerte an den Eingängen M1 und m^2 mit kurzen Zykluszeiten zu erfassen und zu hohen Gesamtwerten am Ausgang MX addieren. Die Auflösung des Bausteins kann wie folgt ermittelt werden.

$MX / 10^{15}$ entspricht der minimalen Auflösung an den Eingängen M1 und M2.

Beispiel:

MX = 10E6 der Verbrauchszähler steht bei 10 MWh (Megawattstunden)

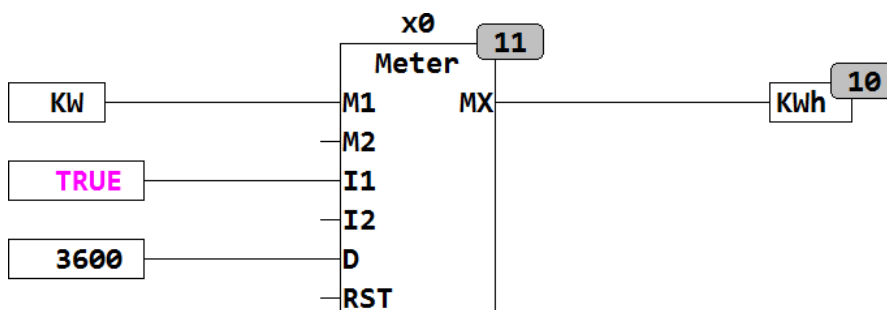
M1 = 0.09 Watt Momentanverbrauch liegt bei 0.1 Watt

D = 3600 Ausgang arbeitet in Wh (Wattstunden)

Zykluszeit beträgt 10ms

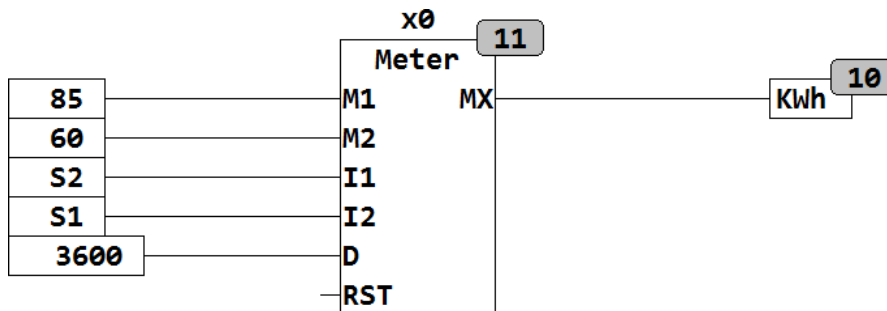
In diesem Beispiel wird je Zyklus ein Wert von $0.09[W] * 0.01 [S] / 3600 = 2.5E-7[Wh]$ zum Ausgang MX addiert. Dies entspricht einer Veränderung an der 14 Dezimalstelle des Ausgangs.

Beispiel 1 Stromverbrauchszähler:



Der Stromverbrauchszähler zählt die Kilowattsekunden am Eingang M1. Durch den Eingang D wird der Ausgang durch 3600 geteilt, sodass der Ausgang Kilowattstunden anzeigt.

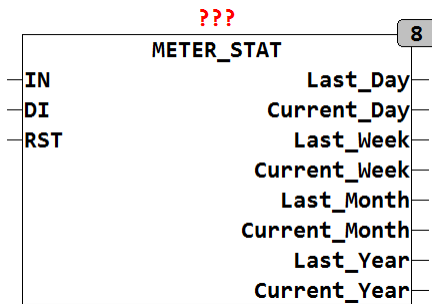
Beispiel 2 Verbrauchsberechnung für einen 2 Stufen Brenner:



In diesem Beispiel ist die Leistung von Stufe 1 (M1) 85KW und Stufe 2 (M2) 60KW. Die Eingänge S1 und S1 (I1 und I2) werden TRUE, wenn die entsprechende Stufe läuft. Durch die Konstante 3600 an D wird der Ausgang durch 3600 geteilt, sodass Kilowattstunden angezeigt werden.

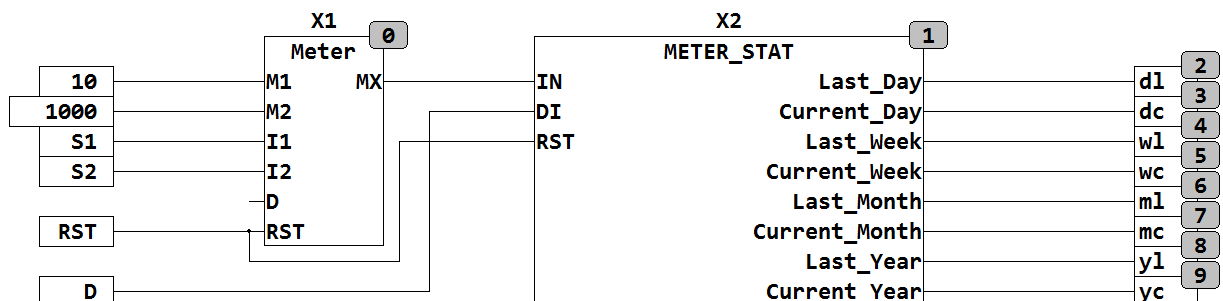
20.6 METER_STAT

Type	Funktionsbaustein
Input	IN : REAL (Eingangssignal) DI : DATE (Datumseingang) RST : BOOL (Reset Eingang)
Output	LAST_DAY : REAL (Verbrauchswert des vergangenen Tages) CURRENT_DAY : REAL (Verbrauchswert des aktuellen Tages) LAST_WEEK : REAL (Verbrauchswert der vergangenen Woche) CURRENT_WEEK : REAL (Verbrauchswert der aktuellen Woche) LAST_MONTH : REAL (Verbrauchswert des vergangenen Monats) CURRENT_MONTH : REAL (Verbrauchswert des aktuellen Monats) LAST_YEAR : REAL (Verbrauchswert des vergangenen Jahres) CURRENT_YEAR : REAL (Verbrauchswert des aktuellen Jahres)



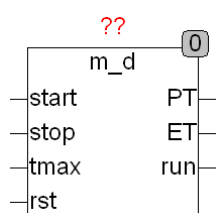
METER_STAT errechnet den Verbrauch des aktuellen Tages, Woche, Monat und Jahr und zeigt den Wert des letzten Vergleichszeitraumes. Der aufaddierte Verbrauchswert liegt am Eingang IN, während am Eingang DI das aktuelle Datum anliegt. Mit dem Eingang RST kann der Zähler jederzeit zurückgesetzt werden.

Das folgende Beispiel zeigt die Anwendung von METER_STAT mit dem Baustein METER:



20.7 M_D

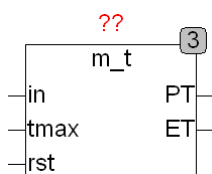
Type	Funktionsbaustein
Input	START : BOOL (Eingangssignal)
	STOP : BOOL (Eingangssignal)
	TMAX : TIME (Timeout für ET)
	RST : BOOL (Reset Eingang)
Output	PT : TIME (gemessene Impulsdauer von der steigenden bis zur fallenden Flanke)
	ET : TIME (Abgelaufene Zeit seit letzter steigender Flanke)



M_D misst die Zeit zwischen einer steigenden Flanke an START und einer steigenden Flanke an STOP. PT ist das Ergebnis der letzten Messung. Der Ausgang ET ist die abgelaufene Zeit seit der letzten steigenden Flanke an START. M_D benötigt eine steigende Flanke, um die Messung zu triggern. Falls beim ersten Aufruf in bereits TRUE ist, wird dies nicht als steigende Flanke gewertet. Auch wenn STOP TRUE ist, wird eine steigende Flanke an START nicht gewertet. Nur wenn alle Startbedingungen (STOP = FALSE, RST := FALSE und steigende Flanke an START) vorliegen geht der Ausgang RUN auf TRUE und eine Messung wird gestartet. Mit TRUE am Eingang RST können die Ausgänge jederzeit auf 0 zurückgesetzt werden. Erreicht ET den Wert TMAX wird automatisch im Baustein ein Reset erzeugt und alle Ausgänge auf 0 zurückgesetzt. TMAX ist intern mit einem Vorgabewert von T#10d belegt und kann im Normalfall unbeschaltet bleiben. TMAX dient dazu einen maximalen Wertebereich für PT vorzugeben.

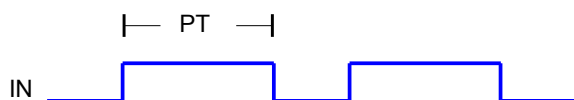
20.8 M_T

Type	Funktionsbaustein
Input	IN : BOOL (Eingangssignal) TMAX : TIME (Timeout für ET) RST : BOOL (Reset Eingang)
Output	PT : TIME (gemessene Impulsdauer von der steigenden bis zur fallenden Flanke) ET : TIME (Abgelaufene Zeit seit letzter Steigender Flanke)



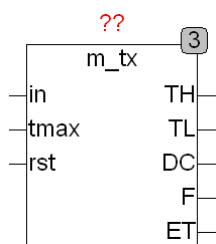
M_T misst die Zeit wie lange IN TRUE war. PT ist die Zeit von der steigenden Flanke des IN Signals bis zur fallenden Flanke des IN Signals. Der Ausgang ET ist die abgelaufene Zeit seit der letzten steigenden Flanke bis zur fallenden Flanke. Solange das Eingangssignal FALSE ist, ist ET = 0. M_T benötigt eine steigende Flanke um die Messung zu triggern. Falls beim ersten Aufruf in bereits TRUE ist, wird dies nicht

als steigende Flanke gewertet. Weitere Beispiele sind in der Beschreibung von M_TX zu finden. Mit TRUE am Eingang RST können die Ausgänge jederzeit auf 0 zurückgesetzt werden. Erreicht ET den Wert TMAX wird automatisch im Baustein ein Reset erzeugt und alle Ausgänge auf 0 zurückgesetzt. TMAX ist intern mit einem Vorgabewert von T#10d belegt und kann im Normalfall unbeschaltet bleiben.



20.9 M_TX

Type	Funktionsbaustein
Input	IN : BOOL (Eingangssignal) TMAX : TIME (Timeout für ET) RST : BOOL (Reset Eingang)
Output	TH : TIME (Overtime des Eingangssignals) TL : TIME (Offtime des Eingangssignals) DC : REAL (Tastverhältnis / Duty Cycle des Eingangssignals) F : REAL (Frequenz des Eingangssignals)

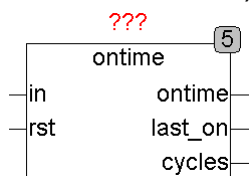


M_TX ermittelt aus dem Eingangssignal IN die Zeit, welche das Signal IN auf TRUE war (TH) und die Zeit die das Signal auf FALSE war (TL). Die Zeiten TH und TL werden nur nach einer steigenden beziehungsweise fallenden Flanke gemessen. Ist IN beim ersten Aufruf des Bausteins bereits high wird dies nicht als steigende Flanke gewertet. Aus den gemessenen Werten TH und TL werden der Duty Cycle und die Frequenz in Hz errechnet. Ein Duty Cycle von 0,4 bedeutet das Signal war 40% TRUE und 60% FALSE. Der Ausgang ET vom Typ TIME wird mit jeder steigenden Flanke bei 0 gestartet und läuft aufwärts, bis die nächste steigende Flanke ihn wieder bei 0 startet. Mit einem TRUE am Eingang RST können die Ausgänge zu jederzeit auf 0 zurückgesetzt werden. Der Eingang TMAX legt fest, nach welcher abgelaufenen Zeit an ET automatisch die Ausgänge zurückgesetzt werden. TMAX ist intern

mit einem Vorgabewert von T#10d belegt und kann im Normalfall offen bleiben. Der Eingang TMAX dient vor allem dazu, bei fehlendem Eingangssignal nach einer definierten Zeit die Ausgänge zurückzusetzen. Ein Beispiel für eine mögliche Anwendung ist die Messung der Drehzahl einer Welle, die nach Ausbleiben der Sensorsignale die Drehzahl (Frequenz) 0 anzeigt. TMAX ist aber mit Vorsicht anzuwenden, da zum Beispiel ein TMAX von 10 Sekunden gleichzeitig die kleinste messbare Frequenz auf 0,1 Hz begrenzt.

20.10 ONTIME

Type	Funktionsbaustein
Input	IN : BOOL (Eingangssignal) RST : BOOL (Reset Eingang)
Output	ONTIME : REAL (Betriebsstunden) LAST_ON : REAL (Länge der letzten Betriebszeit) CYCLES : REAL (Anzahl der Einschaltzyklen seit dem letzten Reset)



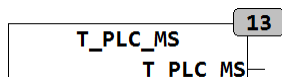
ONTIME ist ein Betriebsstundenzähler. Es wird die gesamte Zeit aufsummiert, die das Signal IN seit dem letzten Reset auf TRUE war. Zusätzlich wird die Einschalt-dauer des letzten Zyklus angezeigt und die Anzahl der gesamten Ein / Aus Zyklen. Die Ausgabewerte sind alle in REAL (in Stunden), um einen Überlauf zu vermeiden. Ein Wert von 36.50 entspricht 36 Stunden und 30 Minuten. Mit dem Eingang RST können die Ausgabewerte jederzeit zurückgesetzt werden.

Alle Ausgangswerte bleiben auch dann erhalten, wenn die SPS ausgeschaltet und später wieder eingeschaltet wird.

20.11 T_PLC_MS

Type Funktion : DWORD

Output DWORD (SPS Timer in Millisekunden)



T_PLC_MS liefert die aktuelle interne SPS Zeit in Millisekunden. Dies hat nichts mit einer eventuell vorhandenen Uhr (Real Time Baustein) zu tun, sondern ist der interne Timer einer SPS, der als Zeitreferenz benutzt wird.

Der Quelltext des Bausteins hat folgende Eigenschaften:

```
FUNCTION T_PLC_MS : DWORD
VAR CONSTANT
    DEBUG : BOOL := FALSE;
    N : INT := 0;
    OFFSET := 0;
END_VAR
VAR
    TEMP : DWORD := 1;
END_VAR
T_PLC_MS := TIME_TO_DWORD(TIME());
IF DEBUG THEN
    T_PLC_MS := SHL(T_PLC_US,N) OR SHL(TEMP,N)-1 + OFFSET;
END_IF;
```

Im Normalbetrieb liest der Baustein mit der Funktion TIME() den internen Timer der SPS aus und liefert diesen zurück. Der Interne Timer der SPS nach IEC Norm hat 1 Millisekunde Auflösung.

Eine weitere Eigenschaft von T_PLC_MS ist ein Debug-Modus, der es erlaubt den Überlauf des SPS internen Timers zu testen und die erstellte Software entsprechend sicher zu verifizieren. Der interne Timer jeder SPS hat unabhängig von Hersteller und Art der Implementierung nach einer festen Zeit einen Überlauf. Das heißt, er läuft gegen FF..FFFF (höchster Wert der im entsprechenden Typ gespeichert werden kann) und beginnt dann wieder bei 000..0000. bei Standard SPS Timern beträgt diese Überlaufzeit $2^{32} - 1$ Millisekunden, was in etwa 49,71 Tagen entspricht. Da es sich bei diesem Timer um einen in Hardware implementierten Timer handelt kann auch sein Anfangswert nicht gesetzt werden, sodass er nach dem Einschalten der SPS immer bei 0 anfängt und bis zum Maximalwert hoch läuft. Nach Erreichen des Maximalwertes entsteht dann der berüchtigte Timer-Überlauf, der fatale Auswirkungen in der Anwendungssoftware hervor ruft, aber nur extrem schwer getestet werden kann.

T_PLC_MS bietet mehrere Möglichkeiten zum Testen des Überlaufs und zeitabhängiger Software. Mit der Konstante DEBUG kann der Test-Modus eingeschaltet werden und dann mittels der Konstanten N und Offset der Timer ab einem bestimmten Wert beginnen, damit gezielt der Überlauf getestet werden kann ohne die 49 Tage abzuwarten. Offset legt hierbei den Wert fest, der zum internen Timer addiert wird. Mit der Konstanten N wird festgelegt, um wie viele Bits der interne Timer Wert nach links verschoben wird und dabei die unteren N Bits mit 1 gefüllt werden. Mit N kann dadurch die Geschwindigkeit des internen Timers um die Faktoren 2,4,8,16 usw. erhöht werden.

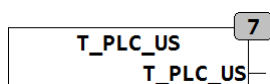
T_PLC_US bietet also alle Möglichkeiten zum Test zeitabhängiger Software, sowohl für die Problematik des Überlaufs, als auch für sehr langsame zeitabhängige Funktionen.

Die Konstanten DEBUG, N und OFFSET wurden absichtlich nicht als Eingänge der Funktion implementiert um eine versehentliche Fehlbedienung zu vermeiden.

20.12 T_PLC_US

Type Funktion : DWORD

Output DWORD (SPS Timer in Mikrosekunden)



T_PLC_US liefert die aktuelle interne SPS Zeit in Mikrosekunden. Dies hat nichts mit einer eventuell vorhandenen Uhr (Real Time Baustein) zu tun, sondern ist der interne Timer einer SPS der als Zeitreferenz benutzt wird.

Der Quelltext des Bausteins hat folgende Eigenschaften:

```
FUNCTION T_PLC_US : DWORD
VAR CONSTANT
    DEBUG : BOOL := FALSE;
    N : INT := 0;
    OFFSET := 0;
END_VAR
VAR
    TEMP : DWORD := 1;
END_VAR
T_PLC_US := TIME_TO_DWORD(TIME())*1000;
IF DEBUG THEN
```

```

T_PLC_US := SHL(T_PLC_US,N) OR SHL(TEMP,N)-1 + OFFSET;
END_IF;

```

Im Normalbetrieb liest der Baustein mit der Funktion TIME() den internen Timer der SPS aus. Da der interne Timer der SPS nach IEC Norm nur mit 1 Millisekunde Auflösung bietet wird der gelesene Wert mit 1000 multipliziert um den Wert in Mikrosekunden zurück zu liefern. Diese Funktion wurde aus Kompatibilitätsgründen so erstellt, damit sie auch auf Steuerungen die keine bessere Auflösung als Millisekunden bieten ein Mikrosekunden Timer zur Verfügung stellt, der dann in anderen Bausteinen verwendet werden kann. Falls die vorhandene SPS Mikrosekunden unterstützt, kann diese Funktion ganz einfach nur an dieser einen Stelle entsprechend angepasst werden und die Genauigkeit ändert sich durch diesen simplen Eingriff in allen Modulen die diese Funktion aufrufen. Die Software bleibt so portierbar und zukunftssicher. Bereits heute unterstützen praktisch alle SPS Controller eine Auflösung in Mikrosekunden. Diese wird allerdings nicht über standardisierte Routinen ausgelesen, sondern herstellerspezifisch und nicht standardisiert zur Verfügung gestellt. Der Baustein T_PLC_US stellt also ein geeignetes Interface zu diesen herstellerspezifischen Timern her.

Eine weitere Eigenschaft von T_PLC_US ist ein Debug-Modus, der es erlaubt den Überlauf des SPS internen Timers zu erzeugen und die erstellte Software entsprechend sicher zu testen. Der interne Timer jeder SPS hat unabhängig von Hersteller und Art der Implementierung nach einer festen Zeit einen Überlauf. Das heißt er läuft gegen FF..FFFF (höchster Wert der im entsprechenden Typ gespeichert werden kann und beginnt dann wieder bei 000..0000. bei Standard SPS Timern beträgt diese Überlaufzeit $2^{32} - 1$ Millisekunden, was etwa 49,71 Tagen entspricht. Da es sich bei diesem Timer um einen in Hardware implementierten Timer handelt kann auch sein Anfangswert nicht gesetzt werden, sodass er nach dem Einschalten der SPS immer bei 0 anfängt und bis zum Maximalwert hoch läuft. Nach Erreichen des Maximalwertes entsteht dann der berüchtigte Timer-Überlauf, der fatale Auswirkungen in der Anwendungssoftware hervor ruft, aber nur extrem schwer getestet werden kann.

T_PLC_US bietet mehrere Möglichkeiten zum testen des Überlaufs und zeitabhängiger Software. Mit der Konstante DEBUG kann der Test Modus eingeschaltet werden und dann mittels der Konstanten N und Offset der Timer ab einem bestimmten Wert beginnen, damit gezielt der Überlauf getestet werden kann ohne die 49 Tage abzuwarten. Offset legt hierbei den Wert fest der zum internen Timer addiert wird. Mit der Konstanten N wird festgelegt, um wie viele Bits der interne Timer Wert nach links verschoben wird und dabei die unteren N Bits mit 1 gefüllt werden. Mit N kann dadurch die Geschwindigkeit des internen Timers um die Faktoren 2,4,8,16 usw. erhöht werden.

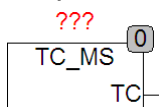
T_PLC_US bietet also alle Möglichkeiten zum Test zeitabhängiger Software, sowohl für die Problematik des Überlaufs, als auch für sehr langsame zeitabhängige Funktionen.

Die Konstanten DEBUG, N und OFFSET wurden absichtlich nicht als Eingänge der Funktion implementiert, um eine versehentliche Fehlbedienung zu vermeiden.

20.13 TC_MS

Type Funktionsbaustein

Output TC : DWORD (letzte Zykluszeit in Millisekunden)

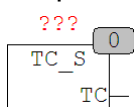


TC_MS ermittelt die letzte Zykluszeit, das ist die Zeit die seit dem letzten Aufruf des Bausteins vergangen ist. Die Zeit wird in Millisekunden geliefert.

20.14 TC_S

Type Funktionsbaustein

Output TC : REAL (letzte Zykluszeit in Sekunden)

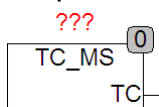


TC_S ermittelt die letzte Zykluszeit, das ist die Zeit die seit dem letzten Aufruf des Bausteins vergangen ist. Die Zeit wird in Sekunden geliefert, hat aber eine Genauigkeit in Mikrosekunden. Der Baustein ruft die Funktion T_PLC_US() auf. T_PLC_US() liefert den SPS internen Timer in Mikrosekunden mit einer Schrittweite von 1000 Mikrosekunden. Wird eine höhere Auflösung gewünscht muss die Funktion T_PLC_US() dem entsprechenden System angepasst werden.

20.15 TC_US

Type Funktionsbaustein

Output TC : DWORD (letzte Zykluszeit in Mikrosekunden)

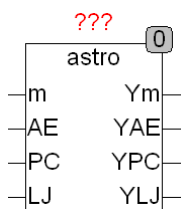


TC_US ermittelt die letzte Zykluszeit, das ist die Zeit die seit dem letzten Aufruf des Bausteins vergangen ist. Die Zeit wird in Mikrosekunden geliefert. Der Baustein ruft die Funktion T_PLC_US() auf. T_PLC_US() liefert den SPS internen Timer in Mikrosekunden mit einer Schrittweite von 1000 Mikrosekunden. Wird eine höhere Auflösung gewünscht muss die Funktion T_PLC_US() dem entsprechenden System angepasst werden.

21 Umrechnungen

21.1 ASTRO

Type	Funktionsbaustein
Input	M : REAL (Entfernung in Meter) AE : REAL (Entfernung in Astronomischen Einheiten) PC : REAL (Entfernung in Parsec) LJ : REAL (Entfernung in Lichtjahren)
Output	YM : REAL (Entfernung in Meter) YAE : REAL (Entfernung in Astronomischen Einheiten) YPC : REAL (Entfernung in Parsec) YLJ : REAL (Entfernung in Lichtjahren)



Der Baustein ASTRO rechnet verschiedene in der Astronomie gebräuchliche Entfernungseinheiten um. Normalerweise wird nur der zu konvertierende Eingang belegt und die restlichen Eingänge bleiben frei. Werden jedoch mehrere Eingänge gleichzeitig mit Werten beaufschlagt, so werden die Werte aller Eingänge entsprechend umgewandelt und dann aufsummiert.

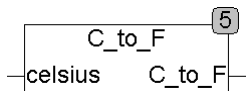
$$1 \text{ AE} = 149,597870 \cdot 10^9 \text{ m}$$

$$1 \text{ PC} = 206265 \text{ AE}$$

$$1 \text{ LJ} = 9,460530 \cdot 10^{15} \text{ m} = 63240 \text{ AE} = 0,30659 \text{ PC}$$

21.2 C_TO_F

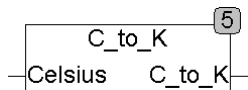
Type Funktion : REAL
 Input CELSIUS : REAL (Temperaturwert in °C)
 Output REAL (Temperaturwert in Fahrenheit)



C_TO_F rechnet einen Temperaturwert von Celsius in Fahrenheit um.

21.3 C_TO_K

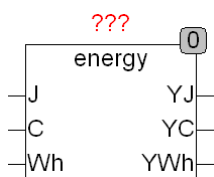
Type Funktion : REAL
 Input CELSIUS : REAL (Temperaturwert in °C)
 Output REAL (Temperaturwert in Kelvin)



C_TO_K rechnet einen Temperaturwert von Celsius in Kelvin um.

21.4 ENERGY

Type Funktionsbaustein
 Input J : REAL (Joule)
 C : REAL (Kalorie)
 WH : REAL (Wattstunden)
 Output YJ : REAL (Joule)
 YC : REAL (Kalorie)
 YWH : REAL (Wattstunden)



Der Baustein ENERGY konvertiert verschiedene in der Praxis gebräuchliche Einheiten für Energie. Normalerweise wird nur der zu konvertierende Eingang belegt und die restlichen Eingänge bleiben frei. Werden jedoch mehrere Eingänge gleichzeitig mit Werten beaufschlagt, so werden die Werte aller Eingänge entsprechend umgewandelt und dann aufsummiert.

1 J = 1 Ws (Watt * Sekunden) = 1 Nm (Newton * Meter)

1 C = 4,1868 J = $1,163 \cdot 10^{-3}$ Wh (Watt * Stunden)

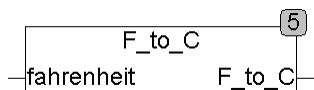
1 Wh = $3,6 \cdot 10^3$ J = 860 C

21.5 F_TO_C

Type Funktion : REAL

Input FAHRENHEIT : REAL (Temperaturwert in Fahrenheit)

Output REAL (Temperaturwert in °C)



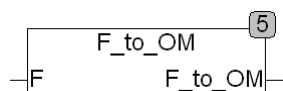
F_TO_C rechnet einen Temperaturwert von Fahrenheit in °C um.

21.6 F_TO_OM

Type Funktion : REAL

Input F : REAL (Frequenz in Hz)

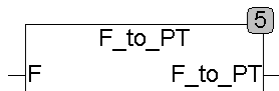
Output REAL (Frequenz in Hz)



F_TO_OM berechnet die Kreisfrequenz Omega aus der Frequenz in Hz.

21.7 F_TO_PT

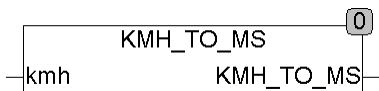
Type Funktion : REAL
 Input F : REAL (Frequenz)
 Output TIME (Periodendauer)



F_TO_PT rechnet einen Frequenzwert von Hz in die entsprechende Periodendauer um.

21.8 KMH_TO_MS

Type Funktion : REAL
 Input KMH : REAL (Geschwindigkeit in m/s)
 Output TIME (Geschwindigkeit in km/h)

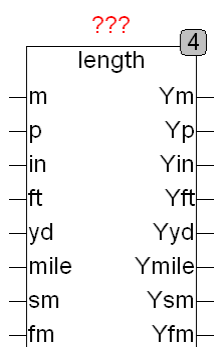


KMH_TO_MS rechnet einem Geschwindigkeitswert von Kilometer / Stunde in Meter / Sekunde um. $KMH_TO_MS := KMH / 3.6$

21.9 LENGTH

Type Funktionsbaustein
 Input M : REAL (Meter)
 P : REAL (Typographischer Punkt)
 IN : REAL (Inch)
 FT : REAL (Foot)
 YD : REAL (Yard)

MILE : REAL (Mile)
 SM : REAL (Internationale Seemeile)
 FM : REAL (Fathom)
 Output YM : REAL (Meter)
 YP : REAL (Typographischer Punkt)
 YIN : REAL (Inch)
 YFT : REAL (Foot)
 YYD : REAL (Yard)
 YMILE : REAL (Mile)
 YSM : REAL (Internationale Seemeile)
 YFM : REAL (Fathom)



Der Baustein LENGTH konvertiert verschiedene in der Praxis gebräuchliche Einheiten für Längeneinheiten. Normalerweise wird nur der zu konvertierende Eingang belegt und die restlichen Eingänge bleiben frei. Werden jedoch mehrere Eingänge gleichzeitig mit Werten beaufschlagt, so werden die Werte aller Eingänge entsprechend umgewandelt und dann aufsummiert.

1 P = 0,376065 mm (Einheit aus dem Druckereigewerbe)

1 IN = 25,4 mm

1 FT = 0,3048 m

1 YD = 0,9144 m

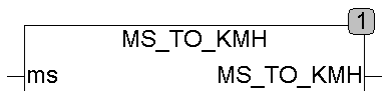
1 MILE = 1609,344 m

1 SM = 1852 m

1 FM = 1,829 m

21.10 MS_TO_KMH

Type Funktion : REAL
 Input KMH : REAL (Geschwindigkeit in km/h)
 Output REAL (Geschwindigkeit in m/s)

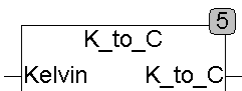


MS_TO_KMH rechnet einem Geschwindigkeitswert von Meter / Sekunde in Kilometer / Stunde um.

$MS_TO_KMH := MS * 3.6$

21.11 K_TO_C

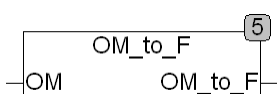
Type Funktion : REAL
 Input KELVIN : REAL (Temperaturwert in Kelvin)
 Output REAL (Temperaturwert in °C)



K_TO_C rechnet einen Temperaturwert von Kelvin in °C um.

21.12 OM_TO_F

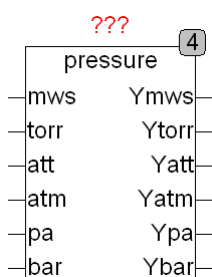
Type Funktion : REAL
 Input OM : REAL (Kreisfrequenz Omega)
 Output REAL (Frequenz in Hz)



OM_TO_F berechnet die Frequenz in Hz aus der Kreisfrequenz Omega.

21.13 PRESSURE

Type	Funktionsbaustein
Input	MWS : REAL (Wassersäule in Meter) TORR : REAL (Torr Bzw. Quecksilbersäule in mm) ATT : REAL (Atmosphäre technisch) ATM : REAL (Atmosphäre physikalisch) PA : REAL (Pascal) BAR : REAL (Bar)
Output	YMWS : REAL (Wassersäule in Meter) YTORR : REAL (Torr Bzw. Quecksilbersäule in mm) YATT : REAL (Atmosphäre technisch) YATM : REAL (Atmosphäre physikalisch) YPA : REAL (Pascal) YBAR : REAL (Bar)



Der Baustein PRESSURE konvertiert verschiedene in der Praxis gebräuchliche Einheiten für Druck. Normalerweise wird nur der zu konvertierende Eingang belegt und die restlichen Eingänge bleiben frei. Werden jedoch mehrere Eingänge gleichzeitig mit Werten beaufschlagt, so werden die Werte aller Eingänge entsprechend umgewandelt und dann aufsummiert.

1 MWS = 1 Meter Wassersäule = 0,0980665 Bar

1 TORR = 1 mm Quecksilbersäule = 0,133322 Bar = 101325 / 760 Pa

1 ATT = 1 kp / cm² = 0,980665 Bar

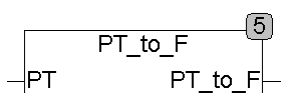
1 ATM = 1,01325 Bar

1 PA = 1 N / m²

1 BAR = 10⁵ Pa

21.14 PT_TO_F

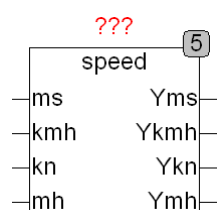
Type Funktion : REAL
 Input PT : TIME (Periodendauer in Sekunden)
 Output REAL (Frequenz in Hz)



PT_TO_F rechnet eine Periodendauer in Sekunden in die entsprechende Frequenz in Hz um.

21.15 SPEED

Type Funktionsbaustein
 Input MS : REAL (Meter / Sekunde)
 KMH : REAL (Kilometer / Stunde)
 KN : REAL (Knoten = Seemeilen / Stunde)
 MH : REAL (Meilen / Stunde)
 Output YMS : REAL (Meter / Sekunde)
 YKMH : REAL (Kilometer / Stunde)
 YKN : REAL (Knoten = Seemeilen / Stunde)
 YMH : REAL (Meilen / Stunde)



Der Baustein SPEED konvertiert verschiedene in der Praxis gebräuchliche Einheiten für Geschwindigkeiten. Normalerweise wird nur der zu konvertierende Eingang belegt und die restlichen Eingänge bleiben frei. Werden jedoch mehrere Eingänge gleichzeitig mit Werten beaufschlagt, so werden die Werte aller Eingänge entsprechend umgewandelt und dann aufsummiert.

1 ms = Meter / Sekunde = 3,6 km/h

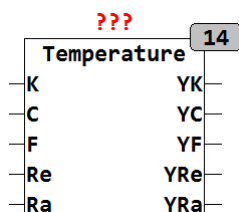
1 kmh = Kilometer / Stunde = 1/3,6 m/s

1 kn = Knoten = 1 Seemeile / Stunde = 0,5144 m/s

1 mh = Meilen / Stunde = 0,44704 m/s

21.16 TEMPERATURE

Type	Funktionsbaustein
Input	K : REAL (Temperatur nach Kelvin Skala) C : REAL (Temperatur nach Celsius Skala) F : REAL (Temperatur nach Fahrenheit Skala) RE : REAL (Temperatur nach Reaumur Skala) RA : Real (Temperatur nach Rankine Skala)
Output	YK : REAL (Temperatur nach Kelvin Skala) YC : REAL (Temperatur nach Celsius Skala) YF : REAL (Temperatur nach Fahrenheit Skala) YRE : REAL (Temperatur nach Reaumur Skala) YRA : Real (Temperatur nach Rankine Skala)



Der Baustein TEMP konvertiert verschiedene in der Praxis gebräuchliche Einheiten für Temperatur. Normalerweise wird nur der zu konvertierende Eingang belegt und die restlichen Eingänge bleiben frei. Werden jedoch mehrere Eingänge gleichzeitig mit Werten beaufschlagt, so werden die Werte aller Eingänge entsprechend umgewandelt und dann aufsummiert.

$$1 \text{ K} = 273.15 \text{ }^{\circ}\text{C}$$

$$1 \text{ }^{\circ}\text{C} = 273.15 \text{ K}$$

$$1 \text{ }^{\circ}\text{F} = \text{ }^{\circ}\text{C} * 1.8 + 32$$

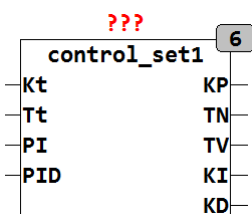
$$1 \text{ Re} = \text{ }^{\circ}\text{C} * 0.8$$

$$1 \text{ Ra} = \text{K} * 1.8$$

22 Regelungstechnik

22.1 CONTROL_SET1

Type	Funktionsblock
Input	KT : REAL (Kritische Verstärkung) TT : REAL (Periodendauer der kritischen Schwingung) PI : BOOL (TRUE wenn Parameter für PI Regler bestimmt sind) PID : BOOL (TRUE wenn Parameter für PID Regler bestimmt sind)
Config	P_K : REAL := 0.5 (Vorgabewert KP für P Regler) PI_K : REAL := 0.45 (Vorgabewert KP für PI Regler) PI_TN : REAL := 0.83 (Vorgabewert TN für PI Regler) PID_K : REAL := 0.6 (Vorgabewert KP für PID Regler) PID_TN : REAL := 0.5 (Vorgabewert TN für PID Regler) PID_TV : REAL := 0.125 (Vorgabewert TV für PID Regler)
Output	KP : REAL (Regelverstärkung KP) TN : REAL (Nachstellzeit des Integrators) TV : REAL (Vorhaltezeit des Differenzierers) KI : REAL (Verstärkungsfaktor des Integrators) KD : REAL (Verstärkungsfaktor des Differenzierers)



CONTROL_SET1 berechnet Einstellparameter für P, PI und PID Controller nach den Ziegler-Nichols Verfahren. Hierbei wird die kritische Verstärkung KT, und die Periodendauer der kritischen Schwingung TT angegeben. Die Parameter werden ermittelt indem der Regler als reiner P-Regler betrieben wird und die Verstärkung solange hochgefahren wird bis eine Dauerschwingung konstanter Amplitude einsetzt. Die ent-

sprechenden Werte K_T und T_T werden dann ermittelt. Nachteil dieses Verfahrens ist das nicht jeder reale Regelkreis an die Stabilitätsgrenze gefahren werden kann, und das diese verfahren für langsame Regelkreise wie Raumregelungen sehr viel Zeit in Anspruch nimmt.

Reglertyp	PI	PID	KP	TN	TV
P-Regler	0	0	$P_K * K_T$		
PI-Regler	1	0	$PI_K * K_T$	$PI_{TN} * T_T$	
PID-Regler	0	1	$PID_K * K_T$	$PID_{TN} * T_T$	$PID_{TV} * T_T$

Die Vorgabewerte der Einstellregeln sind in Config Variablen definiert und können vom Anwender verändert werden. Die folgende Tabelle zeigt die Default Werte

Reglertyp	PI	PID	KP	TN	TV
P-Regler	0	0	$P_K = 0.5$		
PI-Regler	1	0	$PI_K = 0.45$	$PI_{TN} = 0.83$	
PID-Regler	0	1	$PID_K = 0.6$	$PID_{TN} = 0.5$	$PID_{TV} = 0.125$

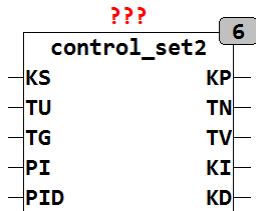
22.2 CONTROL_SET2

Type	Funktionsblock
Input	K_T : REAL (Kritische Verstärkung) T_T : REAL (Periodendauer der kritischen Schwingung) PI : BOOL (TRUE wenn Parameter für PI Regler bestimmt sind) PID : BOOL (TRUE wenn Parameter für PID Regler bestimmt sind)
Config	P_K : REAL := 0.5 (Vorgabewert K_P für P Regler) PI_K : REAL := 0.45 (Vorgabewert K_P für PI Regler) PI_{TN} : REAL := 0.83 (Vorgabewert T_N für PI Regler) PID_K : REAL := 0.6 (Vorgabewert K_P für PID Regler) PID_{TN} : REAL := 0.5 (Vorgabewert T_N für PID Regler) PID_{TV} : REAL := 0.125 (Vorgabewert T_V für PID Regler)
Output	K_P : REAL (Regelverstärkung K_P) T_N : REAL (Nachstellzeit des Integrators)

TV : REAL (Vorhaltezeit des Differenzierers)

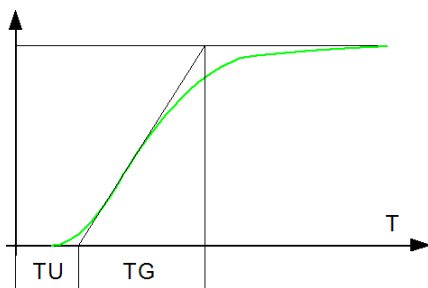
KI : REAL (Verstärkungsfaktor des Integrators)

KD : REAL (Verstärkungsfaktor des Differenzierers)



CONTROL_SET2 berechnet Einstellparameter für P, PI und PID Controller nach den Ziegler-Nichols Verfahren. Hierbei wird die Verzugszeit TU und die Ausgleichzeit TG angegeben. Die Parameter werden ermittelt indem die Sprungantwort der Regelstrecke gemessen wird. TU entspricht der Zeit nach der der Ausgang der Regelstrecke 5% seines Maximalwertes erreicht hat. TG ist die Zeit die Zeit der Wendetangente der Regelstrecke. KS ist Istwertänderung der Regelstrecke / Stellgrößenveränderung.

Die folgende Grafik zeigt die Ermittlung von TU und TG mit dem Wendetangentenverfahren:



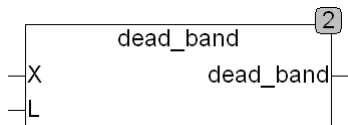
Reglertyp	PI	PID	KP	TN	TV
P-Regler	0	0	$P_K * TG / TU / KS$		
PI-Regler	1	0	$PI_K * TG / TU / KS$	$PI_TN * TU$	
PID-Regler	0	1	$PID_K * TG / TU / KS$	$PID_TN * TU$	$PID_TV * TU$

Die Vorgabewerte der Einstellregeln sind in Config Variablen definiert und können vom Anwender verändert werden. Die folgende Tabelle zeigt die Default Werte:

Reglertyp	PI	PID	KP	TN	TV
P-Regler	0	0	$P_K = 1.0$		
PI-Regler	1	0	$PI_K = 0.9$	$PI_TN = 3.33$	
PID-Regler	0	1	$PID_K = 1.2$	$PID_TN = 2$	$PID_TV = 0.5$

22.3 DEAD_BAND

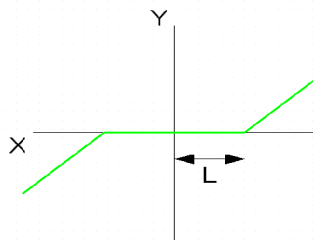
Type Funktionsblock
 Input X : REAL (Eingangswert)
 L : REAL (Lockout Wert)
 Output Y : REAL (Ausgangswert)



DEAD_BAND ist eine lineare Übertragungsfunktion mit Totzone. Die Funktion verschiebt den positiven Teil der Kurve um +L und den negativen Teil der Kurve um -L. DEAD_BAND wird benutzt um Quantisierungsrauschen und andere Rauschanteile aus einem Signal zu filtern. DEAD_Band wird zum Beispiel in Regelkreisen eingesetzt um zu verhindern das der Regler dauernd in kleinen Schritten schaltet und dabei das Stellglied übermäßig belastet und abnutzt.

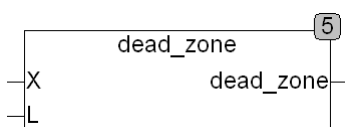
$$\text{DEAD_BAND} = X - \text{SGN}(X) \cdot L \text{ wenn } \text{ABS}(X) > L \text{ wenn } \text{ABS}(X) > L$$

$$\text{DEAD_BAND} = 0 \text{ wenn } \text{ABS}(X) \leq L$$



22.4 DEAD_ZONE

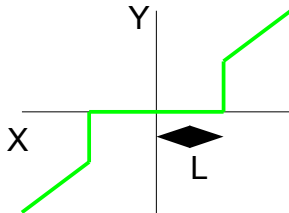
Type Funktion : REAL
 Input X : REAL (Eingangswert)
 L : REAL (Lockout Wert)
 Output REAL (Ausgangswert)



DEAD_ZONE ist eine lineare Übertragungsfunktion mit Totzone. Der Ausgang entspricht dem Eingangssignal, wenn der Absolutwert des Eingangs größer als L ist.

$DEAD_ZONE = X$ wenn $ABS(X) > L$

$DEAD_ZONE = 0$ wenn $ABS(X) \leq L$



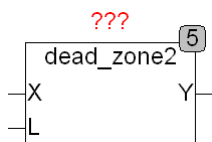
22.5 DEAD_ZONE2

Type Funktionsblock

Input X : REAL (Eingangswert)

L : REAL (Lockout Wert)

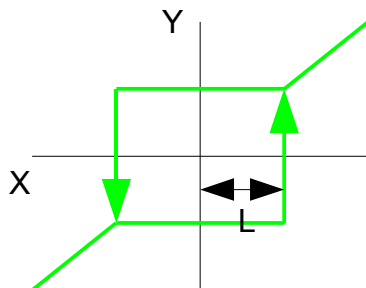
Output Y : REAL (Ausgangswert)



DEAD_ZONE2 ist eine lineare Übertragungsfunktion mit Totzone und Hysterese. Der Ausgang entspricht dem Eingangssignal, wenn der Absolutwert des Eingangs größer als L ist.

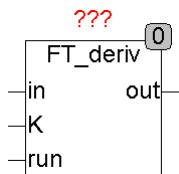
$DEAD_ZONE2 = X$ wenn $ABS(X) > L$

$DEAD_ZONE2 = \pm L$ wenn $ABS(X) \leq L$



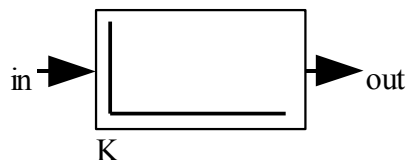
22.6 FT_DERIV

Type	Funktionsblock
Input	IN : REAL (Eingangssignal) K : REAL (Multiplikator) RUN : BOOL (Freigabe Eingang)
Output	OUT : REAL (Ableitung des Eingangssignals $K * \Delta X / \Delta T$)



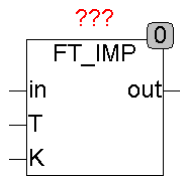
FT_DERIV ist ein D-Glied, oder auch LZI-Übertragungsglied, welches ein differenzierendes Übertragungsverhalten aufweist. Am Ausgang von FT_DERIV steht die Ableitung über die Zeit T in Sekunden zur Verfügung. Wenn das Eingangssignal in einer Sekunde von 3 auf 4 steigt so ist der Ausgang $1 * K$ ($K * \Delta X / \Delta T = 1 * (4 - 3) / 1 = 1$).

Anders ausgedrückt ist die Ableitung des Eingangssignals die momentane Steigung des Eingangssignals. Mit dem Eingang RUN kann FT_DERIV enabled, beziehungsweise disabled werden. FT_DERIV arbeitet intern in Mikrosekunden und wird dadurch auch den Anforderungen sehr schneller SPS Controller mit Zykluszeiten unter einer Millisekunde gerecht.



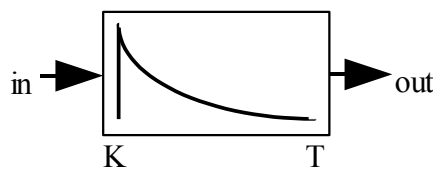
22.7 FT_IMP

Type	Funktionsblock
Input	IN : REAL (Eingangssignal) T : TIME (Zeitkonstante) K : REAL (Multiplikator)
Output	OUT : REAL (Hochpass Mit Zeitkonstante T)



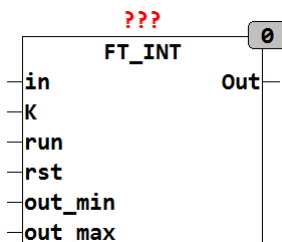
FT_IMP Ist ein Hochpass-Filter mit Zeitkonstante T und Multiplikator K . Eine sprunghafte Änderung am Eingang wird am Ausgang sichtbar, klingt aber nach der Zeit T bereits um 63% und nach $3 * T$ um 95% ab. Somit ist nach einer sprunghaften Änderung des Eingangssignals von 0 auf 10 der Ausgang zum Zeitpunkt der Eingangsänderung 10 und klingt nach $1 * T$ auf 3,7 und nach $3 * T$ auf 0,5 ab und wird dann allmählich wieder 0.

Strukturbild:



22.8 FT_INT

Type	Funktionsblock
Input	IN : REAL (Eingangssignal)
	K : REAL (Multiplikator)
	RUN : BOOL (Freigabe Eingang)
	RST : BOOL (Reset Eingang)
	OUT_MIN : REAL (unteres Ausgangs Limit)
	OUT_MAX : REAL (oberes Ausgangs Limit)
Output	OUT : REAL (Ausgangssignal)



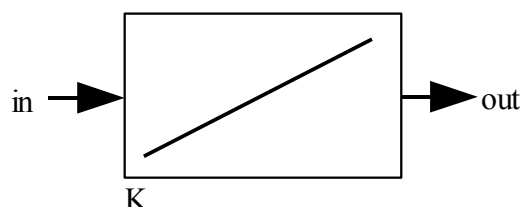
FT_INT ist ein Integratorbaustein der das Integral über das Eingangssignal am Ausgang bereitstellt. Der Eingang K ist ein Multiplikator für das Ausgangssignal. Run

schaltet den Integrator ein, wenn TRUE und Aus, wenn FALSE und RST (Reset) setzt alle internen Werte auf 0. Die Eingänge OUT_MIN und OUT_MAX dienen dazu obere und untere Grenzwerte für den Ausgang des Integrators festzulegen. FT_INT arbeitet intern in Mikrosekunden und wird dadurch auch den Anforderungen sehr Schneller SPS Controller mit Zykluszeiten unter einer Millisekunde gerecht.

Ein Grundsatzproblem bei Integratoren ist die Auflösung, So hat der Ausgang vom Typ Real eine Auflösung von 7-8 Stellen was zur Folge hat das bei einem errechneten Integrationsschritt von 1 bei einem Ausgangswert von größer als hundert Millionen (1E8) dieser Schritt nicht mehr aufaddiert werden kann da er unter die Auflösungsgrenze von maximal 8 Stellen beim Typ Real fällt. FT_INT addiert diese kleinen Werte in einer Zwischenvariable auf und addiert sie erst dann zum Ausgang wenn dieser Wert über der Auflösungsgrenze liegt. So kann FT_INT auch bei Verwendung von REAL als Datentyp exakte Ausgangswerte garantieren.

Beispiel: ein Eingangssignal von 0,0001 bei einer Abtastzeit von 1 Millisekunde und einem Ausgangswert von 100000 würde einen Wert von $0,0001 * 0,001 \text{ Sekunden} = 0,000001$ zum Ausgangswert von 100000 addieren, was unweigerlich wieder den Wert von 100000 ergibt, denn die Auflösung des Datentyps Real kann nur maximal 8 Stellen erfassen. FT_INT wird diesen Wert von 0,000001 solange zwischenspeichern bis die Summe die Auflösungsgrenze überschritten hat und ihn erst dann zum Ausgang addieren. Dies ist vor allem von Vorteil wenn FT_INT als Verbrauchszähler oder ähnlichen Anwendungen zum Einsatz kommen soll. Auch bei Anwendungen in der Regelungstechnik wo der Integrator kleinste Regelabweichungen korrigieren soll wird dieses Verfahren die Genauigkeit drastisch erhöhen.

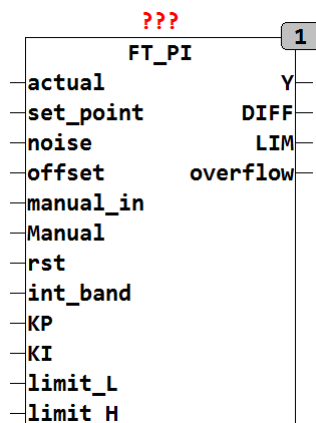
Strukturbild:



22.9 FT_PI

Type	Funktionsbaustein
Input	ACTUAL : REAL (gemessener Wert nach der Strecke) SET_POINT : REAL (Vorgabewert) NOISE : REAL (Rauschunterdrückung) OFFSET : REAL (Offset für den Ausgang)

	MANUAL_IN : REAL (Eingangswert für manuellen Betrieb)
	MANUAL : BOOL (Umschalten auf Handbetrieb, MANUAL = TRUE)
	RST : BOOL (Asynchroner Reset-Eingang)
	INT_BAND : REAL (Regelabweichung bei der der Integrator einschaltet)
	KP : REAL (Proportionaler Anteil des Reglers)
	KI : REAL (Integraler Anteil des Reglers)
	LIMIT_L : REAL (untere Ausgangsbegrenzung)
	LIMIT_H : REAL (obere Ausgangsbegrenzung)
Config	INT_LIMIT_L : REAL (unteres Limit des Integrators)
	INT_LIMIT_H : Real (oberes Limit des Integrators)
Output	Y : REAL (Ausgang des Reglers)
	DIFF : Real (Regelabweichung)
	LIM : BOOL (TRUE, wenn der Ausgang ein Limit erreicht hat)
	OVERFLOW : BOOL (TRUE, wenn der Integrator überläuft)



FT_PI ist ein PI-Regler mit Strukturumschaltung für den Integralteil und manuellem Steuereingang. Der PI Regler arbeitet nach der Formel:

$$Y = KP * DIFF + KI * INTEG(DIFF) + OFFSET$$

wobei $DIFF = SET_POINT - ACTUAL$

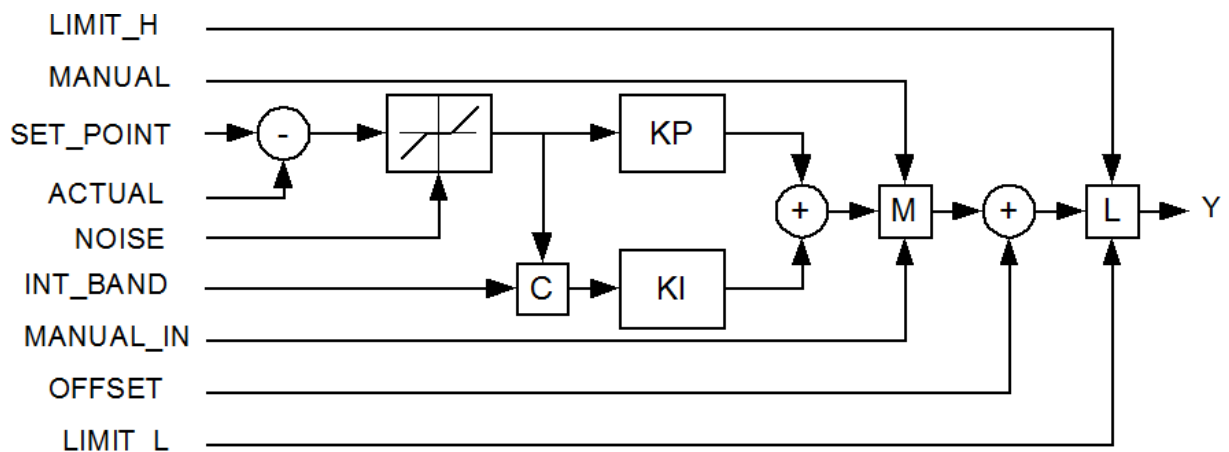
Im Handbetrieb (Manual = TRUE) gilt: $Y = MANUAL_IN + OFFSET$

ACTUAL ist der gemessene Wert nach der Regelstrecke und Set_POINT ist die Sollwertvorgabe für den Regler. Die Eingangswerte LIMIT_H und LIMIT_L begrenzen den Ausgangswert Y, mit den Config Variablen INT_LIMIT_L und INT_LIMIT_H kann der Interne Integrator vor einem Überlauf geschützt werden. Mit RST kann der interne Integrator jederzeit auf 0 gesetzt werden. Der Ausgang LIM signalisiert das der Regler an eine der Grenzen LIMIT_L oder LIMIT_H gelaufen ist und OVERFLOW zeigt an, dass der interne Integrator an seine Limits erreicht hat (Wind-Up). Der PI-Regler arbeitet frei laufend und benutzt zur Berechnung des Integrators die Trapezregel für höchste Genauigkeit und optimale Geschwindigkeit. Mit dem Eingangswert

INT_BAND wird die Strukturumschaltung bestimmt. Mit dem Wert INT_BAND wird festgelegt unterhalb welcher Regelabweichung der Integrator aktiv wird. Die Default-Werte der Eingangsparameter sind wie folgt vordefiniert: $K_P = 1$, $K_I = 1$, $LIMIT_L = -1000$ und $LIMIT_H = +1000$. Mit dem Eingang NOISE wird eine Rauschunterdrückung eingestellt, der Wert am Eingang NOISE legt fest ab welcher Regeldifferenz der Regler einschaltet. Mit NOISE wird vermieden das der Ausgang des Reglers dauern schwankt. Der Wert am Eingang NOISE sollte so bemessen sein das er das Rauschen der Regelstrecke und der Sensoren unterdrückt. Wird zum Beispiel der Eingang NOISE auf 0.1 gesetzt so wird der Regler erst bei Regelabweichungen größer als 0.1 aktiv. Der Ausgang DIFF stellt die gemessene und durch ein Noise Filter (DEAD_BAND) gefilterte Regelabweichung zur Verfügung. DIFF wird in einer Regelstrecke normalerweise nicht benötigt, kann aber zur Beeinflussung der Regelparameter benutzt werden. Der Eingang Offset wird als letzter Wert zum Ausgang addiert, und dient vor allem zum Aufschalten von Störsignalen, deren Wirkung auf den Regelkreis abgeschätzt werden kann.

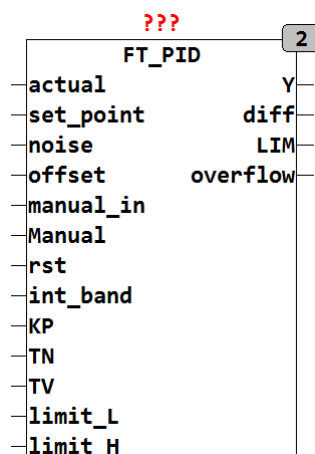
Strukturumschaltung: Die Regelparameter K_P und K_I stellen den PI Regler auf einen Arbeitspunkt ein. Dieser Arbeitspunkt wird aber verlassen, wenn eine Sollwertänderung (ACTUAL) statt findet. Zum Beispiel Tag / Nacht Umschaltung bei Raumregelungen. Der bei Anlauf und sprunghafter Änderung des Sollwertes störende I-Anteil des PI Reglers wird deshalb erst in der Nähe des Sollwertes eingeschaltet und regelt so mit vollem PI-Verhalten nur in der Nähe des Arbeitspunktes. Dieses Verfahren verhindert auch das so genannte „Wind-Up“ des Integrators. Der Integrator wird erst eingeschaltet, wenn der IST-Wert innerhalb INT_BAND vom Sollwert liegt. Wird INT_BAND nicht belegt wird ein Default von 100 aufgeschaltet und der Integrator ist praktisch immer ein. Bei Reglern mit sehr hohen Regelwerten (K_P sehr groß) sollte dieser Wert überprüft und gegebenenfalls angepasst werden.

Die folgende Grafik verdeutlicht die interne Struktur des Reglers:



22.10 FT_PID

Type	Funktionsbaustein
Input	ACTUAL : REAL (gemessener Wert nach der Strecke) SET_POINT : REAL (Vorgabewert) NOISE : REAL (Rauschunterdrückung) OFFSET : REAL (Offset für den Ausgang) MANUAL_IN : REAL (Eingangswert für manuellen Betrieb) MANUAL : BOOL (Umschalten auf Handbetrieb, MANUAL = TRUE) RST : BOOL (Asynchroner Reset-Eingang) INT_BAND : REAL (Regelabweichung bei der der Integrator einschaltet) KP : REAL (Verstärkung des Reglers) TN : REAL (Nachstellzeit des Reglers) TV : REAL (Vorhaltezeit des Reglers) LIMIT_L : REAL (untere Ausgangsbegrenzung) LIMIT_H : REAL (obere Ausgangsbegrenzung)
Config	INT_LIMIT_L : REAL (unteres Limit des Integrators) INT_LIMIT_H : Real (oberes Limit des Integrators)
Output	Y : REAL (Ausgang des Reglers) DIFF : Real (Regelabweichung) LIM : BOOL (TRUE, wenn der Ausgang ein Limit erreicht hat) OVERFLOW : BOOL (TRUE, wenn der Integrator überläuft)



FT_PID ist ein PID-Regler mit Strukturumschaltung für den Integralteil und manuellem Steuereingang. Der PID Regler arbeitet nach der Formel:

$$Y = KP * (DIFF + 1/TN * INTEG(DIFF) + TV *DERIV(DIFF)) + OFFSET$$

wobei $DIFF = SET_POINT - ACTUAL$

Im Handbetrieb (Manual = TRUE) gilt: $Y = MANUAL_IN + OFFSET$

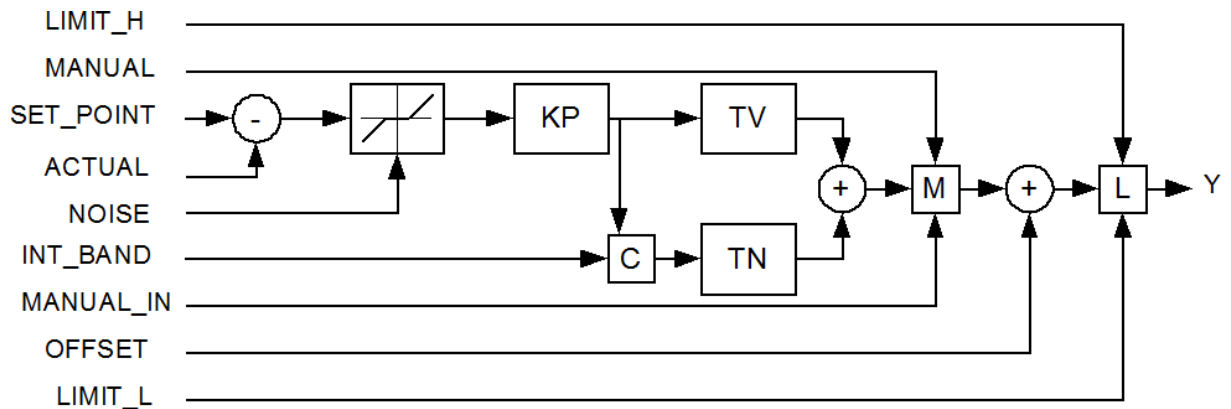
ACTUAL ist der gemessene Wert nach der Regelstrecke und Set_POINT ist die Sollwertvorgabe für den Regler. Die Eingangswerte LIMIT_H und LIMIT_L begrenzen den Ausgangswert Y, mit den Config Variablen INT_LIMIT_L und INT_LIMIT_H kann der Interne Integrator vor einem Überlauf geschützt werden. Mit RST kann der interne Integrator jederzeit auf 0 gesetzt werden. Der Ausgang LIM signalisiert das der Regler an eine der Grenzen LIMIT_L oder LIMIT_H gelaufen ist und OVERFLOW zeigt an, dass der interne Integrator an seine Limits erreicht hat (Wind-Up). Der PID-Regler arbeitet frei laufend und benutzt zur Berechnung des Integrators die Trapezregel für höchste Genauigkeit und optimale Geschwindigkeit. Mit dem Eingangswert INT_BAND wird die Strukturumschaltung bestimmt. Mit dem Wert INT_BAND wird festgelegt unterhalb welcher Regelabweichung der Integrator aktiv wird. Die Default-Werte der Eingangsparameter sind wie folgt vordefiniert: $KP = 1$, $TN = 1$, $TV = 1$, $LIMIT_L = -1000$ und $LIMIT_H = +1000$. Mit dem Eingang NOISE wird eine Rauschunterdrückung eingestellt, der Wert am Eingang NOISE legt fest ab welcher Regeldifferenz der Regler einschaltet. Mit NOISE wird vermieden das der Ausgang des Reglers dauern schwankt. Der Wert am Eingang NOISE sollte so bemessen sein das er das Rauschen der Regelstrecke und der Sensoren unterdrückt. Wird zum Beispiel der Eingang NOISE auf 0.1 gesetzt so wird der Regler erst bei Regelabweichungen größer als 0.1 aktiv. Der Ausgang DIFF stellt die gemessene und durch ein Noise Filter (DEAD_BAND) gefilterte Regelabweichung zur Verfügung. DIFF wird in einer Regelstrecke normalerweise nicht benötigt, kann aber zur Beeinflussung der Regelparameter benutzt werden. Der Eingang Offset wird als letzter Wert zum Ausgang addiert, und dient vor allem zum Aufschalten von Störsignalen, deren Wirkung auf den Regelkreis abgeschätzt werden kann.

Strukturumschaltung:

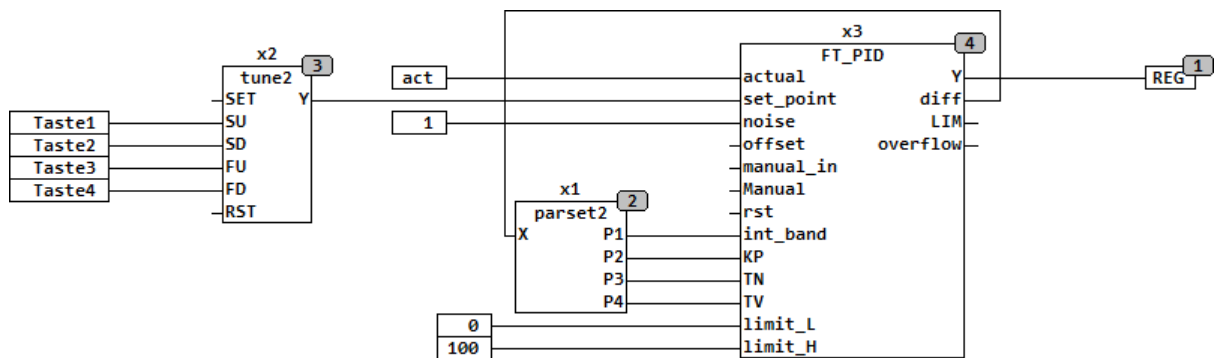
Die Regelparameter KP, TN und TV stellen den PID Regler auf einen Arbeitspunkt ein. Dieser Arbeitspunkt wird aber verlassen, wenn eine Sollwertänderung (ACTUAL) statt findet. Zum Beispiel Tag / Nacht Umschaltung bei Raumregelungen. Der bei Anlauf und sprunghafter Änderung des Sollwertes störende I-Anteil des PID Reglers wird deshalb erst in der Nähe des Sollwertes eingeschaltet und regelt so mit vollem PID-Verhalten nur in der Nähe des Arbeitspunktes. Dieses Verfahren verhindert auch das so genannte „Wind-Up“ des Integrators. Der Integrator wird erst eingeschaltet, wenn der IST-Wert innerhalb INT_BAND vom Sollwert liegt. Wird INT_BAND nicht belegt wird ein Default von 100 aufgeschaltet und der Integrator ist praktisch immer ein. Bei Reglern mit sehr hohen Regelwerten (KP sehr groß) sollte dieser Wert überprüft und gegebenenfalls angepasst werden. Die Regelparameter werden in der Form KP, TN und TV angegeben, falls die Parameter als KP, KI und KD vorliegen können sie entsprechend der folgenden Formel umgerechnet werden:

$$TN = KP/KI \text{ und } TV = KD/KP$$

Die folgende Grafik zeigt die interne Struktur des PID Reglers:

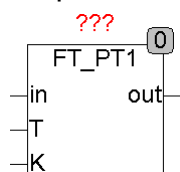


Im folgenden Beispiel wir ein PID Regler gezeigt, dessen SET_POINT durch den Baustein TUNE2 mittels Taster erzeugt wird. Der Ausgang DIFF wird an einen Baustein PARSET2 geleitet welcher abhängig von der Regelabweichung am Ausgang DIFF die Parameter INT_BAND, KP, TN und TV verändert.



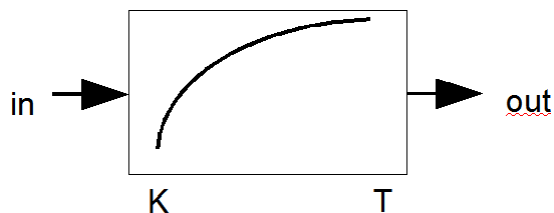
22.11 FT_PT1

Type Funktionsblock
 Input IN : REAL (Eingangssignal)
 T : TIME (Zeitkonstante)
 K : REAL (Multiplikator)
 Output OUT : REAL (Ausgangssignal)

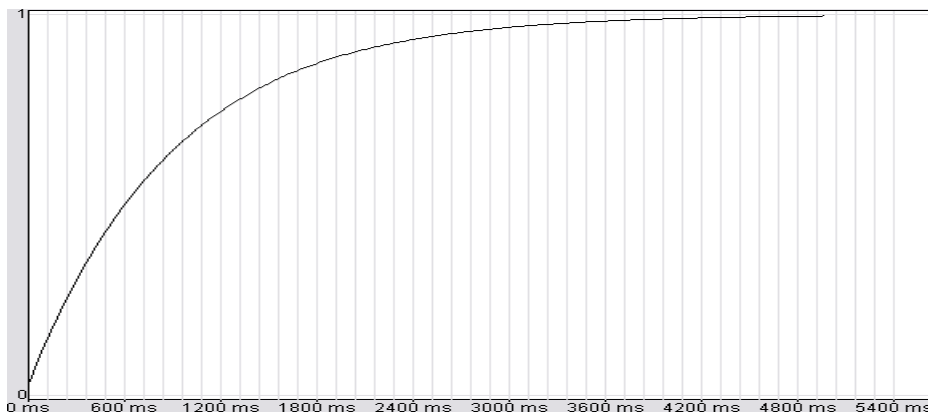


FT_PT1 ist ein LZI-Übertragungsglied mit einem proportionalen Übertragungsverhalten 1. Ordnung, auch als Tiefpass Filter 1. Ordnung bezeichnet. Der Multiplikator K legt den Verstärkungsfaktor (Multiplikator) fest und T die Zeitkonstante. Eine Änderung am Eingang wird am Ausgang gedämpft sichtbar. Das Ausgangssignal steigt innerhalb von T auf 63% des Eingangswerts und nach $3 * T$ auf 95% des Eingangswerts an. Somit ist nach einer sprunghaften Änderung des Eingangssignals von 0 auf 10 der Ausgang zum Zeitpunkt der Eingangsänderung 0, steigt nach $1 * T$ auf 6,3 an und erreicht nach $3 * T$ 9,5 und nähert sich dann asymptotisch dem Wert 10 an. Beim ersten Aufruf wird der Ausgang OUT mit dem Eingangswert IN initialisiert um ein definiertes Anlauf Verhalten zu gewährleisten. Falls der Eingang T gleich $T\#0s$ ist entspricht der Ausgang $OUT = K * IN$.

Strukturbild:

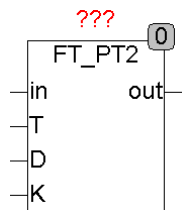


Sprungantwort für $T=1s$, $K=1$



22.12 FT_PT2

Type	Funktionsblock
Input	IN : REAL (Eingangssignal)
	T : REAL (Zeitkonstante)
	D : REAL (Dämpfung)
	K : REAL (Multiplikator)
Output	OUT : REAL (Ausgangssignal)

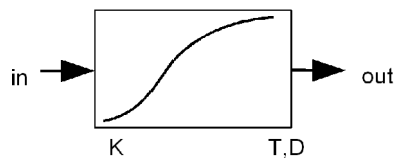


FT_PT2 ist ein LZI-Übertragungsglied mit einem proportionalen Übertragungsverhalten 2. Ordnung, auch als Tiefpass Filter 2. Ordnung bekannt. Der Multiplikator K legt den Verstärkungsfaktor (Multiplikator) fest, T und D die Zeitkonstante und die Dämpfung. Falls der Eingang T gleich T#0s ist entspricht der Ausgang $OUT = K * IN$.

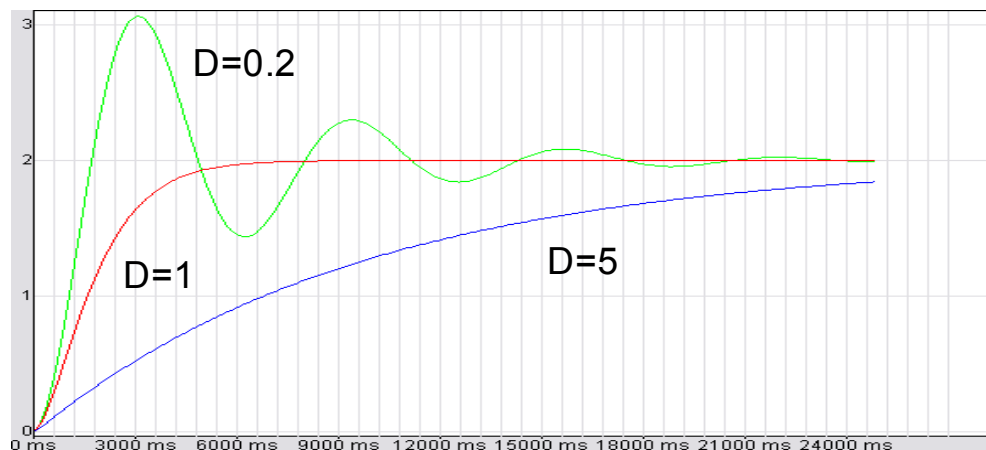
Die entsprechende Funktionalbeziehung in Zeitbereich ist folgende Differenzialgleichung gegeben:

$$T^2 * OUT''(T) + 2 * D * T * OUT'(T) + OUT(T) = K * in(T).$$

Strukturbild:

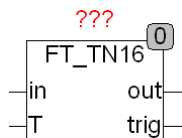


Sprungantwort für $T=1$, $K=2$, $D=0,2 / 1 / 5$

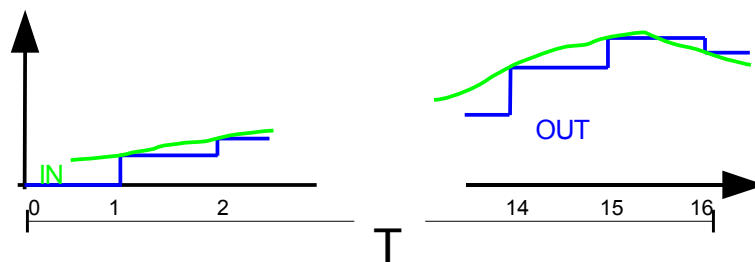


22.13 FT_TN16

Type Funktionsblock
 Input IN : REAL (Eingangssignal)
 T : REAL (Verzögerungszeit)
 Output OUT : REAL (Ausgangssignal)

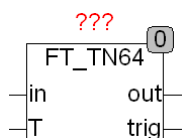


FT_TN16 verzögert ein Eingangssignal um eine einstellbare Zeit T und tastet es in der Zeit T 16 mal ab. Nach jedem update des Ausgangssignals OUT wird TRIG für einen Zyklus TRUE.

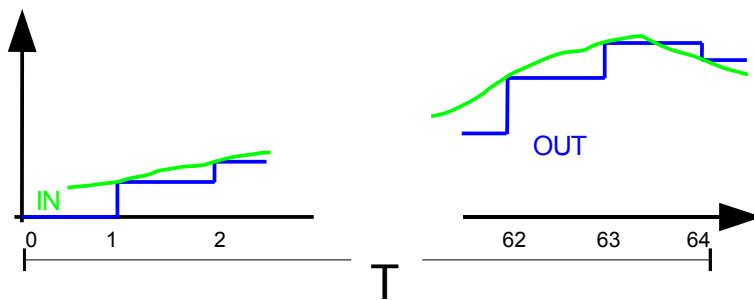


22.14 FT_TN64

Type Funktionsblock
 Input IN : REAL (Eingangssignal)
 T : REAL (Verzögerungszeit)
 Output OUT : REAL (Ausgangssignal)

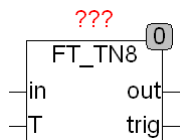


FT_TN64 verzögert ein Eingangssignal um eine einstellbare Zeit T und tastet es in der Zeit T 64 mal ab. Nach jedem update des Ausgangssignals OUT wird TRIG für einen Zyklus TRUE.

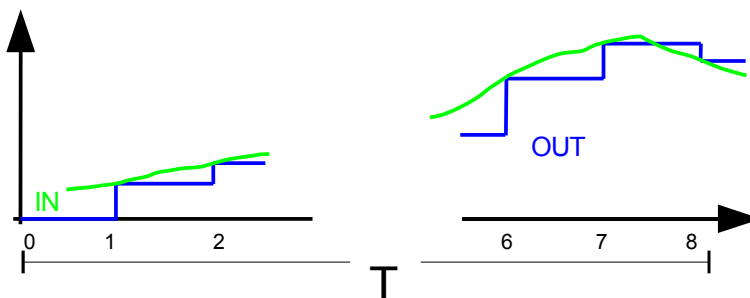


22.15 FT_TN8

Type Funktionsblock
 Input IN : REAL (Eingangssignal)
 T : REAL (Verzögerungszeit)
 Output OUT : REAL (Ausgangssignal)

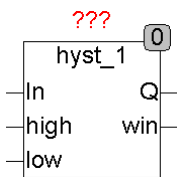


FT_TN8 verzögert ein Eingangssignal um eine einstellbare Zeit T und tastet es in der Zeit T 8 mal ab. Nach jedem update des Ausgangssignals OUT wird TRIG für einen Zyklus TRUE.



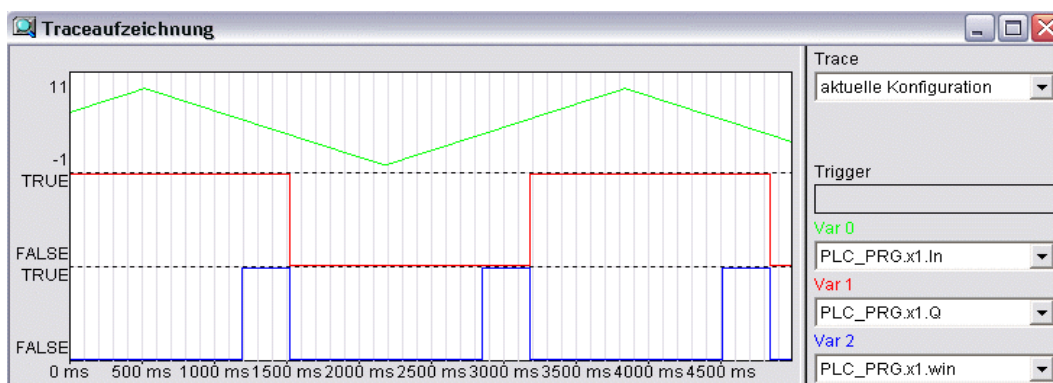
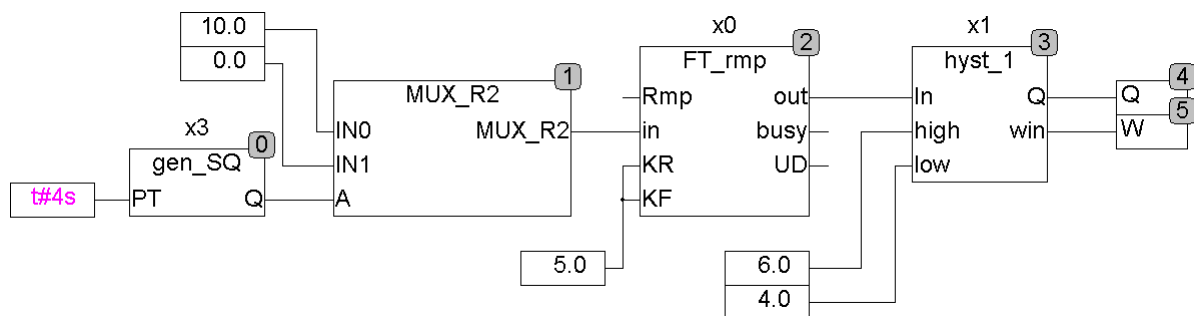
22.16 HYST_1

Type	Funktionsbaustein
Input	IN : REAL (Eingangswert)
	HIGH : REAL (oberer Schwellenwert)
	LOW : REAL (unterer Schwellenwert)
Output	Q : BOOL (Ausgangssignal)
	WIN : BOOL (zeigt an, dass In zwischen LOW und HIGH liegt)



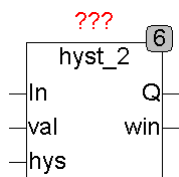
HYST_1 ist ein Hysterese Baustein der mit oberen und unterem Limit arbeitet. Der Ausgang Q wird nur dann TRUE, wenn das Eingangssignal an IN den Wert HIGH überschritten hat. Es bleibt dann solange TRUE, bis das Eingangssignal wieder LOW unterschreitet und Q FALSE wird. Ein weiterer Ausgang WIN zeigt an, ob sich das Eingangssignal zwischen LOW und HIGH befindet.

Das folgende Beispiel zeigt eine Dreiecksgenerator mit nachgeschalteten Hysterese Baustein HYST_1. Die grüne Kurve zeigt das Eingangssignal, Rot den Hysterese Ausgang und Blau den WIN Ausgang.



22.17 HYST_2

Type	Funktionsbaustein
Input	IN : REAL (Eingangswert) VAL : REAL (Mittelwert der Hysterese) HYS : REAL (Breite der Hysterese)
Output	Q : BOOL (Ausgangssignal) WIN : BOOL (zeigt an das In zwischen LOW und HIGH liegt)

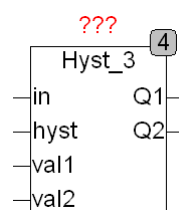


HYST_2 ist ein Hysterese Baustein bei dem Die Schaltschwellen durch einen Mittelwert und die zugehörige Hysterese definiert wird. Die untere Schaltschwelle liegt bei $VAL - HYS / 2$ und die obere Schaltschwelle bei $VAL + HYS / 2$.

Eine eingehende Beschreibung der Hysterese Funktion und ein Anwendungsbeispiel finden Sie unter HYST_1.

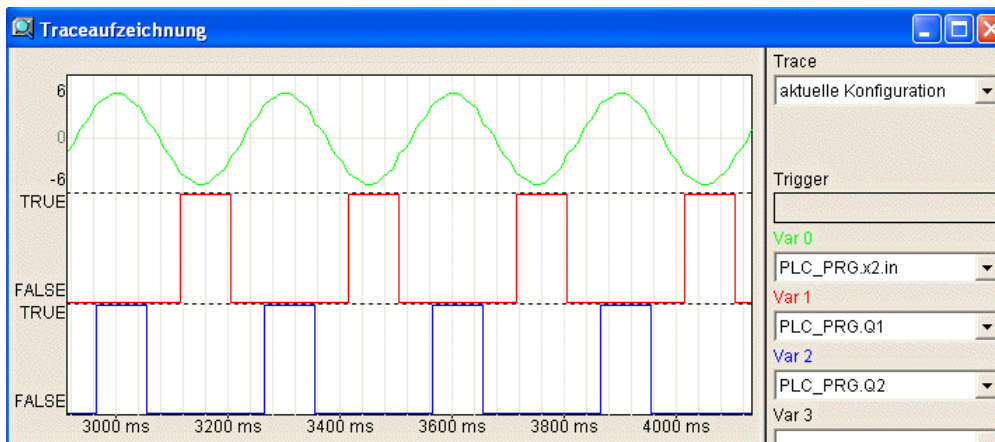
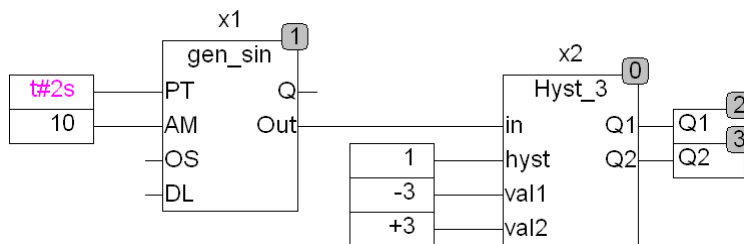
22.18 HYST_3

Type	Funktionsbaustein
Input	IN : REAL (Eingangswert) HYST : REAL (Breite der Hysterese) VAL1 : REAL (Mittelwert der Hysterese 1) VAL2 : REAL (Mittelwert der Hysterese 2)
Output	Q1 : BOOL (Ausgangssignal 1) Q2 : BOOL (Ausgangssignal 2)



HYST_3 ist ein Dreipunktregler. Der Dreipunktregler besteht aus 2 Hysteresefunktionen. Q1 ist eine Hysterese mit val1 als Schwellenwert und HYST als Hysterese. Q1 wird TRUE, wenn IN kleiner ist als $VAL1 - HYST / 2$ und wird FALSE, wenn IN größer ist als $VAL1 + HYST / 2$. Q2 funktioniert analog mit Val2. Der Dreipunktregler wird vor allen dann eingesetzt, wenn man motorische Klappen steuert, die dann mit Q1 auf und mit Q2 ab gesteuert werden. Ist der Wert von IN zwischen VAL1 und VAL2 bleiben beide Ausgänge FALSE und der Motor bleibt stehen.

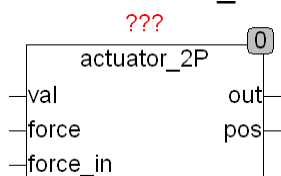
Folgendes Beispiel zeigt den Signalverlauf an einem 3-Punkt Regler:



23 Stellglieder

23.1 ACTUATOR_2P

Type	Funktionsblock
Input	VAL : REAL (Steuereingang 0 - 1) FORCE : BOOL (wenn TRUE: FORCE_IN steuert OUT) FORCE_IN : BOOL (Steuersignal)
Output	OUT : BOOL (Schaltsignal für Ventil) POS : REAL (Analoges Signal 0 – 1 das dem Mengenstrom entspricht)
Config	SELF_ACT_TIME : TIME (Selbstbetätigungszeit) SELF_ACT_CYCLES : INT (Anzahl Zyklen bei Selbstbetätigung) CYCLE_TIME : TIME (Taktrate des Ventils) MIN_ONTIME : TIME (Minimale Einschaltzeit) MIN_OFFTIME : TIME (Minimale Ausschaltzeit)



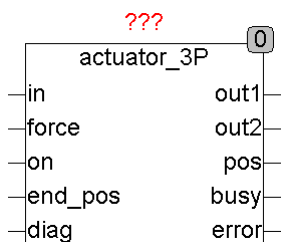
ACTUATOR_2P ist ein Interface für 2-Punkt Aktuatoren wie z. B. Magnetventile. Der 2-Punkt Aktuator kann nur ein / aus Schalten und deshalb wird der Eingangswert VAL in ein Puls / Pause Signal am Ausgang OUT gewandelt. Die Zykluszeit (CYCLE_TIME), minimale Einschaltzeit (MIN_ONTIME) und minimale Ausschaltzeit (MIN_OFFTIME) bestimmen die Schaltzeiten des Ausgangs. Damit ein Festkleben des Ventils durch langes ruhen verhindert wird, kann durch einstellen der Selbstbetätigungszeit (SELF_ACT_TIME) und der Anzahl der Selbstaktivierungszyklen (SELF_ACT_CYCLES) bestimmt werden, nach welcher Zeit wie viele Schaltzyklen automatisch ausgeführt werden, um ein festkleben des Ventils zu verhindern. Mit dem Eingangssignal FORCE kann das Ventil auf eine feste Stellung ein / aus gezwungen werden. Wenn FORCE = TRUE und FORCE_IN = TRUE, dann ist OUT TRUE und wenn FORCE = TRUE und FORCE_IN = FALSE, dann ist OUT FALSE. Der Ausgang POS gibt immer einen REAL-Wert aus, der dem momentanen Mengenfluss durch das Ventil entspricht. Der Eingangswert VAL kann von 0..1 variiert wer-

den. 0 bedeutet das Ventil ist dauernd geschlossen und 1 bedeutet das Ventil ist dauernd offen.

VAL	FORCE	FORCE_IN	POS	OUT
X	FALSE	-	X	Duty-Cycle X
-	TRUE	FALSE	0	FALSE
-	TRUE	TRUE	1	TRUE

23.2 ACTUATOR_3P

Type	Funktionsblock
Input	VAL : REAL (Eingang Steuersignal 0 - 1) FORCE : BOOL (ON / OFF Steuermodus wenn TRUE) ON : BOOL (Steuersignal wenn FORCE = TRUE)
Output	OUT1 : BOOL (Steuersignal für Klappe in Richtung auf) OUT2 : BOOL (Steuersignal für Klappe in Richtung geschlossen) POS : REAL (Wert 0–1 der der Momentanen Klappenstellung entspricht)
Config	MIN_ONTIME : TIME (Minimale Laufzeit für Motor) MIN_OFFTIME : TIME (Minimale Totzeit für Motor) MAX_RUNTIME : TIME (Maximale Laufzeit für den Motor) AUTO_DIAG_TIME : TIME (Zeit nach der Automatisch eine Diagnose durchgeführt wird) CAL_RUNTIME : TIME (Laufzeit des Motors nach der automatisch eine Kalibrierung durchgeführt wird.) SWITCH_AVAILABLE : BOOL (TRUE, wenn ein Endschalter angeschlossen ist)



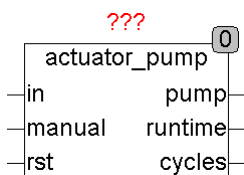
ACTUATOR_3P ist ein 3-Punkt Aktuator Interface zum Ansteuern von Stellmotoren mit Auf / Ab Eingang. Das Signal am Eingang IN wird umgesetzt in Steuerimpulse an den Ausgängen OUT1 und OUT2 die den Motor entsprechend Steuern. Das Eingangssignal IN wird so verarbeitet und die beiden Steuerausgänge (OUT1 und OUT2) so gesteuert, dass ein Eingangswert von 0 Klappe geschlossen, 1 Klappe offen, 0.5 Klappe halb geöffnet usw. bewirkt. Wird der FORCE-Eingang auf TRUE gesetzt, so folgt die Stellung der Klappen nicht mehr dem Eingangssignal IN, sondern folgt dem binären Steuersignal ON. ON=FALSE bedeutet Klappe geschlossen, ON=TRUE bedeutet Klappe 100% offen. Im Force Modus sind keine Zwischenstellungen möglich. Die Klappe kann nur 100% offen oder geschlossen sein. Der Baustein kann auch einen Endschalter verarbeiten. Die Endschalter müssen so angeschlossen werden, dass egal ob oberes oder unteres Ende erreicht wurden, der Eingang END_POS TRUE wird und damit anzeigt, dass die Klappe eine der beiden Endstellungen erreicht hat. Um die Endschalterfunktion in Betrieb zu setzen muss die Config-Variable SWITCH_AVAILABLE auf TRUE stehen, ansonsten wird der Endschalter ignoriert. Ein Diagnose Eingang (DIAG) kann zu jederzeit eine Klappen und Motor-Diagnose starten. Der Baustein durchläuft dann einen Diagnosezyklus und meldet Fehler am Ausgang ERROR. Ein Diagnosezyklus fährt die Klappe zurück auf 0%, vermisst dann die Laufzeit von 0% - 100% und wieder zurück auf 0%. Er prüft auch, ob Endschalter funktionieren (falls diese durch die Config-Variable Switch_available) aktiviert wurden. Nach dem Diagnose-Zyklus fährt die Klappe wieder in die durch den Eingang IN definierte Stellung. Die während der Diagnose gemessenen Laufzeiten werden im Betrieb verwendet um die Klappe extrem genau auf die jeweils geforderte Position zu bewegen. Mit der Config-Variable AUTO_DIAG_TIME wird spezifiziert, nach welcher Zeit eine Diagnose selbständig ohne durch den Eingang DIAG aktiviert zu werden, durchgeführt wird. Nach dem Einschalten wird automatisch immer ein Diagnose-Zyklus durchgeführt. Ist dieser Wert 0, wird keine automatische Diagnose durchgeführt.

Der Ausgang BUSY zeigt an, dass sich die Klappe gerade bewegt oder Diagnose durchgeführt wird und der Ausgang POS ist ein Signal von 0 – 1, dass der momentanen Klappenstellung entspricht. Um ein Flattern der Klappe zu vermeiden und bei großen Motoren ein Auslaufen des Getriebes abzuwarten kann eine minimale Laufzeit (MIN_ONTIME) und eine minimale Totzeit (MIN_OFFTIME) definiert werden.

Eine Klappe wird Üblicherweise Auf und Ab bewegt um verschiedenen Stellungen einzustellen. Je mehr sich eine Klappe bewegt, desto mehr weicht sie von einer idealen absoluten Position ab, weil bei jeder Bewegung ein kleiner Positionsfehler auftritt und sich über viele Bewegungen addiert. Um diesem Fehler entgegenzuwirken kann mit der Config-Variablen CAL_RUNTIME nach einer definierten Laufzeit (aufaddierte Zeit aller Klappenbewegungen) der Klappe eine Kalibrierung automatisch durchgeführt werden. Bei dieser Kalibrierung fährt der Motor in Nullstellung und stellt die Klappe anschließend wieder auf den durch IN spezifizierten Wert. Ein Wert 0 für die CAL_RUNTIME bedeutet, dass keine automatische Kalibrierung durchgeführt wird. Die Config-Variable MAX_RUNTIME stellt die Zeit ein, die der Motor maximal für einen vollen Durchlauf von 0% nach 100% oder umgekehrt benötigen darf.

23.3 ACTUATOR_PUMP

Type	Funktionsblock
Input	IN: BOOL (Steuersignal für Pumpe) MANUAL : BOOL (Manuelles Steuersignal) RST : BOOL (Reset Signal)
Output	PUMP : BOOL (Steuersignal für die Pumpe) RUNTIME : REAL (Betriebsstunden des Motors in Stunden) CYCLES : REAL (Anzahl der Ein / Aus Zyklen der Pumpe)
Config	MIN_ONTIME : TIME (Minimale Laufzeit für Motor) MIN_OFFTIME : TIME (Minimale Totzeit für Motor) RUN_EVERY : TIME (Zeit nach der die Pumpe selbsttätig läuft um ein festsitzen zu vermeiden)

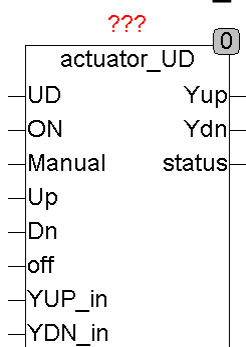


ACTUATOR_PUMP ist ein Pumpeninterface mit Betriebsstundenzähler. Die Pumpe kann sowohl mit IN oder Manual eingeschaltet werden. Die Config-Variablen MIN_ONTIME und MIN_OFFTIME legen eine minimale Einschaltdauer und eine minimale Laufzeit fest. Wird der Eingang IN kürzer als MIN_ONTIME auf TRUE gesetzt, so läuft die Pumpe weiter bis die minimale Laufzeit erreicht ist. Wird die Pumpe kurz hintereinander eingeschaltet, so wartet die Pumpe bis die Zeit MIN_OFFTIME verstrichen ist, bis sie die Pumpe wieder einschaltet. Mit der Config-Variablen RUN_EVERY wird die Zeit definiert, nach der die Pumpe selbsttätig läuft, wenn sie länger als RUN_EVERY stillsteht um ein festsitzen der Pumpe zu vermeiden. Die Pumpe schaltet in diesem Fall selbsttätig ein und läuft für MIN_ONTIME.

Ein interner Betriebsstundenzähler zählt die Laufzeit der Pumpe in Stunden und auch die Anzahl der Schaltzyklen. Beide Werte können mit TRUE am Eingang RST auf Null zurückgesetzt werden. Der Betriebsstundenzähler ist permanent und geht weder bei Stromausfall oder Reset verloren. RUNTIME und CYCLES sind beides REAL-Werte, damit nicht der übliche Overflow wie bei TIME Werten nach 50 Tagen passiert.

23.4 ACTUATOR_UD

Type	Funktionsblock
Input	UD : BOOL (Richtungs Eingang in Auto Mode UP=TRUE) ON : BOOL (TRUE, wenn im Auto Mode) MANUAL : BOOL (TRUE, wenn Manual Mode) UP : BOOL (UP enable in Manual Mode) DN : BOOL (DN enable in Manual Mode) OFF : BOOL (Sicherheits Ausschalter TRUE = Ausgänge FALSE) YUP_IN : BOOL (Rückführungseingang UP Relais) YDN_IN : BOOL (Rückführungseingang DN Relais)
Output	YUP : BOOL (Ausgang für Richtung UP) YDN : BOOL (Ausgang für Richtung DN) STATUS : Byte (ESR kompatibler Status und Fehler Ausgang)
Config	TON : TIME (minimale Einschaltzeit) TOFF : TIME (minimale Ausschaltzeit) OUT_RETURN : BOOL (schaltet die Rückführeingänge YUP_In und YDN_in ein)



ACTUATOR_UD ist eine Wendeschützinterface mit Verriegelung und konfigurierbarem Timing. Mit zusätzlichen Rückführeingängen wird eine Aktivierung verhindert solange ein Relais klemmt. Der Baustein kennt einen Automatik und einen Handbetrieb. Im Automatikmodus (ON = TRUE und Manual = FALSE) entscheidet der Eingang UD über die Richtung und ON über Ein / Aus. Sobald der Manual Eingang TRUE wird beginnt der Manual Modus und die Ausgänge folgen nur den Eingängen UP und DN. UP und DN dürfen nie gleichzeitig TRUE sein, falls trotzdem werden beide Ausgänge FALSE. Mit einem Sicherheitsausschalteingang OFF können sowohl im Manual als auch im Automatik Modus jederzeit die Ausgänge abgeschaltet werden.

Zwei Rückführeingänge YUP_IN und YDN_IN dienen dazu über separate Eingänge den Zustand der Schaltrelais auf den Baustein zurückzuführen und bei versagen ei-

nes Relais das aktivieren des anderen Ausgangs zu vermeiden. Dieser Fehler wird auch durch Fehlermeldungen am Ausgang STATUS gemeldet. Die Rückmeldefunktion ist jedoch nur verfügbar wenn die Config Variable OUT_RETURN auf TRUE gesetzt wird. Status meldet auch alle Aktivitäten des Bausteins um sie für eine Datenaufzeichnung zur Verfügung zu stellen. Der Status Ausgang ist ESR kompatibel und mit anderen ESR Modulen aus unserer Bibliothek kombinierbar. Der Ausgang Status meldet 2 Fehler:

1 : YUP kann nicht gesetzt werden weil YDN_IN TRUE ist.

2 : YDN kann nicht gesetzt werden weil YUP_IN TRUE ist.

Mit den Config Variablen TON und TOFF kann eine Mindesteinschaltzeit und eine Mindesteinschaltzeit zwischen 2 Ausgangsimpulsen definiert werden um das Schalten großer Motoren oder Getriebe die ein An und Auslaufen benötigen zu ermöglichen.

Manual	UP	DN	ON	UD	OFF	YUP	YDN	Status
1	0	0	-	-	0	0	0	102
1	1	0	-	-	0	1	0	103
1	0	1	-	-	0	0	1	104
1	1	1	-	-	0	0	0	102
0	-	-	1	1	0	1	0	111
0	-	-	1	0	0	0	1	112
-	-	-	-	-	1	0	0	101
1	0	0	0	-	0	0	0	110
0	-	-	0	-	-	0	0	110

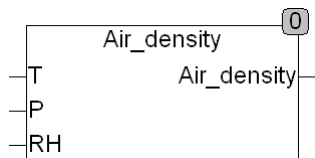
24 Heizung Lüftung und Klima

24.1 AIR_DENSITY

Type Funktion : REAL

Input T : REAL (Temperatur der Luft in °C)
 P : REAL (Luftdruck in Pascal)
 RH : REAL (Luftfeuchtigkeit in %)

Output (Dichte der Luft in kg/m³)



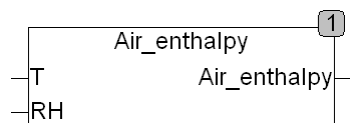
AIR_DENSITY berechnet die Dichte der Luft in kg/m³ abhängig von Druck, Feuchte und Temperatur. Die Temperatur wird in °C, Druck in Pascal und die Feuchte in % (50 = 50%) angegeben.

24.2 AIR_ENTHALPY

Type Funktion : REAL

Input T : REAL (Temperatur der Luft)
 RH : REAL (Relative Feuchte der Luft)

Output (Enthalpie der Luft in J/g)

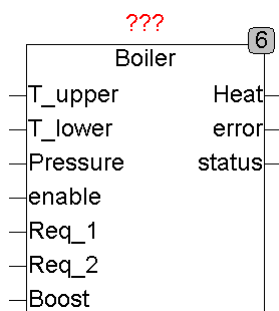


AIR_ENTHALPY berechnet die Enthalpie von Feuchter Luft aus den Angaben T für

Temperatur in Grad Celsius und der relativen Feuchte RH in % (50 = 50%). Die Enthalpie wird in Joule / Gramm berechnet.

24.3 BOILER

Type	Funktionsblock
Input	T_UPPER : REAL (Eingang oberer Temperatursensor) T_LOWER : REAL (Eingang unterer Temperatursensor) PRESSURE : REAL (Eingang Drucksensor) ENABLE : BOOL (Warmwasseranforderung) REQ_1 : BOOL (Anforderungseingang für vordefinierte Temperatur 1) REQ_2 : BOOL (Anforderungseingang für vordefinierte Temperatur 2) BOOST : BOOL (Anforderungseingang für sofortige Bereitstellung)
Output	HEAT : BOOL (Ausgang für Ladekreis) ERROR : BOOL (Fehlersignal) STATUS : Byte (ESR kompatibler Status Ausgang)
Config	T_UPPER_MIN : REAL (Mindesttemperatur für oben) Default = 50 T_UPPER_MAX : REAL (Maximaltemperatur für oben) Default = 60 T_LOWER_ENABLE : BOOL (FALSE, wenn unterer Temperatursensor nicht vorhanden ist) T_LOWER_MAX : REAL (Maximaltemperatur des unten) Default = 60 T_REQUEST_1 : REAL (Temperatur bei Anforderung 1) Default = 70 T_REQUEST_2 : REAL (Temperatur bei Anforderung 2) Default = 50 T_REQUEST_HYS : REAL (Hysterese für Regelung) Default = 5 T_PROTECT_HIGH : REAL (obere Grenztemperatur, Default = 80) T_PROTECT_LOW : REAL (untere Grenztemperatur, Default = 10)

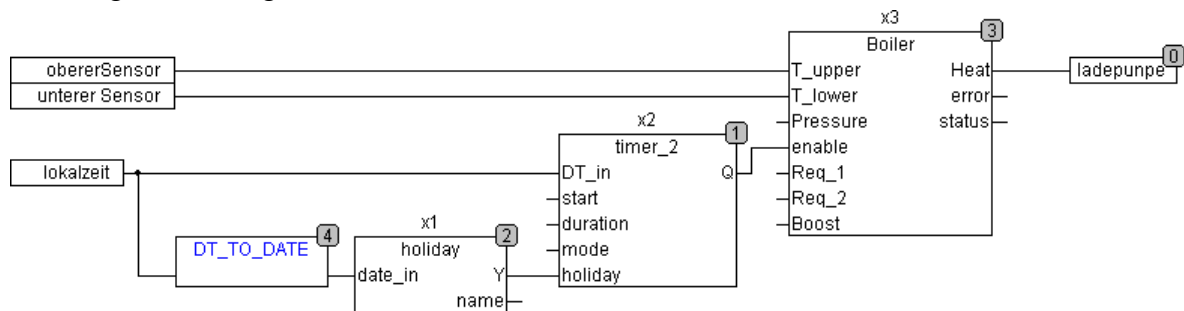


BOILER ist ein Controller für Pufferspeicher wie etwa Warmwasserspeicher. Durch 2 separate Temperatursensor-Eingänge können auch Schichtenspeicher geregelt werden. Mit der Config-Variable `T_LOWER_ENABLE` kann der untere Temperatursensor aus- und eingeschaltet werden. Wenn der Eingang `ENABLE` auf `TRUE` ist, wird der Boiler aufgeheizt (`HEAT = TRUE`) bis die Vorgabetemperatur `T_LOWER_MAX` im unteren Bereich des Puffers erreicht ist und dann die Heizung ausgeschaltet, bis die untere Grenztemperatur des oberen Bereichs (`T_UPPER_MIN`) erreicht wird. Falls `T_LOWER_ENABLE` auf `FALSE` ist, wird der untere Sensor nicht ausgewertet und die Temperatur zwischen `T_UPPER_MIN` und `T_UPPER_MAX` im oberen Bereich geregelt. Ein `PRESSURE`-Eingang schützt den Boiler und verhindert die Ladung, wenn nicht genügend Wasserdruck im Boiler vorhanden ist. Falls ein Drucksensor nicht vorhanden ist, bleibt der Eingang unbeschaltet. Als weitere Schutzfunktion stehen die Vorgabewerte `T_PROTECT_LOW` (Frostschutz) und `T_PROTECT_HIGH` zur Verfügung und verhindern das die Temperatur im Puffer einen oberen Grenzwert nicht übersteigt und ein unterer Grenzwert nicht unterschritten wird. Bei Auftreten eines Fehlers wird der Ausgang `ERROR` auf `TRUE` gesetzt und gleichzeitig ein Statusbyte am Ausgang `Status` gemeldet, welches durch Bausteine wie `ESR_COLLECT` weiter ausgewertet werden kann. Durch eine steigende Flanke am Eingang `BOOST` wird die Puffertemperatur unmittelbar auf `T_UPPER_MAX` (`T_LOWER_ENABLE = FALSE`) beziehungsweise `T_LOWER_MAX` (`T_LOWER_ENABLE = TRUE`) aufgeheizt. `BOOST` kann zur außerplanmäßigen Aufheizung des Boilers, wenn `ENABLE` auf `FALSE` ist, benutzt werden. Die Aufheizung durch `BOOST` ist flankengetriggert und führt bei jeder steigenden Flanke an `BOOST` zu genau einem Aufheizvorgang. Durch eine steigende Flanke an `BOOST` während `ENABLE TRUE` ist wird die Heizung sofort gestartet bis die maximale Temperatur erreicht ist. Der Boiler wird also nachgeladen, um maximale Wärmekapazität bereitzustellen. Die Eingänge `REQ_1` und `REQ_2` dienen dazu, jederzeit eine vordefinierte Temperatur (`T_REQUEST_1` oder `T_REQUEST_2`) bereitzustellen. `REQ` kann zum Beispiel zur Bereitstellung einer höheren Temperatur zur Legionellendesinfektion oder auch zu anderen Zwecken verwendet werden. Die Bereitstellung der Request-Temperaturen erfolgt durch Messung am oberen Temperatursensor und mit einer 2-Punkt Regelung deren Hysterese durch `T_REQUEST_HYS` voreingestellt wird.

Status	
1	oberer Temperatursensor hat die obere Grenztemperatur überschritten
2	oberer Temperatursensor hat die untere Grenztemperatur unterschritten
3	unterer Temperatursensor hat die obere Grenztemperatur überschritten
4	unterer Temperatursensor hat die untere Grenztemperatur unterschritten
5	Wasserdruck im Puffer ist zu gering
100	Standby
101	BOOST Nachladung

102	Standard Nachladung
103	Nachladung auf Request Temperatur 1
104	Nachladung auf Request Temperatur 2

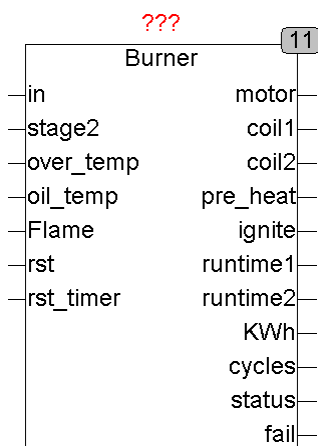
Das folgende Beispiel zeigt die Anwendung von BOILER mit einem TIMER und einer Feiertagsschaltung:



24.4 BURNER

Type	Funktionsblock
Input	IN : BOOL (Steuereingang) STAGE2 : BOOL (Steuereingang Stufe 2) OVER_TEMP : BOOL (Temperaturbegrenzung des Kessels) OIL_TEMP : BOOL (Thermostat der Ölvorerwärmung) FLAME : BOOL (Flammwächter) RST : BOOL (Reset-Eingang Für Störrücksetzung) RST_TIMER : BOOL (Reset für die Betriebszähler)
Output	MOTOR : BOOL (Steuersignal für den Motor) COIL1 : BOOL (Steuersignal für Ölventil Stufe 1) COIL2 : BOOL (Steuereingang für Ölventil Stufe 2) PRE_HEAT : BOOL (Ölvorerwärmung) IGNITE : BOOL (Zündung) RUNTIME1 : REAL (Betriebsstundenzähler Stufe 1) RUNTIME2 : REAL (Betriebsstundenzähler Stufe 2)

KWH : REAL (Kilowattstundenzähler)
 CYCLES : REAL (Anzahl der Brenner Starts)
 STATUS : Byte (ESR Kompatibler Statusausgang)
 FAIL : BOOL (Störmeldung: TRUE, wenn Fehler Auftritt)
 Config PRE_HEAT_TIME : TIME (maximale Zeit für die Ölvorerwärmung)
 PRE_VENT_TIME : TIME (Vorbelüftungszeit)
 PRE_IGNITE_TIME : TIME (Vorzündungszeit)
 POST_IGNITE_TIME : TIME (Nachzündungszeit)
 stage2_DELAY : TIME (Verzögerung Stufe 2)
 SAFETY_TIME : TIME ()
 LOCKOUT_TIME : TIME (Zeit die vergehen muss, bevor mit einem RST eine Störung gelöscht werden kann)
 MULTIPLE_IGNITION : BOOL ()
 KW1 : REAL (Leistung des Brenners auf Stufe 1 in KW)
 KW2 : REAL (Leistung des Brenners auf Stufe 2 in KW)



BURNER ist ein Steuerinterface für Öl- oder Gasbrenner mit Betriebsstundenzähler und Kilowattstundenzähler. Der Baustein steuert einen zweistufigen Brenner mit optionaler Ölvorerwärmung. Der Eingang IN ist der Steuereingang, der den Brenner nur dann startet, wenn der Eingang OVER_TEMP FALSE ist. OVER_TEMP ist der Kesselschutzthermostat, der TRUE wird, wenn die Kesseltemperatur die maximal zulässige Temperatur erreicht hat. Ein Brennerstart beginnt mit der Ölvorerwärmung, indem PRE_HEAT TRUE wird. Dann wird auf ein Signal am Eingang OIL_TEMP gewartet. Falls innerhalb der PRE_HEAT_TIME das Signal OIL_TEMP nicht TRUE wird und die Öltemperatur nicht erreicht wird, wird die Startsequenz unterbrochen und der Ausgang Störung gesetzt. Gleichzeitig wird am Ausgang Status Der Fehler 1 ausgegeben. Nach der Ölvorerwärmung wird der Motor eingeschaltet und damit der Ventilator in Betrieb gesetzt. Anschließend wird nach definierter Zeit die Zündung eingeschaltet und danach das Ölventil geöffnet. Sollte dann nach spezifizierter Zeit (SAFETY_TIME) der Flammwächter nicht ansprechen, so geht der Baustein auf Störung.

Eine Störung wird auch dann signalisiert, wenn der Flammwächter bereits vor der Zündung anspricht. Falls nach erfolgreicher Zündung die Flamme abreißt und die Config-Variable MULTIPLE_IGNITION = TRUE steht, wird sofort wieder gezündet. Eine zweite Stufe wird nach Ablauf der STAGE2_DELAY Zeit automatisch zugeschaltet wenn der Eingang STAGE2 TRUE ist.

Tritt eine Störung auf, so wird der Baustein für eine feste Zeit LOCKOUT_TIME blockiert und erst danach kann ein RST den Betrieb wieder starten. Während der LOCKOUT_TIME muss der RST-Eingang FALSE sein. Ein TRUE am Eingang OVER_TEMP stoppt sofort jede Aktion und meldet den Fehler 9.

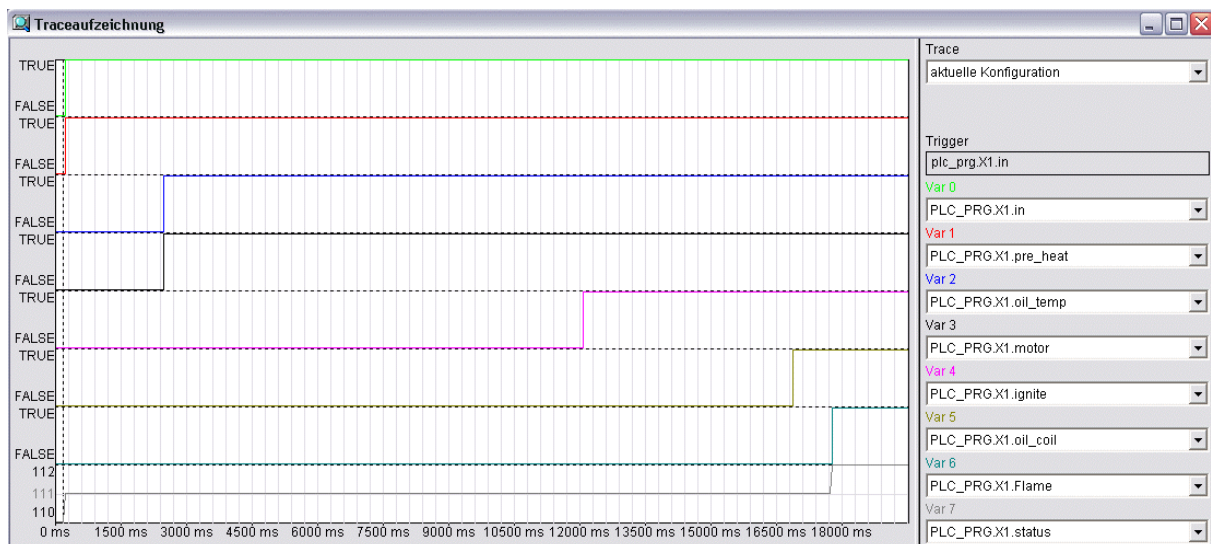
Der Status-Ausgang signalisiert den momentanen Zustand des Bausteins:

- 110 = Warten auf Startsignal (Standby)
- 111 = Startsequenz wird durchlaufen
- 112 = Brenner Läuft auf Stufe 1
- 113 = Brenner läuft auf Stufe 2

Eine Reihe von Fehlerzuständen werden am Ausgang STATUS bereitgestellt, wenn ein Fehler auftritt:

- 1 = Ölvorerwärmung hat innerhalb der PRE_HEAT_TIME nicht angesprochen
- 2 = Flammwächter ist aktiv während der Ölvorerwärmung (PRE_HEAT_TIME)
- 3 = Flammwächter ist aktiv während der Belüftungszeit (PRE_VENTILATION_TIME)
- 4 = Sicherheitszeit (Safety_TIME) wurde ohne Flamme überschritten
- 5 = Flamme ist im Betrieb Abgerissen
- 9 = Kessel Übertemperatur Kontakt hat Ausgelöst

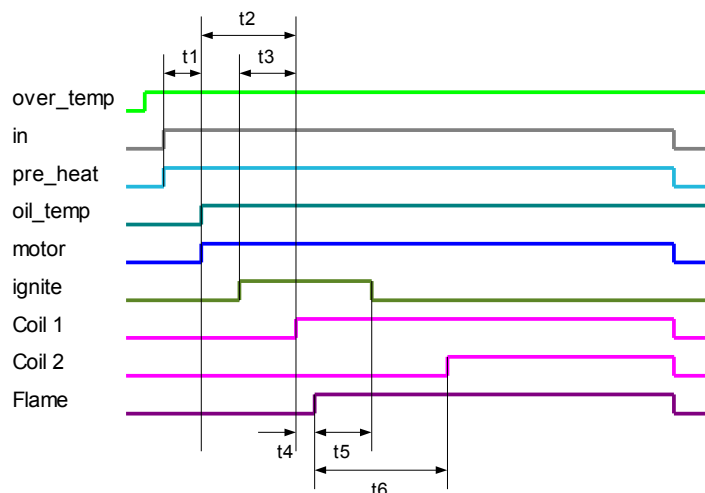
Traceaufzeichnung einer Normalen Startsequenz:



Das Signal IN startet die Sequenz mit dem Ausgang PRE_HEAT. Nach Erreichen der Öltemperatur (OIL_TEMP = TRUE) wird der Motor gestartet und die PRE_VENTILATION_TIME (Zeit von Motor Start bis Ölventil offen ist) abgewartet. Nach einer einstellbaren Zeit (PPR_IGNITION_TIME) vor dem Öffnen des Ölventils wird die Zündung eingeschaltet. Die Zündung bleibt dann solange ein, bis die POST_IGNITION_TIME abgelaufen ist.

In	over temp	Oil temp	Flame	Rst	motor	Oil coil	Pre heat	ignite	Status	fail	
0	0	-	-	0	0	0	0	0	110	0	Wartezustand
1	0	0	0	0	0	0	1	0	111	0	Ölvorwärmphase
1	0	1	0	0	1	0	1	0	111	0	Vorbelüftungsphase
1	0	1	0	0	1	0	1	1	111	0	Vorzündphase
1	0	1	0	0	1	1	1	1	111	0	Ventil Stufe 1 öffnen
1	0	1	1	0	1	1	1	1	112	0	Flamme brennt Nachzündphase
1	0	1	1	0	1	1	1	0	112	0	Brenner läuft
1	0	1	0	0	1	1	1	1	111	0	Nachzündung nach Flammabriß
-	1	-	-	-	-	-	-	-	9	1	Kessel Übertemperatur
1	0	1	1	0	1	0	1	0	3	1	Fremdlichtfehler

Das folgende Zeitdiagramm erläutert die verschiedenen Config-Zeiten und den Ablauf:

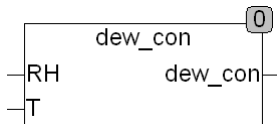


Das Zeitdiagramm gibt den genauen Zeitverlauf wieder:

- t1 = Vorheizzeit (PRE_HEAT_TIME)
- t2 = Vorbelüftungszeit (PRE_VENT_TIME)
- t3 = Vorzündungszeit (PRE_IGNITE_TIME)
- t4 = Sicherheitszeit (SAFETY_TIME)
- t5 = Nachzündungszeit (POST_IGNITE_TIME)
- t6 = Verzögerung für Stufe 2 (STAGE2_DELAY)

24.5 DEW_CON

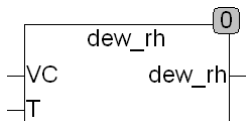
Type Funktion : REAL
 Input RH : REAL (Relative Feuchte)
 T : REAL (Temperatur in °C)
 Output REAL (Wasserdampf Konzentration in Gramm / m³)



Der Baustein DEW_CON berechnet aus der Relativen Feuchte (RH) und der Temperatur (T in °C) die Wasserdampfkonzentration in der Luft. Das Ergebnis wird in Gramm / m³ ermittelt. RH ist in % (50 = 50%) anzugeben und die Temperatur in °C.

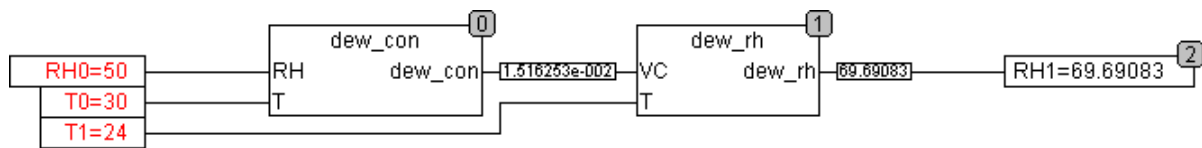
24.6 DEW_RH

Type Funktion : REAL
 Input VC : REAL (Wasserdampfkonzentration in Luft in Gramm / m³)
 T : REAL (Temperatur in °C)
 Output REAL (Relative Luftfeuchtigkeit in %)



Der Baustein DEW_RH berechnet aus der Wasserdampfkonzentration (VC) und der Temperatur (T in °C) die relative Luftfeuchtigkeit in % (50 = 50%). Die Wasserdampfkonzentration wird in Gramm/m³ angegeben. DEW_CON kann für Berechnungen in beide Richtungen (aufheizen und abkühlen) verwendet werden. Wird zu stark abgekühlt, so ist die maximale relative Feuchte auf 100% begrenzt. Für Berechnungen des Taupunktes wird der Baustein DEW_TEMP empfohlen.

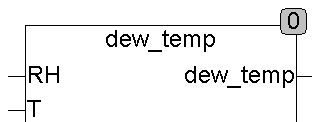
Im folgenden Beispiel wird der Fall berechnet, wenn Luft von 30°C und relativer Feuchte von 50% um 6 Grad abgekühlt wird. Der Baustein DEW_CON liefert die Feuchtigkeitskonzentration in der Ausgangsluft von 30° und DEW_RH berechnet die resultierende relative Luftfeuchtigkeit RH von 69,7%. Diese Berechnungen sind wichtig, wenn Luft abgekühlt oder aufgeheizt wird. In Klimaanlage ist eine resultierende relative Feuchte von 100% wegen Taubildung und den daraus resultierenden Problemen zu vermeiden.



Siehe hierzu auch die Bausteine DEW_CON und DEW_TEMP.

24.7 DEW_TEMP

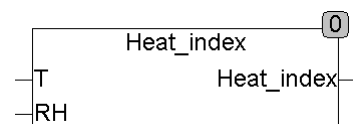
Type Funktion : REAL
 Input RH : REAL (Relative Feuchte)
 T : REAL (Temperatur in °C)
 Output REAL (Taupunkttemperatur)



Der Baustein DEW_TEMP berechnet aus der Relativen Feuchte (RH) und der Temperatur (T in °C) die Taupunkttemperatur. Die Relative Feuchte wird in % angegeben (50 = 50%).

24.8 HEAT_INDEX

Type Funktion : REAL
 Input T : REAL (Temperatur in °C)
 RH : REAL (Relative Feuchte)
 Output REAL (Heat Index Temperatur)

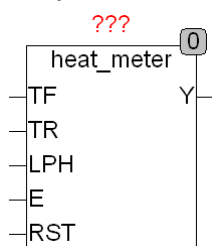


HEAT_INDEX berechnet die bei hohen Temperaturen und hoher Feuchte gefühlte Temperatur. Die Funktion ist definiert für Temperaturen größer 20 °C und relativer

Feuchte > 10%. Für Werte außerhalb des Definitionsbereichs wird die Eingangstemperatur ausgegeben.

24.9 HEAT_METER

Type	Funktion : REAL
Input	TF : REAL (Vorlauftemperatur in °C)
	TR : REAL (Rücklauftemperatur in °C)
	LPH : REAL (Durchflussmenge in L/h bzw. L/Impuls)
	E : BOOL (Enable Signal)
	RST : BOOL (asynchroner Reset Eingang)
Config	CP : REAL (Spezifische Wärmekapazität der 2ten Komponente)
	DENSITY : REAL (Dichte der 2ten Komponente)
	CONTENT : REAL (Anteil, 1 = 100%)
	PULSE_MODE : BOOL (Impulszähler wenn TRUE)
	RETURN_METER : BOOL (Durchflussmesser im Rücklauf wenn TRUE)
	AVG_TIME : TIME (Zeit für die Ermittlung des Momentanverbrauchs)
Output	REAL (Wärmemenge in Joule)



HEAT_METER ist ein Wärmemengenzähler. Die Wärmemenge Y wird in Joule gemessen. Die Eingänge TF und TR sind die Vorlauf und Rücklauftemperatur des Mediums. Am Eingang LPH wird die Durchflussmenge in Liter/Stunde beziehungsweise die Durchflussmenge je Impuls an E spezifiziert. Die Eigenschaft von E wird durch die Config Variable PULSE_MODE bestimmt. PULSE_MODE = FALSE bedeutet das die Wärmemenge kontinuierlich aufaddiert wird solange E auf TRUE ist. PULSE_MODE = TRUE bedeutet das die Wärmemenge mit jeder steigenden Flanke von E aufaddiert wird. Der PULSE_MODE ist bei Verwendung von Wärmezählern einzuschalten, während am Eingang LPH die Flussmenge in Litern je Impuls anzugeben ist und am Eingang E wird der Wärmezähler angeschlossen. Wenn kein Flussmengenmesser Vorhanden ist, so wird am Eingang E das Pumpensignal angeschlossen und am Eingang LPH die Pumpenleistung in Litern / Stunde angegeben.

Bei Verwendung eines Flussmengenmessers mit Analogem Ausgang wird der Ausgang entsprechend auf Liter / Stunde umgerechnet und auf den Eingang LPH gelegt, Der Eingang E wird hierbei auf TRUE gesetzt. Mit den Setup- Variablen CP, DENSITY und CONTENT wird die 2te Komponente des Mediums spezifiziert. Für den Betrieb mit reinem Wasser sind keinerlei Angaben von CP, DENSITY und CONTENT nötig. Wenn ein Gemisch aus Wasser und einem 2ten Medium vorhanden ist werden Mit CP die Spezifische Wärmekapazität in KJ/KgK, mit DENSITY die DICHTe in KG/l und mit CONTENT der Anteil der 2ten Komponente spezifiziert. Ein Anteil von 0.5 bedeutet 50% und 1 wäre entsprechend 100%. Mit der Config Variablen RETURN_METER wird angegeben ob der Durchflussmesser im Vorlauf oder Rücklauf sitzt. RETURN_METER = TRUE steht für Rücklaufmessung und FALSE für Vorlaufmessung. Am Ausgang C stellt der Baustein den Momentanverbrauch zur Verfügung. Der Momentanverbrauch wird in Joule / Stunde angegeben, und in den Zeitabständen von AVG_TIME ermittelt.

Der Baustein hat folgende Vorgabewerte, die aktiv sind wenn die entsprechenden Werte vom Anwender nicht gesetzt werden:

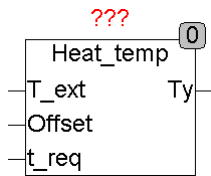
PULSE_MODE = FALSE

RETURN_METER = FALSE

AVG_TIME = T#5s

24.10 HEAT_TEMP

Type	Funktionsbaustein
Input	T_EXT : REAL (Außentemperatur) OFFSET : REAL (Absenkung oder Anhebung der Raumtemperatur) T_REQ : REAL (Temperaturanforderung)
Output	TY : REAL (Heizkreisvorlauftemperatur)
Config	TY_MIN : REAL (Minimale Heizkreistemperatur) TY_MAX : REAL (Maximale Heizkreistemperatur) T_INT : REAL (Raumtemperatur, DEFAULT = 20°C) TY_CONFIG : REAL (Auslegungstemperatur) T_EXT_CONFIG : REAL (T_EXT bei Auslegungstemperatur) C : REAL (Konstante des Heizsystems, DEFAULT = 1,33)



HEAT_TEMP berechnet die Vorlauftemperatur aus der Außentemperatur nach folgender Formel:

$$TY = T_INT + (TY_CONFIG - T_INT) * ((T_INT - T_EXT) / (T_INT - T_EXT_CONFIG))^{1/C}$$

Die Parameter der Heizkurve werden durch die Config-Variablen T_INT (Soll Raumtemperatur), TY_CONFIG (Auslegungsvorlauftemperatur) und T_EXT_CONFIG (Außentemperatur im Auslegungspunkt) vorgegeben. Mit dem Eingang Offset kann die Heizkurve an Raumabsenkung (negativer Offset) oder Raumanhebung (positiver Offset) angepasst werden. Mit den Config-Variablen TY_MIN und TY_MAX kann die Vorlauftemperatur auf einen Minimal- und Maximalwert begrenzt werden. Der Eingang T_REQ dient dazu, externe Temperaturanforderungen wie z.B. vom Boiler zu unterstützen. Ist T_REQ größer als der aus der Heizkurve berechnete Wert für TY, so wird TY auf T_REQ gesetzt. Die Config-Variable C legt die Krümmung der Heizkurve fest. Die Krümmung ist abhängig vom verwendeten Heizsystem.

Konvektoren: C = 1.25 – 1.45

Plattenheizkörper: C = 1.20 – 1.30

Radiatoren: C = 1.30

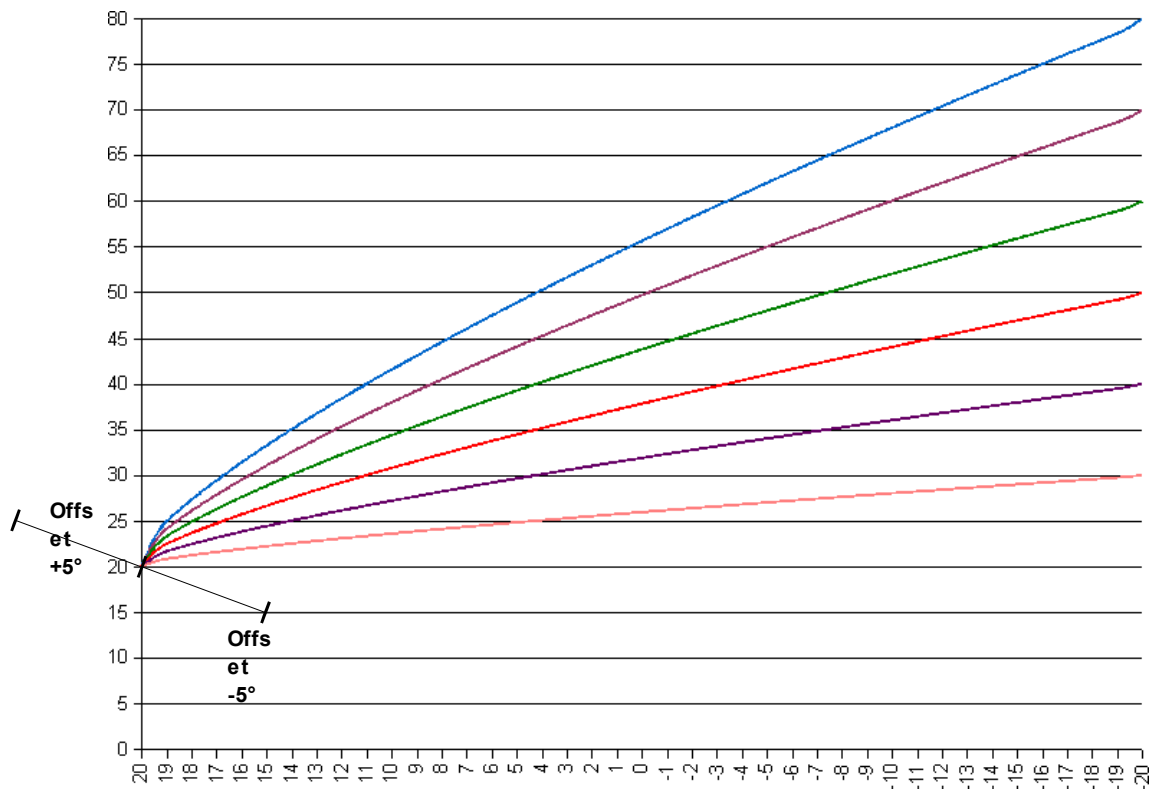
Rohre: C = 1.25

Fußbodenheizung: C = 1.1

Je größer der Wert von C, desto stärker ist die Heizkurve gekrümmt. Ein Wert von 1.0 ergibt eine Gerade als Heizkurve. Typische Heizsysteme liegen zwischen 1.0 und 1.5.

Die Steigung wird definiert durch die Auslegungsvorlauftemperatur bei der Außentemperatur im Auslegungspunkt. OFFSET verschiebt die Heizkurve in 45°. Ein Offset von -5 bedeutet das T_INT auf 15 Grad sinkt und gleichzeitig TY_CONFIG um 5° herabgesetzt wird.

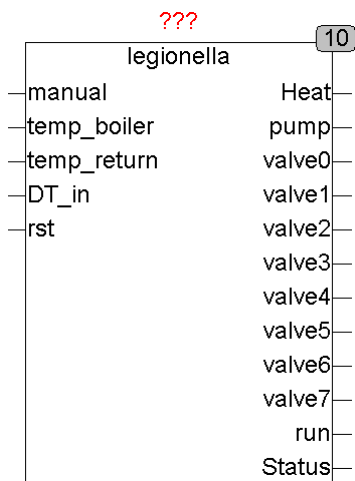
Die Grafik zeigt Heizkurven für Auslegungstemperaturen von 30 – 80 °C Vorlauftemperatur bei -20°C Außentemperatur und bei einem C von 1.33:



24.11 LEGIONELLA

Type	Funktionsbaustein
Input	MANUAL : BOOL (Manual Start Input) TEMP_BOILER : REAL (Boiler Temperatur) TEMP_RETURN : REAL (Temperatur der Zirkulationsleitung) DT_IN : DATE_TIME (Momentane Tageszeit und Datum) RST : BOOL (Asynchroner Reset)
Output	HEAT : BOOL (Steuersignal für Warmwasserheizung) PUMP : BOOL (Steuersignal für Zirkulationspumpe) STATUS : Byte (ESR kompatibler Statusausgang) VALVE0..7 : BOOL (Steuerausgänge für Ventile der Zirkulationskreise)

Config RUN : BOOL (TRUE wenn Sequenz läuft)
 T_START : TOD (Tageszeit zu der die Desinfizierung startet)
 DAY : INT (Wochentag an dem die Desinfizierung startet)
 TEMP_SET : REAL (Temperatur des Boilers)
 TEMP_OFFSET : REAL ()
 TEMP_HYS : REAL ()
 T_MAX_HEAT : TIME (maximale Zeit zum Aufheizen des Boilers)
 T_MAX_RETURN : TIME (maximale Zeit, bis der Eingang
 TEMP_RETURN nach VALVE aktiv wird)
 TP_0 .. 7 : TIME (Desinfektionszeit für Kreise 0..7)



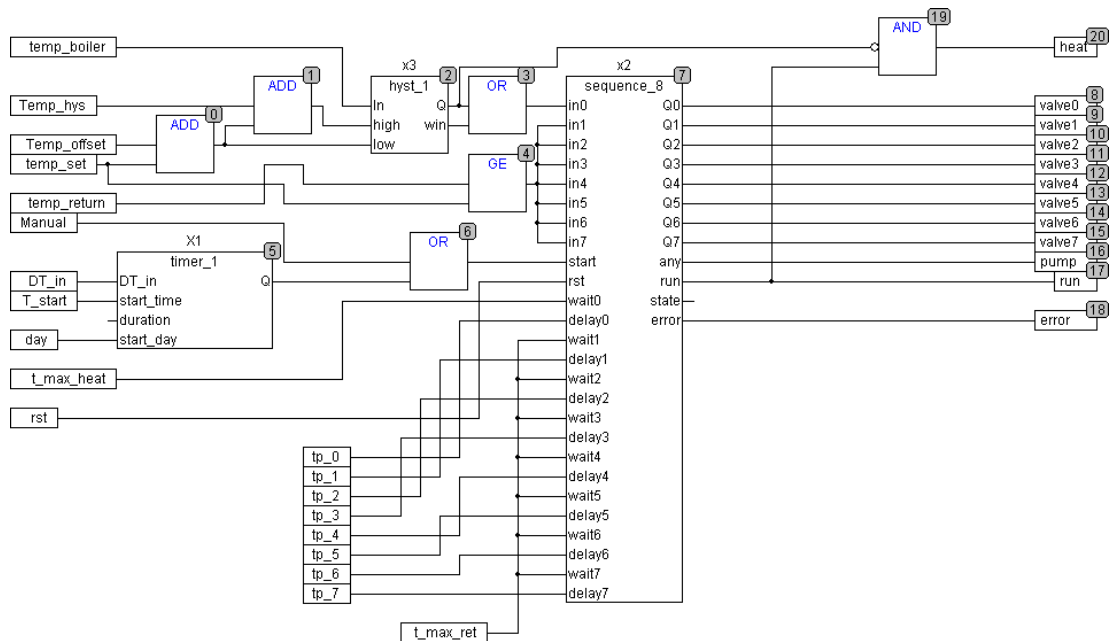
LEGIONELLA hat eine integrierte Schaltuhr, die an einem bestimmten Wochentag (DAY) zu einer bestimmten Tageszeit (T_START) die Desinfektion startet. Hierzu ist die externe Anschaltung der Lokalzeit nötig (DT_IN). Jederzeit kann mit einer steigenden Flanke an MANUAL die Desinfektion auch von Hand gestartet werden.

Der Ablauf eines Desinfektionszyklus wird mit einem internen Start aufgrund von DT_IN, DAY und T_START, oder durch eine steigende Flanke an MANUAL gestartet. Der Ausgang HEAT wird TRUE und steuert die Heizung des Boilers an. Innerhalb der Aufheizzeit T_MAX_HEAT muss dann das Eingangssignal TEMP_BOILER auf TRUE gehen. Wird die Temperatur nicht innerhalb von T_MAX_HEAT gemeldet, geht der Ausgang Status auf Störung. Die Desinfektion läuft aber trotzdem weiter. Nach der Aufheizphase wird die Boilertemperatur gemessen und falls nötig durch TRUE am Ausgang HEAT wieder nachgeheizt. Sobald die Boilertemperatur erreicht ist, wird PUMP TRUE und die Zirkulationspumpe eingeschaltet. Dann werden nacheinander die einzelnen Ventile geöffnet und gemessen, ob innerhalb der Zeit T_MAX_RETURN die Temperatur am Rücklauf der Zirkulationsleitung erreicht wurde. Falls ein Rückflußthermometer nicht vorhanden ist, kann der Eingang T_MAX_RETURN einfach offen bleiben.

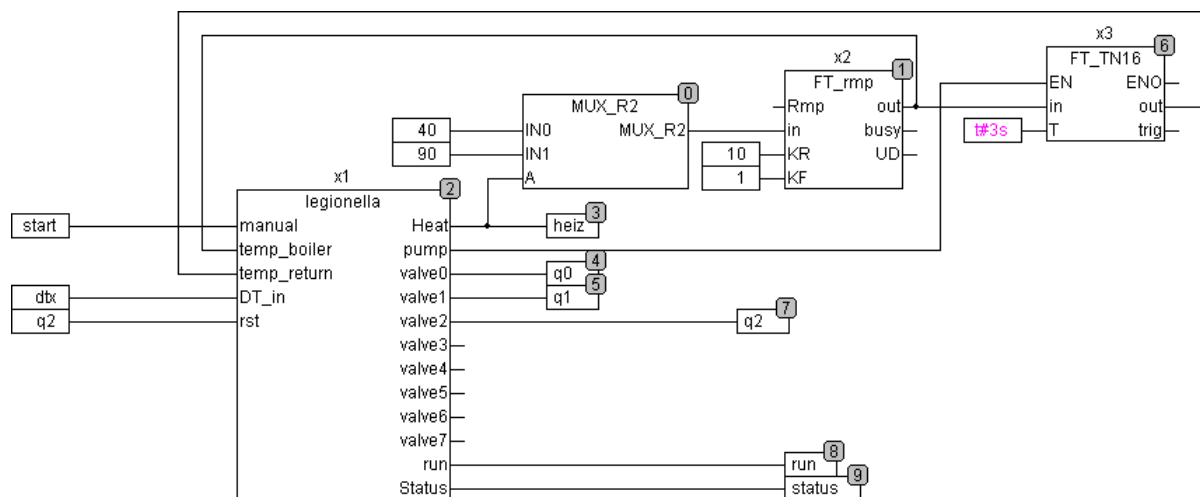
Der Ausgang Status ist ESR kompatibel und kann folgende Meldungen abgeben:

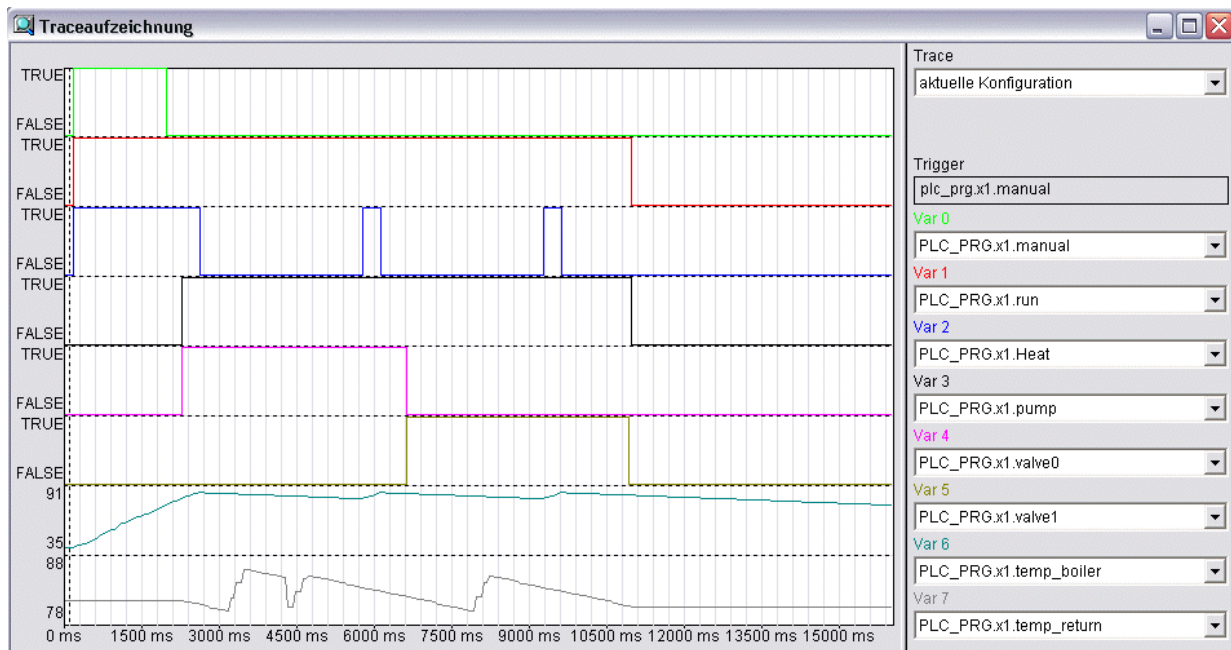
- 110 Wartestellung
111 Sequenz läuft
- 1 Boiler Temperatur wurde nicht erreicht
2 Rücklauftemperatur bei Ventil0 wurde nicht erreicht
3..8 Rücklauftemperatur bei Ventil1..7 wurde nicht erreicht

Schematischer interner Aufbau von LEGIONELLA:



Das Folgende Beispiel zeigt eine Simulation für 2 Desinfektionskreise mit Traceaufzeichnung. In diesem Aufbau ist VALVE2 auf den Eingang RST geschaltet und unterbricht damit die Sequenz nach 2 Kreisen:





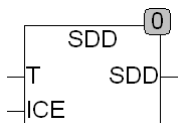
24.12 SDD

Type Funktion : REAL

Input T : REAL (Temperatur der Luft in °C)

ICE : BOOL (TRUE für Luft über Eis und FALSE für Luft über Wasser)

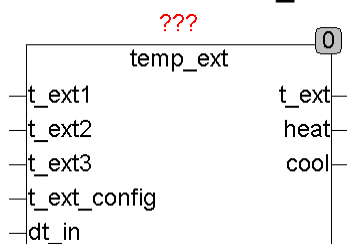
Output (Sättigungsdampfdruck in Pa)



SDD berechnet den Sättigungsdampfdruck für Wasserdampf in Luft. Die Temperatur T wird in °C angegeben. Das Ergebnis kann sowohl für Luft über Eis (ICE = TRUE) und für Luft über Wasser (ICE = FALSE) berechnet werden. Der Gültigkeitsbereich der Funktion liegt bei -30°C bis 70°C über Wasser und bei -60°C bis 0°C über Eis. Die Berechnung wird nach der Magnusformel ausgeführt.

24.13 TEMP_EXT

Type	Funktionsbaustein
Input	T_EXT1 : REAL (Außentemperatur Sensor 1) T_EXT2 : REAL (Außentemperatur Sensor 2) T_EXT3 : REAL (Außentemperatur Sensor 3) T_EXT_CONFIG : BYTE (Abfragemodus) DT_IN : DATE_TIME (Tageszeit)
Output	T_EXT : REAL (Ausgang Außentemperatur) HEAT : BOOL (Heizsignal) COOL : BOOL (Kühlsignal)
Config	T_EXT_MIN : REAL (Minimum Außentemperatur) T_EXT_MAX : REAL (Maximum Außentemperatur) T_EXT_DEFAULT : REAL (Default Außentemperatur) HEAT_PERIOD_START : DATE (Beginn der Heizperiode) HEAT_PERIOD_STOP : DATE (Ende der Heizperiode) COOL_PERIOD_START : DATE (Beginn der Kühlperiode) COOL_PERIOD_STOP : DATE (Ende der Kühlperiode) HEAT_START_TEMP_DAY (Heiztriggertemperatur Tag) HEAT_START_TEMP_NIGHT (Heiztriggertemperatur Nacht) HEAT_STOP_TEMP : REAL (Heizen Stopp Temperatur) COOL_START_TEMP_DAY (Kühl Start Temperatur Tag) COOL_START_TEMP_NIGHT (Kühl Start Temperatur Nacht) COOL_STOP_TEMP : REAL (Kühl Stopp Temperatur) START_DAY : TOD (Anfang des Tages) START_NIGHT : TOD (Anfang der Nacht) CYCLE_TIME : TIME (Abfragezeit für Außentemperatur)



TEMP_EXT verarbeitet bis zu 3 Außentemperaturfühler und stellt eine durch Mode

selektierte Außentemperatur der Heizungsregelung zur Verfügung. Es errechnet Signale für Heizung und Kühlung abhängig von Außentemperatur, Datum und Uhrzeit. Mit dem Eingang T_EXT_CONFIG wird festgelegt, wie der Ausgangswert T_EXT ermittelt wird. Wird T_EXT_CONFIG nicht beschaltet, so ist der Vorgabewert 0. Die Config-Werte T_EXT_MIN und T_EXT_Max legen den Mindestwert und Maximalwert der Außentemperatureingänge fest. Werden diese Grenzen über- oder unterschritten, so wird von einem Fehler im Sensor oder Drahtbruch ausgegangen und an Stelle des Messwertes der Vorgabewert T_EXT_DEFAULT benutzt.

T_EXT_CONFIG	T_EXT
0	Durchschnittswert von T_EXT1, T_ext2 und T_ext3
1	T_EXT1
2	T_EXT2
3	T_EXT3
4	T_EXT_DEFAULT
5	Niedrigster Wert der 3 Eingänge
6	Höchster Wert der 3 Eingänge
7	Mittlerer Wert der 3 Eingänge

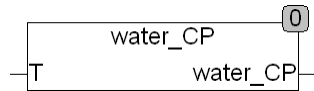
Mit den Config-Variablen HEAT_PERIOD und COOL_PERIOD wird definiert, wann Heizen und wann Kühlen erlaubt ist. Die Entscheidung, ob der Ausgang HEAT oder COOL TRUE wird hängt weiterhin von den CONFIG-Werten HEAT_START-HEAT_STOP und COOL_START und COOL_STOP ab. Diese Werte können separat für Tag und Nacht definiert werden. Der Start für eine Tag- und Nachtperiode kann durch die Config-Variablen START_DAY und START_NIGHT festgelegt werden. Ein Variable CYCLE_TIME legt fest, wie oft die Außentemperatur abgefragt werden soll.

24.14 WATER_CP

Type Funktion : REAL

Input T : REAL (Temperatur des Wassers in °C)

Output (Spezifische Wärmekapazität bei der Temperatur T)



WATER_CP berechnet die spezifische Wärmekapazität von flüssigem Wassers in Abhängigkeit von der Temperatur bei Normaldruck. Die Berechnung ist gültig im Temperaturbereich von 0 bis 100 Grad Celsius und wird in Joule / (Gramm * Kelvin) errechnet. Die Temperatur T wird in °C angegeben.

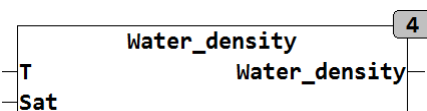
24.15 WATER_DENSITY

Type Funktion : REAL

Input T : REAL (Temperatur des Wassers)

SAT : BOOL (TRUE, wenn das Wasser mit Luft gesättigt ist)

Output (Dichte des Wassers in Gramm / Liter)



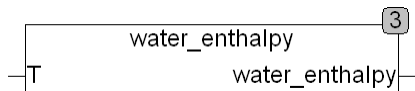
WATER_DENSITY berechnet die Dichte von flüssigem Wassers in Abhängigkeit von der Temperatur bei Normaldruck. Die Temperatur T wird in °C angegeben. Die höchste Dichte erreicht Wasser bei 3,983 °C mit 999,974950 Gramm pro Liter. WATER_DENSITY berechnet die Dichte für flüssiges Wasser, nicht für gefrorenes oder verdampftes Wasser. WATER_DENSITY berechnet die Dichte von luftfreiem Wasser wenn SAT = FALSE und von luftgesättigtem Wasser wenn SAT = TRUE. Die Berechneten Werte werden nach einer Näherungsformel berechnet und liefert Werte mit einer Genauigkeit besser als 0,01% im Temperaturbereich von 0 - 100 °C bei einem konstanten Druck von 1013 mBar.

Die Abweichung der Dichte von mit Luft gesättigtem Wasser wird nach der Formel vom Bignell korrigiert.

Die Abhängigkeit der Wasserdichte vom Druck ist verhältnismäßig gering mit ca. 0,046 kg/m³ je 1 Bar Druckerhöhung Im Bereich bis 50 Bar. Die geringe Druckabhängigkeit hat in praktischen Anwendungsfällen keinen nennenswerten Einfluss.

24.16 WATER_ENTHALPY

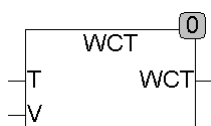
Type Funktion : REAL
 Input T : REAL (Temperatur des Wassers)
 Output (Enthalpie des Wassers in J/Gramm bei der Temperatur T)



WATER_ENTHALPY berechnet die Enthalpie (Wärmeinhalt) von flüssigem Wasser in Abhängigkeit von der Temperatur bei Normaldruck. Die Temperatur T wird in °C angegeben. Die Berechnung ist gültig für eine Temperatur von 0 bis 100 °C und das Ergebnis ist die Wärmemenge die benötigt wird um das Wasser von 0°C auf die Temperatur von T zu erwärmen. Das Ergebnis wird in Joule/Gramm J/g beziehungsweise KJ/Kg ausgegeben. Die Berechnung erfolgt durch lineare Interpolation in Schritten von 10° und erreicht damit eine für nicht wissenschaftliche Anwendungen hinreichende Genauigkeit. Eine mögliche Anwendung von WATER_ENTHALPY ist die Berechnung der Energiemenge die benötigt wird um zum Beispiel einen Pufferspeicher um X (T2 – T1) Grad zu erwärmen. Aus der benötigten Energie kann dann die Laufzeit eines Heizkessels berechnet werden und exakt die benötigte Energie bereitgestellt werden. Da Temperaturmesswerte in der Praxis stark zeitverzögert vorliegen ist mit dieser Methode eine bessere Aufheizung möglich.

24.17 WCT

Type Funktion : REAL
 Input T : REAL (Außentemperatur in °C)
 V : REAL (Windgeschwindigkeit in km/h)
 Output (Windchill Temperatur)

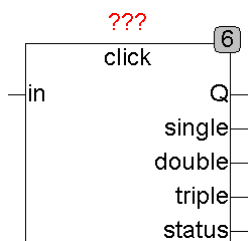


WCT berechnet die Windchill-Temperatur abhängig von der Windgeschwindigkeit in Km/h und der Außentemperatur in °C. Die Windchill-Temperatur ist nur definiert für Windgeschwindigkeiten größer als 5 Km/h und Temperaturen kleiner 10 °C. Für Werte außerhalb des definierten Bereichs wird die Eingangstemperatur ausgegeben.

25 Elektro

25.1 CLICK

Type	Funktionsblock
Input	SET : BOOL (Eingang zum Einschalten des Ausgangs auf 100%) IN : BOOL (Steuereingang für Taster) RST : BOOL (Eingang zum Ausschalten des Ausgangs)
Output	Q : BOOL (Schaltausgang) OUT : Byte (Dimmer Ausgang)
Config	T_DEBOUNCE : TIME (Entprellzeit für Taster) T_SHORT : TIME (Maximale Zeit für kurzen Impuls) T_PAUSE : TIME (Maximale Pause zwischen 2 Impulsen) T_RECONFIG : TIME (Rekonfigurationszeit)

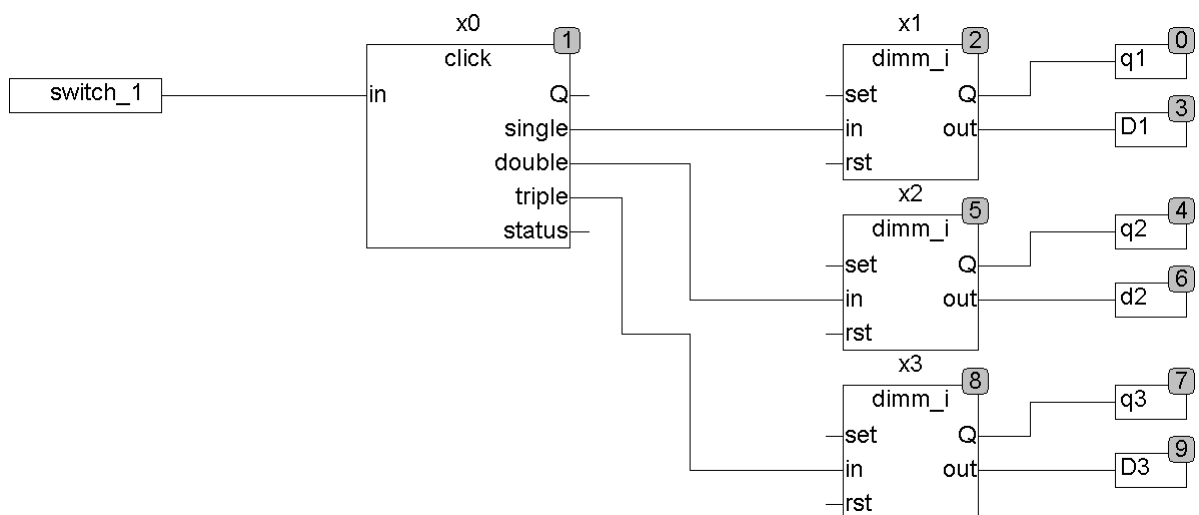


CLICK ist ein Tastinterface das sich selbsttätig auf den angeschlossenen Taster einstellt. Wird ein Taster angeschlossen, so erkennt CLICK selbst, ob es ein Öffner oder Schließer ist und wertet dann nur die jeweils erste Flanke aus. Mit der Config-Variable T_DEBOUNCE wird die Entprellzeit des Tasters festgelegt. Sie ist standardmäßig auf 10ms voreingestellt. Die Zeit T_RECONFIG wird verwendet, um zu entscheiden ob ein Schließer oder Öffner am Eingang IN angeschlossen ist. Bleibt der Eingang länger als diese Zeit in einem Zustand, so wird dies als Ruhstellung angenommen. Der Vorgabewert für T_RECONFIG beträgt 1 Minute. Mit kurz aufeinander folgenden Impulsen wird ein einfacher, doppelter oder dreifacher Puls ausgewertet und schaltet entsprechend den Ausgang SINGLE, DOUBLE oder TRIPLE ein. Ist der Puls länger als die Config-Zeit T_SHORT oder eine Pause zwischen 2 Impulsen länger als T_PAUSE, so wird die Puls-Sequenz unterbrochen und der entsprechende Ausgang gesetzt, bis der Eingangspuls wider inaktiv wird. Der Ausgang Q entspricht dem Ein-

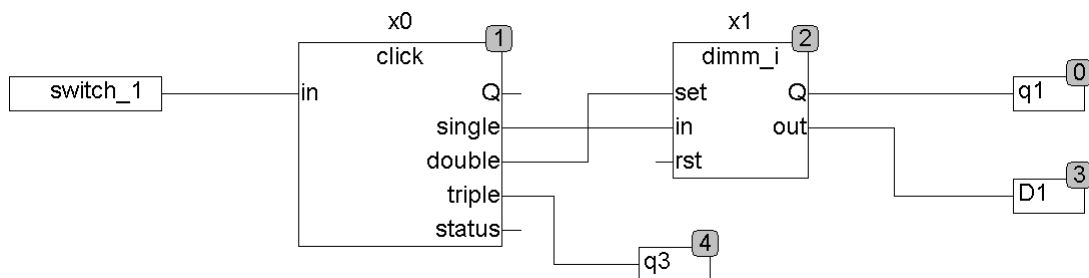
gangsimpuls. Jedoch ist er immer High-aktiv. Ein ESR kompatibler Status Ausgang meldet Zustandsänderungen an nachfolgende ESR kompatible Auswertemodule. Mit kurzen Impulsen wird der Ausgang SINGLE (ein Puls), DOUBLE (2 Impulse) oder TRIPLE (3 Impulse) ausgewählt. Der entsprechende Ausgang bleibt mindestens einen Zyklus aktiv und maximal solange wie der Eingang IN aktiv bleibt.

Status	
110	Eingang inaktiv
111	Ausgang SINGLE aktiviert
112	Ausgang DOUBLE aktiviert
113	Ausgang TRIPLE aktiviert

Beispiel 1 zeigt eine Anwendung von CLICK mit 3 nachfolgenden Dimm Bausteinen. Analog können auch bis zu 3 Schalter oder eine Mischung aus Dimmern oder Schaltern benutzt werden.

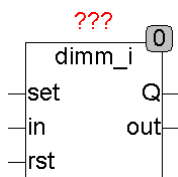


Beispiel 2 zeigt CLICK mit einem Dimmer, wobei sich der Dimmer wie ein Dimmer ohne CLICK verhält, jedoch ein kurzer Doppelklick den Ausgang des Dimmers sofort auf 100% setzt und ein Dreifachklick als weiterer Schaltausgang zur Verfügung steht.



25.2 DIMM_I

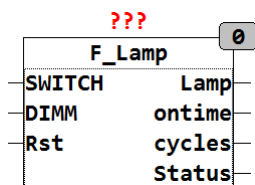
Type	Funktionsblock
Input	SET : BOOL (Eingang zum Einschalten des Ausgangs auf 100%) IN : BOOL (Steuereingang für Taster) RST : BOOL (Eingang zum Ausschalten des Ausgangs)
Output	Q : BOOL (Schaltausgang) OUT : Byte (Dimmer Ausgang)
Config	T_DEBOUNCE : TIME (Entprellzeit für Taster) T_RECONFIG : TIME (Rekonfigurationszeit) T_ON_MAX : TIME (Einschaltbegrenzung) T_DIMM_START : TIME (Reaktionszeit zum Dimmen) T_DIMM : TIME (Zeit für eine Dimm Rampe)



DIMM_I ist ein intelligenter Dimmer der sich selbständig an Öffner oder Schließer anpasst ohne neu konfiguriert werden zu müssen. Der Dimmer kann über Config-Variablen eingestellt werden. Die Zeit T_DEBOUNCE dient zum entprellen des Tasters und ist standardmäßig auf 10ms eingestellt. Die Zeit T_RECONFIG wird verwendet, um zu entscheiden ob ein Schließer oder Öffner am Eingang IN angeschlossen ist. Bleibt der Eingang länger als diese Zeit in einem Zustand, so wird dies als Ruhstellung angenommen. Eine Einschaltbegrenzung T_ON_MAX schaltet den Ausgang automatisch ab, wenn sie überschritten wird. Die Zeiten T_DIMM_Start und T_DIMM legen das Zeitverhalten des Dimmers fest. Mit den Eingängen SET und RST kann der Ausgang jederzeit auf 100% ein- oder ausgeschaltet werden. SET und RST können unter anderem zum Anschluss von Brandmeldeanlagen oder Alarmanlagen benutzt werden. Mit SET können im Brandfall oder bei Einbruch alle Leuchten auf Ein oder bei verlassen des Gebäudes Zentral auf Aus geschaltet werden. Beim Ein- und Ausschalten bleibt der letzte Ausgangswert des Dimmers am Ausgang OUT erhalten, lediglich ein FALSE am Ausgang Q schaltet das Leuchtmittel ab, und TRUE am Ausgang Q schaltet das Leuchtmittel wieder ein.

25.3 F_LAMP

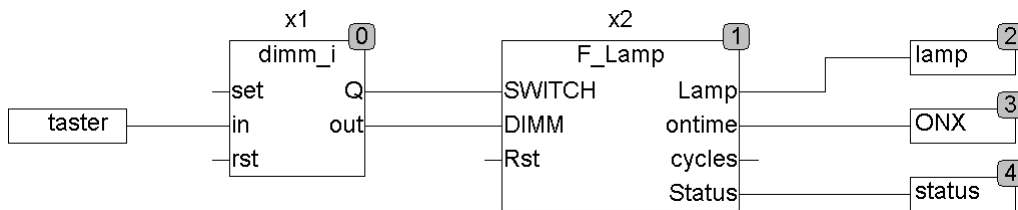
Type	Funktionsblock
Input	SWITCH : BOOL (Schalteingang vom Dimmer) DIMM : Byte (Eingang vom Dimmer) RST : BOOL (Eingang zum Rücksetzen des Betriebsstundenzählers)
Output	LAMP : Byte (Dimmer Ausgang) ONTIME : REAL (Betriebsstundenzähler) CYCLES : REAL (Anzahl der Schaltzyklen des Leuchtmittels)
Config	T_NO_DIMM : REAL (Sperrzeit für Dimmer in Stunden) T_Maintenance : REAL (Meldezeit für Lampenwechsel in Stunden)



F_LAMP ist ein Lampeninterface für Leuchtstofflampen. Der Ausgang LAMP folgt dem Eingang DIMM und SWITCH. Wenn Dimm nicht beschaltet wird ist die Vorgabe 255 und der Ausgang LAMP schaltet zwischen 0 und 255 abhängig von SWITCH. Die Ausgänge ONTIME und CYCLES zählen die Betriebsstunden des Leuchtmittels und die Schaltzyklen. Beide Werte sind remanent gespeichert und bleiben auch bei Stromausfall oder System-Reset erhalten. Ein TRUE am Eingang RST setzt diese beiden Werte auf 0 zurück. Mit der Config-Variablen T_NO_DIMM wird festgelegt, nach welcher Betriebsdauer eines neuen Leuchtmittels mit dem Dimmen begonnen werden darf. Dieser Wert ist, wenn er nicht vom Anwender anders eingestellt wird, auf 100 Stunden voreingestellt. Leuchtstofflampen dürfen während der ersten 100 Betriebsstunden nicht gedimmt werden, sonst wird Ihre Lebensdauer drastisch verkürzt. Durch einen RST beim Lampenwechsel verhindert dieser Baustein das Dimmen in der Anfangsphase. Der Ausgang Status ist ESR kompatibel und kann Betriebszustände melden, aber auch eine Meldung zum Lampenwechsel absetzen. Die voreingestellte Zeit für T_MAINTENANCE beträgt, falls vom Anwender nicht anders eingestellt, 15000 Stunden. Wird T_Maintenance auf 0 gesetzt so wird keine Meldung zum Lampenwechsel generiert.

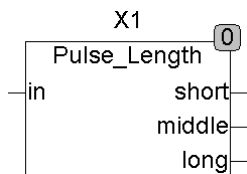
Status	
110	Lampe ausgeschaltet
111	Lampe eingeschaltet Dimmen nicht erlaubt
112	Lampe eingeschaltet Dimmen erlaubt
120	Aufforderung zum Lampenwechsel

Das folgende Beispiel zeigt die Anwendung des Bausteins F_LAMP in Verbindung mit DIMM_I:



25.4 PULSE_LENGTH

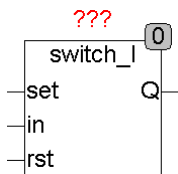
Type	Funktionsblock
Input	IN : BOOL (Eingangspuls)
Output	SHORT : BOOL (Puls wenn $IN < T_SHORT$) MIDDLE : BOOL (Puls wenn $IN \leq T_LONG$ und $IN \geq T_SHORT$) LONG : BOOL (TRUE wenn $IN > T_SHORT$)
Config	T_SHORT : TIME (Maximale Länge für kurzen Puls) T_LONG : TIME (Minimale Länge für Langen Puls)



PULSE_LENGTH setzt bei einem Eingangspuls an IN einen der 3 Ausgänge. Der Ausgang SHORT wird für einen Zyklus TRUE, wenn der Eingangspuls kleiner als T_SHORT ist. Der Ausgang MIDDLE wird für einen Zyklus TRUE, wenn der Eingangspuls zwischen T_SHORT und T_LONG lang ist. Der Ausgang LONG wird gesetzt, sobald der Eingangsimpuls T_LONG überschritten hat und bleibt solange auf TRUE, wie der Eingangsimpuls auf TRUE bleibt.

25.5 SWITCH_I

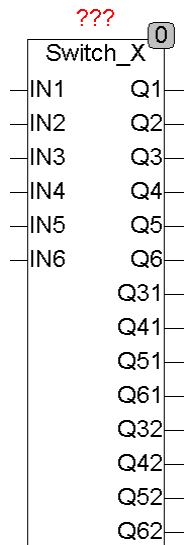
Type	Funktionsblock
Input	SET : BOOL (Eingang zum Einschalten des Ausgangs auf 100%) IN : BOOL (Steuereingang für Taster) RST : BOOL (Eingang zum Ausschalten des Ausgangs)
Output	Q : BOOL (Schaltausgang)
Config	T_DEBOUNCE : TIME (Entprellzeit für Taster) T_RECONFIG : TIME (Rekonfigurationszeit) T_ON_MAX : TIME (Einschaltbegrenzung)



SWITCH_I ist ein intelligenter Schalter der sich selbsttätig auf den angeschlossenen Taster oder Schalter einstellt. Wird ein Schalter angeschlossen, so folgt der Ausgang jeder Schaltflanke des Schalters. Wird jedoch ein Taster angeschlossen, so erkennt SWITCH_I selbst, ob es ein Öffner oder Schließer ist und wertet dann nur die jeweils erste Flanke aus. Die Config-Variable T_ON_MAX legt fest, nach welcher Zeit der Ausgang automatisch wieder ausgeschaltet werden soll. Mit den Eingängen SET und RST kann der Ausgang jederzeit auf 100% ein oder ausgeschaltet werden. Anwendungsbeispiele sind die Meldung von Rauchmeldern oder Alarmanlagen. Die Zeit T_DEBOUNCE dient zum entprellen des Tasters und ist standardmäßig auf 10ms eingestellt. Die Zeit T_RECONFIG wird verwendet, um zu entscheiden, ob ein Schließer oder Öffner am Eingang IN angeschlossen ist. Bleibt der Eingang länger als diese Zeit in einem Zustand, so wird dies als Ruhestellung angenommen.

25.6 SWITCH_X

Type	Funktionsblock
Input	IN1..6 : BOOL (Taster Eingänge)
Output	QX : BOOL (Schaltausgänge)
Config	T_DEBOUNCE : TIME (Entprellzeit für Taster)

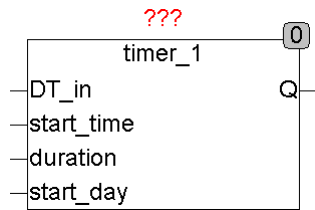


SWITCH_X ist ein Interface für bis zu 6 Taster. Die einzelnen Taster werden mit der Entprellzeit T_DEBOUNCE entprellt und schalten die jeweiligen Ausgänge Q1 bis Q6. IN3 bis IN6 werden direkt auf die Ausgänge geschaltet, wenn sie alleine betätigt werden. IN1 und IN2 erzeugen einen Puls für einen Zyklus nachdem sie betätigt wurden. Wird während IN1 oder IN2 betätigt ist, einer der Eingänge IN3 bis IN 6 betätigt, so wird kein Ausgangsimpuls an Q1 bis Q6 erzeugt, sondern es wird der entsprechende Ausgang Q31 bis Q62 aktiviert. Q42 wird zum Beispiel dann aktiviert, wenn IN4 betätigt wird, während IN2 betätigt ist. Q2 und Q4 werden dann nicht aktiv.

SWITCH_X erlaubt es also auf den Eingängen IN3 bis IN6 eine Dreifachbelegung zu Realisieren und diese durch Betätigen von IN1 oder IN2 und einem weiteren Eingang auszuwählen.

25.7 TIMER_1

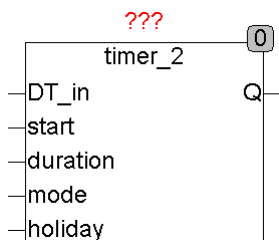
Type	Funktionsblock
Input	DT_IN : DATE_TIME (Datum Zeit Eingang) START_TIME : TOD (Startzeit) DURATION : TIME (Zeitdauer des Ausgangssignals) START_DAY : INT (Wochentag des Ereignisses, 1= Montag)
Output	Q : BOOL (Schaltausgang)



TIMER_1 erzeugt täglich, oder einmal pro Woche an einem festgelegten Wochentag ein Ausgangsereignis mit einer programmierbaren Dauer. DT_IN liefert dem Baustein die Lokalzeit. START_TIME und DURATION legt die Tageszeit und die Dauer des Ereignisses fest. Der Eingang START_DAY legt fest, an welchen Wochentag das Ereignis erzeugt wird (1 = Montag und 7 = Sonntag). Wird START_DAY auf 0 gesetzt, so wird das Ereignis täglich erzeugt. Eine DURATION = 0 legt fest, dass das Ausgangssignal nur für einen Zyklus gesetzt wird. Das erzeugte Ausgangssignal kann auch über Mitternacht verlaufen, oder es kann auch länger als einen Tag sein. TIMER_1 ermittelt die Startzeit für den Impuls und generiert dann unabhängig von der Tageszeit den Ausgangsimpuls mit der durch DURATION definierten Dauer. Die maximale Impulsdauer liegt jedoch bei 49 Tagen (T#49d). TIMER_1 erzeugt limitiert von der Zykluszeit immer die exakte DURATION am Ausgang. TIMER_1 merkt sich an welchen Tag er den letzten Ausgangsimpuls erzeugt hat, so dass sichergestellt ist das nur ein Impuls pro Tag erzeugt wird.

25.8 TIMER_2

Type	Funktionsblock
Input	DT_IN : DATE_TIME (Datum Zeit Eingang) START_TIME : TOD (Startzeit) DURATION : TIME (Zeitdauer des Ausgangssignals) START_DAY : INT (Wochentag des Ereignisses, 1= Montag)
Output	Q : BOOL (Schaltausgang)

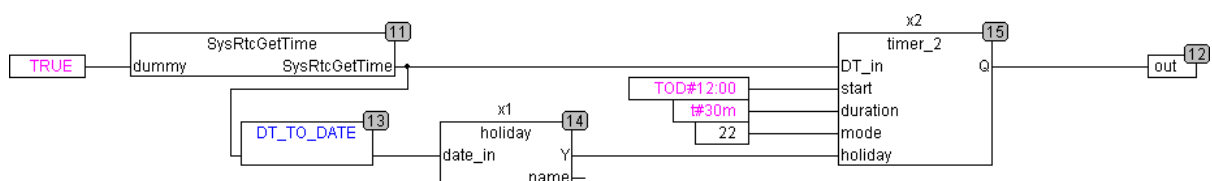


TIMER_2 erzeugt ein Ausgangsereignis mit einer programmierbaren Dauer. DT_IN liefert dem Baustein die Lokalzeit. START_TIME und DURATION legt die Tageszeit

und die Dauer des Ereignisses fest. Der Eingang Mode legt fest, wie oft und an welchem Tagen das Ereignis erzeugt werden soll. HOLIDAY ist ein Eingangssignal, das anzeigt ob der aktuelle Tag ein Feiertag ist. Dieses Signal kann vom Baustein HOLIDAY erzeugt werden.

MODE	Q
0	Es wird kein Ausgangssignal erzeugt
1	nur am Montag
2	nur am Dienstag
3	nur am Mittwoch
4	nur am Donnerstag
5	nur am Freitag
6	nur am Samstag
7	nur am Sonntag
11	jeden Tag
12	alle 2 Tage
13	alle 3 Tage
14	alle 4 Tage
15	alle 5 Tage
16	alle 6 Tage
20	Wochentage (Montag bis Freitag)
21	Samstag und Sonntag
22	Arbeitstage (Wochentage ohne Feiertage)
23	Feiertage und Wochenende
24	Nur an Feiertagen
25	Erster Tag im Monat
26	Letzter Tag im Monat
27	Letzter Tag im Jahr (31. Dezember)
28	Erster Tag im Jahr (1. Januar)

Beispiel für die Anwendung von TIMER_2:



Das Beispiel zeigt die System-Routine (in diesem Fall für einen Wago Controller), die die interne Uhr ausliest und DATE_TIME für TIMER_2 und HOLIDAY bereitstellt. HOLIDAY liefert die Feiertagsinformation an TIMER_2. TIMER_2 liefert in diesem

Beispiel an Wochenenden (Samstag und Sonntag), sowie an Feiertagen (Mode = 22) ein Ausgangssignal jeweils um 12:00 Mittags für eine Dauer von 30 Minuten. TIMER_2 erzeugt limitiert von der Zykluszeit immer die exakte DURATION am Ausgang. TIMER_2 merkt sich an welchen Tag er den letzten Ausgangsimpuls erzeugt hat, so dass sichergestellt ist das nur ein Impuls pro Tag erzeugt wird.

26 Jalousiesteuerung

26.1 Einleitung

Die folgenden Bausteine sind so konzipiert und aufeinander abgestimmt dass Sie einen modularen Aufbau eines Jalousiecontrollers ermöglichen. Dieses modulare System erlaubt einen schnellen und einfachen Aufbau von einfachen bis komplexen Jalousiecontrollern der exakt auf die Anwendung abgestimmt sind. Das System lässt sich auch später jederzeit erweitern und erlaubt einen beliebigen Ausbau des Funktionsumfangs. Die Anwendungen umfassen Jalousien aller Art, Rollläden, und alle Arten von Beschattungseinrichtungen. Die Module sind so gestaltet das sie sich einfach hintereinander schalten lassen und die Reihenfolge der Verschaltung gleichzeitig die Priorität der Funktionen festlegt. Die Signale UP und DN für den Handbetrieb, sowie die Vorgaben für Winkel und Position im Automatikbetrieb (PI und AI) werden dabei von Modul zu Modul weitergereicht, was einen simplen Signalfluss und einen übersichtlichen Aufbau gewährleistet. Eine Besonderheit ist dabei das Die Signale UP und DN wenn beide gleichzeitig TRUE sind den Automatikmodus einschalten.

Alle Bausteine haben die Eingänge UP und DN mit denen der Manuelle Auf und Ab Befehl empfangen wird und durch QU und QD an den nächsten Baustein weitergereicht wird. Werden beide Eingänge UP und DN auf TRUE gesetzt so schalten die betreffenden Module auf Automatik Modus und werten die Eingänge PI und AI (Position- und Winkel- Eingang für die Jalousie aus.

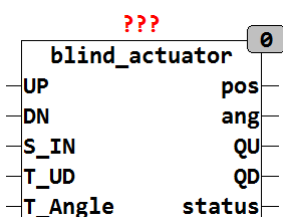
Der Ausgang STATUS gibt ESR kompatible Meldungen über den Zustand des betreffenden Bausteins aus. Wenn keine eigenen Zustandsmeldungen anstehen reicht jeder Baustein die am Eingang S_IN anliegenden Statusmeldungen an den Ausgang STATUS weiter.

Durch die Reihenschaltung einzelner Funktionen kann im Falle einer Fehlersuche auf einfache Weise der Signalfluss und die Funktionsweise überprüft werden. Durch die Reihenfolge der Module wird auch gleichzeitig die Priorität der einzelnen Funktionen festgelegt und kann ohne großen Aufwand vom Anwender jederzeit geändert werden.

Zukünftige oder Kundenspezifische Funktionen können auf diese Weise einfach und schnell in die Vorhandenen Blöcke eingeschaltet werden ohne dabei die vorhandene Programmierung ändern zu müssen.

26.2 BLIND_ACTUATOR

Type	Funktionsblock
Input	UP : BOOL (Eingang AUF) DN : BOOL (Eingang AB) S_IN : BYTE (ESR kompatibler Status Eingang) T_UD : TIME (Laufzeit AUF / AB) T_ANGLE : TIME (Laufzeit der Lamellenverstellung)
Config	T_LOCKOUT : TIME (Totzeit zwischen Richtungswechsel)
Output	POS : BYTE (Position der Jalousie, 0 = unten, 255 = oben) ANG : BYTE (Winkel der Lamelle, 0 = vertikal, 255 = horizontal) QU : BOOL (Motor Auf Signal) QD : BOOL (Motor Ab Signal) STATUS : BYTE (ESR kompatibler Status Ausgang)



BLIND_ACTUATOR ist ein Jalousie / Rollladen Aktor mit Simulation der Position und der Winkelstellung der Lamellen. Die Eingänge UP und DN sind gegenseitig verriegelt, so dass QU und QD nie gleichzeitig aktiv sein können. Mit der Zeit T_LOCKOUT wird die minimale Pause zwischen einem Richtungswechsel festgelegt. zusätzlich bietet BLIND_ACTUATOR noch 2 Ausgänge vom Typ Byte die die jeweilige Stellung und Position der Jalousie simulieren. Für eine exakte Simulation sind die Setup Zeiten T_UD und T_ANGLE entsprechend einzustellen. T_UD legt die Zeit fest die zum Fahren von "geschlossen" auf "offen" (hochfahren) benötigt wird. T_ANGLE spezifiziert die Zeit die zur Verstellung der Lamellen von "senkrecht" nach Horizontal benötigt wird. Der Aktor stellt sicher das beim Hochfahren zuerst die Lamellen auf Horizontal gestellt werden und anschließend erst mit dem Hochfahren begonnen wird. Umgekehrt werden beim Herunterfahren erst die Lamellen auf Senkrecht gestellt und dann mit dem Herunterfahren begonnen. POS = 0 bedeutet Jalousie heruntergefahren, und POS = 255 bedeutet Jalousie ist hochgefahren. Zwischenstellungen werden entsprechend mit Zwischenwerten 0 .. 255 ausgegeben. Der Winkel der Lamellen wird durch den Ausgang ANG ausgegeben, wobei ANG = 0 die vertikale Stellung und ANG = 255 die horizontale Stellung bedeuten, Werte zwischen 0 und 255 geben den entsprechenden Winkel an. Durch die Ausgänge POS und ANG wird die Information über die Jalousie Stellung der Steuerung zur Verfügung gestellt. ANG und POS können jedoch nur sinnvolle Werte liefern wenn die Zeiten T_UD und T_ANGLE exakt für die entsprechende Jalousie angepasst sind. Der Aktor kann,

wenn T_ANGLE auf T#0s gesetzt wird auch für Rollläden aller Arten verwendet werden. Die Eingänge T_UD, T_ANGLE und T_LOCKOUT haben folgende Vorgabewerte:

T_UD = T#10S

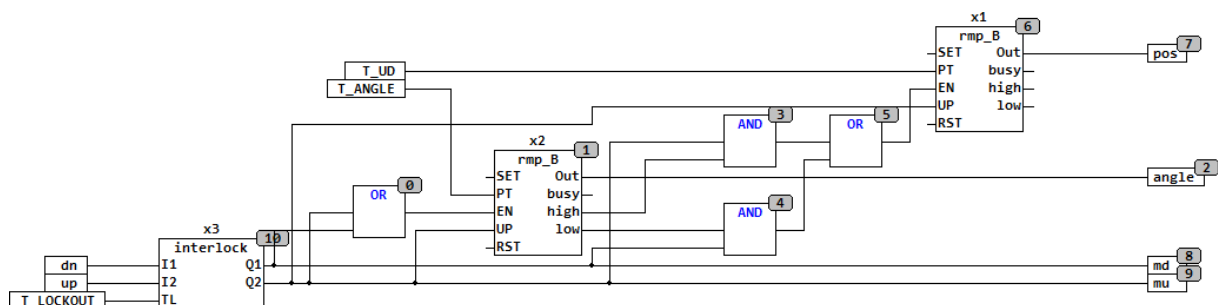
T_ANGLE = T#3S

T_LOCKOUT = T#100MS

Der Eingang S_IN und der Ausgang STATUS sind ESR kompatible Aus und Eingänge, über den Eingang S_IN melden vorgeschaltete Funktionen ihren Status an das Modul, dieser Status wird an den Ausgang STATUS weitergeleitet, und eigene Statusmeldungen werden über STATUS ausgegeben. Wenn am Eingang eine Statusmeldung vorliegt überschreibt diese die eigenen Statusmeldungen, ein Fehler wird mit höchster Priorität ausgegeben.

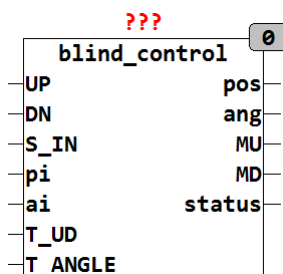
STATUS	Bedeutung
0	keine Meldung
1	Fehler, UP und DN gleichzeitig aktiv
101	Manual UP
102	Manual DN
NNN	weitergereichte Meldung

Die folgende Grafik zeigt den inneren Aufbau und die Funktionsweise des Moduls:



26.3 BLIND_CONTROL

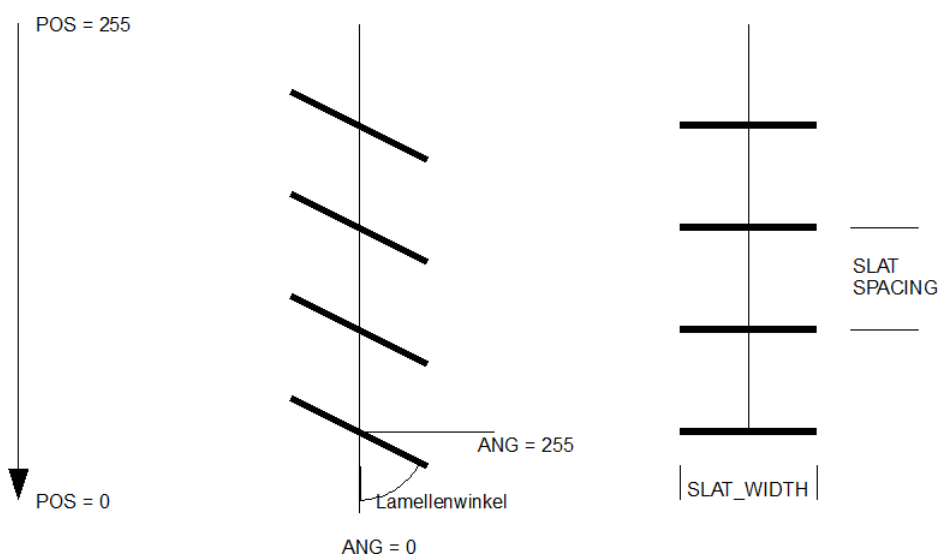
Type	Funktionsblock
Input	UP : BOOL (Eingang AUF) DN : BOOL (Eingang AB) S_IN : BYTE (ESR kompatibler Status Eingang) PI : BYTE (Vorgabe der Position) AI : BYTE (Vorgabe des Lamellenwinkels) T_UD : TIME (Zeit zum Hochfahren von 0 .. 255) T_ANGLE : TIME (Zeit um den Lamellenwinkel von 0 .. 255 zu verstellen)
Config	SENS : BYTE (Auflösung der Regelung) T_LOCKOUT : TIME (Totzeit bei Richtungswechsel der Motoren)
Output	POS : BYTE (Simulierte Jalousiestellung) ANG : BYTE (Simulierter Lamellenwinkel) MU : BOOL (Motor Auf Signal) MD : BOOL (Motor Ab Signal) STATUS : BYTE (ESR kompatibler Status Ausgang)



BLIND_CONTROL regelt die Jalousiestellung und den Lamellenwinkel gemäß den Vorgaben von PI und AI wenn UP und DN gleichzeitig TRUE sind (Automatik Modus). POS und ANG sind dabei die Aktualwerte der Jalousie. BLIND_CONTROL schaltet die Ausgänge MU oder MD solange in der geeigneten Reihenfolge auf TRUE bis die Werte an POS und ANG den Sollwerten an PI und AI entsprechen. Ein interner Sequenzer berücksichtigt dabei dass beim Auf- und Ab-fahren der Jalousie sich der Lamellenwinkel zuerst verstellt und anschließend erst das Hoch- oder Runter-fahren Der Jalousie beginnt. Wird also die Jalousie Hoch- oder Runter-gefahren so verstellt sich zwangsläufig der Lamellenwinkel welcher dann anschließend wieder durch entsprechende Gegenfahrt eingestellt werden muss. Der Eingang SENS legt fest ab welche Regeldifferenz die Regelung aktiv wird und den Ausgang so einstellt das er den Eingängen PI und AI entspricht. Ist SENS = 0 so wird jede Abweichung ausgeregelt, Ist SENS = 5 (Vorgabewert) so wird erst ab einer Differenz von 5 zwischen den Vorgabewerten und den tatsächlichen werten ausgeregelt. Wenn UP und DN nicht beide TRUE sind verlässt BLIND_CONTROL den Automatikmodus und die

Ausgänge QU und QD werden im Handbetrieb direkt von UP und DN gesteuert. BLIND_CONTROL benötigt nicht den Baustein BLIND_ACTUATOR um eine JALOUSIE zu steuern, BLIND_ACTUATOR ist bereits in BLIND_CONTROL integriert. Wenn keine Automatische Regelung der Jalousie benötigt wird so kann auf BLIND_CONTROL verzichtet werden und BLIND_ACTUATOR zum Einsatz kommen. Beim Einsatz von BLIND_CONTROL ist darauf zu achten dass die Zykluszeit für den Baustein kleiner als $T_ANGLE / 512 * SENS$ beträgt. SENS kann durch einen Doppelklick auf das grafische Symbol von BLIND_CONTROL verändert werden, der Vorgabewert ist 5. Wird die Zykluszeit zu groß so beginnt die Jalousie unregelmäßig hin und herzufahren. Wenn eine kleiner Zykluszeit nicht möglich ist kann der Wert von SENS vergrößert werden.

Die folgende Grafik verdeutlicht die Geometrie der Jalousie:

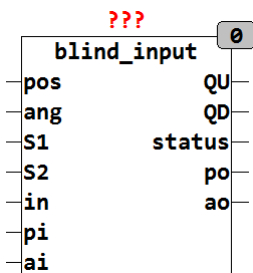


Der Eingang S_IN und der Ausgang STATUS sind ESR kompatible Aus und Eingänge, über den Eingang S_IN melden vorgeschaltete Funktionen Ihren Status an das Modul, dieser Status wird an den Ausgang STATUS weitergeleitet, und eigene Statusmeldungen werden über STATUS ausgegeben.

STATUS	Bedeutung
0	keine Meldung
101	Manual Auf
102	Manual Ab
121	Position Auf
122	Position Ab
123	Lamellenstellung Horizontal
124	Lamellenstellung Vertikal
NNN	weitergereichte Meldungen

26.4 BLIND_INPUT

Type	Funktionsblock
Input	S1 : BOOL (Eingang AUF) S2 : BOOL (Eingang AB) SINGLE : BOOL (Konfiguriert Ein Taster Betrieb) POS : BYTE (Rückführung der Jalousiestellung) ANG : BYTE (Rückführung des Lamellenwinkels) IN : BOOL (Stellt die Jalousie auf die durch AI und PI definierte Position) PI : BYTE (Position wenn IN = TRUE) AI : BYTE (Winkel wenn IN = TRUE)
Config	SINGLE_SWITCH : BOOL (Konfigurations Bit für Einzeltasterbetrieb) CLICK_MODE : BOOL (Konfigurations Bit für Click Taster Betrieb) CLICK_TIME : BOOL (Timeout für Click Modus) CLICK_TIMEOUT : TIME (Timeout für eine Bewegung ohne Abbruch) MANUAL_TIMEOUT : TIME (Timeout des Handbetriebs)
Output	QU : BOOL (Motor Auf Signal) QD : BOOL (Motor Ab Signal) STATUS : BYTE (ESR kompatibler Status Ausgang)



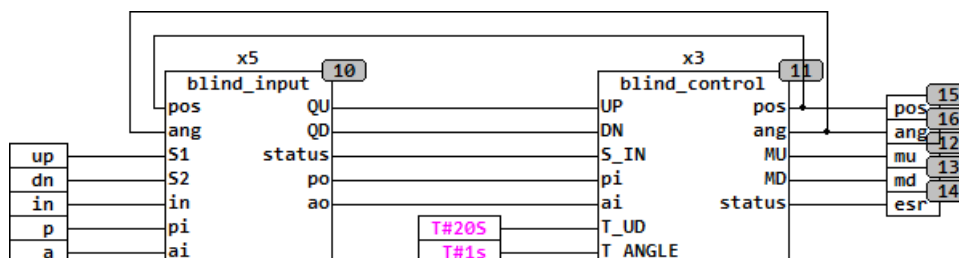
BLIND_INPUT dient als Taster Interface zur Bedienung von Jalousien. Der Baustein unterstützt 3 Modi, Handbetrieb, Automatikbetrieb und gesteuerter Betrieb. wenn IN = FALSE (Handbetrieb) werden die Eingänge S1 und S2 benutzt um die Ausgänge QU und QD zu steuern. Wenn die Config Variable SINGLE_SWITCH = TRUE ist, dann wird der Eingang S2 ignoriert, und die gesamte Steuerung erfolgt über den Taster S1. S1 schaltet dann abwechselnd QU und QD so dass durch aufeinander folgendes Drücken des Tasters S1 zwischen Auf und Ab Bewegung gewechselt wird. Der interne Vorgabewert ist FALSE (2 Taster Konfiguration). Die Config Variable MANUAL_TIMEOUT definiert nach welcher Ruhezeit (Zeit ohne Signal auf S1 oder S2) der Baustein selbständig in den Automatikbetrieb wechselt. Wird dieser Wert nicht spezifiziert so wird der Interne Vorgabewert von 1 Stunde verwendet. Wenn der Eingang IN = TRUE ist, werden die Ausgänge QU und QD auf Automatik (beide TRUE)

gesetzt und die Eingänge PI und AI auf die Ausgänge PS und AS geschaltet. Die Eingänge POS und ANG sind die Rückführungseingänge für die aktuelle Position der Jalousie. Diese Werte werden von dem Modul BLIND_CONTROL bereitgestellt. Mit der Config Variable CLICK_MODE wird ein 2 Taster Betrieb festgelegt, jedoch startet ein kurzer Tastendruck die Richtung Auf für S1 und Ab für S2 und ein zweiter kurzer Tastendruck beendet die entsprechende Richtung oder kehrt die Richtung um. Diese Einstellung ist für Rollläden mit langer Laufzeit Sinnvoll, oder um mit einem kurzen Tastendruck in eine Endstellung zu fahren. wird der Tastendruck länger als die Config Zeit CLICK_TIME so wird für diesen Tastendruck der CLICK Modus verlassen und die Jalousie fährt solange wie die Taste gedrückt bleibt. Ist ein Tastendruck kürzer als CLICK_TIME so Führt die Jalousie weiter bis ein weiterer Klick die Fahrt beendet. Der Vorgabewert für CLICK_TIME ist 200 Millisekunden und die Vorgabe für CLICK_MODE ist TRUE. Wenn die Config Variable CLICK_MODE auf TRUE ist hat die Einstellung SINGLE_SWITCH keine Wirkung. Mit der über CLICK_TIMEOUT eingestellten Zeit wird die Laufzeit begrenzt die durch einen einfachen Click gestartet wird aber nicht mit einem weiteren Click beendet wird. Der CLICK_TIMEOUT ist mit T#60s vorbelegt und sollte solange sein das die Jalousie sicher aus jeder beliebigen Stellung die Endstellung erreichen kann.

Der Ausgang STATUS ist ESR kompatibel und gibt Statusmeldungen über Zustandsänderungen aus.

STATUS	Bedeutung
0	keine Meldung
131	Forced Input Active
132	Manual Auf
133	Manual Ab
134	Auto Mode nach Timeout

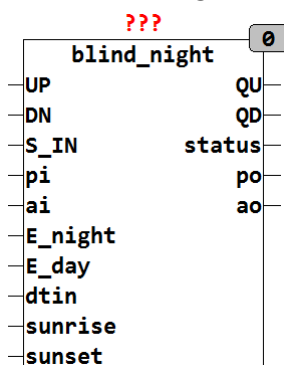
Das folgende Beispiel zeigt den Aufbau eines Jalousiecontrollers mit dem Baustein BLIND_INPUT und BLIND_CONTROL:



Die Verwendung weiterer BLIND Module ist optional und dient dazu den Funktionsumfang zu erweitern. BLIND_INPUT und BLIND_CONTROL ergeben bereits eine vollwertige Jalousiesteuerung.

26.5 BLIND_NIGHT

Type	Funktionsblock
Input	UP : BOOL (Eingang AUF) DN : BOOL (Eingang AB) S_IN : BYTE (ESR kompatibler Status Eingang) PI : BYTE (Eingangswert der Jalousiestellung im Automatikbetrieb) AI : BYTE (Eingangswert des Lamellenwinkels im Automatikbetrieb) E_NIGHT : BOOL (Automatische Nachtschaltung ein) E_DAY : BOOL (Automatische Tagschaltung ein) DTIN : DT (Datum und Tageszeit in Weltzeit UTC) SUNRISE : TOD (Zeit für Sonnenaufgang) SUNSET : TOD (Zeit für Sonnenuntergang)
Config	SUNRISE_OFFSET : TIME (Verzögerung bei Sonnenuntergang) SUNSET_OFFSET : TIME (Verzögerung bei Sonnenaufgang) NIGHT_POSITION : BYTE (Position für Nachtschaltung) NIGHT_ANGLE : BYTE (Winkel für Nachtschaltung)
Output	QU : BOOL (Motor Auf Signal) QD : BOOL (Motor Ab Signal) STATUS : BYTE (ESR kompatibler Status Ausgang) PO : BYTE (Ausgangswert der Jalousiestellung im Automatikbetrieb) AO : BYTE (Ausgangswert des Lamellenwinkels im Automatikbetrieb)



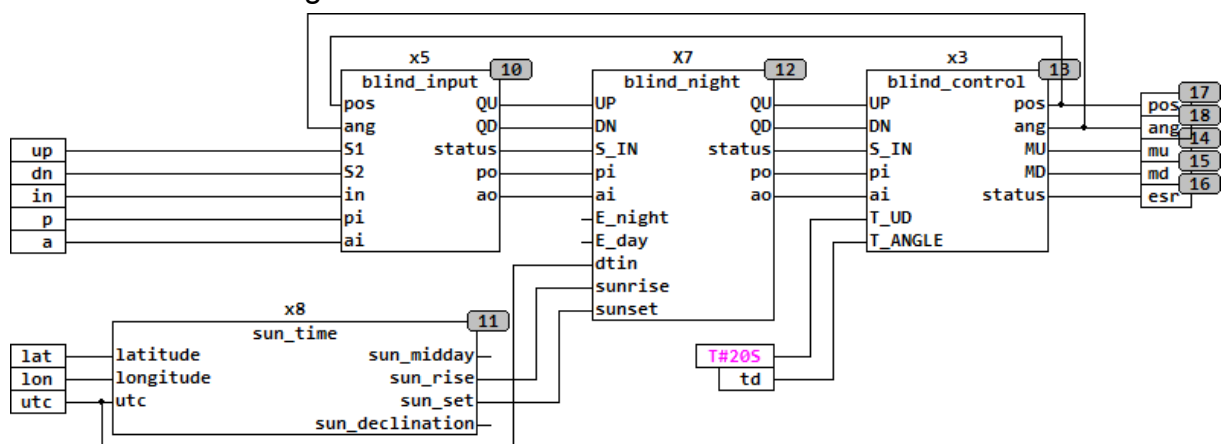
BLIND_NIGHT dient dazu die Rollläden oder Jalousie bei Nacht zu schließen. Der Baustein schließt automatisch nach Sonnenuntergang mit einer Verzögerung von SUNSET_OFFSET die Jalousie und fährt die Jalousie nach Sonnenaufgang mit einer Verzögerung von SUNRISE_OFFSET wieder auf die Position die vor der Nachtbeschattung bestanden hat zurück. Das Schließen und Öffnen kann separat mit den

Eingängen E_NIGHT für schließen und E_DAY für öffnen Freigeschaltet werden. Wird zum Beispiel E_NIGHT auf TRUE gestellt und E_DAY nicht so fährt am Abend bei Dämmerung die Jalousie herunter, jedoch muss sie am nächsten Morgen manuell Hochgefahren werden. Werden E_NIGHT und E_DAY nicht beschaltet so werden beide intern auf TRUE gesetzt. Damit die entsprechenden Zeiten ermittelt werden können benötigt der Baustein die Tageszeit nach Weltzeit (DTIN) und die entsprechenden Zeiten für Sonnenaufgang (SUNRISE) und Sonnenuntergang (SUNSET). Die Zeiten für Sonnenaufgang und Sonnenuntergang liefert der Baustein SUN_TIME aus der Bibliothek unter Zeit- Datums-Funktionen. UP, DN und SIN sind die Eingänge von anderen BLIND Modulen und werden im Tagesbetrieb an die Ausgänge QU, QD und STATUS weitergegeben. Die Signale PI, AI und PO, AO reichen die Werte für die Position und den Lamellenwinkel der Jalousie an die folgenden Bausteine weiter. Im Nachtbetrieb werden an den Ausgängen PO und AO die Werte für den Nachtbetrieb ausgegeben, jegliche manuelle Betätigung löscht den Automatischen Nachtbetrieb. BLIND_NIGHT beendet den Nachtbetrieb indem es auf die zuletzt (vor Beginn der Nachtbeschattung) an den Eingängen PI und AI anliegende Position fährt. Die Jalousie wird also nur dann hochgefahren wenn sie bei Beginn der Nachtbeschattung auch oben war.

Der Eingang S_IN und der Ausgang STATUS sind ESR kompatible Aus und Eingänge, über den Eingang S_IN melden vorgeschaltete Funktionen Ihren Status an das Modul, dieser Status wird an den Ausgang STATUS weitergeleitet, und eigene Statusmeldungen werden über STATUS ausgegeben.

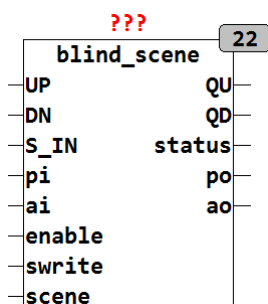
STATUS	Bedeutung
0	keine Meldung
141	Nachtbetrieb
NNN	weitergereichte Meldungen

Die folgende Grafik zeigt die Verschaltung von BLIND_NIGHT mit anderen Modulen zur Jalousiesteuerung:



26.6 BLIND_SCENE

Type	Funktionsblock
Input	UP : BOOL (Eingang AUF) DN : BOOL (Eingang AB) S_IN : BYTE (ESR kompatibler Status Eingang) PI : BYTE (Eingangswert der Jalousiestellung im Automatikbetrieb) AI : BYTE (Eingangswert des Lamellenwinkels im Automatikbetrieb) ENABLE : BOOL (Freigabeeingang für Szenen) SWRITE : BOOL (Schreibeingang für Szenen) SCENE : BYTE (Nummer der Szene)
Output	QU : BOOL (Motor Auf Signal) QD : BOOL (Motor Ab Signal) STATUS : BYTE (ESR kompatibler Status Ausgang) PO : BYTE (Ausgangswert der Jalousiestellung im Automatikbetrieb) AO : BYTE (Ausgangswert des Lamellenwinkels im Automatikbetrieb)



BLIND_SCENE speichert bis zu 16 Szenen bestehend aus jeweils aktueller Jalousiestellung und Winkel ab und kann diese Szenen bei Abruf wieder herstellen. jede einzelne Szene kann aktiv oder inaktiv sein, abhängig davon ob beim Speichern der Szene der Eingang ENABLE TRUE war oder nicht (ENABLE = TRUE bedeutet aktiv). Eine Szene wird abgerufen indem die Nummer der Szene (0 .. 15) am Eingang SCENE angelegt wird und gleichzeitig ENABLE auf TRUE gesetzt wird. Eine Szene kann nur dann abgerufen werden wenn die beiden Eingänge UP und DN gleichzeitig TRUE sind (Automatik Modus). Dadurch ist sichergestellt das eine aktive Szene immer von der Manuellen Betriebsart überschrieben wird.

Die folgende Tabelle verdeutlicht die Funktionsweise von BLIND_SCENE:

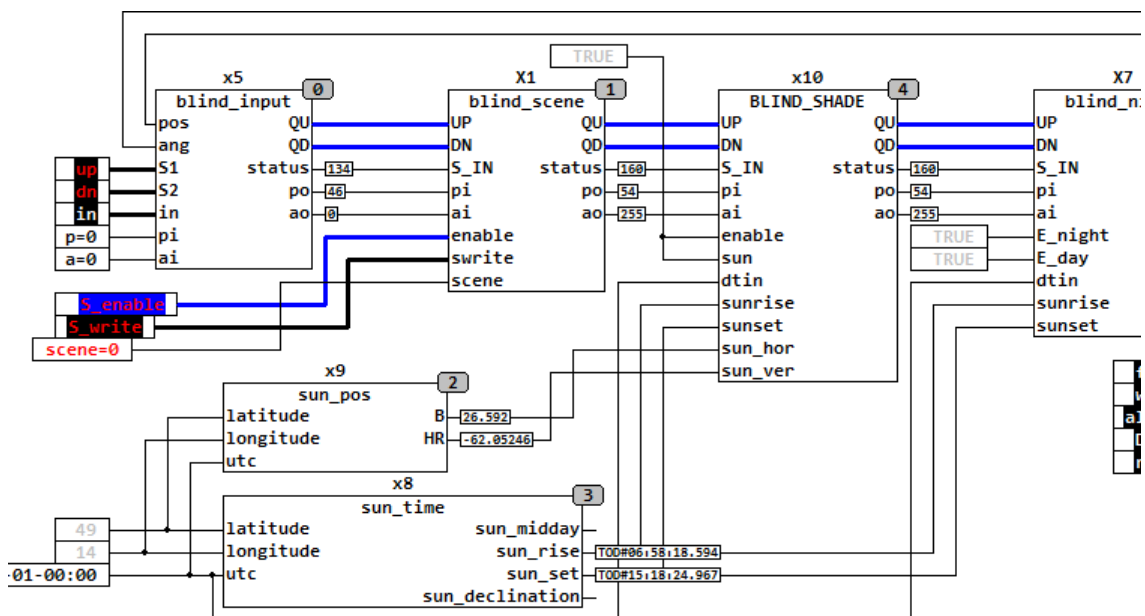
UP	DN	ENABLE	SWRITE	SCENE	QU	QD	PO	AO	
1	1	0	0	-	1	1	PI	AI	no scene

-	-	1	1	y	-	-	-	-	write scene number y
-	-	0	1	y	-	-	-	-	disable scene number y
1	1	1	0	y	1	1	-	-	recall scene number y

Der Eingang S_IN und der Ausgang STATUS sind ESR kompatible Aus und Eingänge , über den Eingang S_IN melden vorgeschaltete Bausteine Ihren Status an das Modul, dieser Status wird an den Ausgang STATUS weitergeleitet, und eigene Statusmeldungen werden über STATUS Ausgegeben.

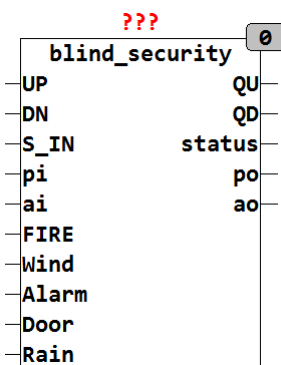
STATUS	Bedeutung
160 .. 175	Szenen 0..15 aktiv
176	Szene geschrieben
NNN	weitergereichte Meldungen

Die folgende Grafik zeigt die Anwendung von BLIND_SCENE mit anderen Modulen zur Ansteuerung einer Jalousie:



26.7 BLIND_SECURITY

Type	Funktionsblock
Input	UP : BOOL (Eingang AUF) DN : BOOL (Eingang AB) S_IN : BYTE (ESR kompatibler Status Eingang) PI : BYTE (Eingangswert der Jalousiestellung im Automatikbetrieb) AI : BYTE (Eingangswert des Lamellenwinkels im Automatikbetrieb) FIRE : BOOL (Eingang für Brandalarm) WIND : BOOL (Eingang für Windalarm) ALARM : BOOL (Eingang für Einbruchsmeldung) DOOR : BOOL (Eingang für Türkontakt) RAIN : BOOL (Eingang für Regenmelder)
Config	ALARM_UP : BOOL (Vorgaberichtung bei ALARM, Default = Up) WIND_UP : BOOL (Vorgaberichtung bei Wind, Default = Up) RAIN_UP : BOOL (Vorgaberichtung bei Regen, Default = Down)
Output	QU : BOOL (Motor Auf Signal) QD : BOOL (Motor Ab Signal) STATUS : BYTE (ESR kompatibler Status Ausgang) PO : BYTE (Ausgangswert der Jalousiestellung im Automatikbetrieb) AO : BYTE (Ausgangswert des Lamellenwinkels im Automatikbetrieb)



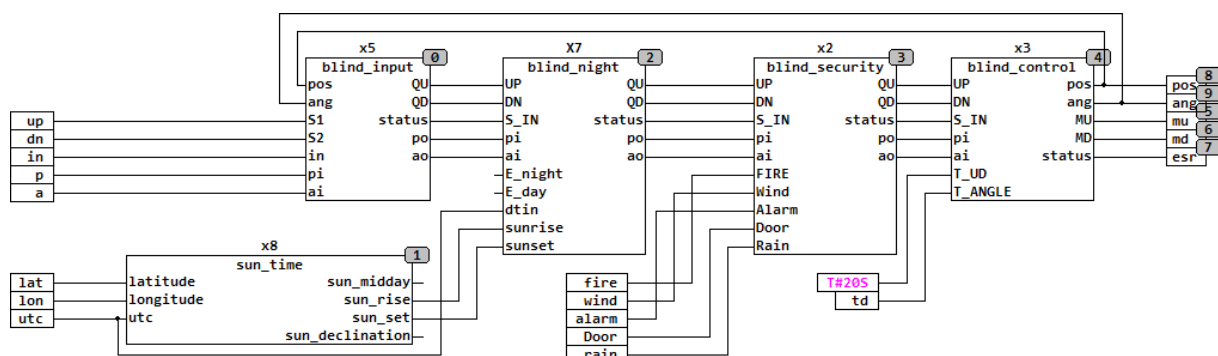
BLIND_SECURITY stellt sicher das Jalousien bei bestimmten Ereignissen entweder nach oben oder nach unten gefahren werden. Die Eingänge UP und DN steuern über die Ausgänge QU und QD ein nachgeschaltetes Modul BLIND_ACTUATOR. Mit den Eingängen FIRE, WIND, ALARM und RAIN werden die Eingänge UP und DN überschrieben und die Jalousie entweder ganz nach oben oder ganz nach unten gefahren. Hierbei hat FIRE die höchste Priorität, gefolgt von WIND, Alarm und mit der niedrigsten Priorität RAIN. Rain kann als einziger auch von den manuellen Eingängen

gen UP und DN überschrieben werden. Sollte also der Benutzer entscheiden dass trotz Regen die Jalousie offen bleiben soll, so muss er lediglich den Regenschutz durch einen kurzen Tastendruck auf UP oder DN unterbrechen. FIRE fährt die Jalousie nach oben, während RAIN, Wind und Alarm für Auf oder Ab konfigurierbar sind. ALARM ist mit der Config Variablen ALARM_UP sowohl für Hoch- als auch Runterfahrt konfigurierbar, Die Config Variable WIND_UP legt fest ob bei Wind nach oben oder runter gefahren wird. Mit der Variable RAIN_UP wird festgelegt welche Stellung bei Regen angefahren wird. Die Vorgabewerte sind UP für Alarm, UP für Wind und DN für Regen. Die Config Variablen können durch einen Doppelklick auf das Symbol jederzeit verändert werden.

Der Eingang S_IN und der Ausgang STATUS sind ESR compatible Aus und Eingänge , über den Eingang S_IN melden vorgeschaltete Funktionen Ihren Status an das Modul, dieser Status wird an den Ausgang STATUS weitergeleitet, und eigene Statusmeldungen werden über STATUS Ausgegeben.

STATUS	Bedeutung
0	keine Meldung
111	Feuer
112	Wind
113	Einbruch Alarm
114	Türalarm
115	Regen
NNN	weitergereichte Meldungen

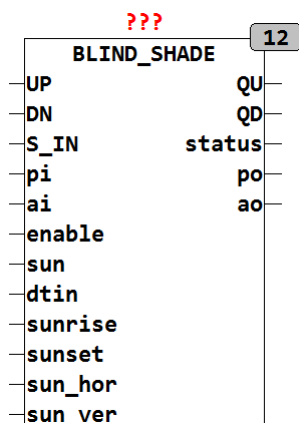
Die folgende Grafik zeigt die Anwendung von BLIND_SECURITY mit BLIND_ACTUATOR zur Steuerung einer Jalousie:



BLIND_SECURITY muss unbedingt direkt vor BLIND_CONTROL eingesetzt werden. Sollten andere Module zwischen BLIND_SECURITY und BLIND_CONTROL eingebaut werden so sind die Sicherheitsfunktionen nicht mehr gewährleistet.

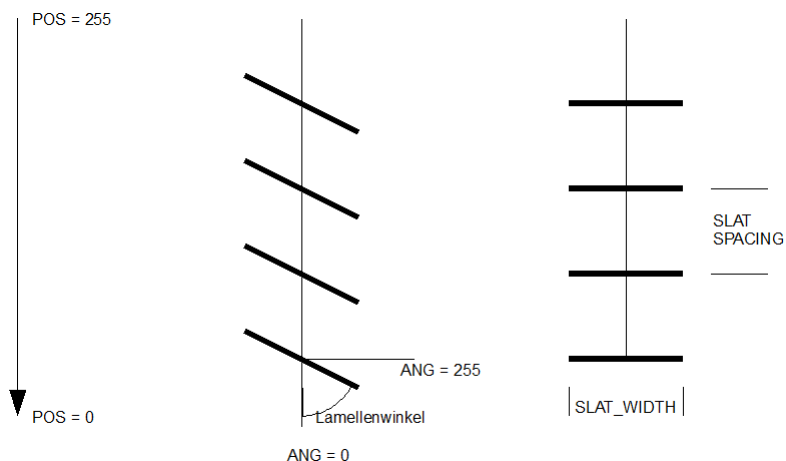
26.8 BLIND_SHADE

Type	Funktionsblock
Input	UP : BOOL (Eingang AUF) DN : BOOL (Eingang AB) S_IN : BYTE (ESR kompatibler Status Eingang) PI : BYTE (Eingangswert der Jalousiestellung im Automatikbetrieb) AI : BYTE (Eingangswert des Lamellenwinkels im Automatikbetrieb) DTIN : DT (Datum und Tageszeit in Weltzeit UTC) SUNRISE : TOD (Zeit für Sonnenaufgang) SUNSET : TOD (Zeit für Sonnenuntergang) SUN_HOR : REAL (Horizontaler Sonnenwinkel, Süd = 180°) SUN_VER : Vertikaler Sonnenwinkel, 0° = Horizont)
Config	SUNRISE_OFFSET : TIME (Verzögerung bei Sonnenuntergang) SUNSET_PRESET : TIME (Verzögerung bei Sonnenaufgang) DIRECTION : REAL (Fassadenausrichtung, 180° = Südfassade) ANGLE_OFFSET : REAL (Horizontaler Öffnungswinkel für Beschattung) SLAT_WIDTH : REAL (Breite der Lamellen in mm) SLAT_SPACING : REAL (Abstand der Lamellen in mm) SHADE_DELAY : TIME (Verzögerungszeit der Beschattung)
Output	QU : BOOL (Motor Auf Signal) QD : BOOL (Motor Ab Signal) STATUS : BYTE (ESR kompatibler Status Ausgang) PO : BYTE (Ausgangswert der Jalousiestellung im Automatikbetrieb) AO : BYTE (Ausgangswert des Lamellenwinkels im Automatikbetrieb)

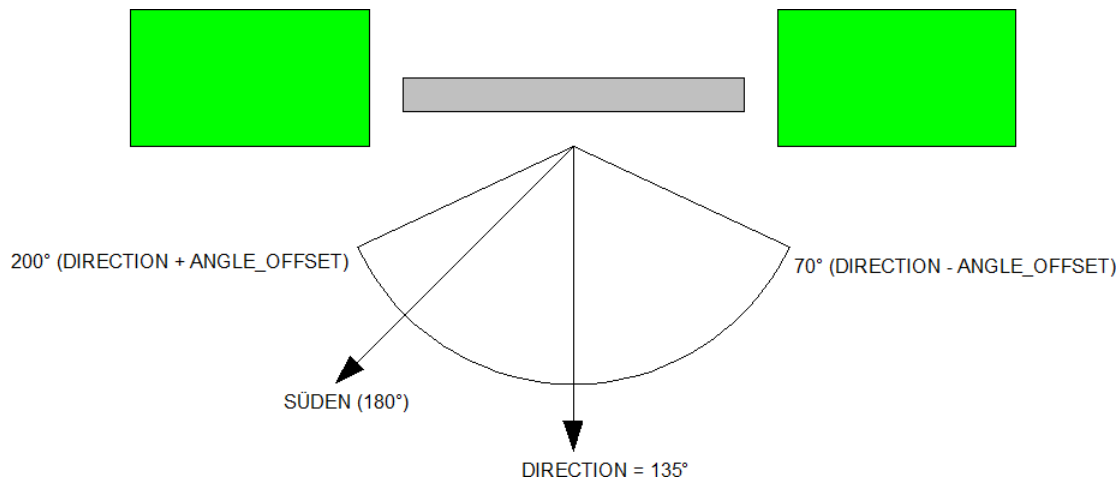


BLIND_SHADE berechnet aus dem momentanen Sonnenstand den geeigneten Lamellenwinkel um eine optimale Beschattung zu gewährleisten. Die Lamellen werden dem Sonnenstand nachgeführt, so dass über den Tagesverlauf immer Beschattung sichergestellt ist. Mit dem Eingang ENABLE wird die Funktion freigeschaltet wenn gleichzeitig UP und DN (Automatik Modus) aktiv sind. Der Baustein wertet weiterhin den EINGANG SUN aus welcher durch TRUE Sonnenschein anzeigt. Wird SUN oder ENABLE FALSE so schaltet sich der Baustein automatisch ab. SUNRISE_OFFSET mit welchem Zeitversatz nach Sonnenaufgang die Beschattung aktiv wird. SUNSET_PRESET legt mit welcher Zeitspanne vor Sonnenuntergang die Beschattung ausgesetzt wird. Die Beschattung ist dann Aktiv wenn SUN = TRUE, ENABLE = TRUE, UP = TRUE, DN = TRUE, der horizontale Sonnenwinkel sich innerhalb des Bereichs DIRECTION - ANGLE_OFFSET und DIRECTION + ANGLE_OFFSET befindet und die Tageszeit sich innerhalb des durch SUNRISE, SUNRISE_OFFSET, SUNSET und SUNSET_PRESET definierten Zeitbereichs befindet. DIRECTION legt die Ausrichtung der Fassade fest, 180° bedeutet Fassade zeigt genau nach Süden, 90° liegt im Osten und 270° im Westen. Mit der Setup Variable BLIND_SHADE wird festgelegt wie lange nachdem SUN auf FALSE geht die Beschattung eingestellt bleibt. Der Vorgabewert liegt bei 60 Sekunden. SHADE_DELAY Verhindert das bei Teilbewölkung die Jalousie dauernd auf und ab fährt. Beim Einsatz von BLIND_SHADE ist darauf zu achten dass die Zykluszeit für den Baustein kleiner als $T_ANGLE / 512 * SENS$ beträgt. SENS ist hierbei der SENS Wert des BLIND_CONTROLLERS. Wird die Zykluszeit zu groß so beginnt die Jalousie unregelmäßig hin und herzufahren.

Die folgende Grafik beschreibt die Geometrie der Jalousie:

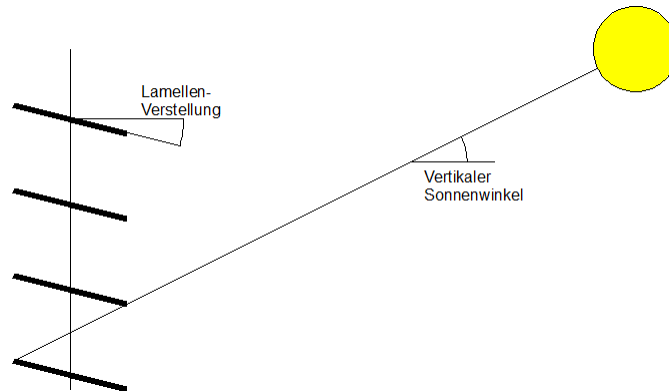


Die folgende Grafik zeigt eine nach Süd-Ost gerichtete Fassade mit $DIRECTION = 135^\circ$ und $ANGLE_OFFSET = 65^\circ$:



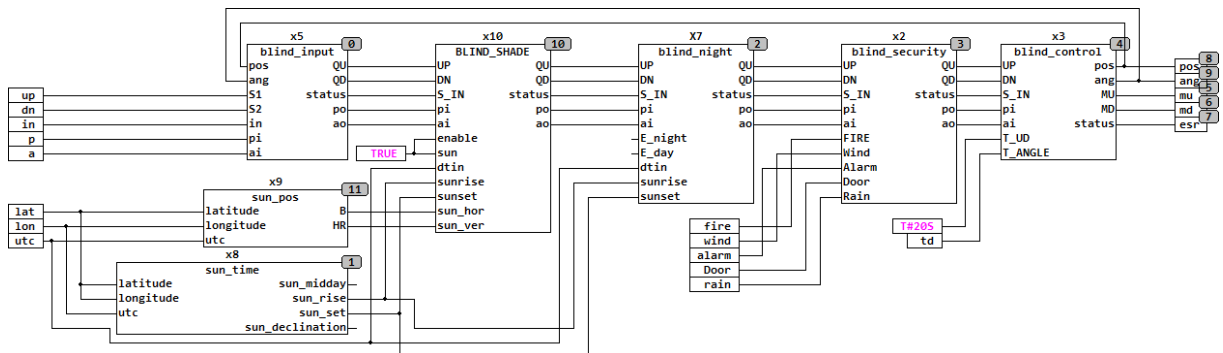
Die Beschattungsfunktion berechnet den Lamellenwinkel so dass die Lamellen immer nur soweit geschlossen werden dass die Sonne abgeschattet wird, aber dennoch soviel Licht wie möglich in den Raum gelangt. Aus den Angaben $DIRECTION$ und $ANGLE_OFFSET$ wird berechnet wann der horizontale Einstrahlwinkel der Sonne eine Beschattung erfordert. Je nach Stärke der Mauer und Breite des Fensters kann der $ANGLE_OFFSET$ so eingestellt werden das unnötige Beschattung vermieden wird. Mit $DIRECTION$ wird die Himmelsrichtung der Fassade angegeben. Mithilfe der Abmessungen der Lamellen, Breite und Abstand in Millimeter ($SLAT_WIDTH$ und $SLAT_SPACING$) wird berechnet wie weit die Lamellen geneigt werden müssen um die Sonneneinstrahlung zu verhindern. Ziel ist es dabei die Lamellen nur soweit wie unbedingt nötig zu neigen damit optimale Lichtverhältnisse garantiert sind. Um bei Sonnenaufgang und Sonnenuntergang die Stimmung und Lichtverhältnisse nicht zu beeinflussen kann ein $OFFSET$ vom Sonnenaufgang und ein $PRESET$ vor dem Sonnenuntergang eingestellt werden. Mit einen $OFFSET$ von 30 Minuten und einem $PRESET$ von 60 Minuten wird zum Beispiel erst 30 Minuten nach Sonnenaufgang mit der Beschattung begonnen und bereits 60 Minuten vor Sonnenuntergang die Beschattung beendet. Der Eingang SUN des Moduls dient Dazu einen Sonnenintensitätssensor oder einen beliebigen geeigneten Sensor anzuschließen der die Funktion unterbricht wenn keine Sonnenstrahlung vorliegt.

In der Folgenden Grafik wird die Abschattung verdeutlicht:



Der Eingang S_IN und der Ausgang STATUS sind ESR kompatible Aus und Eingänge , über den Eingang S_IN melden vorgeschaltete Funktionen Ihren Status an das Modul, dieser Status wird an den Ausgang STATUS weitergeleitet, und eigene Statusmeldungen werden über STATUS ausgegeben. BLIND_SHADE meldet am STATUS Ausgang den STATUS 151 wenn die Beschattungsfunktion aktiv ist.

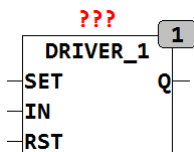
Das folgende Beispiel zeigt die Anwendung von BLIND_SHADE innerhalb einer Jalousiesteuerung:



27 Automation

27.1 DRIVER_1

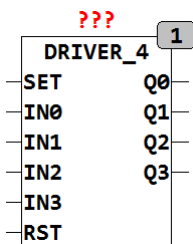
Input	SET : BOOL (asynchroner Set Eingang)
	IN : BOOL (Schalteingang)
	RST : BOOL (asynchroner Reset Eingang)
Config	TOGGLE_MODE : BOOL (Mode des Eingangs IN)
	TIMEOUT : TIME (Maximale Einschaltdauer der Ausgänge)
Output	Q0 : BOOL (Schaltausgang)



DRIVER_1 ist ein Treiberbaustein dessen Ausgang Q durch den Eingang IN wenn TOGGLE_MODE = FALSE gesetzt werden kann. Der Ausgang bleibt dann solange auf TRUE bis er entweder durch einen asynchronen Reset (RST) auf FALSE gesetzt wird oder bis die maximale Schaltzeit (TIMEOUT) abgelaufen ist. Weitere Impulse am Eingang IN verlängern dabei die TRUE Phase am Ausgang indem mit jeder steigenden Flanke an IN der Timeout von Neuem beginnt. Wenn TOGGLE_MODE = TRUE, dann schaltet der Ausgang Q mit jeder steigenden Flanke an IN den Zustand zwischen TRUE und FALSE. Auch im TOGGLE_MODE wird mit TIMEOUT die maximale TRUE Phase am Ausgang Q begrenzt. Wird TIMEOUT auf T#0s gesetzt (Default) dann ist kein Timeout aktiv. Die asynchronen SET und RST Eingänge setzen den Ausgang Q auf TRUE oder FALSE. Der Baustein DRIVER_4 stellt bei gleicher Funktionalität 4 Schaltausgänge zur Verfügung.

27.2 DRIVER_4

Type	Funktionsblock
Input	SET : BOOL (asynchroner Set Eingang) IN0..IN3 : BOOL (Schalteingänge) RST : BOOL (asynchroner Reset Eingang)
Config	TOGGLE_MODE : BOOL (Mode des Eingangs IN) TIMEOUT : TIME (Maximale Ontime der Ausgänge)
Output	Q0 .. Q3 : BOOL (Schaltausgänge)

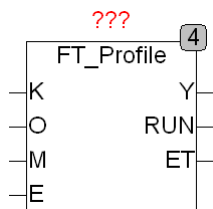


DRIVER_1 ist ein Treiberbaustein dessen Ausgänge Q durch die Eingänge IN geschaltet werden. eine detaillierte Beschreibung des Bausteins befindet sich unter DRIVER_1. DRIVER_4 stellt im Gegensatz zu DRIVER_1 4 Schaltausgänge zur Verfügung, hat aber ansonsten die gleiche Funktionalität.

27.3 FT_PROFILE

Type	Funktionsblock
Input	K : REAL (Multiplikator) O : REAL (Offset) M : REAL (Zeitmultiplikator) E : BOOL (Startsignal)
Output	Y : REAL (Signalausgang) RUN : BOOL (TRUE, wenn Ausgangssignal erzeugt wird) ET : TIME (Zeit seit Start des Ausgangsprofils)
Config	VALUE_0 : REAL (Ausgangswert des Ausgangs zum Startzeitpunkt) TIME_1 : TIME (Zeitpunkt wenn die Rampe VALUE_1 erreicht)

VALUE_1 : REAL (Wert den die Rampe zum Zeitpunkt TIME_1 erreicht)
 TIME_2 : TIME (Zeitpunkt wenn die Rampe VALUE_2 erreicht)
 VALUE_2 : REAL (Wert den die Rampe zum Zeitpunkt TIME_2 erreicht)
 TIME_3 : TIME (Zeitpunkt wenn die Rampe VALUE_3 erreicht)
 VALUE_3 : REAL (Wert den die Rampe zum Zeitpunkt TIME_3 erreicht)
 TIME_10 : TIME (Zeitpunkt wenn die Rampe VALUE_10 erreicht)
 VALUE_10 : REAL (Wert den die Rampe zum Zeitpunkt TIME_10 erreicht)
 TIME_11 : TIME (Zeitpunkt wenn die Rampe VALUE_11 erreicht)
 VALUE_11 : REAL (Wert den die Rampe zum Zeitpunkt TIME_11 erreicht)
 TIME_12 : TIME (Zeitpunkt wenn die Rampe VALUE_12 erreicht)
 VALUE_12 : REAL (Wert den die Rampe zum Zeitpunkt TIME_12 erreicht)
 TIME_13 : TIME (Zeitpunkt wenn die Rampe VALUE_13 erreicht)
 VALUE_13 : REAL (Wert den die Rampe zum Zeitpunkt TIME_13 erreicht)



FT_PROFILE erzeugt ein zeitabhängiges Ausgangssignal. Das Ausgangssignal wird definiert durch Zeit – Wertepaare. FT_PROFILE erzeugt ein Ausgangssignal Y, indem die Wertepaare durch Rampen verbunden werden. Ein typische Anwendung für FT_PROFILE ist die Erzeugung eines Temperaturprofils für einen Glühofen, aber auch jede Anwendung die ein zeitabhängiges Steuersignal benötigt stellt ein Anwendungsgebiet dar. Das zeitabhängige Ausgangssignal wird durch eine steigende Flanke an E gestartet und läuft dann selbsttätig ab. Nach dem Wertepaar (TIME_10, VALUE_10) verharrt das Ausgangssignal auf VALUE_10, bis der Eingang E FALSE wird. Mit einer Flanke an E kann also das Signal gestartet werden und zusätzlich kann der Eingang E auch benutzt werden um das Signal beliebig lange zu strecken. Dadurch ist es möglich einen Verlauf bis zum Wert VALUE_3 zu erzeugen, mit E zu strecken und nach der fallenden Flanke von E wieder einen Verlauf zurück zum Ausgangswert zu erzeugen. Mit den Eingängen K, O und M kann das Ausgangssignal dynamisch gestreckt und skaliert werden.

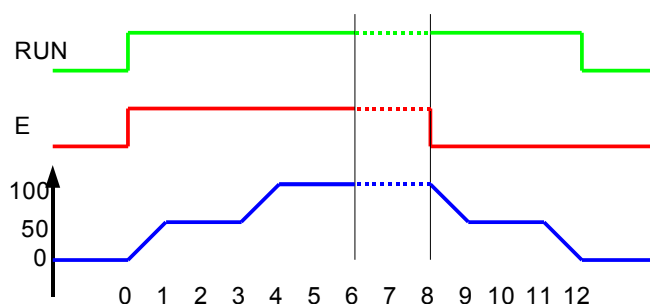
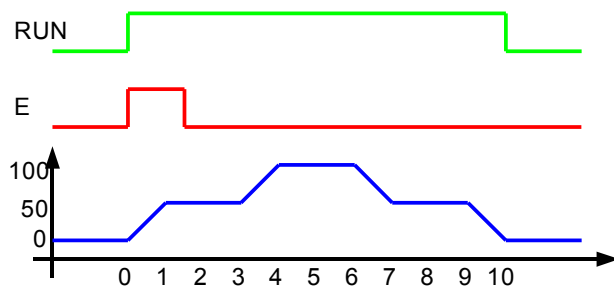
$$Y = \text{erzeugter Wert} * K + O$$

Der Eingang M dient zum Strecken des Signals über die Zeit. Der tatsächliche Zeitverlauf entspricht dem über Setup definierten Zeitverlauf multipliziert mit M. Um geradlinige Rampen zu gewährleisten wirkt eine Zeitstreckung durch M nur nach Abschluss einer Flanke. Der Ausgang RUN wird mit einer steigenden Flanke von E auf TRUE gesetzt und wird erst nach Abschluss des Zeitprofils wieder FALSE. Am Ausgang ET kann die seit Start verstrichene Zeit abgelesen werden.

Die folgenden Graphiken zeigen das Ausgangssignal für die Wertepaare:

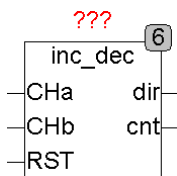
VALUE_0 = 0
 TIME_1, VALUE_1 = 1s, 50
 TIME_2, VALUE_2 = 3s, 50
 TIME_3, VALUE_3 = 4s, 100
 TIME_10, VALUE_10 = 6s, 100
 TIME_11, VALUE_11 = 7s, 50
 TIME_12, VALUE_12 = 9s, 50
 TIME_13, VALUE_13 = 10s, 0

Die Schaubilder stellen das Ausgangssignal sowohl mit gestreckter Phase 3 durch E dar, als auch ohne Streckung.



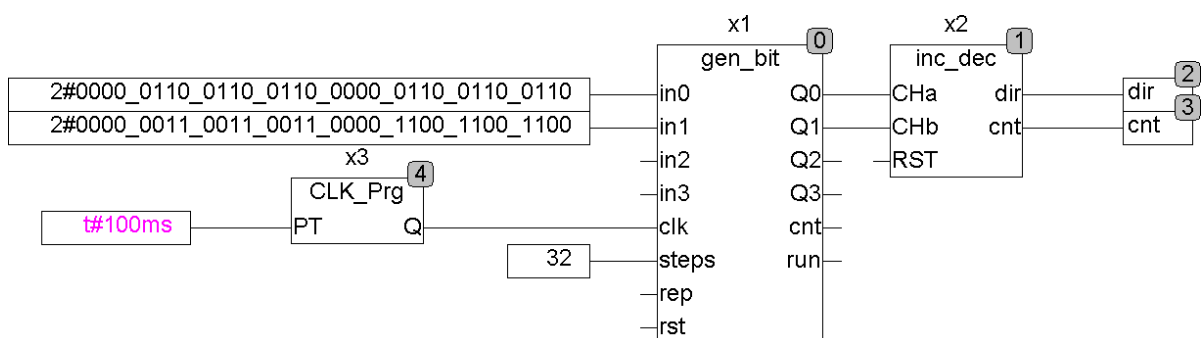
27.4 INC_DEC

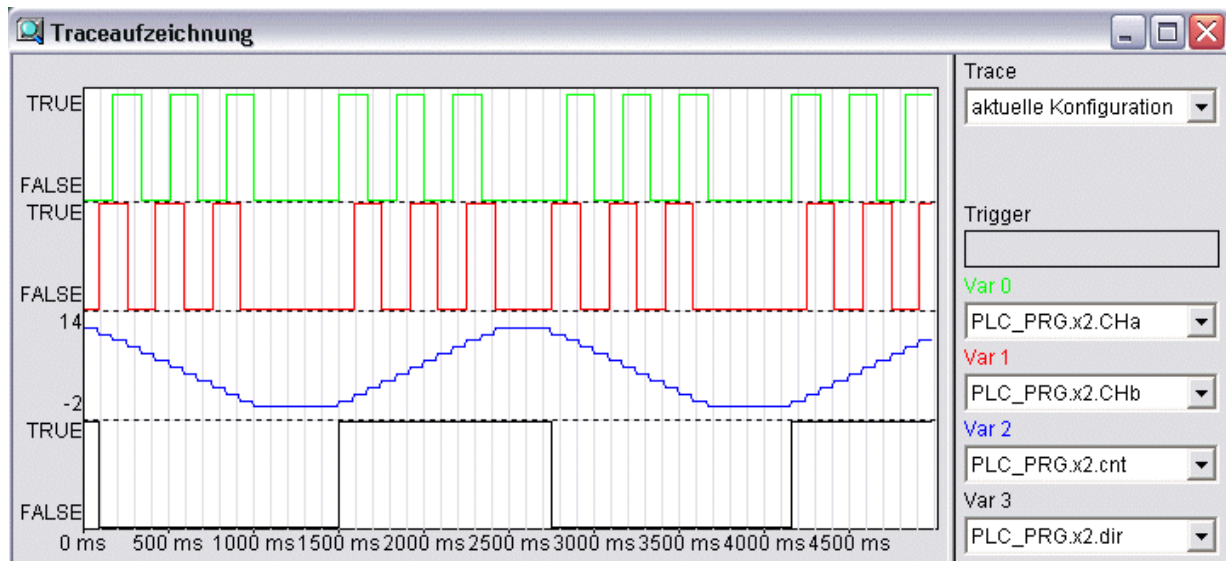
Type	Funktionsbaustein
Input	CHA : BOOL (Kanal A des Gebers) CHB : BOOL (Kanal B des Gebers) RST : BOOL (Reset)
Output	DIR : BOOL (Drehrichtung) CNT : INT (Zählerwert)



INC_DEC ist ein Decoder für Inkrementalgeber. Inkrementalgeber (Drehgeber) liefern 2 überlappende Impulse, Kanal A und Kanal B. Aus den beiden Kanälen wird die Drehrichtung und der Drehwinkel dekodiert. INC_DEC detektiert jede einzelne Flanke des Drehgebers, sodass 4-fache Auflösung erreicht wird. Der Ausgang DIR zeigt die Drehrichtung an, und am Ausgang CNT steht ein Integer-Wert bereit, der die Anzahl der gezählten Impulse ausgibt. Für eine volle Umdrehung eines Drehgebers mit 100 Impulsen zählt CNT bis 400, weil an jeder Flanke beider Kanäle gezählt wird, sodass 4-fache Auflösung erreicht wird. Ein RST Eingang erlaubt jederzeit den Zähler auf 0 zu stellen. Der Zähler zählt aufwärts, wenn DIR = TRUE und abwärts, wenn DIR = FALSE.

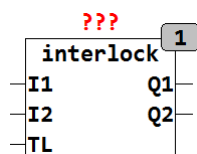
Im Folgenden Beispiel wird ein Bitmustergenerator GEN_BIT benutzt um einen Drehgeber zu simulieren, der immer abwechselnd 3 Schritte im Uhrzeigersinn und 3 im Gegenuhrzeigersinn macht. In der Traceaufzeichnung ist zu sehen, wie der INC_DEC die Bewegung in 12 Zahlschritte zerlegt und die Richtung dekodiert.





27.5 INTERLOCK

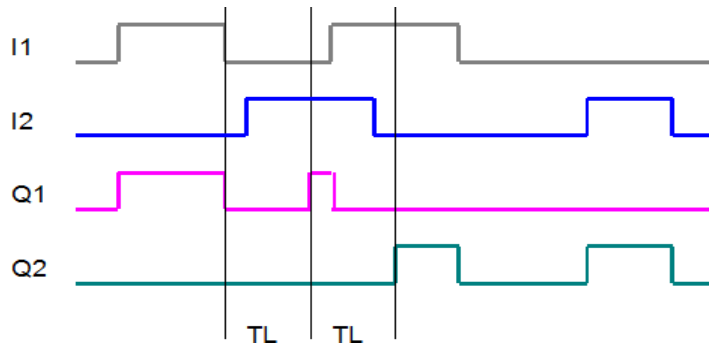
Type Funktionsblock
 Input I1 : BOOL (Eingang 1)
 I2 : BOOL (Eingang 2)
 TL : TIME (Totzeit)
 Output Q1 : BOOL (Ausgang 1)
 Q2 : BOOL (Ausgang 2)



Der Baustein INTERLOCK hat 2 Eingänge I1 und I2 die jeweils die Ausgänge Q1 und Q2 schalten. Q1 und Q2 sind jedoch gegenseitig verriegelt so dass immer nur ein Ausgang auf TRUE stehen kann. Die Zeit TL legt eine Totzeit zwischen den Beiden Ausgängen fest. Ein Ausgang kann nur dann TRUE werden wenn der andere Ausgang für mindestens für die Zeit TL FALSE war.

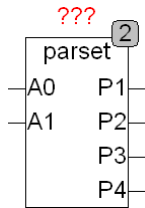
I1	I2	Q1	Q2
0	0	0	0

0	1	0	1
1	0	1	0
1	1	0	0



27.6 PARSET

Type	Funktionsbaustein
Input	A0 : BOOL (Auswahl Eingang 0) A1 : BOOL (Auswahl Eingang 1)
Config	X01, X11, X21, X31 : REAL (Werte für Parameter P1) X02, X12, X22, X32 : REAL (Werte für Parameter P2) X03, X13, X23, X33 : REAL (Werte für Parameter P3) X04, X14, X24, X34 : REAL (Werte für Parameter P4) TC : TIME (Rampenzeit zu einem neuen Wert an den Ausgängen P)
Output	P1 : REAL (Parameter 1 Ausgang) P2 : REAL (Parameter 2 Ausgang) P3 : REAL (Parameter 3 Ausgang) P4 : REAL (Parameter 4 Ausgang)

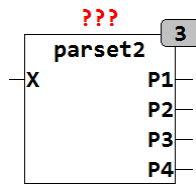


PARSET wählt aus bis zu 4 Parametersätzen je einen aus und liefert die Werte an den Ausgängen P1 bis P4. Die Werte für die Parametersätze werden mit Setup Variablen definiert. Wird die Setup Variable TC auf einen Wert > 0 gesetzt, so schalten die Ausgänge nicht sprunghaft auf einen neuen Wert, sondern laufen in einer Rampe auf den neuen Wert zu so dass der Endwert nach der Zeit TC erreicht wird. Dies erlaubt den weichen Übergang zwischen verschiedenen Parametersätzen. Die Auswahl der Parameter wird durch die Eingänge A0 und A1 gesteuert.

A1,A0	P1	P2	P3	P4
00	X01	X02	X03	X04
01	X11	X12	X13	X14
10	X21	X22	X23	X24
11	X31	X32	X33	X34

27.7 PARSET2

Type	Funktionsbaustein
Input	X : REAL (Signaleingang)
Config	X01, X11, X21, X31 : REAL (Werte für Parameter P1) X02, X12, X22, X32 : REAL (Werte für Parameter P2) X03, X13, X23, X33 : REAL (Werte für Parameter P3) X04, X14, X24, X34 : REAL (Werte für Parameter P4) L1, L2, L3 : REAL (Limits für die Parameterumschaltung) TC : TIME (Rampenzeit zu einem neuen Wert an den Ausgängen P)
Output	P1 : REAL (Parameter 1 Ausgang) P2 : REAL (Parameter 2 Ausgang) P3 : REAL (Parameter 3 Ausgang) P4 : REAL (Parameter 4 Ausgang)

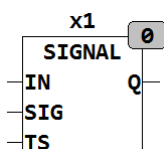


PARSET2 wählt aus bis zu 4 Parametersätzen je einen aus und liefert die Werte an den Ausgängen P1 bis P4. Die Werte für die Parametersätze werden mit Setup Variablen definiert. Wird die Setup Variable TC auf einen Wert > 0 gesetzt, so schalten die Ausgänge nicht sprunghaft auf einen neuen Wert, sondern laufen in einer Rampe auf den neuen Wert zu so dass der Endwert nach der Zeit TC erreicht wird. Dies erlaubt den weichen Übergang zwischen verschiedenen Parametersätzen. Die Auswahl der Parameter wird durch die Steuergröße X und die Schaltschwellen L1 bis L3 festgelegt.

X	P1	P2	P3	P4
$X < L1$	X01	X02	X03	X04
$L1 < X < L2$	X11	X12	X13	X14
$L2 < X < L3$	X21	X22	X23	X24
$X \geq L3$	X31	X32	X33	X34

27.8 SIGNAL

Type Funktionsbaustein
 Input IN : BOOL (Freigabeeingang)
 SIG : BYTE (Bitpattern)
 TS : TIME (Schaltzeit)
 Output Q : BOOL (Ausgangssignal)

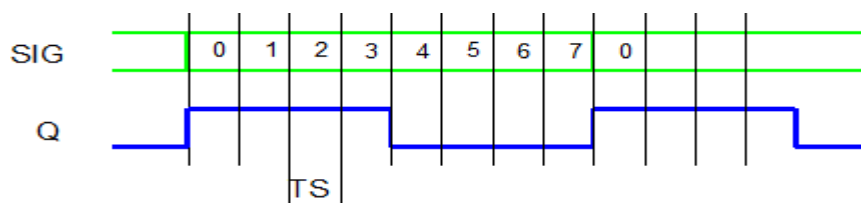


SIGNAL erzeugt ein Ausgangssignal Q das dem Bitpattern in SIG entspricht. Das Bitpattern wird in TS langen Schritten ausgegeben. Durch verschiedene Bitmuster in SIG können verschiedene Ausgangssignale erzeugt werden. Wird der Eingang IN

auf TRUE geschaltet beginnt der Baustein den Ausgang Q entsprechend dem in SIG bereitgestellten Bitpattern einzuschalten. Durch die Anpassung des Bitpattern können verschiedene Ausgangssignale erzeugt werden. Ein Pattern von 10101010 erzeugt ein Ausgangssignal mit 50% Duty Cycle und einer Frequenz die $1 / 2 \cdot S$ entspricht. Ein Pattern von 11110000 erzeugt hingegen ein Ausgangssignal von 50% DC und einer Frequenz von $1 / 8 \cdot TS$. Der Start eines Ausgangssignals ist zufällig. Die Bitsequenz beginnt ab einem beliebigen Bit sobald der Eingang IN auf TRUE geht. Wird am Eingang TS keine Zeit vorgegeben so verwendet der Baustein intern eine Vorgabe von 1024ms je Zyklus (ein Zyklus ist der Durchlauf aller 8 Bits einer Sequenz). Typische Anwendungen für SIGNAL ist die Signalerzeugung für Sirenen oder Signallampen.

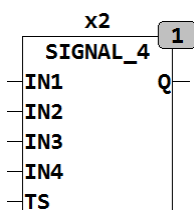
Die folgende Grafik verdeutlicht die Funktionsweise von SIGNAL für

SIG = 2#1111_0000:



27.9 SIGNAL_4

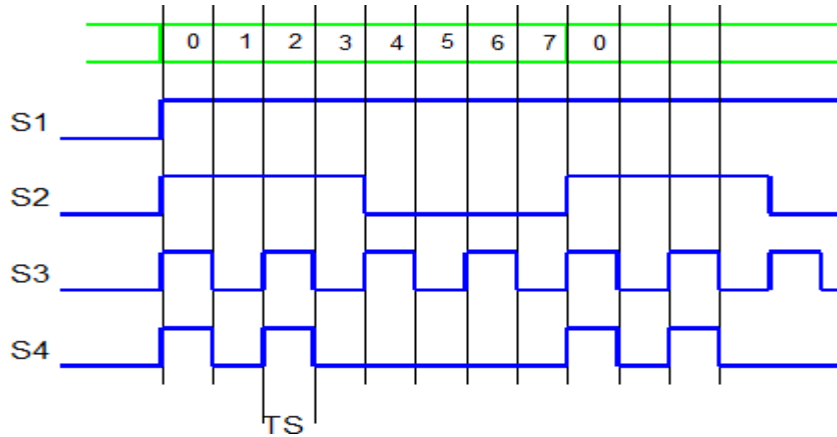
Type	Funktionsbaustein
Input	IN1..IN4 : BOOL (Eingang für Bitpattern S1..S4) TS : TIME (Schaltzeit)
Config	S1 .. S4 : BYTE (Bitpattern S1 .. S4)
Output	Q : BOOL (Ausgangssignal)



SIGNAL_4 erzeugt ein Ausgangssignal Q das einem von 4 Bitpattern (S1 .. S4) entspricht. Das Bitpattern wird in TS langen Schritten ausgegeben. Die Eingänge IN1..IN4 sind priorisierte Eingänge. Ein TRUE an IN1 überschreibt allen anderen Eingänge, IN2 überschreibt IN3 und IN4 hat die niedrigste Priorität. Eine weiterführende Beschreibung der Funktion von SIGNAL_4 befindet sich unter SIGNAL. Die 4 ver-

schiedenen Bitpattern sind in Config Variablen S1 .. S4 abgelegt und können vom Anwender jederzeit angepasst werden.

Der Baustein hat per folgende Bitpattern voreingestellt, welche aber vom Anwender bei Bedarf verändert werden können:



S1 = 2#1111_1111

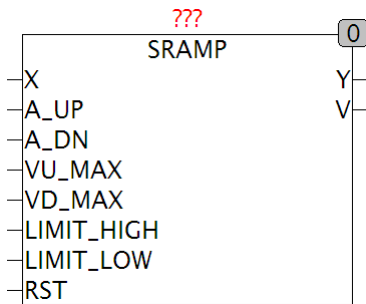
S2 = 2#1111_0000

S3 = 2#1010_1010

S4 = 2#1010_0000

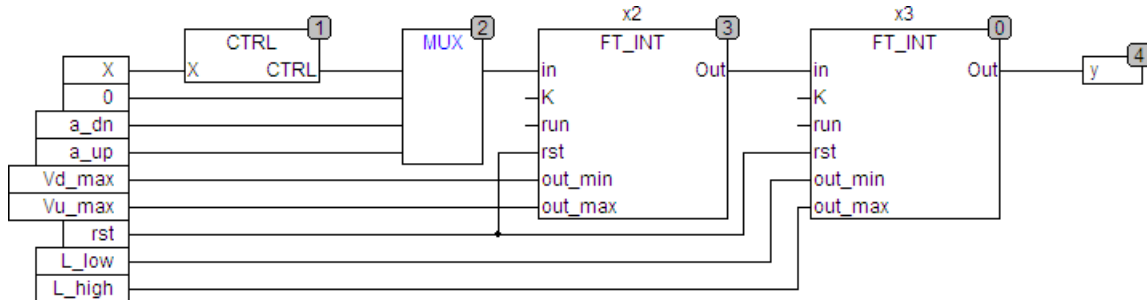
27.10 SRAMP

Type	Funktionsbaustein
Input	X: REAL (Eingangssignal) A_UP : REAL (Maximale Beschleunigung Aufwärts) A_DN : REAL (Maximale Beschleunigung Abwärts) VU_MAX : REAL (Maximale Geschwindigkeit Aufwärts) VD_MAX : REAL (Maximale Geschwindigkeit Abwärts) LIMIT_HIGH : REAL (Ausgangslimit High) LIMIT_LOW : REAL (Ausgangslimit Low) RST : BOOL (Asynchroner Reset)
Output	Y : REAL (Ausgangssignal) V : REAL (Momentane Geschwindigkeit des Ausgangssignals)

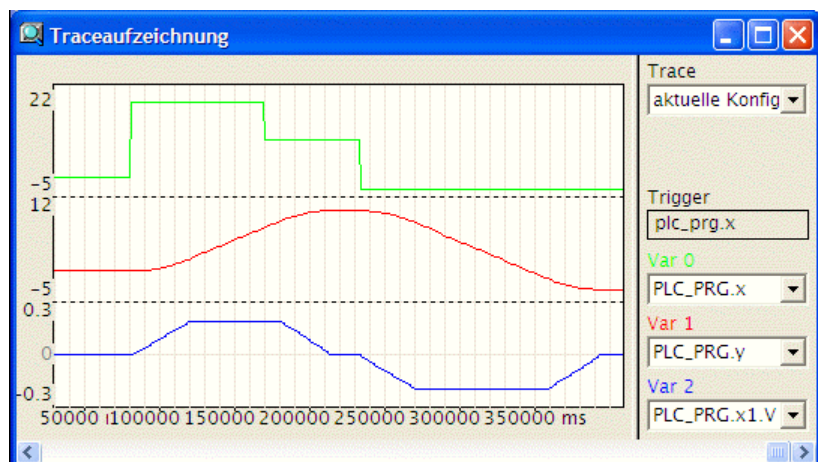


SRAMP erzeugt ein Ausgangssignal das durch einstellbare Parameter begrenzt wird. Das Ausgangssignal folgt dem Eingangssignal und wird dabei durch maximale Geschwindigkeit (VU_MAX und VD_MAX), oberer und unterer Grenzwert (LIMIT_LOW und LIMIT_HIGH), sowie eine maximale Beschleunigung (A_UP und A_DN) begrenzt. SRAMP wird verwendet um zum Beispiel Motoren anzusteuern. Der Ausgang V gibt die momentane Geschwindigkeit des Ausgangs an.

Im folgenden Schema ist die Interne Arbeitsweise von SRAMP ersichtlich. Ein Rampengenerator X2 stellt die Geschwindigkeit mit der sich der Ausgang verändern darf ein und ein Zweiter Rampengenerator X3 steuert den Ausgang.



Die Trace Aufzeichnung zeigt ein Beispiel für SRAMP. Der Eingang (grün) steigt von 0 auf 20 und gleich danach auf 10 wobei der Ausgang mit der maximal zulässigen Beschleunigung auf die maximal zulässige Geschwindigkeit ansteigt, anschließend wird verdeutlicht das der Eingang auch während des Signalverlaufs sich ändern kann. In diesem Beispiel wird rechtzeitig wieder abgebremst, so das exakt bei 10 der Ausgang zum Stillstand kommt. Nach Erreichen des Endwertes 10 schaltet der Eingang auf -3 und der Ausgang Y folgt entsprechend.

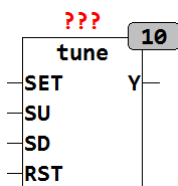


In diesem Beispiel wird rechtzeitig wieder abgebremst, so das exakt bei 10 der Ausgang zum Stillstand kommt. Nach Erreichen des Endwertes 10 schaltet der Eingang auf -3 und der Ausgang Y folgt entsprechend.

Die Eingangswerte für A_UP und VU_MAX müssen mit positivem Vorzeichen angegeben werden, A_DN und VD_MAX benötigen ein negatives Vorzeichen.

27.11 TUNE

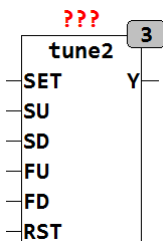
Type	Funktionsbaustein
Input	SET : BOOL (Asynchroner Set Eingang) SU, SD : BOOL (Eingänge für Auf und Ab) RST : BOOL (Asynchroner Reset Eingang)
Config	SS : REAL (Schrittweite für kleiner Schritt) Limit_L : REAL (unterer Grenzwert) Limit_H : REAL (oberer Grenzwert) RST_VAL : REAL (Ausgangswert nach Reset) SET_VAL : REAL (Ausgangswert nach SET) T1 : TIME (Zeit nach der die erste Rampe anläuft) T1 : TIME (Zeit nach der die zweite Rampe anläuft) S1 : REAL (Geschwindigkeit für erste Rampe) S2 : REAL (Geschwindigkeit für zweite Rampe)
Output	Y : REAL (Ausgangssignal)



TUNE setzt mithilfe von Auf- und Ab- Tastern ein Ausgangssignal Y. Durch entsprechende Config Variablen kann die Schrittweite individuell programmiert werden. Ein oberer und unterer Grenzwert für den Ausgang Y kann mittels LIMIT_L und LIMIT_H vorgegeben werden. mit den Tastern SU und SD werden Schritte Auf oder Ab erzeugt. Wird eine Taste länger als die Zeit T1 gedrückt gehalten, so wird der Ausgang Y kontinuierlich Auf oder Ab verstellt. Die Geschwindigkeit mit der der Ausgang verstellt wird ist hierbei mit S1 vorgegeben. S1 und S2 geben die Einheiten je Sekunde an. Wird eine Taste länger als die Zeit T2 gedrückt gehalten, so schaltet der Baustein automatisch auf eine zweite Geschwindigkeit S2 um. Mit den Eingängen RST und SET kann der Ausgang jederzeit auf einen durch RST_VAL beziehungsweise SET_VAL vorgegebenen Wert gestellt werden.

27.12 TUNE2

Type	Funktionsbaustein
Input	SET : BOOL (Asynchroner Set Eingang) SU, SD : BOOL (Eingänge für Auf und Ab in kleinen Schritten) FU, FD : BOOL (Eingänge für Auf und Ab in großen Schritten) RST : BOOL (Asynchroner Reset Eingang)
Config	SS : REAL (Schrittweite für kleine Schritte) FS : REAL (Schrittweite für große Schritte) Limit_L : REAL (unterer Grenzwert) Limit_H : REAL (oberer Grenzwert) RST_VAL : REAL (Ausgangswert nach Reset) SET_VAL : REAL (Ausgangswert nach SET) TR : TIME (Zeit nach der die Rampe anläuft) S1 : REAL (Geschwindigkeit für kleine Rampe) S2 : REAL (Geschwindigkeit für große Rampe)
Output	Y : REAL (Ausgangssignal)



TUNE2 setzt mithilfe von Auf- und Ab- Tastern ein Ausgangssignal Y. Durch entsprechende Config Variablen kann die Schrittweite für kleine und große Schritte individuell programmiert werden. Ein oberer und unterer Grenzwert für den Ausgang Y kann mittels LIMIT_L und LIMIT_H vorgegeben werden. mit den Tastern SU und SD werden kleine Schritte Auf oder Ab erzeugt. Die Taster FU und FD erzeugen jeweils große Schritte am Ausgang Y. Wird eine Taste länger als TR gedrückt gehalten, so wird der Ausgang Y kontinuierlich Auf oder Ab verstellt. Die Geschwindigkeiten mit der der Ausgang verstellt wird ist hierbei für die beiden Tastenpaare mit S1 und S2 individuell einstellbar. S1 und S2 geben die Einheiten je Sekunde an. S1 ist die Geschwindigkeit für die Taster SU und SD, und S2 entsprechend für FU und FD. Mit den Eingängen RST und SET kann der Ausgang jederzeit auf einen durch RST_VAL beziehungsweise SET_VAL vorgegebenen Wert gestellt werden.

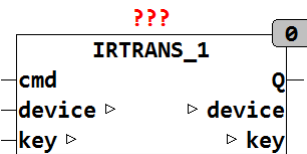
28 Geräte Treiber

28.1 IRTRANS

Die Bausteine IRTRANS_? stellen ein Interface für Infrarot Transmitter der Firma IR-Trans GmbH zur Verfügung. IRTrans bietet Transmitter für RS232 und TCP/IP an, welche alle mit den folgenden Treiberbausteinen betrieben werden können. Der grundsätzliche Anschluss an RS232 oder TCP/IP hat mit den entsprechenden Hersteller-routinen zu erfolgen. Die Interfacebausteine setzen auf ein Buffer Interface auf welches in einen Buffer (Array of Byte) die Daten und in einem Counter die Länge des Datenpakets in Bytes zur Verfügung stellt. Die IRTrans Geräte erlernen beliebige IR Tastencodes und setzen diese mittels einer konfigurierbaren Datenbank in ASCII Strings um. Mit der Ethernet Variante werden diese Strings dann über UDP versandt und können von einer SPS empfangen und ausgewertet werden. So können zum Beispiel vollautomatisch die Jalousien heruntergefahren werden wenn jemand den Fernseher einschaltet ohne das dafür eine zusätzliche Bedienung nötig wäre. Die SPS kann über Diesen Weg beliebig vielen Fernsteuerungen in unterschiedlichen Räumen zuhören und entsprechende Aktionen daraus ableiten. Umgekehrt ist natürlich auch die Aussendung von Tastencodes über die Transmitter Module möglich.

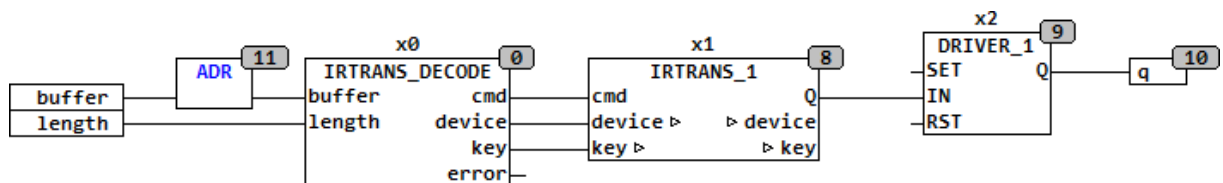
28.1.1 IRTRANS_1

Type	Funktionsblock
Input	CMD : BOOL (TRUE wenn Daten zum Auswerten Anliegen) DEVICE : STRING (Name der Fernsteuerung) KEY : STRING (NAME der Taste)
Config	DEVICE_CODE : STRING (zu dekodierender Fernsteuerungsname) KEY_CODE : STRING (zu dekodierender Tastencode)
Output	Q : BOOL (Ausgangssignal)



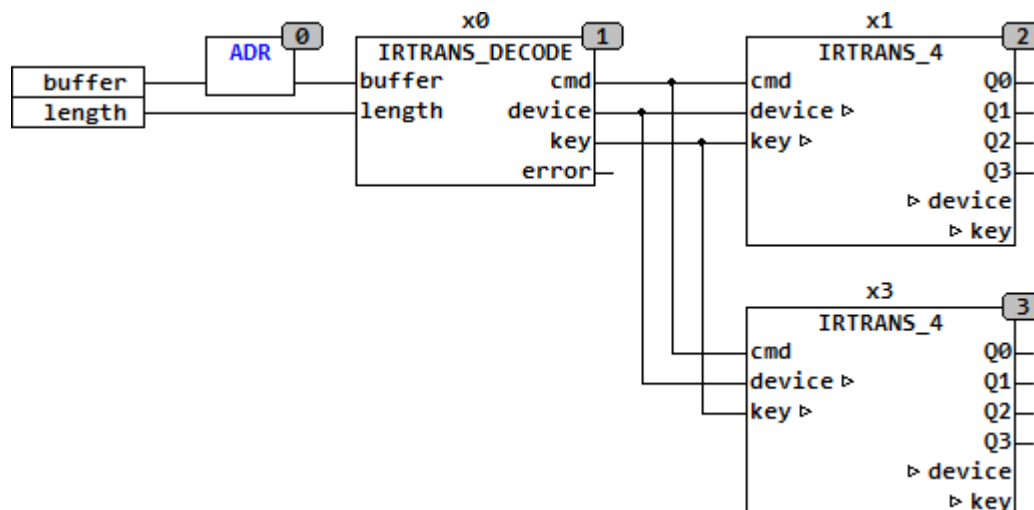
IRTRANS_1 überprüft wenn CMD = TRUE ob die Zeichenkette am Eingang DEVICE dem DEVICE_CODE entspricht und die Zeichenkette am Eingang KEY dem KEY_CODE entspricht. Wenn die Codes übereinstimmen und CMD = TRUE ist dann wird der Ausgang Q für einen Zyklus auf TRUE gesetzt.

Das folgende Beispiel zeigt die Anwendung von IRTRANS_1:



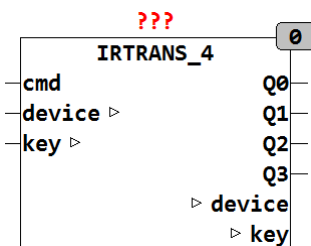
In diesem Beispiel berechnet ADR() die Adresse des Datenpuffers und gibt sie zusammen mit der Länge an IRTRANS_DECODE. Der Decoder ermittelt aus gültigen Datenpaketen den String DEVICE und KEY und reicht diese per CMD an IRTRANS_1. IRTRANS_1 oder alternativ auch IRTRANS_4 und IRTRANS_8 überprüft dann ob DEVICE und KEY übereinstimmen und schaltet dann den Ausgang Q für einen Zyklus auf TRUE. im Beispiel wird damit ein DRIVER_1 gesteuert der es der Fernsteuerung erlaubt mit jedem empfangenen Protokoll den Ausgang umzuschalten.

Wenn mehrere Key Codes ausgewertet werden sollen können alternativ die Bausteine IRTRANS_4 oder IRTRANS_8 verwendet werden oder mehrere dieser Bausteine parallel betrieben werden.



28.1.2 IRTRANS_4

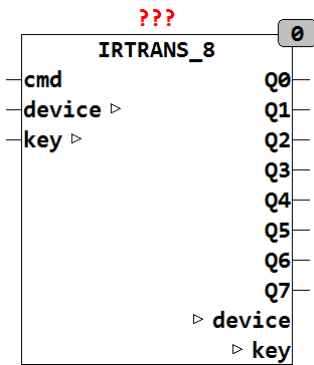
Type	Funktionsblock
Input	CMD : BOOL (TRUE wenn Daten zum Auswerten Anliegen) DEVICE : STRING (Name der Fernsteuerung) KEY : STRING (NAME der Taste)
Config	DEVICE_CODE : STRING (zu dekodierender Fernsteuerungsname) KEY_CODE_0..7 : STRING (zu dekodierender Tastencode)
Output	Q : BOOL (Ausgangssignal)



IRTRANS_4 überprüft wenn CMD = TRUE ob die Zeichenkette am Eingang DEVICE dem DEVICE_CODE entspricht und die Zeichenkette am Eingang KEY einem KEY_CODE entspricht. Wenn die Codes übereinstimmen und CMD = TRUE ist dann wird der entsprechende Ausgang Q für einen Zyklus auf TRUE gesetzt. Weitergehende Informationen zur Funktion des Bausteins befinden sich unter IRTRANS_1.

28.1.3 IRTRANS_8

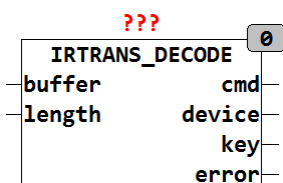
Type	Funktionsblock
Input	CMD : BOOL (TRUE wenn Daten zum Auswerten Anliegen) DEVICE : STRING (Name der Fernsteuerung) KEY : STRING (NAME der Taste)
Config	DEVICE_CODE : STRING (zu dekodierender Fernsteuerungsname) KEY_CODE_0..7 : STRING (zu dekodierender Tastencode)
Output	Q0..Q7 : BOOL (Ausgangssignal)



IRTRANS_8 überprüft wenn CMD = TRUE ob die Zeichenkette am Eingang DEVICE dem DEVICE_CODE entspricht und die Zeichenkette am Eingang KEY einem KEY_CODE entspricht. Wenn die Codes übereinstimmen und CMD = TRUE ist dann wird der entsprechende Ausgang Q für einen Zyklus auf TRUE gesetzt. Weitergehende Informationen zur Funktion des Bausteins befinden sich unter IRTRANS_1.

28.1.4 IRTRANS_DECODE

Type	Funktionsblock
Input	BUFFER : Pointer to ARRAY of BYTE (Datenpuffer) LENGTH : INT (Länge der Daten im Puffer)
Output	CMD : BOOL (TRUE wenn gültige Daten am Ausgang anliegen) DEVICE : STRING (Name der Fernsteuerung) KEY : STRING (Name des Tastencodes) ERROR : BOOL (TRUE wenn ein ungültiges Datenpaket vorliegt)



IRTRANS_DECODE empfängt die in BUFFER vorliegenden Daten mit der Länge LENGTH, überprüft ob ein gültiges Datenpaket vorliegt und dekodiert aus dem Datenpaket den Namen der Fernsteuerung und den Namen der Taste. Wenn ein gültiges Datenpaket dekodiert wurde liegt der Name der Fernsteuerung am Ausgang DEVICE und der Name der Taste am Ausgang KEY. Der Ausgang CMD signalisiert das neue Ausgangsdaten vorliegen. Der Ausgang ERROR wird dann gesetzt wenn ein Datenpaket empfangen wurde das nicht dem Format entspricht. Der Eingang BUFFER ist als Pointer ausgeführt um mit unterschiedlich großen Eingangspuffern arbei-

ten zu können. Die Adresse des BUFFERS kann mit der Funktion ADR() ermittelt werden.

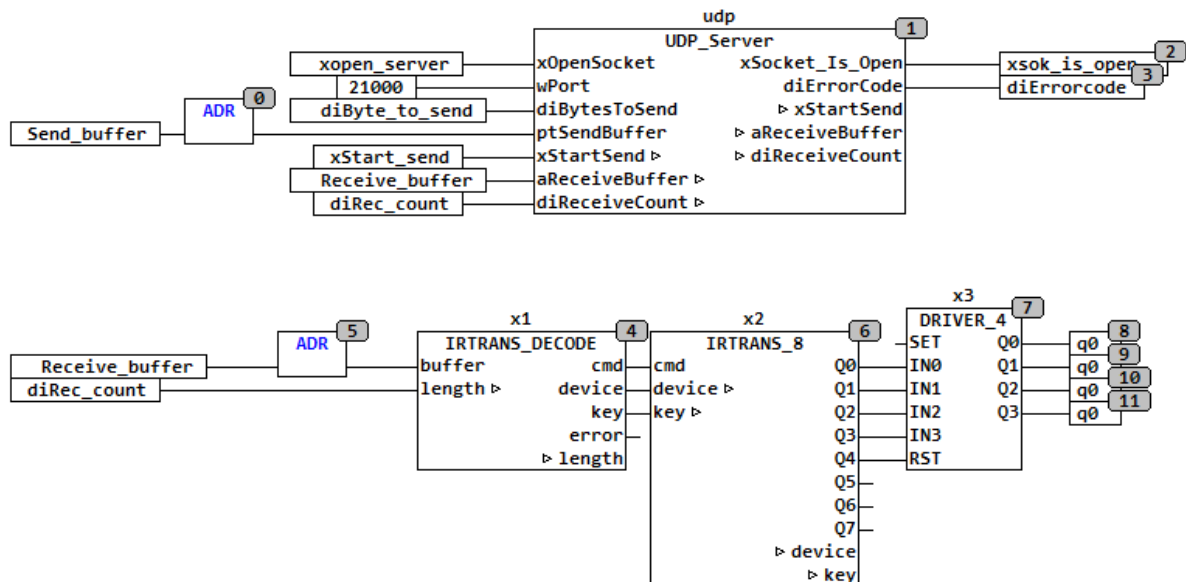
Das Format ist wie folgt definiert:

'Name der Fernsteuerung','Name des Tastencodes'\$R\$N

Ein Datenpaket besteht aus dem Namen der Fernsteuerung, gefolgt von einem Komma und anschließend der Name des Tastencodes. Das Datenpaket wird mit einem Carriage Return und einem Line Feed abgeschlossen.

Damit IRTRANS_DECODE funktioniert muss in der IRTrans Konfiguration die Check box BROADCAST IR RELAY angekreuzt sein und in der entsprechenden Device Datenbank unter DEFAULT ACTION muss der String '%r,%c\r\n' eingetragen sein. IRTRANS_DECODE wertet genau diesen String aus und dekodiert %r als Name der Fernbedienung und %c als die gedrückte Taste.

Das folgende Beispiel zeigt die Anwendung von IRTRANS_DECODE mit einem UDP Server:



Der UDP Server liefert das Empfangene Datenpaket im RECEIVE_BUFFER ab und hinterlegt in DIREC_COUNT die Länge der Empfangsdaten in Bytes. IRTRANS_DECODE wird aktiv wenn DIREC_COUNT > 0 wird und überprüft dann ob die Empfangsdaten dem Format eines IRTRANS Empfangspakets entsprechen. Sind die Daten in RECEIVE_BUFFER gültige Empfangsdaten wird der Name der Fernsteuerung in DEVICE und der Tastencode in KEY abgespeichert, DIREC_COUNT auf 0 gesetzt und der Ausgang CMD für einen Zyklus auf TRUE gesetzt. Wenn die Eingangsdaten nicht dem vorgeschriebenen Format entsprechen wird ERROR auf TRUE gesetzt und die Länge in DIREC_COUNT wird nicht auf 0 gesetzt damit gegebenenfalls andere Decoder parallel die Daten im RECEIVE_BUFFER auswerten können. In der IRTRANS Web Konfiguration muss als Broadcast Adresse die IP Adresse der SPS eingetragen werden.

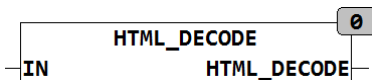
IRTRANS Web Konfiguration:

		LED D v
	☒ Broadcast IR Relay	
UDP Broadcast Target	192.168.124.1	
UDP Broadcast Port	21000	
Store Settings		

29 Netzwerk und Kommunikation

29.1 HTML_DECODE

Type Funktion : STRING(255)
 Input IN : STRING (Zeichenkette)
 Output STRING(255) (Zeichenkette)



HTML_DECODE wandelt reservierte Zeichen welche in der Form &name; im HTML Code gespeichert sind in die originalen Zeichen um. Zusätzlich werden alle codierten Sonderzeichen in den entsprechenden ASCII Code umgewandelt. Sonderzeichen können in HTML durch folgende Zeichenfolge repräsentiert sein:

- &#NN; wobei NN die Position des Zeichens innerhalb der Zeichentabelle in dezimaler Schreibweise darstellt.
- &#xNN; oder &#XNN wobei NN die Position des Zeichens innerhalb der Zeichentabelle in hexadezimaler Schreibweise darstellt.

&Name; Sonderzeichen haben Namen wie zum Beispiel € für €.

Die reservierten Zeichen in HTML sind:

- & wird codiert als &
- > wird codiert als >
- < wird codiert als <
- " wird codiert als "

Beispiele:

HTML_DECODE('1 ist > als 0') = '1 ist > als 0';

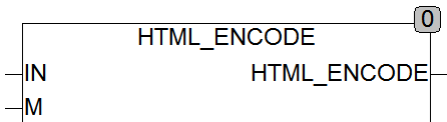
HTML_DECODE('&#D79;&#D83;&#D67;&#D65;&#D84;') = 'OSCAT';

HTML_DECODE('&#xH4F;&#xH53;&#xH43;&#xH41;&#xH54;') = 'OSCAT';

HTML_DECODE('&#XH4F;&#XH53;&#XH43;&#XH41;&#XH54;') = 'OSCAT';

29.2 HTML_ENCODE

Type Funktion : STRING(255)
 Input IN : STRING (Zeichenkette)
 M : BOOL (Mode)
 Output STRING(255) (Zeichenkette)



HTML_ENCODE wandelt in HTML reservierte Zeichen um in die Form &Name;. Wird der Eingang M auf TRUE gesetzt werden zusätzlich alle Zeichen mit dem Code 160-255 und 128 in die &Name Konvention umgesetzt.

Vorsicht ist bei der Anwendung von Zeichensätzen geboten weil diese nicht auf allen Systemen identisch sind und Abweichungen speziell bei Sonderzeichen häufig vorkommen. So ist zum Beispiel nicht bei allen Systemen das € Zeichen auf Position 128 in der Zeichentabelle.

Die reservierten Zeichen in HTML sind:

& wird codiert als &

> wird codiert als >

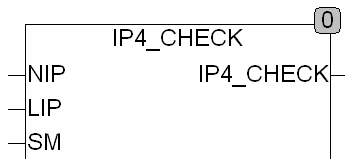
< wird codiert als <

" wird codiert als "

HTML_ENCODE wandelt die Zeichenkette '1 ist > als 0' um in '1 ist >als 0'.

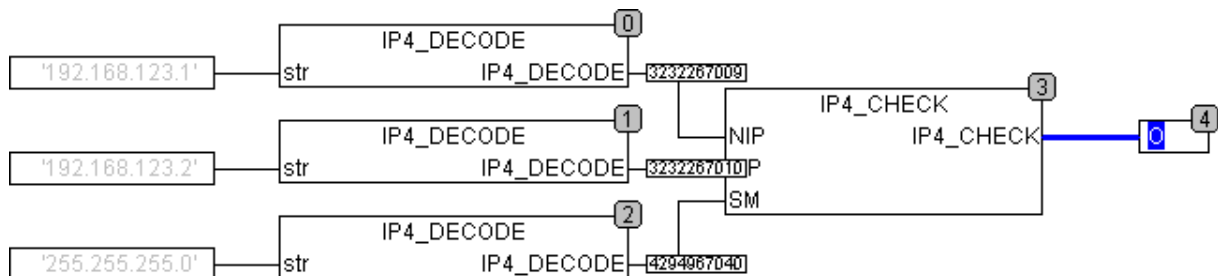
29.3 IP4_CHECK

Type Funktion : BOOL
 Input NIP : DWORD (Netzwerk IP Adresse)
 LIP : DWORD (Lokale IP Adresse)
 SM : DWORD (Subnet Maske)
 Output BOOL (TRUE wenn NIP und LIP im gleichen Subnet liegen)



IP4_CHECK prüft ob eine Netzwerkadresse NIP und die Lokale Adresse LIP im gleichen Subnet liegen. Beiden Adressen werden zuerst mit der Subnet Maske Maskiert und dann auf Gleichheit geprüft. Es werden nur die Bits auf Gleichheit geprüft die in der Subnet Maske TRUE sind. Die Netzwerk Adressen müssen dem Ipv4 Format entsprechen und als DWORD vorliegen. Sollen IP Adressen die als String vorliegen geprüft werden müssen diese vorher in DWORD gewandelt werden.

Folgendes Beispiel zeigt 2 IP Adressen und eine Subnet Maske die als String vorliegen und nach entsprechender Umwandlung in DWORD geprüft werden. Der Ausgang wird TRUE weil beide Adressen im gleichen Subnet liegen.



29.4 IP4_DECODE

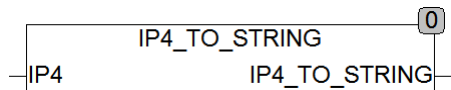
Type Funktion : DWORD
 Input STR : BOOL (Zeichenkette die die IP Adresse enthält)
 Output DWORD (dekodierte IP v4 Adresse)



IP4_DECODE dekodiert die in STR enthaltene Zeichenkette als IP v4 Adresse und gibt diese als DWORD zurück. Eine Rückgabe von 0 bedeutet das eine ungültige Adresse oder die Adresse '0.0.0.0' ausgewertet wurde. IP4 kann auch zum Auswerten einer Subnet Maske des IP v4 Formates verwendet werden.

29.5 IP4_TO_STRING

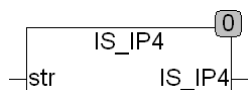
Type Funktion : STRING(15)
 Input STR : BOOL (Zeichenkette die die IP Adresse enthält)
 Output DWORD (dekodierte IP v4 Adresse)



IP4_TO_STRING wandelt die als DWORD gespeicherte IP4 Adresse in IN in einen STRING um. Das Format entspricht 'NNN.NNN.NNN.NNN'.

29.6 IS_IP4

Type Funktion : BOOL
 Input STR : STRING (zu prüfende Zeichenkette)
 Output BOOL (TRUE wenn STR eine gültige IP v4 Adresse enthält)



IS_IP4 prüft ob die Zeichenkette STR eine gültige IP v4 Adresse enthält, wenn nicht wird FALSE zurückgegeben. Ein gültige IP v4 Adresse besteht aus 4 Zahlen von 0 – 255 die mit je einem Punkt getrennt sind. Die Adresse 0.0.0.0 wird dabei als falsch eingestuft.

IS_IP4(0.0.0.0) = FALSE

IS_IP4(255.255.255.255) = TRUE

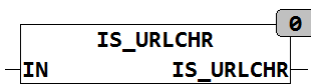
IS_IP4(256.255.255.255) = FALSE

IS_IP4(0.1.2.) = FALSE

IS_IP4(0.1.2.3.) = FALSE

29.7 IS_URLCHR

Type Funktion : BOOL
 Input STR : STRING (zu prüfende Zeichenkette)
 Output BOOL (TRUE wenn STR eine gültige IP v4 Adresse enthält)



IS_URLCHR prüft ob die Zeichenkette nur zulässige Zeichen für eine URL Kodierung enthält. Enthält die Zeichenkette reservierte Zeichen gibt Sie FALSE zurück, andernfalls TRUE.

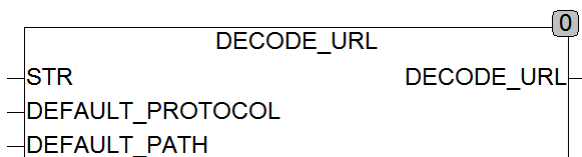
Für eine URL sind folgende Zeichen zulässig:

[A..Z]
 [a..z]
 [0..9]
 [-._~]

alle anderen Zeichen sind reserviert oder nicht zulässig.

29.8 STRING_TO_URL

Type Funktion : URL
 Input STR : STRING (Unified Resource Locator)
 DEFAULT_PROTOCOL : STRING (Ersatzprotokoll)
 DEFAULT_PATH : STRING (Ersatzpfad)
 Output URL (URL)



DECODE_URL zerlegt eine URL (Uniform Resource Locator) in seine Bestandteile und speichert diese in dem Datentyp URL. Wird in STR kein Pfad oder kein Protokoll spezifiziert, so setzt die Funktion die fehlenden Werte Automatisch mit den spezifizierten Ersatzwerten.

Eine URL setzt sich wie folgt zusammen:

protokoll://user:password@domain:port/path?query#anker

Beispiel: <ftp://hugo:nono@oscat.de:1234/download/manual.html>

einige Bestandteile der URL sind optional wie zum Beispiel User, Passwort, Anker, Query ...

29.9 URL_DECODE

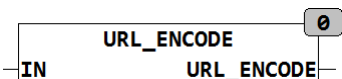
Type Funktion : STRING(255)
 Input IN : STRING (Zeichenkette)
 Output STRING(255) (Zeichenkette)



URL_DECODE wandelt die als %HH codierten Sonderzeichen in der Zeichenkette IN in den entsprechenden ASCII Code um. In einer URL Kodierung dürfen nur die Zeichen [A..Z, a..z, 0..9, -._~] vorkommen. Anderen Zeichen können mit einem % Zeichen gefolgt vom 2 Zeichen langen Hexadezimalcode des Zeichens dargestellt werden. Das reservierte Zeichen '#' wird dabei als '%23' kodiert.

29.10 URL_ENCODE

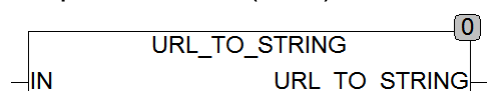
Type Funktion : STRING(255)
 Input IN : STRING (Zeichenkette)
 Output STRING(255) (Zeichenkette)



URL_ENCODE wandelt reservierte Zeichen in der Zeichenkette IN in die Zeichenkette '%HH' um. In einer URL Kodierung dürfen nur die Zeichen [A..Z, a..z, 0..9, -._~] vorkommen. Anderen Zeichen können mit einem % Zeichen gefolgt vom 2 Zeichen langen Hexadezimalcode des Zeichens dargestellt werden. Das reservierte Zeichen '#' wird dabei als '%23' kodiert.

29.11 URL_TO_STRING

Type	Funktion : STRING(255)
Input	STR : STRING (Unified Resource Locator) DEFAULT_PROTOCOL : STRING (Ersatzprotokoll) DEFAULT_PATH : STRING (Ersatzpfad)
Output	URL (URL)



URL_TO_STRING erzeugt aus den in IN gespeicherten Daten vom Typ URL einen Unified Resource Locator als String zusammen.

Eine URL setzt sich wie folgt zusammen:

protokoll://user:password@domain:port/path?query#anker

Beispiel: <ftp://hugo:nono@oscat.de:1234/download/manual.html>

einige Bestandteile der URL sind optional wie zum Beispiel User, Passwort, Anker, Query ...

30 BUFFER Management

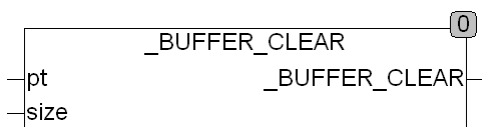
30.1 _BUFFER_CLEAR

Type Funktion : BOOL

Input PT : POINTER TO BYTE (Adresse des Puffers)

 SIZE : UINT (Größe des Puffers)

Output BOOL (Returns TRUE)



Die Funktion `_BUFFER_CLEAR` initialisiert ein beliebiges Array of Byte mit 0. Beim Aufruf wird der Funktion ein Pointer auf das zu initialisierende Array und dessen Größe in Bytes übergeben. Unter CoDeSys lautet der Aufruf: `_BUFFER_CLEAR(ADR(Array), SIZEOF(Array))`, wobei Array der Name des zu manipulierenden Arrays ist. ADR ist eine Standardfunktion, die den Pointer auf das Array ermittelt und SIZEOF ist eine Standardfunktion, die die Größe des Arrays ermittelt. Die Funktion liefert nur TRUE zurück. Das durch den Pointer angegebene Array wird direkt im Speicher manipuliert.

Diese Art der Bearbeitung von Arrays ist äußerst effizient, da kein zusätzlicher Speicher benötigt wird und keine Übergabewerte kopiert werden müssen.

Beispiel: `_BUFFER_CLEAR(ADR(bigarray), SIZEOF(bigarray))`
 initialisiert bigarray mit 0.

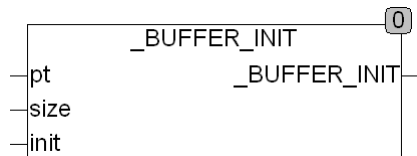
30.2 **_BUFFER_INIT**

Type Funktion : BOOL

Input PT : POINTER TO BYTE (Adresse des Puffers)

 SIZE : UINT (Größe des Puffers)

Output BOOL (Returns TRUE)



Die Funktion `_BUFFER_INIT` initialisiert ein beliebiges Array of Byte mit dem Wert `INIT`. Beim Aufruf wird der Funktion ein Pointer auf das zu initialisierende Array und dessen Größe in Bytes übergeben. Unter CoDeSys lautet der Aufruf: `_BUFFER_INIT(ADR(Array), SIZEOF(Array), INIT)`, wobei Array der Name des zu manipulierenden Arrays ist. `ADR` ist eine Standardfunktion, die den Pointer auf das Array ermittelt und `SIZEOF` ist eine Standardfunktion, die die Größe des Arrays ermittelt. Die Funktion liefert nur `TRUE` zurück. Das durch den Pointer angegebene Array wird direkt im Speicher manipuliert.

Diese Art der Bearbeitung von Arrays ist äußerst effizient, da kein zusätzlicher Speicher benötigt wird und keine Übergabewerte kopiert werden müssen.

Beispiel: `_BUFFER_INIT(ADR(bigarray), SIZEOF(bigarray),3)`
 initialisiert bigarray mit 3.

30.3 **_BUFFER_INSERT**

Type Funktion : BOOL

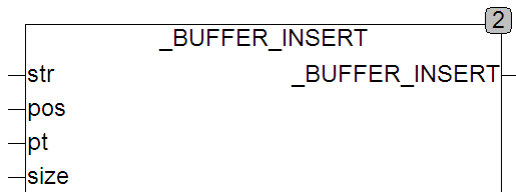
Input STR : STRING (zu kopierender String)

 POS : INT (Position ab der der String in den Puffer kopiert wird)

 PT : POINTER TO BYTE (Adresse des Puffers)

 SIZE : UINT (Größe des Puffers)

Output BOOL (Returns TRUE)



Die Funktion `_BUFFER_INSERT` kopiert einen String in ein beliebiges Array of Byte und Verschiebt den Rest des Array um die Länge des Strings. Der String wird ab einer beliebigen Position POS im Puffer abgelegt. Das erste Element im Array hat die Positionsnummer 0. Beim Aufruf wird der Funktion ein Pointer auf das zu bearbeitende Array und dessen Größe in Bytes übergeben. Unter CoDeSys lautet der Aufruf: `_BUFFER_INSERT(STR, POS, ADR(Array), SIZEOF(Array))`, wobei Array der Name des zu manipulierenden Arrays ist. ADR ist eine Standardfunktion, die den Pointer auf das Array ermittelt und SIZEOF ist eine Standardfunktion, die die Größe des Arrays ermittelt. Die Funktion liefert nur TRUE zurück. Das durch den Pointer angegebene Array wird direkt im Speicher manipuliert.

Diese Art der Bearbeitung von Arrays ist äußerst effizient, da kein zusätzlicher Speicher benötigt wird und keine Übergabewerte kopiert werden müssen.

Beispiel: `_BUFFER_INSERT(STR, POS, ADR(bigarray), SIZEOF(bigarray))`

30.4 STRING_TO_BUFFER

Type Funktion : BOOL

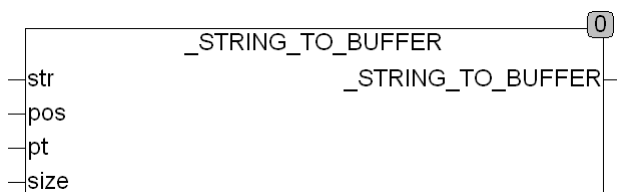
Input STR : STRING (zu kopierender String)

 POS : INT (Position ab der der String in den Puffer kopiert wird)

 PT : POINTER TO BYTE (Adresse des Puffers)

 SIZE : UINT (Größe des Puffers)

Output BOOL (Returns TRUE)



Die Funktion `_STRING_TO_BUFFER` kopiert einen String in ein beliebiges Array of Byte. Der String wird ab einer beliebigen Position POS im Puffer abgelegt. Das erste Element im Array hat die Positionsnummer 0. Beim Aufruf wird der Funktion ein Pointer auf das zu bearbeitende Array und dessen Größe in Bytes übergeben. Unter CoDeSys lautet der Aufruf: `_STRING_TO_BUFFER(STR, POS, ADR(Array), SI-`

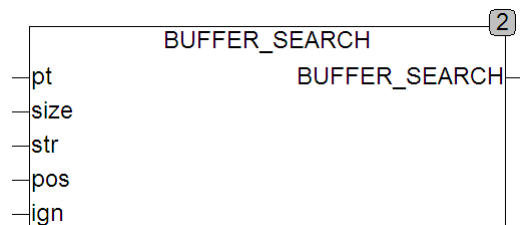
ZEOF(Array)), wobei Array der Name des zu manipulierenden Arrays ist. ADR ist eine Standardfunktion, die den Pointer auf das Array ermittelt und SIZEOF ist eine Standardfunktion, die die Größe des Arrays ermittelt. Die Funktion liefert nur TRUE zurück. Das durch den Pointer angegebene Array wird direkt im Speicher manipuliert.

Diese Art der Bearbeitung von Arrays ist äußerst effizient, da kein zusätzlicher Speicher benötigt wird und keine Übergabewerte kopiert werden müssen.

Beispiel: `_STRING_TO_BUFFER(STR, POS, ADR(bigarray), SIZEOF(bigarray))`

30.5 BUFFER_SEARCH

Type	Funktion : INT
Input	PT : POINTER (Adresse des Puffers) SIZE : UINT (Größe des Puffers) STR : STRING (Suchstring) POS : INT (Position ab der gesucht wird) IGN : BYTE (Dieses Zeichen wird beim Suchen ignoriert)
Output	INT (Position an der die Zeichenkette gefunden wurde)



Die Funktion BUFFER_SEARCH durchsucht ein beliebiges Array of Byte auf den Inhalt einer Zeichenkette und meldet die Position des ersten Zeichens der Zeichenkette im Array wenn eine Übereinstimmung gefunden wird. Der Puffer wird ab einer beliebigen Position POS durchsucht. Das erste Element im Array hat die Positionsnummer 0. Beim Aufruf wird der Funktion ein Pointer auf das zu bearbeitende Array und dessen Größe in Bytes übergeben. Unter CoDeSys lautet der Aufruf: `BUFFER_SEARCH(ADR(Array), SIZEOF(ARRAY), STR, POS, IGN)`, wobei ARRAY der Name des Arrays ist. ADR ist eine Standardfunktion, die den Pointer auf das Array ermittelt und SIZEOF ist eine Standardfunktion, die die Größe des Arrays ermittelt. Die Funktion liefert die aus dem Puffer kopierte Zeichenkette als STRING zurück. Diese Art der Bearbeitung von Arrays ist äußerst effizient, da kein zusätzlicher Speicher benötigt wird und keine Übergabewerte kopiert werden müssen. Die Funktion BUFFER_SEARCH unterstützt Wildcard Zeichen (IGN) enthält die zu suchende

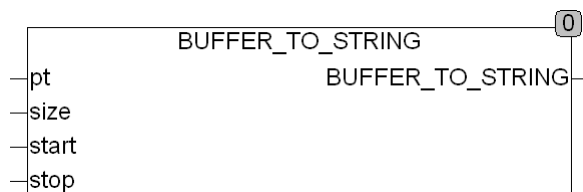
Zeichenkette STR ein Zeichen mit dem Code IGN so wird an dieser Stelle der Zeichenkette jedes beliebige Zeichen im Puffer akzeptiert.

Beispiel: BUFFER_SEARCH(ADR(Array), SIZEOF(ARRAY), 'FIND?0', 0, 63)

Findet FIND10, FIND20, FIND30 im Array, aber findet keine Übereinstimmung bei FIND11, FIND 22 Die zu suchende Zeichenkette kann auch mehrere Wildcards enthalten. Ein Wildcard Zeichen entspricht immer exakt einem Zeichen im Puffer.

30.6 BUFFER_TO_STRING

Type	Funktion : STRING
Input	PT : POINTER TO BYTE (Adresse des Puffers) SIZE : UINT (Größe des Puffers) START : INT (Position ab der der String aus dem Puffer kopiert wird) STOP : INT (Ende des Strings im Puffer)
Output	STRING (Zeichenkette die aus dem Puffer kopiert wurde)



Die Funktion BUFFER_TO_STRING extrahiert einen String aus einem beliebigen Array of Byte. Der String wird ab einer beliebigen Position START aus dem Puffer kopiert und endet an der Position STOP. Das erste Element im Array hat die Positionsnummer 0. Beim Aufruf wird der Funktion ein Pointer auf das zu bearbeitende Array und dessen Größe in Bytes übergeben. Unter CoDeSys lautet der Aufruf: BUFFER_TO_STRING(ADR(Array), SIZEOF(ARRAY), START, STOP), wobei ARRAY der Name des Arrays ist. ADR ist eine Standardfunktion, die den Pointer auf das Array ermittelt und SIZEOF ist eine Standardfunktion, die die Größe des Arrays ermittelt. Die Funktion liefert die aus dem Puffer kopierte Zeichenkette als STRING zurück. Diese Art der Bearbeitung von Arrays ist äußerst effizient, da kein zusätzlicher Speicher benötigt wird und keine Übergabewerte kopiert werden müssen. Beispiel: BUFFER_TO_STRING(ADR(Array), SIZEOF(ARRAY), START, STOP)

Verzeichnis der Funktionsbausteine

ACOSH.....	30	BUFFER_TO_STRING.....	387
ACOTH.....	30	BURNER.....	312
ACTUATOR_2P.....	303	BYTE_OF_BIT.....	174
ACTUATOR_3P.....	304	BYTE_OF_DWORD.....	174
ACTUATOR_PUMP.....	306	BYTE_TO_BIT.....	175
ACTUATOR_UD.....	307	BYTE_TO_RANGE.....	221
AGDF.....	31	BYTE_TO_STRB.....	124
AIN.....	216	BYTE_TO_STRH.....	124
AIN1.....	217	C_TO_F.....	275
AIR_DENSITY.....	309	C_TO_K.....	275
AIR_ENTHALPY.....	309	C0.....	20
ALARM_2.....	258	CABS.....	70
AOUT.....	219	CACOS.....	71
AOUT1.....	220	CACOSH.....	71
ARRAY_AVG.....	62	CADD.....	71
ARRAY_GAV.....	63	CALIBRATE.....	261
ARRAY_HAV.....	64	CAPITALIZE.....	125
ARRAY_MAX.....	64	CARG.....	72
ARRAY_MIN.....	65	CASIN.....	72
ARRAY_SDV.....	66	CASINH.....	73
ARRAY_SPR.....	66	CATAN.....	73
ARRAY_SUM.....	67	CATANH.....	73
ARRAY_TREND.....	68	CCON.....	74
ARRAY_VAR.....	68	CCOS.....	74
ASINH.....	31	CCOSH.....	74
ASTRO.....	274	CDIV.....	75
ATAN2.....	31	CEIL.....	32
ATANH.....	32	CEIL2.....	33
B_TRIG.....	156	CEXP.....	75
BAR_GRAPH.....	259	CHAR_NAME_1.....	21
BCD_TO_INT.....	171	CHAR_NAME_2.....	21
BIT_COUNT.....	171	CHAR_NAME_3.....	21
BIT_LOAD_B.....	172	CHAR_NAME_4.....	21
BIT_LOAD_DW.....	172	CHARCODE.....	125
BIT_LOAD_W.....	173	CHARNAME.....	126
BIT_OF_DWORD.....	173	CHECK_PARITY.....	175
BLIND_ACTUATOR.....	340	CHR.....	127
BLIND_CONTROL.....	342	CINV.....	75
BLIND_INPUT.....	344	CIRCLE_A.....	91
BLIND_NIGHT.....	346	CIRCLE_C.....	91
BLIND_SCENE.....	348	CLICK.....	329
BLIND_SECURITY.....	350	CLK_1MS.....	156
BLIND_SHADE.....	352	CLK_2MS.....	157
BOILER.....	310	CLK_4MS.....	157
BUFFER_SEARCH.....	386	CLK_8MS.....	157

CLK_DIV.....	157	DIFFER.....	36
CLK_N.....	159	DIMM_I.....	331
CLK_PRG.....	160	DRIVER_1.....	356
CLK_PULSE.....	160	DRIVER_4.....	357
CLOG.....	76	DST.....	106
CMP.....	33	DW_TO_REAL.....	182
CMUL.....	76	DWORD_OF_BYTE.....	183
CODE.....	127	DWORD_TO_STRB.....	128
CONE_V.....	92	DWORD_TO_STRF.....	129
CONTROL_SET1.....	283	DWORD_TO_STRH.....	129
CONTROL_SET2.....	284	E1.....	20
COSH.....	34	EASTER.....	107
COTH.....	34	EL.....	20
COUNT_BR.....	192	ELLIPSE_A.....	92
COUNT_CHAR.....	128	ELLIPSE_C.....	93
CPOL.....	76	ENERGY.....	275
CPOW.....	77	ERF.....	36
CRC_CHECK.....	176	ERFC.....	37
CRC_GEN.....	177	ESR_COLLECT.....	23
CSET.....	77	ESR_MON_B8.....	25
CSIN.....	78	ESR_MON_R4.....	26
CSINH.....	78	ESR_MON_X8.....	27
CSQRT.....	78	EVEN.....	37
CSUB.....	79	EXEC.....	130
CTAN.....	79	EXP10.....	38
CTANH.....	79	EXPN.....	38
CYCLE_TIME.....	262	EXTENDED_ASCII.....	21
D_TRIG.....	161	F_LAMP.....	332
D_TRUNC.....	35	F_LIN.....	83
DATE_ADD.....	102	F_LIN2.....	83
DAY_OF_DATE.....	103	F_POLY.....	84
DAY_OF_MONTH.....	103	F_POWER.....	84
DAY_OF_YEAR.....	103	F_QUAD.....	85
DAY_TO_TIME.....	104	F_TO_C.....	276
DAYS_DELTA.....	104	F_TO_OM.....	276
DCF77.....	105	F_TO_PT.....	277
DEAD_BAND.....	286	FACT.....	39
DEAD_ZONE.....	286	FADE.....	224
DEAD_ZONE2.....	287	FF_D2E.....	193
DEC_2.....	179	FF_D4E.....	194
DEC_4.....	180	FF_DRE.....	195
DEC_8.....	181	FF_JKE.....	196
DEG.....	35	FIB.....	39
DELAY.....	221	FIFO_16.....	153
DELAY_4.....	222	FIFO_32.....	154
DEW_CON.....	316	FILL.....	130
DEW_R.....	316	FIND_CHAR.....	131
DEW_TEMP.....	317	FIND_CTRL.....	131

FIND_NONUM.....	132	HYPOT.....	44
FIND_NUM.....	132	HYST_1.....	300
FINDB.....	133	HYST_2.....	301
FINDB_NONUM.....	133	HYST_3.....	301
FINDB_NUM.....	134	INC.....	44
FINDP.....	134	INC_DEC.....	360
FIX.....	135	INT_TO_BCD.....	183
FLOOR.....	40	INTERLOCK.....	361
FLOOR2.....	40	IP4_CHECK.....	377
FRACT.....	41	IP4_DECODE.....	378
FT_AVG.....	85	IP4_TO_STRING.....	379
FT_DERIV.....	288	IRTRANS_1.....	370
FT_IMP.....	288	IRTRANS_4.....	372
FT_INT.....	289	IRTRANS_8.....	372
FT_MIN_MAX.....	86	IRTRANS_DECODE.....	373
FT_PI.....	290	IS_ALNUM.....	136
FT_PID.....	293	IS_ALPHA.....	136
FT_PROFILE.....	357	IS_CC.....	137
FT_PT1.....	295	IS_CTRL.....	137
FT_PT2.....	297	IS_HEX.....	138
FT_RMP.....	87	IS_IP4.....	379
FT_TN16.....	298	IS_LOWER.....	138
FT_TN64.....	298	IS_NCC.....	139
FT_TN8.....	299	IS_NUM.....	139
GAMMA.....	41	IS_SORTED.....	69
GAUSS.....	42	IS_UPPER.....	140
GAUSSCD.....	42	IS_URLCHR.....	380
GDF.....	43	ISC_ALPHA.....	140
GEN_BIT.....	162	ISC_CTRL.....	141
GEN_RDM.....	205	ISC_HEX.....	142
GEN_RDT.....	206	ISC_LOWER.....	142
GEN_RMP.....	206	ISC_NUM.....	143
GEN_SIN.....	207	ISC_UPPER.....	143
GEN_SQ.....	164	K_TO_C.....	279
GEN_SQR.....	209	KMH_TO_MS.....	277
GN.....	20	LANGEVIN.....	45
GOLD.....	43	LATCH.....	198
H.....	316	LATCH4.....	198
HEAT_INDEX.....	317	LEAP_OF_DATE.....	109
HEAT_METER.....	318	LEAP_YEAR.....	110
HEAT_TEMP.....	319	LEGIONELLA.....	321
HEX_TO_DWORD.....	135	LENGTH.....	277
HOLIDAY.....	107	LINEAR_INT.....	88
HOUR.....	108	LOCAL_TIME.....	110
HOUR_TO_TIME.....	109	LOWERCASE.....	144
HOUR_TO_TOD.....	109	LTIME.....	111
HTML_DECODE.....	376	LTIME_TO_UTC.....	112
HTML_ENCODE.....	377	M_D.....	265

M_T.....	266	R2_ADD2.....	81
M_TX.....	267	R2_MUL.....	82
MATRIX.....	184	RAD.....	49
MAX3.....	46	RANGE_TO_BYTE.....	230
MESSAGE_8.....	144	RANGE_TO_WORD.....	230
METER.....	262	RDM.....	49
METER_STAT.....	264	RDM2.....	50
MID3.....	47	RDMDW.....	50
MIN3.....	46	REAL_TO_DW.....	188
MINUTE.....	112	REAL_TO_STRF.....	146
MINUTE_TO_TIME.....	113	REPLACE_ALL.....	147
MIRROR.....	146	REPLACE_CHARS.....	147
MIX.....	225	REPLACE_UML.....	148
MODR.....	47	RES_NI.....	251
MONTH.....	113	RES_PT.....	252
MS_TO_KMH.....	279	RES_SI.....	253
MUL_ADD.....	48	REVERSE.....	188
MULTI_IN.....	250	RMP_B.....	211
MULTIME.....	114	RMP_SOFT.....	214
MUX_2.....	186	RMP_W.....	215
MUX_4.....	186	RND.....	51
MUX_R2.....	225	ROUND.....	52
MUX_R4.....	226	RTC_2.....	115
NEG.....	48	RTC_MS.....	116
NETWORK_BUFFER_SIZE.....	21	SCALE.....	231
OFFSET.....	226	SCALE_B.....	231
OFFSET2.....	228	SCALE_B2.....	232
OM_TO_F.....	279	SCALE_B4.....	234
ONTIME.....	268	SCALE_B8.....	235
OSCAT_VERSION.....	28	SCALE_D.....	236
OVERRIDE.....	229	SCALE_R.....	236
PARITY.....	187	SCALE_X2.....	237
PARSET.....	362	SCALE_X4.....	238
PARSET2.....	363	SCALE_X8.....	239
PERIOD.....	114	SDD.....	324
PERIOD2.....	115	SECOND.....	117
PI05.....	20	SECOND_TO_TIME.....	117
PI1.....	20	SEL2_OF_3.....	240
PI2.....	20	SEL2_OF_3B.....	240
PN.....	20	SELECT_8.....	197
POLYNOM_INT.....	89	SENSOR_INT.....	254
PRESSURE.....	280	SEQUENCE_4.....	165
PT_TO_F.....	281	SEQUENCE_8.....	168
PULSE_LENGTH.....	333	SET_DATE.....	118
PWM_DC.....	210	SET_DT.....	119
PWM_PW.....	211	SET_TOD.....	119
R2_ABS.....	80	SGN.....	52
R2_ADD.....	81	SH.....	241

SH_1.....	242	TIMER_2.....	336
SH_2.....	243	TK.....	20
SH_T.....	245	TO_LOWER.....	149
SHL1.....	189	TO_UML.....	149
SHR_4E.....	199	TO_UPPER.....	150
SHR_4UDE.....	200	TOGGLE.....	204
SHR_8PLE.....	201	TONOF.....	169
SHR_8UDE.....	202	TP_R.....	170
SHR1.....	189	TRIANGLE_A.....	94
SIGMOID.....	53	TRIM.....	151
SIGN_I.....	53	TRIM1.....	151
SIGN_R.....	54	TRIME.....	151
SIGNAL.....	364	TUNE.....	368
SIGNAL_4.....	365	TUNE2.....	369
SINC.....	54	UG.....	20
SINH.....	54	UPPERCASE.....	152
SPEED.....	281	URL_DECODE.....	381
SPHERE_V.....	93	URL_ENCODE.....	381
SQRTN.....	55	URL_TO_STRING.....	382
SRAMP.....	366	V3_ABS.....	95
STACK_16.....	154	V3_ADD.....	96
STACK_32.....	155	V3_ANG.....	96
STAIR.....	246	V3_DPRO.....	96
STAIR2.....	247	V3_NORM.....	97
STATUS_TO_ESR.....	28	V3_NUL.....	97
STORE_8.....	203	V3_PAR.....	98
STRING_LENGTH.....	21	V3_REV.....	98
STRING_TO_BUFFER.....	385	V3_SMUL.....	99
STRING_TO_URL.....	380	V3_SUB.....	99
SUN_POS.....	120	V3_XANG.....	100
SUN_TIME.....	121	V3_XPRO.....	100
SWAP_BYTE.....	190	V3_YANG.....	101
SWAP_BYTE2.....	190	V3_ZANG.....	101
SWITCH_I.....	334	WATER_CP.....	327
SWITCH_X.....	334	WATER_DENSITY.....	327
T_PLC_MS.....	269	WATER_ENTHALPY.....	328
T_PLC_US.....	270	WCT.....	328
TANH.....	55	WEEKDAY.....	122
TC_MS.....	272	WINDOW.....	56
TC_S.....	272	WINDOW2.....	56
TC_US.....	272	WORD_OF_BYTE.....	190
TEMP_EXT.....	325	WORD_OF_DWORD.....	191
TEMP_NI.....	255	WORD_TO_RANGE.....	249
TEMP_PT.....	256	WORK_WEEK.....	122
TEMP_SI.....	257	YEAR.....	123
TEMPERATURE.....	282	YEAR_BEGIN.....	123
TICKER.....	148	_ARRAY_ABS.....	57
TIMER_1.....	335	_ARRAY_ADD.....	58

_ARRAY_INIT.....	58	_ARRAY_SORT.....	62
_ARRAY_MEDIAN.....	59	_BUFFER_CLEAR.....	383
_ARRAY_MUL.....	60	_BUFFER_INIT.....	384
_ARRAY_SHUFFLE.....	61	_BUFFER_INSERT.....	384